

The Backend Engineer's Library

Databases and Storage Systems

Volume 5

Contents

The Relational Model and SQL Semantics

Storage Engines: B-Trees vs LSM-Trees

Indexing in Depth

Query Processing and Optimization

Transactions and ACID

Isolation Levels and MVCC

Write-Ahead Logging and Crash Recovery

Replication: Physical, Logical, Sync, and Async

Partitioning and Sharding

Distributed Transactions: 2PC, Sagas, and Alternatives

NoSQL: Key-Value, Document, Wide-Column, and Graph

NewSQL and Distributed SQL

Specialized Stores: Search, Time-Series, and Analytics

Operating Databases: Pooling, Migrations, and Scaling

Chapter 1 — The Relational Model and SQL Semantics

What this chapter covers. This volume descends, over fourteen chapters, from the logical surface of a database to its physical depths — storage engines, indexes, query optimizers, transactions, replication, sharding — and then back up to the distributed SQL systems that reassemble all of it across machines. Before any of that, we need the surface itself, stated precisely. This chapter is about the relational model as Codd actually defined it in 1970, and about SQL as it actually behaves — which differs from the model in ways that produce real production bugs. We start with the one idea that made everything else in this volume possible: the separation of the logical data model from physical storage. We then work through the parts of SQL that senior engineers most often get subtly wrong: NULL and three-valued logic, bag versus set semantics, the logical evaluation order of a SELECT, grouping correctness, the NULL-poisoned difference between `NOT IN` and `NOT EXISTS`, window frames, and recursive queries. We treat relational algebra not as theory for its own sake but as the vocabulary of query plans — every `EXPLAIN` output you will read in Chapter 4 is an algebra tree, and you cannot read plans fluently without it. We close with normalization as a pragmatic engineering discipline rather than a purity contest, and with the distributed-systems lens: the schema decisions this chapter teaches you to make are precisely the decisions that determine, years later, whether your database can be sharded at all.

Learning goals — after this chapter you should be able to:

- State the relational model precisely: relations as sets of tuples over typed attributes, and why *data independence* is the foundational idea rather than an implementation detail.
- Define candidate, primary, and foreign keys, and the four constraint families — entity, referential, domain, and CHECK — including how each treats NULL.
- Reason in three-valued logic: predict the result of any predicate over NULLs, and identify the four classic 3VL bug patterns in real queries.

- Translate between SQL and relational algebra — selection, projection, the join varieties including semi and anti joins, and aggregation — well enough to read a query plan.
- Recite the logical evaluation order of a SELECT and use it to explain concrete errors, such as why a SELECT alias is invisible to WHERE.
- Use window functions with explicit frames, and explain why ROWS and RANGE frames differ in the presence of ties.
- Write a correct WITH RECURSIVE query and explain its working-table semantics.
- Carry a schema from unnormalized to BCNF via functional dependencies, name the anomaly each step removes, and argue honestly about when to denormalize.

Codd's model: relations, tuples, and the great separation

E. F. Codd's 1970 paper, *A Relational Model of Data for Large Shared Data Banks*, is nine pages long and is plausibly the highest-leverage systems paper ever published. To see why, you have to see what it was written against. The databases of 1970 — IBM's hierarchical IMS, the CODASYL network model — exposed physical structure directly to applications. A program navigated from record to record by following pointers; queries were expressed as traversal code. If the database administrator changed an access path, added an index, or reordered records, application code broke. Codd names this failure precisely in the paper's opening: the data dependencies of ordering, indexing, and access path. His proposal was to eliminate all three at once.

The model itself is austere. A **relation** is a *set* of **tuples**. Each tuple assigns a value to each of a fixed collection of named, typed **attributes**; the type of an attribute — its **domain** — is the set of values it may take. Three consequences of the word "set" do all the work, and each one is a promise the model makes to the optimizer:

1. **No duplicate tuples.** A relation cannot contain the same tuple twice; a fact stated twice is the same fact.
2. **No tuple ordering.** A relation has no first row. Any order you observe is an accident of physical storage, and you may not depend on it.

3. **No positional attributes.** Attributes are identified by name, not position. There is no "third column."

Every value in a tuple is atomic with respect to the model — the model gives you no way to reach inside a value and address its parts relationally. (This is the seed of first normal form, which we return to later.)

The payoff of this austerity is **data independence**, and it is the single most important idea in this volume. Because the application speaks only in terms of relations, attributes, and predicates — never in terms of files, pages, pointers, or indexes — the physical layer underneath is free to change without breaking anything above it. Codd distinguished what we now call *physical* data independence (storage layout, access paths, and indexes can change under a fixed schema) from *logical* data independence (the schema itself can evolve, with views insulating applications from the change).

It is worth pausing on how much rode on this bet, because we live in its consequences. Everything in Chapters 2 through 4 of this volume — heap files versus clustered B-trees versus LSM-trees, every index type, every join algorithm, cost-based optimization itself — exists *below* the line Codd drew, and could therefore be invented, deployed, and replaced for five decades without applications changing a line of SQL. Your query from 1995 runs against a storage engine designed in 2020. Row stores became column stores for analytics; nested-loop joins became hash joins; single machines became clusters — and the interface held. No other interface in computing has absorbed this much implementation churn. When Chapter 12 shows you distributed SQL systems executing ordinary SQL across dozens of sharded nodes, that is not a new idea succeeding; it is the 1970 idea being cashed out one more time.

Keys

The model identifies tuples by value, not by pointer or position, so it needs a vocabulary for identifying values.

A **superkey** is any set of attributes whose values uniquely identify a tuple within the relation. A **candidate key** is a *minimal* superkey — remove any attribute and uniqueness is lost. A relation can have several candidate keys: an `employees` relation might be uniquely identified by `employee_id` and also by `email`. The **primary key** is the candidate key you designate as the relation's canonical identifier; the others become

alternate keys, typically enforced with UNIQUE constraints. A **foreign key** is a set of attributes in one relation whose values must appear as a candidate key value in another (or be NULL) — it is how the relational model expresses relationships without pointers: by shared values.

The distinction between *natural* keys (attributes with real-world meaning, like `email`) and *surrogate* keys (synthetic identifiers, like a `bigint` from a sequence or a UUID) is an engineering decision the model is silent on. The working consensus for OLTP systems is surrogate primary keys with natural keys enforced as UNIQUE — real-world identifiers change (emails, usernames, even national ID numbers), and a primary key that changes propagates updates through every referencing foreign key. The choice *between* surrogate types (sequential versus random UUID) is a physical-layer question with real consequences for B-tree insertion patterns, and Chapter 3 takes it up.

Integrity constraints

Constraints are the model's mechanism for making the database, rather than every application that writes to it, the enforcer of invariants. Four families:

- **Entity integrity:** no component of a primary key may be NULL. A tuple you cannot identify is not a fact.
- **Referential integrity:** every non-NULL foreign key value must match an existing referenced key. SQL lets you choose what happens on violation-by-deletion: `ON DELETE RESTRICT` (refuse), `CASCADE` (propagate), `SET NULL` (orphan explicitly). Treat `CASCADE` with suspicion in large systems — a delete that fans out across tables is a foot-gun at scale, and an invisible one in code review.
- **Domain constraints:** attribute values come from the declared type — and SQL types are coarse domains at best. `NOT NULL` belongs here; so do enumerated types.
- **CHECK constraints:** arbitrary row-level predicates — `CHECK (price_cents >= 0)`, `CHECK (starts_at < ends_at)`.

One asymmetry deserves a flag now, because it foreshadows the next section: a WHERE clause keeps a row only when its predicate is TRUE, but a CHECK constraint rejects a row only when its predicate is FALSE. A CHECK that evaluates to UNKNOWN — because some operand is NULL — *passes*. `CHECK (price_cents >= 0)` happily admits a NULL price. If you

meant to forbid that, you needed `NOT NULL` too. This is not a quirk; it is your first encounter with three-valued logic, which has a whole section below because it has earned one.

The engineering argument for declaring constraints in the database rather than only in application code is the same argument as for types: the invariant is enforced at the last possible moment, against *all* writers — every service, every migration script, every engineer with a psql prompt — not just the well-behaved ones. In a multi-service organization the database constraint is frequently the only invariant that is actually true.

NULL and three-valued logic, honestly

NULL is the most bug-productive feature in SQL, and the bugs are not caused by NULL itself but by engineers reasoning in two-valued logic about a three-valued system. The rules are small. Learn them once, properly.

NULL is not a value. It is a marker meaning *no value here* — absent, unknown, inapplicable (SQL deliberately refuses to distinguish these). Because NULL is not a value, comparing anything to it — including another NULL — cannot yield TRUE or FALSE. It yields the third truth value, **UNKNOWN**:

```
SELECT NULL = NULL;    -- NULL (i.e., UNKNOWN – not TRUE)
SELECT NULL <> NULL;    -- NULL
SELECT 1 = NULL;       -- NULL
```

The connectives extend to three values the way "unknown" intuitively should: `NOT UNKNOWN` is UNKNOWN; `UNKNOWN AND TRUE` is UNKNOWN but `UNKNOWN AND FALSE` is FALSE (a conjunction with a false conjunct is false no matter what the unknown turns out to be); `UNKNOWN OR TRUE` is TRUE but `UNKNOWN OR FALSE` is UNKNOWN. To *test* for NULL you must leave ordinary comparison and use the dedicated predicates `IS NULL` / `IS NOT NULL`, or the two-valued comparison `IS [NOT] DISTINCT FROM`, which treats two NULLs as equal and never returns UNKNOWN.

That is the entire theory. The damage comes from four places where UNKNOWN silently changes query results. Each of the following is a bug I have seen shipped.

Trap 1: WHERE eliminates UNKNOWN rows. WHERE keeps rows whose predicate is TRUE. Rows evaluating to FALSE or UNKNOWN are dropped. The consequence: a filter and its negation do not partition the table.

```
CREATE TABLE tickets (  
  id          bigint PRIMARY KEY,  
  severity    int,           -- NULL until triaged  
  title       text NOT NULL  
);  
INSERT INTO tickets VALUES (1, 1, 'db down'), (2, 3, 'typo'), (3,  
NULL, 'untriaged');  
  
SELECT count(*) FROM tickets WHERE severity >= 2;  -- 1 (ticket 2)  
SELECT count(*) FROM tickets WHERE severity < 2;  -- 1 (ticket 1)  
-- Ticket 3 appears in NEITHER. "Everything is either >= 2 or < 2" is  
-- two-valued reasoning, and it is false here.
```

The real-world version is a dashboard that splits traffic into "EU" and "non-EU" and quietly loses every row with a NULL region — the two panels no longer sum to the total, and nobody notices for a quarter.

Trap 2: NOT IN with a NULL in the list returns nothing. This is the classic, and it is worth deriving rather than memorizing. `x NOT IN (a, b, c)` means `x <> a AND x <> b AND x <> c`. If any list element is NULL, that conjunct is UNKNOWN, so the whole conjunction is either FALSE (if x matches some element) or UNKNOWN (if it doesn't) — it can never be TRUE. WHERE keeps only TRUE. Result: zero rows, always.

```
CREATE TABLE customers (id bigint PRIMARY KEY, referrer_id bigint);  
-- NULLable  
INSERT INTO customers VALUES (1, NULL), (2, 1), (3, 1);  
  
-- "Customers who never referred anyone":  
SELECT id FROM customers  
WHERE id NOT IN (SELECT referrer_id FROM customers);  
-- Returns ZERO rows: the subquery yields {NULL, 1}, and NOT IN over a  
-- set containing NULL cannot be TRUE for any id.  
  
-- Correct: NOT EXISTS is a genuine anti-join and is NULL-safe.  
SELECT c.id FROM customers c  
WHERE NOT EXISTS (  
  SELECT 1 FROM customers r WHERE r.referrer_id = c.id  
);  
-- Returns 2 and 3.
```


The insidious part is the failure mode over time: the query is *correct* while the subquery column happens to contain no NULLs, then returns empty forever after the first NULL row is inserted. The rule of thumb is blunt: never write `NOT IN (subquery)` against a nullable column; write `NOT EXISTS`.

Trap 3: aggregates skip NULLs — except `COUNT(*)`. All standard aggregates (`SUM`, `AVG`, `MIN`, `MAX`, `COUNT(expr)`) ignore NULL inputs. `COUNT(*)` counts rows regardless.

```
CREATE TABLE reviews (id bigint PRIMARY KEY, rating int);
-- NULL = no rating left
INSERT INTO reviews VALUES (1, 5), (2, 4), (3, NULL), (4, NULL);

SELECT count(*)      AS rows,          -- 4
       count(rating) AS rated,         -- 2
       avg(rating)   AS avg_rating    -- 4.5 (not 2.25!)
FROM reviews;
```

Whether 4.5 or 2.25 is "the average rating" is a product question, but the database has silently answered it for you. Note also that aggregates over an *empty* input yield NULL, not zero — `SUM` of no rows is NULL — which then feeds Trap 1 in any enclosing predicate; `COALESCE(SUM(x), 0)` is the standard antidote.

Trap 4: comparisons in joins. Join predicates are predicates: a NULL join key matches nothing, including another NULL. Rows with NULL keys silently vanish from inner joins. When Chapter 8's replication and Chapter 11's service-merging patterns have you combining data from multiple sources — where "field not populated by that source" is routine — this is the trap that multiplies. Two datasets that each look complete produce a join that is quietly missing every row either side left NULL.

Two final notes for calibration. First, `GROUP BY`, `DISTINCT`, and `ORDER BY` do *not* use UNKNOWN-producing comparison — they treat NULLs as a single group / duplicate class, and sort them together (last by default in Postgres ascending order; `NULLS FIRST/LAST` controls it). SQL is not even consistently three-valued, which is part of why it is hard. Second, the standard's own committee has been ambivalent about NULL for forty years, and C. J. Date has argued for decades that 3VL was a mistake. You do not get to relitigate it; you get to declare `NOT NULL` wherever a value is genuinely required — the cheapest bug-prevention available in a

schema — and reach for `IS DISTINCT FROM` when you need two-valued comparison.

Relational algebra: the vocabulary of query plans

Codd gave the model two equivalent query languages: relational *calculus* (declarative — describe the tuples you want) and relational *algebra* (operational — a set of operators that each take relations and produce a relation). SQL descends from the calculus side, but the algebra is what you must know, for one compelling reason: **query plans are algebra trees**. When Chapter 4 shows the optimizer parsing your SQL, rewriting it, and choosing among physical plans, every intermediate form is relational algebra. `EXPLAIN` is printed algebra. Learn the operators and plans stop being wall-of-text and start being sentences.

The operators, with their SQL correspondences:

Algebra	Symbol	SQL surface
Selection	σ	<code>WHERE</code> — filter rows by predicate
Projection	π	<code>SELECT</code> column list — keep named attributes (set semantics: dedup)
Cartesian product	$\times$	<code>CROSS JOIN</code>
Inner join	$\bowtie$	<code>JOIN ... ON</code> — product plus selection, fused
Outer joins	$\bowtie\!\!\!\bowtie$ $\bowtie\!\!\!\bowtie$	<code>LEFT/RIGHT/FULL JOIN</code> — preserve unmatched rows, NULL-padded
Semi join	$\ltimes$	<code>WHERE EXISTS (...)</code> / <code>IN (...)</code> — filter left by existence of a match
Anti join	$\rhd$	<code>WHERE NOT EXISTS (...)</code> — filter left by absence of a match
Union / Intersect / Except	$\cup$ $\cap$ $-$	<code>UNION</code> / <code>INTERSECT</code> / <code>EXCEPT</code>
Rename	ρ	<code>AS</code> aliases
Grouping/ aggregation	γ	<code>GROUP BY</code> with aggregates (an extension beyond Codd's original set)

Semi and anti joins deserve emphasis because they exist in every optimizer and in almost no SQL textbooks. A **semi join** returns each left row at most once if *any* matching right row exists — it never duplicates left rows and never produces right columns, which is exactly the semantics of `EXISTS`. An **anti join** returns left rows with *no* match — the semantics of `NOT EXISTS`. When you write `EXISTS`, the optimizer does not run the subquery per row like a naive interpreter; it plans a semi join and picks a physical algorithm (hash, merge, indexed nested loop) like any other join. This is also where Trap 2 comes home: the optimizer can convert `NOT IN` to an anti join *only* when it can prove the relevant columns are `NOT NULL` — otherwise it must preserve the deranged 3VL semantics with a slower plan. Your `NOT NULL` declarations are optimizer input.

The property that makes the algebra an algebra is **closure**: every operator consumes relations and produces a relation. Closure is why operators compose into arbitrary trees, why a view or CTE can be substituted anywhere a table can, and why the optimizer may legally rewrite your tree into any equivalent one — pushing a selection below a join, reordering joins, splitting an aggregation. Hold onto closure; the distributed-systems lens at the end of this chapter rests on it.

Here is the correspondence made concrete. The query: revenue per customer for large completed orders since a date, for customers in a given region.

```
SELECT c.name, sum(o.amount_cents) AS revenue_cents
FROM customers c
JOIN orders o ON o.customer_id = c.id
WHERE o.status = 'completed'
      AND o.placed_at >= DATE '2026-01-01'
      AND c.region = 'emea'
GROUP BY c.name;
```

```
flowchart BT
    CUST["customers"] --> SC["selection: region = 'emea'"]
    ORD["orders"] --> S0["selection: status = 'completed'"]
    AND["placed_at >= 2026-01-01"]
    SC --> J["join: o.customer_id = c.id"]
    S0 --> J
    J --> G["group by c.name"]
    aggregate["sum of amount_cents"]
    G --> P["projection: name, revenue_cents"]
```

Read bottom-up: base relations at the leaves, result at the root. Notice the tree already shows one rewrite the optimizer will always make — the selections have been *pushed down* below the join, so the join sees only EMEA customers and completed recent orders rather than filtering after a full join. Chapter 4 develops the full rewrite repertoire; the point here is that the rewrites are *provable* precisely because both trees denote the same algebra expression.

SQL semantics that bite

SQL is not the relational algebra, and the deviations are exactly where production bugs live.

Bags, not sets

SQL tables and query results are **bags** (multisets): duplicates are allowed and ordinary operations preserve them. This was a pragmatic choice — eliminating duplicates costs a sort or hash over the whole result, and users often want duplicate-preserving behavior for aggregation — but it means the model's "no duplicate tuples" promise holds only where you enforce it with keys. The operational consequences:

- `SELECT` (projection) does not deduplicate; `SELECT DISTINCT` does, at the cost of a hash/sort you should notice in plans.
- `UNION` deduplicates (set semantics); `UNION ALL` does not (bag semantics) and is therefore cheaper. The number of times `UNION` appears in a codebase where `UNION ALL` was meant is large, and each one is a silent sort of the combined result. Default to `UNION ALL` unless you specifically need dedup — and if inputs are disjoint by construction, dedup is pure waste.
- Duplicates interact with aggregation: `count(x)` versus `count(DISTINCT x)` answer different questions, and a join that fans out rows (one-to-many) before an aggregate silently inflates sums — the classic "revenue doubled after we joined in shipments" bug. Aggregate first in a subquery or use `DISTINCT` deliberately, but know which question you are answering.

The logical evaluation order of a `SELECT`

A `SELECT` statement is written in one order and *means* another. The standard defines a logical evaluation order — every conforming

implementation must behave *as if* it evaluated clauses in this sequence (the optimizer will do something cleverer, but observably equivalent):

```
flowchart LR
    F["FROM"] --> W["WHERE"]
    W --> G["GROUP BY"]
    G --> H["HAVING"]
    H --> S["SELECT list"]
    S --> D["DISTINCT"]
    D --> O["ORDER BY"]
    O --> L["LIMIT / OFFSET"]
```

This one diagram resolves a whole family of "why won't SQL let me..." questions mechanically:

- **Why can't WHERE see a SELECT alias?** `SELECT price * qty AS total ... WHERE total > 100` fails because WHERE runs before the SELECT list exists. Repeat the expression, or compute it in a subquery/CTE and filter outside. (`ORDER BY` runs *after* SELECT, which is why it *can* use the alias.)
- **Why HAVING at all?** WHERE filters rows before grouping; HAVING filters *groups* after aggregation. `WHERE count(*) > 5` is meaningless — there are no groups yet — and `HAVING price > 100` on a non-grouped column is equally confused in the other direction.
- **Why can't WHERE contain a window function?** Windows are computed with the SELECT list, after WHERE. Filtering on `row_number()` requires wrapping the query and filtering the outer level — the top-N-per-group idiom below.
- **Why does ON versus WHERE matter for outer joins?** For an inner join, a predicate in `ON` and the same predicate in `WHERE` are equivalent. For a `LEFT JOIN` they are not: `ON` decides *what matches* (unmatched left rows survive, NULL-padded), while `WHERE` filters the joined result — and a WHERE predicate on right-side columns silently turns your left join back into an inner join, because the NULL-padded rows evaluate to UNKNOWN and are dropped (Trap 1 again, wearing a join costume).
- **Why is LIMIT without ORDER BY nondeterministic?** LIMIT runs last, over a result whose order is — per the model — undefined. It cuts *some* N rows. The order you observed in development was a physical accident (a sequential scan of a fresh heap); the different order in production (an index scan, a parallel scan, post-vacuum layout) is equally legal. Relatedly, `OFFSET`-based pagination without a

total order over a *unique* key skips or repeats rows across pages; keyset pagination (`WHERE (placed_at, id) > (:last_at, :last_id)` `ORDER BY placed_at, id LIMIT 50`) fixes both correctness and the $O(\text{offset})$ cost.

GROUP BY correctness

Once rows are grouped, each output row represents a *group*, and the `SELECT` list may only reference things that are single-valued per group: the grouping columns, aggregates, and — this is the subtle part — columns *functionally dependent* on the grouping columns. The standard (since SQL:1999) permits the functional-dependence case; Postgres implements the most useful instance — group by a table's primary key and you may select any column of that table:

```
-- Legal in Postgres: c.id is the primary key, so c.name is
-- functionally dependent on the grouping column.
SELECT c.id, c.name, count(o.id)
FROM customers c LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id;
```

MySQL's history here is instructive. For years it accepted *any* non-grouped column and returned a value from some arbitrary row of the group — nondeterministic results, silently. Since 5.7 the `ONLY_FULL_GROUP_BY` SQL mode is on by default and it now enforces the rule (with its own functional-dependence analysis). If you operate an older or laxer-configured MySQL, every such query is a latent wrong-answer bug that will surface when the optimizer changes its access path.

Subqueries: correlated, uncorrelated, and the EXISTS/IN split

An **uncorrelated** subquery references nothing from the outer query; logically it runs once. A **correlated** subquery references outer columns; logically it runs per outer row — though, per data independence, the optimizer is free to *decorrelate* it into a join, and good optimizers usually do. The semantics you must actually memorize is the `NULL` behavior:

- `x IN (subquery)` : `TRUE` if `x` equals some result value; `FALSE` only if the subquery result is `NULL`-free and contains no match; otherwise `UNKNOWN`.
- `x = ANY (...)` is identical to `IN`. `x <> ALL (...)` is identical to `NOT IN` — and inherits its `NULL` pathology.

- **EXISTS (subquery)** : TRUE or FALSE, *never UNKNOWN*. It asks only "is the result nonempty," so NULLs inside the subquery are irrelevant.

Hence the working rule from Trap 2, now fully justified: **EXISTS / NOT EXISTS** are the two-valued, optimizer-friendly, NULL-safe way to express semi and anti joins; **IN** is acceptable sugar for the positive case; **NOT IN** is safe only against provably NOT NULL columns, and your reviewer should not have to prove it.

Window functions

Window functions, standardized in SQL:2003, compute a value for each row using a *window* of related rows — without collapsing rows the way GROUP BY does. They replaced a generation of self-join gymnastics: before them, "running total" meant joining a table to itself on `t2.date <= t1.date` and aggregating — an $O(n^2)$ construction that optimizers struggled with and juniors got wrong; "top 3 per group" meant a correlated count subquery. Window functions express both directly, and the engine computes them in a single pass over sorted partitions.

A window specification has three parts, each defaulting sensibly and each worth making explicit:

- **PARTITION BY** — splits rows into independent groups; the function restarts per partition. Omitted: one partition of everything.
- **ORDER BY** — orders rows *within* the partition, defining "before" for running computations and rank.
- **Frame** — for aggregate windows, which rows around the current row are aggregated: **ROWS / RANGE / GROUPS BETWEEN ... AND ...**.

```
SELECT
  customer_id,
  placed_at,
  amount_cents,
  sum(amount_cents) OVER (
    PARTITION BY customer_id
    ORDER BY placed_at
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total_cents,
  row_number() OVER (PARTITION BY customer_id ORDER BY placed_at) AS order_seq
FROM orders;
```

```

flowchart TB
    subgraph PART["Partition: customer_id = 42, ordered by placed_at"]
        R1["row 1: 500"] --> R2["row 2: 300"] --> R3["row 3: 700"]
        R3 --> R4["row 4: 200"] --> R5["row 5: 900"]
    end
    FR["frame for row 3 with ROWS UNBOUNDED PRECEDING: rows 1..3 - running total = 1500"] -.-> R3
    FR2["frame with ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING: rows 2..4 - moving sum = 1200"] -.-> R3

```

The frame is where the bites are. When you write `ORDER BY` in a window and *omit* the frame, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — and `RANGE` includes all *peers* of the current row (rows equal under the `ORDER BY`). If two orders share a timestamp, both get the running total *including both*, which is rarely what a "running total" means, and the bug only manifests when ties exist — i.e., not in your test data. `ROWS` counts physical rows and does what you almost always mean; write the frame explicitly. (`GROUPS`, added in SQL:2011 and available in Postgres, frames by peer groups — occasionally exactly right, mostly a curiosity.)

The ranking trio differ only on ties, and the difference is a one-line table:

Input scores	row_number()	rank()	dense_rank()
95	1	1	1
90	2	2	2
90	3	2	2
85	4	4	3

`row_number` breaks ties arbitrarily (nondeterministically, unless the `ORDER BY` is total — another accidental-order trap); `rank` leaves gaps; `dense_rank` does not. The canonical top-N-per-group, exploiting the evaluation order (windows compute after `WHERE`, so the filter must sit outside):


```

SELECT *
FROM (
    SELECT o.*,
           row_number() OVER (PARTITION BY customer_id
                              ORDER BY amount_cents DESC, id) AS rn
    FROM orders o
) ranked
WHERE rn <= 3;

```

Note the `, id` tiebreaker making the ordering total and the result deterministic. Chapter 4 discusses how engines evaluate windows (sort per partition specification, shared between compatible windows) and Chapter 3 shows when an index can supply the order for free.

CTEs, recursion, and LATERAL

A common table expression — `WITH name AS (...)` — names a subquery for reuse and readability; by closure, it can stand anywhere a table can. One dialect note that matters operationally: Postgres before version 12 always materialized CTEs (an optimization fence — predicates could not be pushed into them); since 12 they inline by default, with `MATERIALIZED` / `NOT MATERIALIZED` as explicit overrides. Old advice about CTEs being slow in Postgres, and old code relying on the fence, both need rechecking.

`WITH RECURSIVE` (SQL:1999) is the real power tool: it computes a fixpoint, which is how SQL — whose plain algebra cannot express transitive closure — walks graphs and hierarchies. The semantics is precise and worth stating exactly, because mental models like "the CTE calls itself" predict wrong results. The engine maintains a **working table**:

1. Evaluate the non-recursive term; its rows become both the working table and the first slice of the result.
2. Evaluate the recursive term with the recursive self-reference bound to the *current working table only* — not the accumulated result. Its output becomes the new working table and is appended to the result.
3. Repeat step 2 until the working table is empty.

So the recursion is breadth-first iteration by levels, and the self-reference sees exactly one level back. A real hierarchy walk — services and the services they depend on, with depth and path, plus the cycle guard you must never omit on real (i.e., dirty) data:

```

CREATE TABLE service_deps (
    service      text NOT NULL,
    depends_on text NOT NULL,
    PRIMARY KEY (service, depends_on)
);

-- Everything 'checkout' transitively depends on:
WITH RECURSIVE reachable AS (
    SELECT d.depends_on,
           1 AS depth,
           ARRAY['checkout', d.depends_on] AS path
    FROM service_deps d
    WHERE d.service = 'checkout'

    UNION ALL

    SELECT d.depends_on,
           r.depth + 1,
           r.path || d.depends_on
    FROM reachable r
    JOIN service_deps d ON d.service = r.depends_on
    WHERE NOT d.depends_on = ANY (r.path) -- cycle guard
)
SELECT DISTINCT depends_on, min(depth) AS min_depth
FROM reachable
GROUP BY depends_on
ORDER BY min_depth;

```

Without the guard, one cycle in the data (service A depends on B depends on A — which *happens*, that's why you're writing this query) makes the working table never empty and the query run until it exhausts memory or a timeout kills it. `UNION` instead of `UNION ALL` deduplicates each level against the whole result and can terminate some cyclic queries, but carrying the path is the honest approach when you need it anyway. Postgres 14 added `CYCLE` and `SEARCH` clauses that generate this machinery for you.

LATERAL, briefly, because it completes the subquery story: a `LATERAL` subquery in the `FROM` clause may reference columns of tables to its left — it is a correlated subquery promoted to a full table expression, evaluated per left row. Its killer application is top-N-per-group when the groups come from another table, where it often beats the window formulation by doing N index probes instead of scanning and ranking everything:

```

SELECT c.name, recent.placed_at, recent.amount_cents
FROM customers c
CROSS JOIN LATERAL (
  SELECT o.placed_at, o.amount_cents
  FROM orders o
  WHERE o.customer_id = c.id
  ORDER BY o.placed_at DESC
  LIMIT 3
) recent;

```

Normalization, pragmatically

Normalization has a reputation as academic hair-splitting. It is not. It is the discipline of making a schema unable to contradict itself, and the anomalies it prevents are concrete production bugs. The theory rests on one concept: a **functional dependency** (FD) $X \rightarrow Y$ holds when tuples agreeing on attributes X must agree on attributes Y — X determines Y . FDs are facts about the *domain*, not the data you happen to have; you elicit them by asking "can this ever differ?" of the people who own the semantics.

Start with the failure. A team stores order lines in one wide table:

```

CREATE TABLE order_lines_flat (
  order_id      bigint,
  customer_id   bigint,
  customer_email text,
  product_id    bigint,
  product_name  text,
  unit_price_cents int,
  quantity      int,
  PRIMARY KEY (order_id, product_id)
);

```

The FDs: `order_id → customer_id` (an order belongs to one customer), `customer_id → customer_email`, `product_id → product_name`, `unit_price_cents`. Three anomaly families follow directly, and each is a real bug class:

- **Update anomaly:** a customer's email is stored once per order line. Changing it means updating many rows; miss one (a partial failure, a concurrent writer — Chapter 5) and the database now asserts two emails for one customer. Which is correct? The schema cannot say.

- **Insert anomaly:** you cannot record a new product's name and price until someone orders it — there is no row to put it in. Teams "solve" this with dummy orders, manufacturing fake facts to store real ones.
- **Delete anomaly:** deleting the last order line that references a product deletes your only record of its name and price. Removing one fact destroyed an unrelated one.

The normal forms are increasingly strict rules about where FDs may point; informally: **1NF** — atomic values, no repeating groups (the model's baseline); **2NF** — no non-key attribute depends on a *proper subset* of a composite key (`product_name` depends on `product_id` alone, violating it above); **3NF** — no non-key attribute depends on another non-key attribute (`customer_email` depends on `customer_id`, which depends on `order_id` — a transitive dependency); **BCNF** — the clean unifying statement: *for every nontrivial FD $X \rightarrow Y$, X is a superkey*. Every arrow comes from a key. The decomposition, guided by the FDs — one relation per determinant:

```

flowchart LR
    subgraph BEFORE["Before: one table, redundant facts"]
        FLAT["order_lines_flat  
order_id, customer_id, customer_email,  
product_id, product_name, unit_price_cents, quantity  
email repeated per line – update anomaly  
product needs an order to exist – insert anomaly  
last line deletion loses product – delete anomaly"]
    end
    subgraph AFTER["After BCNF: every fact stated once"]
        CUST2["customers  
id PK, email"]
        ORD2["orders  
id PK, customer_id FK"]
        PROD2["products  
id PK, name, unit_price_cents"]
        LINES2["order_lines  
order_id FK, product_id FK, quantity  
PK order_id, product_id"]
        ORD2 --> CUST2
        LINES2 --> ORD2
        LINES2 --> PROD2
    end
    BEFORE --> AFTER

```

```

CREATE TABLE customers (
  id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email text NOT NULL UNIQUE
);
CREATE TABLE products (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        text NOT NULL,
  unit_price_cents int NOT NULL CHECK (unit_price_cents >= 0)
);
CREATE TABLE orders (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES customers(id),
  placed_at   timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE order_lines (
  order_id  bigint NOT NULL REFERENCES orders(id),
  product_id bigint NOT NULL REFERENCES products(id),
  quantity  int NOT NULL CHECK (quantity > 0),
  PRIMARY KEY (order_id, product_id)
);

```

Every fact now lives in exactly one place; the anomalies are structurally impossible rather than procedurally avoided. (One deliberate wrinkle: if order lines must record the price *as sold* — a temporal fact distinct from the product's current price — then `sold_price_cents` on `order_lines` is not denormalization, it is a different fact. Knowing your FDs is what lets you make that distinction confidently.)

Two honest caveats before the pragmatics. First, BCNF and dependency preservation can genuinely conflict: the textbook case is a relation like `assignments(customer, product, engineer)` with FDs `{customer, product} → engineer` and `engineer → product` — 3NF but not BCNF, and the BCNF decomposition into `(engineer, product)` and `(customer, engineer)` can no longer enforce the first FD with keys alone. When this arises (rarely), 3NF plus a documented constraint is a defensible stopping point. Second, higher forms (4NF, 5NF) address multivalued and join dependencies; they matter occasionally, mostly when someone crams two independent many-to-many relationships into one table, and we leave them to the references.

Now the guidance, stated plainly. **Normalization is about write correctness; denormalization is about read performance.** A normalized schema makes updates cheap and safe (touch one row) and reads potentially expensive (joins); a denormalized one inverts the deal,

buying join-free reads at the price of redundancy that some mechanism — triggers, application code, an async pipeline — must now keep consistent, and that mechanism *is* the update anomaly, readmitted deliberately and hopefully managed. Sometimes that trade is right: read-heavy paths at scale, precomputed aggregates, the fan-out patterns of Chapter 9, and the read-model side of CQRS designs (Volume 7, Chapter 5 — Data Modeling — treats this in design terms). The modern answer is a sequenced policy, not a side in a war: **normalize until it hurts — as the default, in the system of record, because correctness bugs compound and storage-layer reads are cheaper than your intuition says (Chapters 2-3) — then denormalize where measured**, deliberately, with the consistency mechanism written down and owned. Denormalizing up front, on anticipated performance grounds, buys real anomalies today against hypothetical latency tomorrow; it is premature optimization applied to the one layer where mistakes are hardest to unwind — as Chapter 14 will show, schema migrations at scale are among the most dangerous operations in production databases.

Declarative SQL and the optimizer's bargain

SQL is declarative: you state *what* — a predicate over relations — and the system chooses *how*: which indexes, which join algorithms, which order. This is data independence applied to computation, and it is a bargain with two faces.

The bright face: fifty years of physical innovation arrived without application rewrites, and the optimizer applies plan-quality expertise at every query, every time, adapting to data statistics as tables grow — something no hand-written imperative traversal does.

The dark face: the abstraction leaks under pressure, and it leaks in performance, not correctness. Two semantically equivalent formulations can differ by orders of magnitude — the `NOT IN` versus `NOT EXISTS` pair above is exactly such a case (one is anti-joinable, the other must preserve 3VL semantics); so is a filter the optimizer can push into an index versus one wrapped in a function call that blinds it; so is the correlated subquery a given engine happens not to decorrelate; so is a stale statistics estimate flipping a hash join to a nested loop, turning a 10 ms query into a 10 s one — a 1000× swing with zero code change. The practitioner's response is not to memorize incantations, which rot as optimizers evolve, but to understand the model well enough to read the plan: know the algebra,

know what rewrites are legal, know what information the optimizer has (constraints, statistics, your NOT NULLs) and what you have denied it. `EXPLAIN` is the conversation; Chapter 4 teaches you to hold up your end. Everything in this chapter — algebra trees, semi/anti joins, functional dependencies, NULL-ability — is precisely the vocabulary that conversation is conducted in.

Standard SQL and the dialects you actually run

"SQL" is ISO/IEC 9075, a multi-part standard — Part 1 (Framework) and Part 2 (Foundation, the several-thousand-page core that defines everything in this chapter), plus parts you will meet later such as Part 11 (information schema) and, in SQL:2023, Part 16 (property-graph queries). Editions are named by year: SQL:1999 brought recursion; SQL:2003 brought window functions; SQL:2016 brought JSON; SQL:2023 is current as of this writing. No engine implements all of it, every engine extends it, and portability is a spectrum. The divergences worth carrying in your head for the two engines you most likely run:

Concern	Postgres	MySQL
Identifier quoting	<code>"double quotes"</code> (standard); folds unquoted to lowercase	<code>`backticks`</code> (unless <code>ANSI_QUOTES</code>); case sensitivity of table names varies by OS/filesystem
String literals	<code>'single quotes'</code> only	<code>'single'</code> and, non-standard, <code>"double"</code> by default
Upsert	<code>INSERT ... ON CONFLICT (col) DO UPDATE SET ...</code> (also standard <code>MERGE</code> since PG 15)	<code>INSERT ... ON DUPLICATE KEY UPDATE ...</code> (fires on <i>any</i> unique key, not a named target)
Returning modified rows	<code>RETURNING *</code> on <code>INSERT/UPDATE/DELETE</code>	Not in MySQL (MariaDB has <code>RETURNING</code>); use <code>LAST_INSERT_ID()</code> and a re-read
Text comparison	Case-sensitive by default	Default collations are case-insensitive: <code>'a' = 'A'</code> is true

Concern	Postgres	MySQL
GROUP BY laxity	FD-based (primary key) per standard	<code>ONLY_FULL_GROUP_BY</code> on by default since 5.7; historically returned arbitrary rows

The upsert row deserves one concrete example because everyone writes one eventually and the dialects genuinely differ in semantics, not just spelling — `ON CONFLICT` names its arbiter constraint, `ON DUPLICATE KEY` triggers on any unique violation:

```
-- Postgres: idempotent counter bump keyed on a named constraint,
-- returning the resulting row in the same round trip.
INSERT INTO page_views (page_id, day, views)
VALUES (:page_id, current_date, 1)
ON CONFLICT (page_id, day)
DO UPDATE SET views = page_views.views + 1
RETURNING views;
```

The pragmatic posture: write standard SQL where the standard suffices, use extensions deliberately and knowingly (an ORM will not save you — it merely picks a dialect for you), and when a query must be portable, test it on every engine it must be portable *to*, because the silent divergences (collation, GROUP BY laxity, quoting) are worse than the loud ones.

The distributed-systems lens

Everything in this chapter was defined for one node. All of it bears on many nodes, and the connections are structural, not analogical.

Data independence is the abstraction distributed SQL exploits. Your application speaks predicates over relations and never names a page, a file — or a *node*. That last omission is the entire opening that Chapter 12's NewSQL systems (Spanner, CockroachDB, and kin) walk through: partitioning, replica placement, and distributed execution are just more physical decisions below Codd's line, and the line was drawn in 1970 wide enough to hide them. The same SQL runs; the "storage engine" now spans a fleet. This is not a coincidence; it is the payout of the model's central bet, and it is why SQL — pronounced dead in 2009 — is the interface that distributed transactional systems converged back to.

Closure is why queries can be split across nodes. Because every algebra operator yields a relation, a plan tree can be *cut* at any edge:

evaluate the subtree below the cut on the node that holds the data, ship the resulting relation, continue above the cut elsewhere. Predicate pushdown to a shard, partial aggregation on each partition followed by a merge aggregate, shipping semi-join filters — every distributed execution strategy in Chapters 9 and 12 is an application of the closure property plus the rewrite rules of Chapter 4. An algebra without closure could not be partitioned this way; you would be shipping opaque program state instead of relations.

Constraints stop at the shard boundary. A foreign key is cheap when the referencing and referenced rows share a node: check an index, done. When a shard key splits them, every referential check becomes a cross-node round trip, and enforcing it transactionally requires the distributed-transaction machinery of Chapter 10 — so sharded systems commonly downgrade FKs to advisory or drop them entirely, and uniqueness on anything but the shard key becomes a global-index problem (Chapter 9). Read the implication in the correct direction: the *logical* schema decisions you make today — which relationships are enforced by keys, which invariants span rows, which candidate key you choose — determine which shard keys are even *available* later. Co-locate what must be transactionally consistent; a schema whose critical invariants all cross any plausible partition boundary is a schema that cannot be sharded without being redesigned. Chapter 9 makes this concrete; Volume 7, Chapter 5 treats it as a design method.

NULL bugs multiply when data is merged. Within one schema you can fight NULL with NOT NULL. But federate data — join replicas mid-catchup, merge per-service extracts in a warehouse, stitch API responses where "field absent" became NULL — and you reintroduce NULLs at every seam: the outer joins that combine sources *manufacture* NULLs for every unmatched row by design. Every trap in this chapter then fires at once: `NOT IN` against a column that is NULL-free in service A but not in the merged view; two-panel dashboards that no longer sum to the total; aggregates silently skipping the rows one source failed to populate. Teams that merge data across systems relearn three-valued logic empirically, in production, at the seams. The cheaper path is the one this chapter taught: know the 3VL rules cold, and treat every merged dataset as maximally nullable until proven otherwise.

Key takeaways

- **The relational model is a set of promises to the layers below.** Relations are sets of tuples over typed, named attributes: no duplicates, no order, no positions. Applications speak predicates; storage is free to change. That data independence is why fifty years of storage and optimizer innovation — and now distributed execution — arrived without breaking applications.
- **Constraints make the database the enforcer of invariants** against all writers, not just well-behaved ones. Know the asymmetry: WHERE keeps only TRUE, but CHECK rejects only FALSE — a CHECK evaluating to UNKNOWN passes, so CHECK does not imply NOT NULL.
- **SQL logic is three-valued, and the four traps are mechanical:** WHERE drops UNKNOWN rows (filters and their negations don't partition a table); NOT IN over a list containing NULL returns nothing, ever; aggregates skip NULLs and return NULL on empty input; NULL join keys match nothing. Declare NOT NULL wherever true; write NOT EXISTS, never NOT IN, against nullable columns.
- **Relational algebra is the language of query plans.** Selection, projection, joins — including the semi and anti joins that EXISTS/NOT EXISTS denote — and aggregation compose into trees because of closure, and EXPLAIN is printed algebra. Your NOT NULL declarations are optimizer input.
- **SQL is bags, evaluated in a fixed logical order:** FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This one fact explains alias visibility, HAVING's existence, why window filters need a wrapper, why WHERE on a left join's right side makes it inner, and why LIMIT without ORDER BY is nondeterministic. UNION ALL unless you mean UNION.
- **Window functions need explicit frames.** The implicit frame with ORDER BY is RANGE ... CURRENT ROW, which includes peers and corrupts running totals under ties; write ROWS explicitly. ROW_NUMBER breaks ties arbitrarily, RANK gaps, DENSE_RANK doesn't.
- **WITH RECURSIVE iterates a working table level by level** — the self-reference sees one level back, not the accumulated result — and on real data it needs a cycle guard.

- **Normalize by functional dependencies until BCNF** ("every arrow from a key"), because the anomalies are real bug classes; **denormalize where measured**, knowing you are readmitting the update anomaly and must own the mechanism that manages it. Normalization is write correctness; denormalization is read performance.
- **Equivalent queries can differ 1000×**. SQL states what, the optimizer picks how, and the leaks are in performance, not correctness. Understanding the model — and reading plans — beats memorizing dialect tricks that rot.
- **Schema design is shard design, years early**. Constraints and invariants that span partition boundaries become expensive or impossible; the logical layer determines later shardability.

Further reading

- Codd, E. F., "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM* 13(6), June 1970 — the nine pages this volume stands on. <https://dl.acm.org/doi/10.1145/362384.362685>
- ISO/IEC 9075, *Information technology — Database languages — SQL* — the standard; Part 2 (Foundation) defines the semantics in this chapter. Latest edition SQL:2023. (Paywalled by ISO; the PostgreSQL documentation's conformance appendix is the practical index to it.)
- PostgreSQL documentation — the best free SQL-semantics reference in existence, dialect notwithstanding: queries and SELECT evaluation (<https://www.postgresql.org/docs/current/sql-select.html>), window functions (<https://www.postgresql.org/docs/current/tutorial-window.html>), `WITH` queries (<https://www.postgresql.org/docs/current/queries-with.html>).
- Date, C. J., *SQL and Relational Theory: How to Write Accurate SQL Code*, 3rd ed. (O'Reilly, 2015) — the sharpest available treatment of where SQL deviates from the model, including a sustained argument against NULL that will vaccinate you even if you don't accept it.
- Date, C. J., *An Introduction to Database Systems*, 8th ed. (Addison-Wesley, 2003) — the standard comprehensive text on the model, dependencies, and normal forms.

- Chamberlin, D. and Boyce, R., "SEQUEL: A Structured English Query Language," *SIGFIDET*, 1974 — where SQL itself began, at IBM San Jose alongside System R.
- Melton, J. and Simon, A. R., *SQL:1999 — Understanding Relational Language Components* (Morgan Kaufmann, 2001) — dated but honest standardese-to-English translation, by the standard's longtime editor.
- Winand, M., *Modern SQL* — a well-maintained site tracking which standard features (window functions, recursion, LATERAL, and beyond) each engine actually implements. <https://modern-sql.com>
- Chapter 4 — Query Processing and Optimization — where the algebra trees of this chapter meet cost models and become physical plans.
- Chapter 9 — Partitioning and Sharding — what happens to keys and constraints at the shard boundary; and Chapter 12 — NewSQL — data independence cashed out across a fleet.
- Volume 7, Chapter 5 — Data Modeling — the design-method view of normalization and denormalization in system design.

Chapter 2 — Storage Engines: B-Trees vs LSM-Trees

What this chapter covers. Chapter 1 ended with Codd's promise: the application sees relations and predicates, and the physical layout of bytes on disk is somebody else's problem. This chapter is about the somebody else. The storage engine is the layer that maps logical rows onto pages on a device, and everything above it — indexing (Chapter 3), query execution (Chapter 4), transactions (Chapters 5–7) — inherits its cost structure. Two designs dominate the field, and they are close to duals of each other. The **B+ tree** keeps data sorted in place and updates it in place; it has been the default answer since the 1970s and still is, in Postgres, InnoDB, WiredTiger, and SQL Server. The **log-structured merge tree** never updates anything in place; it accumulates sorted immutable files and continuously merges them in the background; it is the engine inside LevelDB, RocksDB, Cassandra, ScyllaDB, and, through RocksDB's embedding, a surprising fraction of modern infrastructure. We

develop both in mechanism-level detail — the fanout arithmetic that makes a B+ tree four levels deep for a hundred billion rows, the buffer pool and latch protocols around it, the memtable/SSTable/compaction machinery of the LSM and the write-stall failure mode that machinery produces in real fleets — and then compare them honestly, using read, write, and space amplification as the framework. We close by mapping the theory onto the engines you actually run, examining why memory-mapped I/O keeps tempting engine authors and keeps disappointing them, and tracing the LSM's central trick — immutability — out into distributed systems, where the same move shows up as Kafka log segments and lakehouse files.

Learning goals — after this chapter you should be able to:

- Explain why the page is the unit of I/O, and derive from device characteristics — HDD seek costs, SSD erase blocks and the FTL — why both major engine designs look the way they do.
- Describe slotted pages, heap files, and tuple layout, including how oversized values are handled by TOAST-style overflow.
- Walk a search, an insert, and a page split through a B+ tree; do the fanout arithmetic that predicts tree height; and explain fill factor, deferred deletion, buffer-pool pinning, and latch coupling.
- Trace a write and a read through an LSM tree — WAL, memtable, SSTable levels — and explain what bloom filters buy on the read path and what they cannot buy.
- Compare size-tiered and leveled compaction in terms of read, write, and space amplification, and explain tombstones, resurrection hazards, and write stalls as operational realities.
- Choose between a B-tree and an LSM engine for a given workload using measured characteristics rather than slogans, and state the caveats on "LSM writes faster, B-tree reads faster."
- Explain the mmap argument from the CIDR 2022 paper accurately, and why most serious engines manage their own buffer pool.

The engine beneath the SQL

Strip away the parser, the planner, the transaction manager, and what remains is a component with a brutally simple contract: store key-value-ish records durably, find them again by key, scan them in key order, and

do all of this faster than the underlying device's naive cost model would suggest is possible. That component is the storage engine. In MySQL it is literally pluggable — InnoDB, MyRocks, and (historically) MyISAM implement the same handler API. In Postgres it is the heap plus the index access methods. In MongoDB it is WiredTiger. In half the distributed databases built in the last decade it is an embedded RocksDB with a replication layer glued on top.

The reason this layer deserves a chapter of mechanism rather than a paragraph of names is that its data structure choice is not an implementation detail — it determines the shape of the whole system's performance envelope. Whether your p99 write latency degrades smoothly or in cliffs, whether a bulk load is cheap or generates a week of background I/O debt, whether disk usage is predictable or breathes by tens of percent: these are storage-engine properties, and you cannot tune them away from above.

Disks dictate the design

Both designs are answers to the same question: given what durable storage devices actually charge for, how should we arrange bytes? Volume 1 covered the storage hierarchy and SSD internals and Volume 2 covered the page cache and fsync; here is the summary that matters for engine design.

Rotating disks charge for seeks. A random read on a 7,200 rpm disk costs a head seek plus rotational latency — several milliseconds, call it on the order of 100–200 random operations per second — while sequential transfer runs at a couple of hundred megabytes per second. The ratio between sequential and random throughput is roughly four orders of magnitude. Every structure in this chapter descends from that ratio: B-trees exist to bound the number of random reads per lookup, and log-structured storage exists to convert random writes into sequential ones.

SSDs abolish the seek but not the asymmetry. Flash reads any page in tens of microseconds regardless of location, so random *reads* are cheap — this is what makes LSM read amplification survivable at all. But flash cannot overwrite in place: data is written in pages (a few KiB) and erased only in much larger erase blocks (megabytes). The flash translation layer hides this by remapping writes and garbage-collecting erase blocks, and small scattered writes fragment the blocks, forcing the

FTL to copy live data around — **device-level write amplification** that costs throughput and flash lifetime (Volume 1, Chapter 6). Large sequential writes are the friendliest possible pattern for the FTL. So even on SSDs, the log-structured write pattern is not obsolete; it moved from "necessary for performance" to "kind to the device."

The page is the unit of I/O. Devices and kernels transfer blocks, not bytes; a database adopts a fixed page size — 8 KiB in Postgres, 16 KiB in InnoDB, variable-but-page-shaped in others — and does all I/O, caching, and locking accounting in pages. Reading one 100-byte row costs a page. Dirtying one 100-byte row costs writing a page. Every amplification factor in this chapter is measured against that granularity.

Pages, slots, and heap files

Before either tree structure, the humblest layout: the **heap file**, an unordered bag of pages, each page holding whatever tuples were placed there. This is Postgres's table storage — the "heap" in its documentation is exactly this, with B-tree indexes layered on as separate structures (Chapter 3 develops the consequences).

Within a page, nearly every engine uses the **slotted page** layout. A header at the front holds metadata and a *slot array* — an array of (offset, length) pointers that grows forward — while tuple data grows backward from the end of the page; the free space in the middle is consumed from both sides. The indirection is the point: a tuple is addressed externally as (page number, slot number), so the engine can compact the page — sliding tuples around to defragment free space — without invalidating any external reference, because only the slot array entries change. Postgres calls these addresses TIDs, or ctids; a tuple's identity is (page, slot), and indexes store exactly that.

Tuples themselves are a header (transaction visibility information — Chapter 6 will make heavy use of it), a null bitmap, and the attribute values serialized per the catalog's column order, with alignment padding. Variable-length values carry a length word. The interesting edge case is the value that does not fit: a page must hold at least a handful of tuples for the structure to work, so engines cap in-page tuple size and spill oversized attributes elsewhere. Postgres's mechanism is **TOAST**: when a tuple exceeds roughly 2 KiB, the engine first tries compressing wide values in place, then moves them out of line into a companion TOAST

table, chunked, leaving an 18-byte pointer in the main tuple. InnoDB does the analogous thing with overflow pages for long `VARCHAR` / `BLOB` values — the row in the B-tree leaf holds a 20-byte pointer to an overflow chain. The operational consequence is the same in both: queries that never touch the wide column never pay for it, and queries that do pay extra page fetches per row.

A heap file alone supports `INSERT` (append to a page with room) and full scans. What it cannot do is find a key without scanning, or return rows in order. Everything else in this chapter is about adding order.

B+ trees

Structure, and the arithmetic of fanout

The B-tree family, introduced by Bayer and McCreight in 1970, is the ordered structure designed for page-granular storage. What databases actually use is the **B+ tree** variant, with three defining properties:

1. **All data lives in the leaves.** Internal pages hold only separator keys and child pointers — they are pure routing information. This is what makes fanout high.
2. **High fanout, uniform shallow depth.** Every leaf sits at the same depth, and each internal page has hundreds to a thousand-plus children, so the depth is tiny.
3. **Leaves are chained to their siblings.** Once a range scan finds its starting leaf, it walks sibling links in key order without touching the upper tree again.

The fanout arithmetic is worth doing once with real numbers, because it explains why B-trees won the last fifty years. Take InnoDB: 16 KiB pages, a `BIGINT` primary key. An internal page entry is an 8-byte key plus a ~6-byte page pointer plus record overhead — call it 16–20 bytes, so an internal page holds on the order of **1,000 separators** after headers and slack. Suppose rows are 160 bytes, so a leaf holds about 100 rows. Then:

- Height 2 (root → leaf): $1,000 \text{ leaves} \times 100 \text{ rows} \approx$ **100 thousand rows**.
- Height 3: $1,000 \times 1,000 \text{ leaves} \times 100 \approx$ **100 million rows**.
- Height 4: $1,000^3 \text{ leaves} \times 100 \approx$ **100 billion rows**.

Four levels covers any table you will ever host on one machine. Better: the root is one page and the second level is ~1,000 pages — a few tens of megabytes — so the top two levels are effectively always in the buffer pool, and often the third as well. A point lookup on a multi-billion-row table costs **one or two actual disk reads**. That logarithm with base ~1,000 rather than base 2 is the entire reason we do not use binary search trees on disk: a binary tree over a billion keys is 30 levels — 30 random reads — deep.

Search, insert, and the split

Search is the obvious descent: binary-search the root's separators to pick a child, repeat until a leaf, binary-search the leaf. Insert is search plus placement — and the interesting case is the full leaf.

Work a small example with toy fanout (four keys per page). A leaf holds [12, 23, 31, 47], its parent routes to it via separator 12. Insert key 27:

1. Descend to the leaf; it is full.
2. **Split**: allocate a new page, move the upper half there. Old leaf: [12, 23]; new leaf: [27, 31, 47]. Wire the new leaf into the sibling chain.
3. **Promote** a separator — the new leaf's low key, 27 — into the parent, with a pointer to the new page.
4. If the parent is now over-full, it splits the same way, promoting a separator to *its* parent. Splits propagate upward; if the root itself splits, a new root is created above it, and this is the only way a B+ tree ever grows taller. Growth at the root is why every leaf stays at the same depth — the tree is self-balancing by construction.

```

flowchart TD
    subgraph BEFORE["Before: insert 27 into a full leaf"]
        P1["parent: ... 12 ..."]
        L1["leaf: 12 23 31 47 – FULL"]
        P1 --> L1
    end
    subgraph AFTER["After the split"]
        P2["parent: ... 12 | 27 ..."]
        L2["leaf: 12 23"]
        L3["new leaf: 27 31 47"]
        P2 --> L2
        P2 --> L3
        L2 -.->|"sibling link"| L3
    end
    BEFORE ==>|"split leaf, promote 27"| AFTER

```

Amortized, splits are rare — with fanout 100+, fewer than a percent of inserts split — but they are not free, and their placement is workload-dependent. Monotonically increasing keys (an auto-increment ID, a timestamp) always split the rightmost leaf, which engines special-case into an efficient append pattern. Random keys (UUIDv4) splash inserts across the whole key space, splitting everywhere, touching cold pages, and leaving every leaf half-full immediately after its split. Chapter 3 turns this into concrete primary-key guidance.

Fill factor is the knob that pre-arranges slack: build pages only N% full so future inserts land in existing pages instead of splitting immediately. Postgres B-tree indexes default to `fillfactor = 90`; a table rewritten by `VACUUM FULL` or bulk-loaded can set table fillfactor below 100 to leave room for updates on the same page (which matters for HOT updates, below).

```

-- Postgres: leave 10% slack in index pages, 20% in the heap
CREATE INDEX orders_created_idx ON orders (created_at) WITH
(fillfactor = 90);
ALTER TABLE orders SET (fillfactor = 80);

```

Deletion, and what real engines actually do

Textbook B-tree deletion mirrors insertion: if a deletion leaves a page under half full, borrow from a sibling or merge with it, possibly propagating merges upward. Real engines mostly do not bother. Merging is complex under concurrency, and workloads that delete heavily often re-insert into the same key range soon after. So production engines **defer or**

skip rebalancing: a delete marks the entry dead; a page that becomes completely empty is unlinked and put on a free list for reuse; pages that are merely sparse simply stay sparse. Postgres's B-trees never merge partially empty pages — `VACUUM` reclaims wholly-empty ones — and InnoDB merges only when a page falls below a threshold, with page reuse doing most of the work. The consequence is familiar to every operator: a B-tree index on a table with churning keys accumulates bloat — low-density pages that inflate the index's size and cache footprint — and the honest fix is periodic reindexing, not faith in the deletion algorithm. Graefe's survey (Further reading) documents just how far production B-trees diverge from the textbook on this point.

The buffer pool

The tree lives on disk; the engine works on it through a **buffer pool** — a cache of page frames in memory that the engine manages itself rather than delegating to the OS page cache (the reasons to insist on managing it are the subject of the mmap section below). The protocol around it:

- A page needed by an operation is fetched into a frame and **pinned** — the pin count prevents eviction while any operator holds a reference. Unpin when done.
- A modified page is marked **dirty** in the frame; the write to disk happens later. Crucially, the WAL rule — log record describing the change must reach disk before the dirty page does — is what makes "later" safe; Chapter 7 builds crash recovery on exactly this.
- Eviction picks unpinned victims by an approximate-LRU policy (CLOCK variants, or InnoDB's midpoint-insertion LRU, which resists a table scan flushing the whole pool).
- A **checkpoint** periodically forces dirty pages out so that recovery does not need to replay the entire log; the tension between checkpoint I/O bursts and steady-state latency is a Chapter 7 topic, but note it now as the B-tree's version of background-work interference — the same role compaction plays for LSMs.

The buffer pool is why the fanout arithmetic above translates into real latency: the hit rate on the tree's upper levels is effectively 100%, so tree height minus cached levels equals disk reads per lookup.

Latches, briefly

Concurrent threads descending and splitting the same tree need coordination, and database literature uses a terminology split worth internalizing: **locks** protect logical database content on behalf of transactions and are held for transaction-scale durations (Chapters 5–6); **latches** are the short-duration mutexes protecting in-memory structures like buffer frames — exactly the primitives of Volume 4, Chapter 2, under a different name.

The classic protocol is **latch coupling** (crabbing): latch the child before releasing the parent, so a splitting page can never be yanked out from under a descent. Read descents take shared latches; inserts take exclusive latches only where a split might propagate, which is rare, so optimistic variants first descend assuming no split and restart if wrong. Postgres uses the Lehman-Yao B-link design, where every page carries a high key and a right-sibling pointer, letting a reader that lands on a just-split page simply chase the right-link instead of holding multiple latches — a lock-free-flavored trick that trades a little read-path work for much better concurrency. The details are engine-specific; the point is that a production B-tree's concurrency control is a substantial fraction of its code and a real source of contention on hot pages — rightmost-leaf contention under append-heavy insert load being the canonical case.

Write amplification and torn pages

Now the B-tree's structural cost. An in-place engine that changes one 100-byte row must eventually write the whole 16 KiB page: **~160× write amplification at the page level** for that single-row update, before the device's own FTL amplification stacks on top. A well-behaved workload amortizes this — many rows changed per page between flushes — but a random-update workload over a large working set approaches the worst case, with every logical write costing a full-page physical write to a random location.

It gets worse at the crash boundary. A 16 KiB page write is not atomic on devices whose atomic unit is 4 KiB or 512 bytes: a crash mid-write leaves a **torn page** — half old, half new, checksum-invalid, and unrecoverable from the WAL alone if the WAL records only the small logical change. Engines pay for protection: Postgres's `full_page_writes` logs the entire page image the first time a page is dirtied after each checkpoint (so recovery can restore a known-good base before replaying logical

changes), and InnoDB's **doublewrite buffer** writes pages first to a sequential staging area, syncs, then writes them home — a torn home-location write can be repaired from the staging copy. Both mechanisms are pure write amplification purchased for crash atomicity; Chapter 7 treats them fully. Keep the shape in mind: it is precisely the cost the LSM design will avoid by never overwriting anything.

LSM trees

The log-structured merge tree, formalized by O'Neil, Cheng, Gawlick, and O'Neil in 1996 and popularized in its modern form by Google's Bigtable and LevelDB, starts from the opposite premise: **never update in place**. All writes are sequential appends; sorted order is restored later, in bulk, in the background.

The write path

A write — put or delete alike — touches two structures:

1. **Append to the WAL.** Durability first: the record goes to a sequential log and (per the configured sync policy) is fsynced. This is the only I/O on the write path, and it is an append.
2. **Insert into the memtable.** An in-memory sorted structure — typically a skiplist, chosen because it supports concurrent sorted insertion without rebalancing (Volume 14 covers the structure) — absorbs the write. The write is now done, from the client's perspective: **no page read, no tree descent, no random I/O**.

When the memtable reaches its size limit (RocksDB default: 64 MiB), it is made immutable, a fresh memtable takes over, and a background thread **flushes** the immutable memtable to disk as an **SSTable** — a Sorted String Table, an immutable file of key-ordered entries — into level 0. The WAL segment covering that memtable can then be discarded.

```

flowchart LR
    W["write: put or delete"] --> WAL["WAL append  
sequential, fsync per policy"]
    WAL --> MT["memtable  
skiplist, in memory"]
    MT --> IMM["full: 64 MiB| IMM[immutable memtable]"]
    IMM --> L0["background flush"| L0["L0 SSTables  
overlapping key ranges"]
    L0 --> L1["compaction"| L1["L1 SSTables  
disjoint ranges, ~256 MiB total"]
    L1 --> L2["compaction"| L2["L2 - 10x larger"]
    L2 --> LN["compaction"| LN["L3..Ln - 10x each"]

```

Note what the design bought: every byte of I/O on the write path is sequential, writes are batched by the memtable into large flushes, and a burst of writes to the same key collapses into one entry at flush time. This — not magic — is why LSM ingest throughput embarrasses B-trees.

SSTables

An SSTable is more than a sorted dump. A RocksDB block-based table contains:

- **Data blocks** (~4 KiB default), each holding a run of key-value entries, prefix-compressed and then block-compressed (immutable sorted data compresses very well — adjacent keys share structure — which is a real, measurable LSM advantage).
- A **block index**: the last key of each block, so a point lookup binary-searches the index (cached) and reads exactly one data block.
- A **bloom filter** over the file's keys (Volume 14, Chapter 4). At the common sizing of ~10 bits per key, the false-positive rate is about 1%, meaning a lookup for a key *not* in this file skips the file entirely 99% of the time at the cost of a few hash probes in memory.
- Footer metadata, properties, and per-block checksums.

Immutability is what makes all of this cheap: the filter and index are computed once at file creation and never maintained, and the file can be cached, copied, or shipped over a network with no coordination whatsoever. Hold that thought for the distributed-systems lens.

The read path

The price appears on reads. A point lookup must consult, in recency order: the active memtable, any immutable memtables awaiting flush, **every** L0 file (L0 files come straight from memtable flushes, so their key ranges overlap), and then at most one file per deeper level (within L1+, compaction maintains disjoint key ranges, so binary search over file metadata finds the one candidate). The first hit wins — recency order is the versioning.

That is **read amplification**: one logical read fans out into many structure probes. Bloom filters are what make it survivable for point reads: each SSTable probe first consults the filter, so the expected number of data-block reads stays near one even with dozens of files in scope. Range scans get no such rescue — a filter cannot answer "what keys exist between a and b" — so a range read must open a **merging iterator** across the memtable and every relevant file in every level, doing a heap-merge as it advances. Range-heavy workloads feel LSM read amplification with full force; prefix bloom filters (RocksDB `prefix_extractor`) claw some of it back for prefix-bounded scans.

```
flowchart TD
    subgraph BT["B+ tree point read"]
        Q1["lookup key"] --> R1["root - cached"]
        R1 --> I1["internal - cached"]
        I1 --> LF["leaf page"]
    end
    LF -- "1-2 disk reads total" --> end
    subgraph LSM["LSM point read"]
        Q2["lookup key"] --> M1["memtable"]
        M1 -->|"miss"| M2["immutable memtables"]
        M2 -->|"miss"| F0["each L0 file"]
        F0 -->|"miss"| F1["one file per level L1..Ln"]
    end
    F1 -- "bloom filter first" --> HIT["first hit wins"]
    HIT -- "~1 block read if filters work" --> end
    HIT -- "many probes regardless" --> end
```

Compaction: the heart of the machine

Left alone, flushes would pile up files forever and reads would drown. **Compaction** is the background process that merges SSTables into fewer, larger, sorted runs, dropping overwritten versions and (eventually)

tombstones. It is not a maintenance chore bolted on; it is the other half of the design — the deferred sorting work the write path skipped. Two strategies dominate.

Size-tiered compaction (Cassandra's STCS default; RocksDB's "universal" style) collects runs of similar size and merges them into one run of the next size class. Each key is rewritten roughly once per size class it passes through — logarithmically few times — so **write amplification is low**. The costs: many runs of overlapping key ranges exist simultaneously, so **read amplification is high**; and merging N similar-size runs needs scratch space for the output while inputs still exist, so **space amplification is high** — transiently up to $\sim 2\times$ for a full-overlap merge, and steady-state disk usage carries obsolete versions across tiers. Cassandra operators budget headroom (the classic guidance: keep $\sim 50\%$ free for worst-case STCS) for exactly this.

Leveled compaction (LevelDB/RocksDB default; Cassandra's LCS option) organizes each level L1+ as a single sorted run — many fixed-size files (RocksDB `target_file_size_base`, 64 MiB) with disjoint key ranges — with each level capped at $\sim 10\times$ the previous (`max_bytes_for_level_base` = 256 MiB for L1, `max_bytes_for_level_multiplier` = 10). When a level overflows, one file is merged into the overlapping files of the next level down. Reads touch at most one file per level and few levels exist, and obsolete versions are purged aggressively: **read and space amplification are low** (steady-state space overhead around 10%, since $\sim 90\%$ of data sits in the last level). The cost: a file pushed into the next level overlaps ~ 10 files there and rewrites them all, so each key is rewritten roughly the multiplier per level transit — **write amplification on the order of the multiplier times the level count**, commonly cited in the tens for a large store.


```

flowchart TD
    subgraph ST["Size-tiered"]
        A1["4 small runs, similar size"] -->|"merge"| A2["1 medium run"]
        A3["4 medium runs"] -->|"merge"| A4["1 large run"]
        A5["low write amp"]
    end
    high_read_amp["high read amp: overlapping runs"]
    high_space_amp["high space amp: 2x during merge"]
    end_ST["end"]
    subgraph LV["Leveled"]
        B0["L0: overlapping flush files"] -->|"pick file, merge into overlap"| B1["L1: one sorted run, disjoint files"]
        B1 -->|"same, 10x down"| B2["L2: 10x larger"]
        B3["low read amp: 1 file per level"]
    end
    low_space_amp["low space amp: ~10 percent"]
    high_write_amp["high write amp: ~10x per level"]
    end_LV["end"]

```

This three-way tension is not an artifact of these two algorithms; it is the **RUM conjecture** (Athanassoulis et al., 2016): for Read, Update (write), and Memory/space overheads, an access method can be excellent on two only by paying on the third. B-trees minimize read amplification and pay in write amplification; size-tiered LSMs minimize write amplification and pay in read and space; leveled LSMs buy read and space back and re-pay in writes. There is no fourth quadrant. Engine selection and compaction tuning are the act of choosing which overhead your workload can best afford.

The knobs are real and worth seeing. RocksDB, leveled defaults with a bloom filter:

```

rocksdb::Options opts;
opts.write_buffer_size = 64 << 20;           // memtable size
opts.max_write_buffer_number = 4;           // memtables before
write stall
opts.compaction_style = rocksdb::kCompactionStyleLevel;
opts.level0_file_num_compaction_trigger = 4; // L0 files that
trigger compaction
opts.level0_slowdown_writes_trigger = 20;    // L0 files: begin
throttling writes
opts.level0_stop_writes_trigger = 36;        // L0 files: block
writes entirely
opts.max_bytes_for_level_base = 256 << 20;  // L1 target size
opts.max_bytes_for_level_multiplier = 10;    // each level 10x
the previous
opts.target_file_size_base = 64 <<
20;          // SSTable file size at L1
opts.max_background_jobs = 4;                // flush +
compaction parallelism

rocksdb::BlockBasedTableOptions table;
table.filter_policy.reset(rocksdb::NewBloomFilterPolicy(10.0)); // ~1%
FP
table.block_size = 4 * 1024;
opts.table_factory.reset(rocksdb::NewBlockBasedTableFactory(table));

```

Tombstones: deletes are writes

An immutable-file design cannot remove a key by touching the files that contain it. So a delete **writes a tombstone** — a marker record that shadows older versions on the read path. The key's space is reclaimed only when compaction merges the tombstone down far enough to have provably met every older version beneath it; only at the bottom level can the tombstone itself finally be dropped. Three operational consequences follow. Deletes cost write bandwidth like any other write — a mass delete is a mass *ingest*, followed by a compaction bill. Read performance can *degrade* after bulk deletes, because scans must stream past tombstones to find live data — range-scan-over-a-graveyard is a classic Cassandra incident pattern. And tombstone garbage collection is genuinely hazardous in replicated systems: Cassandra retains tombstones for `gc_grace_seconds` (default ten days) so that anti-entropy repair can propagate the delete to replicas that missed it; if a replica stays unrepaired past that window, the tombstone is collected while the stale replica still holds the old value, and repair then happily copies the value back — **resurrection of deleted data**. Chapter 8 and Volume 6 return to this class of problem; note here only that it is a storage-engine GC policy

interacting with a replication protocol, and neither layer can see the whole hazard alone.

Write stalls: the failure mode

Compaction is a debt system: the foreground accepts writes cheaply by borrowing sorting work from the future. When sustained ingest exceeds compaction throughput, the debt compounds — L0 files accumulate, read amplification climbs — and the engine must eventually force the foreground to slow down. RocksDB does this in stages: at 20 L0 files (`level0_slowdown_writes_trigger`) it throttles write throughput; at 36 (`level0_stop_writes_trigger`) it **blocks writes entirely** until compaction catches up; parallel triggers exist for pending-compaction bytes and for running out of memtables while flushes are behind. This is the LSM's signature production failure: a system that benchmarked beautifully absorbs a traffic spike or a bulk load, then minutes later — when the memtables and L0 fill — p99 write latency goes from microseconds to seconds, not gradually but at a trigger threshold. The B-tree's background-work interference (checkpoint flushing) is comparatively gentle; the LSM's is a cliff. Monitoring L0 file count and pending-compaction bytes is not optional for anyone operating an LSM in anger, and the deepest tuning insight is that stall triggers are not the problem — they are the *messenger* announcing that steady-state ingest exceeds what compaction, as configured, can digest.

The comparison, honestly

The slogan — **LSMs write faster, B-trees read faster** — is directionally right and numerically useless. The defensible version:

Dimension	B+ tree	LSM tree
Point read	1-2 page reads; upper levels cached; stable latency	Multi-structure probe; bloom filters make it ~1 block read typical, worse on misses/cold cache
Range scan	Find leaf, walk sibling links — near-optimal	Merging iterator across levels; tombstones and overlap tax it

Dimension	B+ tree	LSM tree
Write path	Read-modify-write of pages; random I/O on random keys	Sequential append + memory insert; batched flushes
Write amplification	Page-size $\div$ row-size per touched page, plus full-page/doublewrite protection	Compaction-driven: low (size-tiered) to tens of $\times$ (leveled)
Space	Fragmentation and bloat from splits/sparse pages; predictable otherwise	Leveled: $\sim 10\%$ overhead; size-tiered: large and breathing
Compression	Per-page, modest — in-place updates fight it	Excellent — immutable sorted files compress at will
Background load	Checkpoint flushing, vacuum/purge	Compaction: continuous CPU + I/O, cliff-shaped stalls
Tail latency	Steady; degrades smoothly under pressure	Good until stall triggers; then a cliff
Concurrency hot spots	Rightmost-leaf latch contention on append workloads	Foreground writes rarely contend; compaction competes for the device

The caveats that keep the slogan honest:

- **A B-tree with a big enough buffer pool reads from memory**, at which point the LSM's read amplification is also mostly memory probes and the read gap narrows sharply. The comparison bites hardest when the working set exceeds RAM.
- **An LSM's write advantage is measured at the foreground**, but the compaction bill is paid on the same device. Total device writes per logical write can *exceed* the B-tree's under leveled compaction; what the LSM buys is that the foreground write never waits for a random read, and the deferred work is sequential and schedulable.
- **Deletes and overwrites shift the balance.** Update-heavy and delete-heavy workloads inflate LSM read paths (versions, tombstones) until compaction digests them; the B-tree handles them in place.

- **Space accounting differs in kind.** A B-tree's disk usage is boring; a size-tiered LSM's disk usage has *dynamics*, and capacity planning must budget for the worst compaction transient, not the average.

Measured workloads beat slogans. If your choice matters, benchmark with your key distribution, your value sizes, your read/write/scan/delete mix, at your working-set-to-RAM ratio, for long enough that compaction reaches steady state — an LSM benchmark that ends before compaction debt comes due is fiction.

The engines you actually run

InnoDB (MySQL) is a B+ tree engine in which the table *is* a B+ tree, clustered by primary key: leaf pages hold full rows in PK order. Secondary indexes are separate B+ trees whose leaves hold the primary key as the row pointer — so every secondary lookup is two tree descents, and a fat primary key physically inflates every secondary index. Chapter 3 turns this into design rules. The doublewrite buffer (above) handles torn pages; the change buffer batches secondary index maintenance for pages not in memory.

PostgreSQL stores tables as heap files, unclustered; all indexes, including on the primary key, are secondary structures pointing at (page, slot) TIDs. Because MVCC (Chapter 6) writes a new tuple version for every update, an update would normally require touching every index; the **HOT** (heap-only tuple) optimization avoids exactly that when no indexed column changed and the new version fits on the same page — which is what table fillfactor below 100 is buying room for.

RocksDB (Facebook's fork of Google's LevelDB) is the LSM described throughout this chapter, and its importance is mostly as an *embedded* engine: MyRocks puts it behind MySQL's handler API (built for Facebook's write-heavy, space-sensitive UDB tier), Kafka Streams and Flink use it for local state stores, and CockroachDB (historically), TiKV, and YugabyteDB build distributed databases over it or over engines patterned on it. When you operate "a modern distributed database," you are very often operating a fleet of RocksDB instances with a consensus protocol on top.

WiredTiger is MongoDB's default engine since 3.2: a B-tree engine (an LSM mode exists but is not the default), with checkpoints plus a journal for durability, and per-record MVCC rather than page-level versioning.

Cassandra and ScyllaDB are LSM engines at distributed scale — memtables, SSTables, and pluggable compaction (size-tiered default, leveled and time-window options; time-window compaction exists because time-series data expires in bulk, and dropping a whole SSTable whose window has expired is infinitely cheaper than compacting tombstones). ScyllaDB reimplements the same design in C++ with a shard-per-core architecture, which changes the constant factors, not the storage theory.

The mmap temptation

Every few years an engine is built on `mmap` : map the data files, let the OS page cache be the buffer pool, dereference pointers instead of calling `read()` . The appeal is obvious — the buffer pool, eviction policy, and I/O scheduling come free. Crotty, Leis, and Pavlo's CIDR 2022 paper ("Are You Sure You Want to Use MMAP in Your DBMS?") catalogs why serious engines keep abandoning it, and the argument deserves accurate summary because each point is a mechanism, not a taste:

1. **Transactional safety.** The OS may flush a dirty mapped page *at any time*, in any order. The engine's WAL discipline requires that no dirty page reach disk before its log records; with `mmap` the engine has surrendered control of exactly that ordering, and must reclaim it with copy-on-write tricks, shadow paging, or OS-specific pinning — at which point the "free" buffer pool is no longer free.
2. **I/O stalls the engine cannot see.** A page miss is a page fault: the faulting thread blocks inside a memory access, with no opportunity for async I/O, prefetch hints beyond `madvise` , or scheduling another task meanwhile. A query touching a cold page stalls a worker thread invisibly.
3. **Error handling.** Explicit `read()` returns an error code at a known call site; a failed mapped read delivers `SIGBUS` at an arbitrary instruction. Checksumming pages on read — routine in buffer-pool engines — has no natural hook when any dereference may transparently re-fetch from disk.
4. **Performance at modern device speeds.** The paper's microbenchmarks show `mmap`-based access falling well behind explicit I/O on fast NVMe under concurrency, bottlenecked on kernel-

side costs: page-table contention, TLB shutdowns during eviction, and single-threaded kernel reclaim machinery.

The history matches: MongoDB retired its mmap-based MMAPv1 engine in favor of WiredTiger; InfluxDB and others hit the stalls and moved off. LMDB is the honest counterexample — a copy-on-write B-tree deliberately designed around mmap's constraints, single-writer by design — which shows the approach can work when the entire architecture bends around it. The paper's conclusion, and this book's, is that for a general-purpose engine the answer to the title question is no: a real buffer pool is the price of controlling write ordering, I/O scheduling, and error handling — which is to say, of controlling the things Chapter 7 is about.

The distributed-systems lens

The storage engine looks like the most single-node topic imaginable. It is not; its design decisions propagate to fleet scale along at least four paths.

Immutability is the same move everywhere. An SSTable is a sorted, immutable, checksummed, self-describing file. So is a Kafka log segment (Volume 10, Chapter 3). So is a Parquet file in an object-store lakehouse. The property being purchased is identical in all three: an immutable file can be **replicated by copying, cached without invalidation protocol, and tiered to cheaper storage** — you ship files, not diffs, and a receiver needs no merge logic, no locks, and no conversation to trust what it received. Cassandra streams SSTables between replicas during bootstrap and repair; RocksDB-based systems ship snapshot files to bring new replicas online; "zero-copy" backup of an LSM is a hard link. A B-tree, mutating pages in place, offers no such unit — backing it up or replicating it means either a coordinated snapshot or shipping the WAL — which is a large part of why log-structured designs keep winning inside distributed systems even where a B-tree would serve single-node performance equally well.

Compaction debt is a capacity-planning input, not a tuning detail.

A bulk load — a migration, a backfill, a region failover doubling traffic — deposits compaction debt on *every replica of every shard simultaneously*, and the fleet pays it back concurrently: a **compaction storm** of background I/O and CPU across the cluster, hours after the event that caused it, competing with foreground reads precisely when the system is

recovering. Fleet capacity models for LSM-based systems must reserve headroom for compaction throughput, not just foreground QPS, and operational runbooks for bulk ingest (throttling, pre-splitting, temporarily switching compaction strategy) exist because this bill always comes due.

Write stalls couple to replication and backpressure. When a follower's engine hits its stall trigger, the follower stops absorbing the replication stream; lag grows (Volume 6's consistency implications follow), quorum writes slow or fail, and — if the system lacks end-to-end backpressure (Volume 10, Chapter 7) — queues upstream absorb the difference until they do not. A single node's `level0_stop_writes_trigger` firing can thus surface as cluster-wide tail latency. The engine's flow-control behavior is part of the distributed system's flow-control behavior, whether the architecture acknowledges it or not.

Storage-compute separation splits the engine across a network. Aurora's design takes the page/WAL layering of this chapter and Chapter 7 and cuts it at the interface: the compute node runs the B-tree and buffer pool but ships only WAL records to a distributed storage service, which materializes pages, replicates six ways, and serves page reads back on buffer-pool miss — "the log is the database." Disaggregated LSM systems do the analogous split by putting SSTables in object storage with local caches. Both are only possible because the engine layer already had a clean page/log seam to cut along; Chapters 8 and 12 develop these architectures. When you read them, you will be reading this chapter's diagrams with a network drawn through the middle.

Key takeaways

- The storage engine maps rows onto **pages**, the unit of I/O; device economics — HDD seeks, SSD erase-block/FTL behavior — are the design forces behind both major engine families.
- **Slotted pages** buy tuple movability via slot indirection; heap files hold unordered tuples; oversized values spill to TOAST/overflow storage so the page structure survives.
- A **B+ tree** routes through high-fanout internal pages to leaves holding all data, chained for range scans. Fanout ~1,000 makes it 3–4 levels for millions to billions of rows, and with upper levels cached, a point read costs 1–2 disk reads.

- Real B-trees **split eagerly and merge lazily** — deferred deletion and page reuse mean bloat is normal and reindexing is a legitimate maintenance operation, not an admission of failure.
- B-tree costs: **page-granular write amplification** on small updates, torn-page protection (full-page writes, doublewrite), buffer-pool checkpointing, and latch contention on hot pages.
- An **LSM tree** writes via WAL append plus memtable insert — purely sequential I/O — and restores sorted order later through **compaction** of immutable SSTables carrying block indexes and bloom filters.
- The LSM read path fans out across memtables, L0, and one file per deeper level; **bloom filters rescue point reads; nothing rescues range scans**, which must merge iterators.
- **Size-tiered vs leveled compaction** is the RUM conjecture in the flesh: low write amp with high read/space amp, or the reverse. You choose which amplification to pay; you do not choose whether.
- **Deletes are writes** (tombstones), tombstone GC races replication repair (resurrection), and **write stalls** are the LSM's signature cliff — monitor L0 counts and compaction debt.
- "LSM writes faster, B-tree reads faster" survives only with caveats about working-set-to-RAM ratio, total device writes, and steady-state compaction. **Benchmark your workload past compaction steady state or not at all.**
- **mmap is not a buffer pool**: uncontrolled flush ordering, invisible stalls, SIGBUS error delivery, and kernel-side scaling limits are why engines that tried it retreated.
- Immutable SSTables generalize: **ship files, not diffs** is the same design move as Kafka segments and lakehouse files, and compaction debt and stall behavior are fleet-level operational inputs, not node-level trivia.

Further reading

- O'Neil, P., Cheng, E., Gawlick, D., and O'Neil, E., "The Log-Structured Merge-Tree (LSM-Tree)," *Acta Informatica* 33(4), 1996 — the original formulation.

- Bayer, R. and McCreight, E., "Organization and Maintenance of Large Ordered Indices," *Acta Informatica* 1, 1972 — the B-tree's origin.
- Graefe, G., "Modern B-Tree Techniques," *Foundations and Trends in Databases* 3(4), 2011 — the definitive survey of how production B-trees differ from textbook ones: latching, deferred maintenance, logging interactions.
- Lehman, P. and Yao, S. B., "Efficient Locking for Concurrent Operations on B-Trees," *ACM TODS* 6(4), 1981 — the B-link design Postgres uses.
- Athanassoulis, M. et al., "Designing Access Methods: The RUM Conjecture," *EDBT 2016* — the read/update/memory trade-off framework used in this chapter.
- Crotty, A., Leis, V., and Pavlo, A., "Are You Sure You Want to Use MMAP in Your DBMS?," *CIDR 2022* — the mmap argument, with microbenchmarks. <https://db.cs.cmu.edu/mmap-cidr2022/>
- Chang, F. et al., "Bigtable: A Distributed Storage System for Structured Data," *OSDI 2006* — the paper that gave us memtables and SSTables by name.
- The RocksDB Wiki — compaction styles, write stalls, tuning; the most detailed public documentation of a production LSM. <https://github.com/facebook/rocksdb/wiki>
- Dong, S. et al., "RocksDB: Evolution of Development Priorities in a Key-Value Store Serving Large-Scale Applications," *ACM TOS 2021* — write amplification vs space amplification priorities, from the engine's authors.
- PostgreSQL documentation, "Database Physical Storage" — page layout, TOAST, fillfactor. <https://www.postgresql.org/docs/current/storage.html>
- MySQL 8.4 Reference Manual, "InnoDB On-Disk Structures" — clustered indexes, doublewrite buffer, change buffering.
- Kleppmann, M., *Designing Data-Intensive Applications* (O'Reilly, 2017), Chapter 3 — a complementary treatment of the same comparison.
- Volume 1, Chapter 6 — SSD internals: the FTL and device-level write amplification.

- Volume 2 — the page cache, `fsync`, and `mmap` semantics this chapter's I/O discussion rests on.
- Volume 4, Chapter 2 — the latch primitives beneath latch coupling.
- Chapter 7 — WAL, checkpoints, full-page writes, and crash recovery done properly.

Chapter 3 — Indexing in Depth

What this chapter covers. Chapter 2 built the storage engines — the B+ trees and LSM trees that hold table data on disk. This chapter is about the structures we layer on top of them to make queries fast, and about the price we pay for doing so. An index is a redundant, derived copy of part of your data, maintained in an order the base table does not have, and every one of them taxes every write to the table forever after. We begin with that accounting, because it is the frame in which every other decision sits. We then get precise about the two physical models that dominate practice — InnoDB's clustered primary key, where the row is a B+ tree entry, versus Postgres's heap, where every index is secondary — because the model determines the cost of a lookup, the cost of an update, and even which primary keys are safe to choose. From there we work through composite indexes and the leftmost-prefix rule, covering indexes and index-only scans, selectivity and why the optimizer ignores indexes you were sure it would use, and the full variety of index types beyond the B-tree: hash, partial, expression, GiST, GIN, BRIN, and bitmap. We then face the maintenance realities — page splits, MVCC bloat, online index builds and their failure modes — and close with a method: design indexes from queries, not from tables, and verify with `EXPLAIN`. The distributed-systems lens examines what happens to secondary indexes when the table is sharded, which is where indexing stops being a local data-structure problem and becomes a distributed-consistency problem.

Learning goals — after this chapter you should be able to:

- State precisely what an index is and account for its cost on every `INSERT`, `UPDATE`, and `DELETE`, not just its benefit on reads.
- Explain the difference between a clustered index and a secondary index, trace a lookup through each, and predict from the model why

a fat or random primary key hurts InnoDB in ways it does not hurt Postgres.

- Apply the leftmost-prefix rule to decide which predicates a composite index can serve, and order columns correctly: equality first, then range, then sort.
- Design covering indexes that serve queries without touching the table, and explain why a Postgres index-only scan sometimes still fetches heap pages.
- Choose among B-tree, hash, partial, expression, GiST, GIN, and BRIN indexes based on the query shape and data distribution, and explain what a bitmap index scan is doing.
- Build indexes online with `CREATE INDEX CONCURRENTLY`, know its failure modes, and detect the invalid indexes it can leave behind.
- Explain why secondary indexes become fundamentally harder under sharding, and compare local scatter-gather indexes with global asynchronously-maintained ones — with DynamoDB GSIs as the worked example.

What an index actually is

Strip away the syntax and an index is three things at once:

1. **A redundant copy.** The indexed column values exist twice: once in the table, once in the index. Some indexes copy more (a covering index may duplicate half the row); none copy less than the key plus a pointer back to the row.
2. **A derived ordering.** The copy is arranged in an order the table does not have — sorted by key in a B-tree, hashed into buckets, inverted into posting lists. The ordering is the entire point: it converts an $O(N)$ scan into an $O(\log N)$ descent, or an $O(1)$ probe, or a merge of pre-sorted lists.
3. **A maintained invariant.** The database promises that the copy is consistent with the table within the transaction semantics of the engine. That promise is kept by doing extra work on every write, inside the same transaction, holding the same locks, generating the same WAL (Chapter 7).

The third property is where the accounting lives, and it is the one that engineers who think of indexes as free speedups forget. Every `INSERT`

into a table with five indexes performs six structure modifications: one heap or clustered-tree insertion, plus five index insertions, each of which is a B-tree descent, possibly a page split, and always WAL. Every `DELETE` marks entries dead in every index (immediately or deferred to vacuum, depending on the engine). Every `UPDATE` is the interesting case: in the worst case it is a delete-plus-insert in every index; in the best case — an update that touches no indexed column, in an engine with an optimization for that case — it can skip index maintenance entirely. We will see below that whether you get the best case or the worst is a property of your physical model and your schema, not of luck.

So the fundamental trade is: **an index spends write throughput, storage, and cache space to buy read latency**. All three costs are real:

- **Write amplification.** One logical row write becomes $1 + K$ physical structure writes for K indexes touched. On a write-heavy table, index count is often the dominant term in write cost.
- **Space.** Indexes on a heavily-indexed OLTP table routinely exceed the size of the table itself. That space is not just disk — it is backup size, replication volume, and restore time.
- **Buffer-pool dilution.** Index pages compete with table pages for the same cache (Volume 1 treats the memory hierarchy generally; Chapter 2 covered the buffer pool). Ten indexes on a hot table mean ten sets of hot internal pages resident in memory. An index that is rarely used still evicts pages that would have served real queries.

The corollary is a discipline: **measure before adding, and account for the write side**. An index is justified by the queries it serves, quantitatively — how often they run, how much they improve — against the writes it taxes. The end of this chapter shows how to find indexes that lost that bargain.

One more framing note. This chapter is mostly about B-trees because most indexes are B-trees, but "index" is broader than "B-tree." The structural details of B+ trees themselves — node layout, fanout arithmetic, why fanout of a few hundred makes trees three or four levels deep at a billion keys — are in Chapter 2 and Volume 14. Here we take the structure as given and study how it is *used*.

Clustered versus secondary: two physical models

The single most consequential fact about an index is what its leaf entries point at. There are two mainstream answers, and they produce systems with different performance characteristics, different failure modes, and different rules for choosing primary keys.

The clustered model: InnoDB

In InnoDB (MySQL's default engine), the table *is* an index. Rows are stored in the leaf pages of a B+ tree ordered by primary key — the **clustered index**. There is no separate heap; asking "where is the row?" and "where is the primary-key entry?" is the same question. If you declare no primary key, InnoDB uses the first unique non-null index, and failing that generates a hidden 6-byte row ID — so the clustering always happens, with or without your participation.

Every other index on the table is a **secondary index**, and here is the critical design decision: InnoDB secondary leaf entries do not store a physical row address. They store the row's **primary key value**. A lookup through a secondary index is therefore *two* B-tree descents: one down the secondary index to find the PK, then one down the clustered index to find the row. This indirection is deliberate — because rows move when clustered-index pages split, a physical pointer would need updating in every secondary index on every split, whereas a logical PK reference never goes stale.

Two practical consequences fall straight out of this design:

- **Fat primary keys inflate every secondary index.** The PK value is stored in every leaf entry of every secondary index. A 4-byte integer PK adds 4 bytes per secondary entry; a 36-character UUID string adds 36. On a table with six secondary indexes, the difference is paid six times per row, in disk, in cache, and in every backup.
- **Primary-key insertion order is a first-class performance concern.** Rows physically live in PK order, so *where* new rows land in the tree is determined by the PK you generate — which brings us to page splits and the UUID problem, treated in the maintenance section below. The short version: monotonically increasing PKs (`AUTO_INCREMENT`, UUIDv7) append to the rightmost leaf and pack pages tightly; random PKs (UUIDv4) insert uniformly across the

whole tree, splitting pages everywhere and dragging the entire tree through the buffer pool.

The heap model: PostgreSQL

Postgres stores rows in a **heap**: pages of rows in no particular order, addressed by **TID** (tuple identifier — a page number and a slot within the page, displayed as `(block, offset)`). Every index, including the one backing the primary key, is a secondary structure whose leaf entries map key values to TIDs. There is no clustered index; `CLUSTER` physically rewrites the table in index order once, but nothing maintains that order afterward.

A lookup is one B-tree descent to a TID, then one heap-page fetch — flatter than InnoDB's double descent, and the PK's width is nobody's business but the PK index's own. But the heap model interacts with Postgres's MVCC design (Chapter 6) in ways that create its own distinctive costs, and two of them matter enough to name here.

HOT updates. In Postgres, an `UPDATE` never modifies a row in place; it writes a new row version and leaves the old one for vacuum (Chapter 6). Naively, that means every update must insert a new entry into *every* index, because the new version has a new TID — even if no indexed column changed. The **HOT** (heap-only tuple) optimization avoids this: if the update changes no indexed column *and* the new version fits on the same heap page, the old version simply points to the new one in a chain within the page, and no index is touched at all. Index scans land on the chain head and walk to the live version.

HOT is why "just index it, updates are cheap" is false in Postgres in a specific, measurable way: **indexing a frequently-updated column disqualifies every update to that column from HOT**, which converts a cheap same-page update into a new-TID update that must insert into every index on the table — not just the new one. The `n_tup_hot_upd` versus `n_tup_upd` columns of `pg_stat_user_tables` tell you your HOT ratio; a common operational trick is lowering a hot table's `fillfactor` so pages retain the free space HOT needs.

Index-only scans and the visibility map. If a query needs only columns present in the index, the engine would like to answer it from the index alone. In Postgres there is a complication: index entries carry no visibility information — you cannot tell from the index whether a tuple is visible to

your snapshot (Chapter 6). Checking would require fetching the heap tuple, defeating the purpose. The escape hatch is the **visibility map**, a bitmap with one bit per heap page meaning "every tuple on this page is visible to all transactions." Vacuum sets the bit; any subsequent write to the page clears it. An index-only scan consults the map: pages marked all-visible are skipped entirely; others force a heap fetch for that tuple. This is why `EXPLAIN ANALYZE` reports `Heap Fetches` for an index-only scan, and why an index-only scan on a freshly-loaded, never-vacuumed table performs like a plain index scan: the visibility map is empty until vacuum runs. If you are counting on index-only scans, you are also counting on vacuum keeping up.

```

flowchart TD
    subgraph INNO["InnoDB – clustered"]
        Q1["query: WHERE email = ..."] --> S1["secondary index on email  
descent 1"]
        S1 --> PK["leaf entry holds  
PRIMARY KEY value"]
        PK --> C1["clustered PK B+ tree  
descent 2"]
        C1 --> R1["row lives in the  
clustered leaf page"]
    end
    subgraph PG["PostgreSQL – heap"]
        Q2["query: WHERE email = ..."] --> S2["index on email  
one descent"]
        S2 --> TID["leaf entry holds TID  
block and offset"]
        TID --> H["heap page fetch"]
        H --> V["visibility check via  
tuple header – MVCC"]
        V --> R2["row version returned"]
    end
    end

```

Which model when

Neither model dominates. Clustering makes PK-range scans superb — `WHERE user_id = ? ORDER BY created_at` on a PK of `(user_id, created_at)` reads physically contiguous pages, which is why InnoDB schemas often deliberately design composite PKs around the dominant access path. The heap model makes secondary lookups one hop shorter, makes PK choice a non-event for storage, and (via HOT) makes non-indexed-column updates cheap — while paying for MVCC with vacuum and visibility machinery. SQL Server offers both (clustered and

nonclustered indexes on the same table, plus heaps), and its guidance — narrow, static, ever-increasing clustering keys — is exactly the guidance the InnoDB analysis above predicts. What you must not do is carry intuitions from one model to the other unexamined: "UUIDs are fine as PKs" and "adding an index only slows writes to that column" are each true in one model and false in the other.

Composite indexes and the leftmost-prefix rule

A composite (multi-column) index on (a, b, c) is a B-tree sorted by a, then by b within equal a, then by c within equal (a, b) — exactly the ordering of a three-column ORDER BY, or of a phone book sorted by last name, then first name, then street. Everything about which queries the index can serve follows from that one fact, and it is worth deriving rather than memorizing, because the derived version handles the cases the memorized rule mangles.

A B-tree can efficiently find the *starting point* of any contiguous range of its sort order and scan forward. So the question "can this index serve this predicate?" becomes: **does the predicate describe a contiguous range of the index's sort order?**

Consider CREATE INDEX ON orders (customer_id, status, placed_at):

Predicate	Contiguous in index order?	What the index can do
customer_id = 42	Yes — one run of entries	Seek, scan the run
customer_id = 42 AND status = 'shipped'	Yes	Seek deeper, smaller run
customer_id = 42 AND status = 'shipped' AND placed_at > now() - '7 days'	Yes	Seek on all three, tight run
customer_id = 42 AND placed_at > ... (no status)	No — matching entries are scattered across all statuses within customer 42	Seek on customer_id, scan its whole run, filter on placed_at

Predicate	Contiguous in index order?	What the index can do
<code>status = 'shipped'</code> alone	No — scattered across all customers	Cannot seek; useless (barring skip scan, below)
<code>customer_id > 100 AND status = 'shipped'</code>	No — range on the first column, so <code>status</code> order restarts within each customer	Seek to <code>customer_id = 100</code> , scan the range, filter on <code>status</code>

Three rules crystallize out of the table:

1. **The leftmost-prefix rule.** The index serves seeks on `(a)`, `(a,b)`, `(a,b,c)` — prefixes — not on `(b)`, `(c)`, or `(b,c)`. An index on `(a,b,c)` is a strictly better version of an index on `(a)` and largely subsumes `(a,b)`; separate indexes on those prefixes are usually redundant (an anti-pattern we return to).
2. **A range predicate stops the seek.** Columns after the first range-compared column can no longer narrow the descent; they can only filter entries already being scanned. (Postgres will still pass later-column conditions into the index scan as filter conditions — cheaper than heap filtering — but the run being scanned is sized by the columns up to and including the range.)
3. **Therefore: equality columns first, then the range column, then sort columns.** For `WHERE customer_id = ? AND status = ? AND placed_at > ? ORDER BY placed_at`, the right index is `(customer_id, status, placed_at)`: two equality columns narrow the run, and within that run entries are already sorted by `placed_at` — the range and the `ORDER BY` are served by the same trailing column, and the sort disappears from the plan entirely. Put `placed_at` second and you get a larger scan plus a filter; put it first and the index barely helps at all.

The equality/range/sort ordering heuristic resolves most composite-index design questions, but note the tension it hides: when different queries need different orders, one index cannot serve both seek shapes, and you must decide whether the second query merits a second index — which is a write-cost question, not an indexing question.

```

flowchart TD
    IDX["composite index on  
customer_id, status, placed_at"] --> P1["WHERE customer_id = 42  
SEEK – leftmost prefix"]
    IDX --> P2["WHERE customer_id = 42  
AND status = 'shipped'  
SEEK – two-column prefix"]
    IDX --> P3["WHERE customer_id = 42  
AND status = 'shipped'  
AND placed_at > t  
SEEK all three, sorted output"]
    IDX --> P4["WHERE customer_id = 42  
AND placed_at > t  
SEEK on customer only,  
FILTER on placed_at"]
    IDX --> P5["WHERE status = 'shipped'  
NO SEEK – not a prefix"]
    style P1 fill:#e6f4e6,stroke:#2e7d32,color:#000
    style P2 fill:#e6f4e6,stroke:#2e7d32,color:#000
    style P3 fill:#e6f4e6,stroke:#2e7d32,color:#000
    style P4 fill:#fff8e1,stroke:#f9a825,color:#000
    style P5 fill:#fdecea,stroke:#c62828,color:#000

```

Skip scans, briefly. The "not a prefix, therefore useless" verdict has a qualified exception. If the leading column has very few distinct values, the engine can iterate over each distinct value of `a` and perform a seek on `(a, b)` for each — logically rewriting `WHERE b = ?` into "for each `a`: `WHERE a = :a AND b = ?`". Oracle has done this for years as the *index skip scan*; MySQL 8.0 added a skip-scan optimization; PostgreSQL 18 (2025) added B-tree skip scan for omitted low-cardinality prefix columns. It is a genuine improvement and a poor thing to design around: it pays one descent per distinct leading value, so it only wins when that cardinality is tiny. Treat it as a safety net, not a license to put the wrong column first.

Covering indexes and index-only scans

If the index contains every column a query reads — predicate columns *and* selected columns — the engine never needs the table at all. That is a **covering index** serving an **index-only scan** (MySQL calls it "using index" in `EXPLAIN`), and it is the single biggest lever on read-heavy hot paths, because it eliminates the per-row heap or clustered-index hop, which is where most of the random I/O in an index lookup lives.

You can cover by widening the key, but key columns cost comparisons on every descent and, past a point, tree depth. Postgres 11+ and SQL Server offer the cleaner tool: **INCLUDE columns**, stored in leaf entries only, not part of the key, not sorted, not usable for seeking — pure payload.

```
-- The query to serve, thousands of times per second:
-- SELECT order_id, total_cents FROM orders
-- WHERE customer_id = $1 AND status = 'shipped'
-- ORDER BY placed_at DESC LIMIT 20;
```

```
CREATE INDEX orders_cust_status_placed_cover
ON orders (customer_id, status, placed_at)
INCLUDE (order_id, total_cents);
```

```
Limit (cost=0.43..26.10 rows=20 width=24)
      (actual time=0.031..0.074 rows=20 loops=1)
  -> Index Only Scan Backward using orders_cust_status_placed_cover
      on orders
        (cost=0.43..318.42 rows=248 width=24)
        (actual time=0.030..0.068 rows=20 loops=1)
        Index Cond: ((customer_id = 41972) AND (status = 'shipped'))
        Heap Fetches: 2
        Buffers: shared hit=7
Planning Time: 0.198 ms
Execution Time: 0.096 ms
-- (trimmed)
```

Read the plan: **Index Only Scan Backward** serves both the predicate and the descending sort from the index; seven buffer hits total; and **Heap Fetches: 2** is the visibility map earning its keep — two tuples sat on pages not yet marked all-visible and forced heap checks. On a table where vacuum is behind, that number climbs and the "index-only" scan quietly degrades into an ordinary one.

In InnoDB, remember, every secondary index implicitly includes the primary key columns — so an index on **(customer_id, status, placed_at)** on a table with PK **order_id** already covers the query above (minus **total_cents**). Covering design in MySQL means widening the key, and the implicit PK suffix is part of your budget whether you wanted it or not.

The cost side is the usual one, amplified: an **INCLUDE d** column is one more column whose update disqualifies HOT and forces index maintenance. Cover the two or three hot queries that dominate your read traffic; do not cover speculatively.

Selectivity, cardinality, and why the optimizer ignores your index

An index is only worth using when the predicate is **selective** — when it matches a small fraction of the table. The reasoning is mechanical, not aesthetic. A secondary-index lookup costs a descent plus, per matching row, a heap or clustered hop that is effectively a random page read. A sequential scan costs one pass of sequential reads, which per page is far cheaper. There is a crossover: match 0.1% of rows and the index wins enormously; match 30% and the index's per-row random I/O costs more than scanning everything. The optimizer estimates the matched fraction from statistics — histograms, most-common-value lists, distinct counts (`n_distinct`) — and chooses by cost. Chapter 4 covers the machinery; what matters here is the design consequence:

Low-cardinality columns make poor B-tree index keys on their own. An index on `status` when 95% of orders are `'delivered'` will be used for `status = 'refunded'` and correctly ignored for `status = 'delivered'` — and if the statistics are stale, misused for both. Such columns earn their keep as *leading equality columns of composite indexes* (where they narrow a run that later columns subdivide) or as *partial-index predicates* (next section), not as standalone indexes.

A subtler statistic is **correlation**: how closely the index's logical order tracks the heap's physical order. Postgres tracks this per column (`pg_stats.correlation`). Scanning an index range over a well-correlated column (say, `created_at` on an append-only table) touches a few contiguous heap pages; the same range over an uncorrelated column touches a scattered page per row. The planner costs these very differently, and it is one reason two indexes of identical selectivity can have wildly different observed value — and the entire premise of BRIN indexes below.

The index bestiary

The B-tree is the default for a reason: it serves equality, ranges, prefixes, and ordering, all from one structure. The remaining varieties exist because some query shapes fit it badly.

Hash indexes

A hash index buckets entries by hash of the key: $O(1)$ equality probes, and *nothing else* — no ranges, no ordering, no prefixes, because hashing deliberately destroys order. In practice their niche in general-purpose engines is narrow: Postgres hash indexes have been crash-safe (WAL-logged) only since Postgres 10 and win over B-trees mainly for equality on long keys, where storing a 4-byte hash code per entry beats storing the key. The places hashing genuinely dominates are elsewhere: in-memory stores (Redis's keyspace), hash-organized memory-optimized tables (SQL Server Hekaton), LSM memtables and their bloom filters (Chapter 2), and InnoDB's *adaptive hash index* — an automatic in-memory hash layer that InnoDB builds over hot B-tree pages on its own. Default to B-trees; let the engine hash where hashing helps.

Partial (filtered) indexes

A partial index indexes only rows matching a predicate:

```
-- 97% of rows are soft-deleted history; queries only ever touch live
rows.
CREATE INDEX users_live_email ON users (email) WHERE deleted_at IS
NULL;

-- Only pending jobs are polled; the queue table is 99% completed rows.
CREATE INDEX jobs_pending ON jobs (queued_at) WHERE state = 'pending';
```

The wins compound: the index is a fraction of the full size (better cache residency, faster descents), writes to non-matching rows skip it entirely, and it can express things a full index cannot — `CREATE UNIQUE INDEX ... ON users (email) WHERE deleted_at IS NULL` enforces "one live account per email" while permitting any number of deleted ones. The planner uses a partial index only when it can *prove* the query's predicate implies the index's, so keep the predicate simple and literally matched in queries. Postgres and SQL Server (filtered indexes) support these; MySQL does not — the closest MySQL idiom is indexing a generated column or accepting a full index. The soft-delete pattern above is the canonical use, and if your tables carry `deleted_at`, most of your indexes probably want to be partial.

Expression (functional) indexes

An index normally stores column values, so `WHERE lower(email) = lower($1)` cannot use a plain index on `email` — the index is sorted by `email`, not by `lower(email)`, and the function's output order need not match. This is a systematic trap: **any function or type cast wrapped around an indexed column in a predicate silently discards the index**, and it appears constantly in the wild — `lower()` for case-insensitive matching, `date(created_at) = '2026-08-14'`, implicit casts when a string parameter meets an integer column. Two fixes: rewrite the predicate to leave the column bare (`created_at >= '2026-08-14' AND created_at < '2026-08-15'`), or index the expression itself:

```
CREATE INDEX users_email_lower ON users (lower(email));
-- Now: WHERE lower(email) = lower($1)  -- matches the indexed
expression, uses the index
```

The expression in the query must match the indexed expression as the planner sees it. MySQL 8.0 supports functional index parts directly; before that, the idiom was an indexed generated column. Postgres additionally gathers statistics on the indexed expression, which can improve estimates for such predicates.

GiST and R-tree ideas: multi-dimensional data

B-trees need a total order, and multi-dimensional data has none that respects proximity — sort points by longitude and nearby latitudes scatter. The R-tree family answers with a tree of *bounding rectangles*: each node covers a region containing all its children, and a query descends only into subtrees whose regions intersect the search region. Postgres's **GiST** (Generalized Search Tree) is a framework generalizing exactly this — the access method supplies the tree mechanics; an operator class supplies "consistent," "union," and "penalty" functions per data type — which is how one structure serves PostGIS geometries, range types ("find bookings overlapping this interval," including exclusion constraints that forbid overlapping rows), and nearest-neighbor ordering. MySQL implements R-tree `SPATIAL` indexes on geometry columns.

GIN and inverted indexes: containment

For "which rows *contain* X" — an array element, a JSONB key, a word in a document — the right structure is inverted: a B-tree of *elements*, each

pointing to a **posting list** of the rows containing it. That is Postgres's **GIN** (Generalized Inverted Index), and it is the standard index for `jsonb` containment (`@>`), array overlap, and full-text `tsvector` matching:

```
CREATE INDEX docs_body_fts ON docs USING gin (to_tsvector('english',
body));
CREATE INDEX events_payload ON events USING gin (payload
jsonb_path_ops);

-- served: WHERE payload @> '{"type": "refund", "region": "eu"}'
```

The cost profile is distinctive: one row insert updates one posting list *per element the row contains* — a 40-word document touches ~40 lists — so GIN writes are expensive, and GIN mitigates this with a *pending list* of recent entries merged in batches (the `fastupdate` option), trading some lookup work for write throughput. This is a small preview of a large topic: dedicated search engines are, architecturally, freestanding inverted indexes with their own storage and scoring; Chapter 13 treats them properly.

BRIN: block-range indexes

A BRIN index stores, for each *range of table blocks* (128 pages by default), only the min and max of the indexed column in that range — a few bytes summarizing a megabyte. A query prunes ranges whose min/max exclude the predicate and scans the survivors. The entire bet is **physical correlation**: on an append-only events table where `created_at` increases with page number, each range's min/max window is tight and pruning is near-perfect — a multi-terabyte table indexed in megabytes, with negligible write cost. On an uncorrelated column, every range's window spans the whole domain, nothing prunes, and the index is worthless. BRIN is the cheapest index that ever works and the most workload-dependent: check `pg_stats.correlation` before relying on one, and remember that updates and deletes erode the correlation BRIN depends on.

Bitmap indexes and bitmap scans

A **bitmap index** materializes, per distinct value, a bit array with one bit per row. Bitwise AND/OR across values and columns is extremely fast, which suits analytic queries combining many low-cardinality predicates; Oracle offers persistent bitmap indexes for exactly that warehouse niche.

The write costs are severe (concurrent OLTP writes on bitmap-indexed columns serialize badly), so no OLTP-first engine stores them persistently.

Postgres instead builds bitmaps *transiently, at query time*: a **bitmap index scan** walks an ordinary B-tree and constructs an in-memory bitmap of matching TIDs; `BitmapAnd / BitmapOr` nodes combine bitmaps from *multiple separate indexes*; then a **bitmap heap scan** visits matching heap pages in physical order — turning scattered random I/O into a sequential-ish sweep. If the bitmap outgrows `work_mem` it degrades to page granularity (lossy), and the heap scan rechecks the predicate per row (the `Recheck Cond` line):

```
Bitmap Heap Scan on orders (cost=41.2..1180.6 rows=312 width=64)
  Recheck Cond: ((status = 'refunded') AND (region = 'eu'))
  Heap Blocks: exact=214
-> BitmapAnd (cost=41.2..41.2 rows=312 width=0)
    -> Bitmap Index Scan on orders_status_idx
        Index Cond: (status = 'refunded')
    -> Bitmap Index Scan on orders_region_idx
        Index Cond: (region = 'eu')
-- (trimmed)
```

This is how single-column indexes get combined when no composite index matches — useful, but a purpose-built composite index still beats it for a hot query, because the bitmap machinery costs setup time and loses the index's ordering (no cheap `ORDER BY ... LIMIT` from a bitmap).

Index type	Serves	Refuses	Distinctive cost
B-tree	Equality, range, prefix, ordering	Containment, similarity	The default; per-write descent + possible split
Hash	Equality only	Everything else	Niche in disk engines; no ordering ever
Partial B-tree	Predicate-matching subset	Rows outside predicate	Near-zero for non-matching writes
Expression	Function-wrapped predicates	Bare-column predicates on same data	Function evaluated per write

Index type	Serves	Refuses	Distinctive cost
GiST / R-tree	Overlap, containment, nearest-neighbor, ranges	Total-order semantics	Looser structure; heavier maintenance
GIN	Element containment, full-text	Ranges over the whole value	One update per contained element; pending list
BRIN	Ranges over physically-correlated columns	Anything uncorrelated	Almost free; value collapses without correlation
Bitmap (scan)	AND/OR of several indexes	Ordered limited output	Built per-query; lossy past work_mem

Maintenance realities

Indexes are not write-once. They are long-lived mutable structures whose health degrades in predictable ways, and several of the worst production database incidents are index-maintenance incidents.

Page splits, fill factor, and the UUID problem

A B-tree page that must accept an entry but has no room **splits**: allocate a new page, move half the entries, update the parent (which may itself split, recursively). Splits are the mechanism by which trees grow, and they are fine in moderation; the pathology is in *where* inserts land.

Sequential keys — auto-increment IDs, timestamps, UUIDv7 — always insert at the rightmost leaf. Engines special-case this: rather than wastefully splitting the rightmost page 50/50 (the left half would never receive another insert), they split lopsidedly or just allocate a fresh page, yielding densely packed pages. Only the right edge of the tree is hot, so the working set for inserts is a handful of pages.

Random keys — UUIDv4, hashes — land uniformly across the entire key space. Every leaf page is eventually the insertion target, so every leaf page eventually splits 50/50, leaving the tree at roughly 50-70% page

density: the same data occupies up to twice the pages, halving the effective value of every buffer-pool megabyte. Worse, the insert working set is *the whole tree*: with a clustered InnoDB PK, that means the whole table must be cache-resident to insert without read I/O, and a table that outgrows the buffer pool hits a throughput cliff — each insert becomes a random disk read (to fetch the target leaf) plus dirty-page writeback. This failure mode is gradual, then sudden, and it has a name in most InnoDB shops: "we used UUIDv4 primary keys."

```
flowchart TD
    subgraph SEQ["sequential keys – rightmost inserts"]
        A1["leaf: 100..199 FULL"] --> A2["leaf: 200..299 FULL"]
        A2 --> A3["rightmost leaf: 300..347"]
    end
    all_inserts_land_here[all inserts land here]
    fills_densely_then_new_page[fills densely, then new page]
    end
    subgraph RND["random keys – inserts everywhere"]
        B1["leaf: aa..cf 50% full"]
        B2["leaf: d0..e7 62% full"]
        B3["leaf: e8..ff 55% full"]
        B4["insert key b7"] --> B1
        B1 --> B5["page FULL: split 50/50"]
    end
    allocate_page_move_half[allocate page, move half,]
    update_parent_wal[update parent, WAL it all]
    end
```

Mitigations, in order of preference: use sequential keys (**UUIDv7** gives you a time-ordered prefix with UUID's coordination-free generation — the practical best-of-both since its 2024 standardization in RFC 9562); if random keys are imposed, keep them *out of the InnoDB PK* (make them a unique secondary; splits then afflict one narrow index, not the table); and tune **fillfactor** — the fraction of each page filled at build time (Postgres B-tree default 90%, InnoDB fills ~15/16) — to pre-reserve split headroom, at a permanent density cost. Note the inversion for the heap model: in Postgres a random UUID PK bloats *one index*, not the table, so the same choice that is severe in InnoDB is merely wasteful in Postgres.

Bloat: MVCC's tax on indexes

In MVCC engines (Chapter 6), deleted and updated rows leave dead versions behind, and dead heap tuples imply dead index entries pointing at them. Postgres indexes shed dead entries in three ways: opportunistically, when a scan discovers a dead tuple and sets a hint (the `LP_DEAD` "killed tuple" bit) so future scans skip it; in bulk, when `VACUUM`

scans each index and removes entries for the dead TIDs it collected (often the dominant cost of vacuuming a big table); and structurally never — **B-tree pages do not merge**. A page emptied by deletes can be recycled whole, but a tree that once held 500M rows and now holds 50M retains its inflated structure, half-empty pages and all. That persistent inflation is **bloat**, it degrades cache efficiency and scan cost, and the fix is a rebuild: `REINDEX CONCURRENTLY` (Postgres 12+) or, in MySQL, rebuilding via `OPTIMIZE TABLE` /online DDL. Postgres 13's B-tree deduplication and 14's bottom-up deletion substantially blunted the classic version-churn bloat, but did not repeal it: a monitoring page for index bloat (via `pgstattuple` or the community bloat queries) belongs in any serious Postgres operation. High-churn queue tables — small, hot, constantly inserted and deleted — are the classic worst case, sometimes carrying indexes a hundred times larger than their live data.

Building indexes online — and the invalid-index failure mode

A plain `CREATE INDEX` takes a lock that blocks all writes to the table for the duration of the build — unacceptable on a large production table. `CREATE INDEX CONCURRENTLY` (CIC) is the Postgres answer, and its mechanics explain its failure modes. It builds in phases: register the index as not-ready so concurrent writes begin maintaining it; scan and build from a snapshot; then a second pass to insert rows that arrived during the build; with waits in between for every transaction that might hold an older snapshot to finish. The costs follow directly: roughly two table scans instead of one, and progress gated on the *oldest running transaction* — a long-running analytics query or an idle-in-transaction connection stalls CIC indefinitely.

The failure mode every operator learns eventually: **if CIC fails or is cancelled — deadlock, statement timeout, a unique violation discovered mid-build, an operator's Ctrl-C — it leaves behind an INVALID index**. The invalid index is never used by the planner, but it is *still maintained by every write*: you keep the entire write tax and receive none of the read benefit, silently, until someone notices. Check for them (`\d` marks them `INVALID`; programmatically, `pg_index.indisvalid = false`), then `DROP INDEX` and retry — re-running CIC does not repair the existing invalid one. MySQL's equivalent story is InnoDB online DDL (`ALGORITHM=INPLACE, LOCK=NONE`), which logs concurrent DML during the build and applies it at the end; different mechanics, same theme — online index builds trade doubled work and operational sharp edges for

availability, and you should treat them as long-running migrations with monitoring, not as DDL that happens to be slow.

The cost of too many indexes

The failure mode at the portfolio level is accumulation. Indexes are added by many hands over years — one per incident, one per ORM annotation, one per "just in case" — and almost never removed, because removal feels risky and benefit is invisible. The compounding costs are the ones from the opening accounting: write amplification (every write touches every index — twelve indexes means a single-row insert performs twelve tree descents and up to twelve splits), buffer-pool dilution (each index's hot pages evict something else's), doubled vacuum work in Postgres, and — less obviously — **optimizer confusion**: more indexes mean more plans to consider, more near-tie decisions made on estimated costs, and more opportunities for a plan to flip to a subtly worse index after a statistics refresh (Chapter 4). Redundant near-duplicates — (a) alongside (a, b) alongside (a) INCLUDE (b) — are pure waste on the write side and plan-instability fuel on the read side. Index count is a budget; spend it on evidence.

Designing indexes from queries, not tables

Everything above converges on one method, and it inverts the way indexes are usually created. Indexes designed by staring at the schema — "customers will be looked up by email, index email" — accumulate into the portfolio problem just described. Indexes designed from the *query workload* stay few and earn their keep. The method:

1. **Extract the access patterns.** From `pg_stat_statements`, MySQL's performance schema digests, or your APM: the queries that dominate load or matter for latency, with their real predicate shapes, sorts, and limits. Ten queries usually cover the great majority of load.
2. **Design composite keys around the critical predicate + sort.** Per query: equality columns first, then the range column, then sort columns; then decide whether covering (INCLUDE or a wider key) is worth the write cost for that path. Then *merge*: find the smallest index set where one index serves several queries via prefixes, and delete every index another subsumes.

3. **Verify with EXPLAIN , then with EXPLAIN (ANALYZE, BUFFERS)** — first that the plan uses the index as designed (seek, not filter; sort absorbed, not re-sorted), then that the actual row counts and buffer touches match the theory. Chapter 4 teaches plan-reading in earnest; the two lines that matter here are `Index Cond` (predicates that narrowed the descent — your seek) versus `Filter` (predicates applied per-row after the fact — index not helping), and the presence or absence of a `Sort` node above the scan.
4. **Audit continuously for unused indexes.** The database counts index usage for you:

```
-- Postgres: indexes never or rarely used since stats reset
SELECT schemaname, relname, indexrelname, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;

-- MySQL:
-- SELECT * FROM sys.schema_unused_indexes;
```

Before dropping, mind the caveats: stats reset on `pg_stat_reset()` and (before Postgres 15's persistence improvements) on crash; check replicas, which serve different traffic; and remember that unique indexes enforcing constraints are load-bearing at zero scans. Postgres 14+ lets you de-risk the drop entirely: `ALTER INDEX ... NO INHERIT` —no— the safe idiom is marking the index invisible to the planner first (MySQL 8.0 `ALTER TABLE ... ALTER INDEX ... INVISIBLE` ; in Postgres, no built-in equivalent short of dropping, so rely on scan counts over a long window and keep the DDL to recreate it).

The anti-patterns to hunt during the audit are the mirror images of the rules: an index per column ("index everything, let the optimizer sort it out" — it cannot combine them better than a designed composite, and each one taxes writes); redundant prefixes (`(a)` next to `(a,b)`); indexes on low-cardinality columns alone; expression-blind indexes that no production predicate can ever match; and the unused residue of queries that no longer exist.

The distributed-systems lens: indexes meet sharding

Everything so far assumed the index and the table live on the same node, maintained in the same transaction, by the same engine. Shard the table (Chapter 9) and that assumption breaks, because an index is *an alternative ordering of the data* — and a table can only be physically partitioned by one key. Rows are placed by the shard key; a secondary index is precisely a query path that ignores the shard key. Something has to give, and there are exactly two places to put the pain.

Local secondary indexes (per-shard): each shard indexes its own rows. Writes stay single-shard — row and index entries co-located, one local transaction, no coordination — but a query on the indexed column alone cannot know which shards hold matches, so it must **scatter-gather**: query every shard, merge the results. Read cost scales with shard count, not result size; tail latency is the max over N shards (Volume 6 treats this amplification); and a `LIMIT 20 ORDER BY` must fetch candidates from every shard to merge. This is the model of most sharded SQL deployments and of Cassandra's native secondary indexes — and it is why Cassandra's documentation warns against them for high-cardinality columns on large clusters.

Global secondary indexes (partitioned by the *indexed* value): the index is itself a distributed table, sharded by indexed column. Reads become single-shard again — a targeted lookup lands on exactly the index partition holding the value. The pain moves to writes: a row on shard 3 whose indexed value maps to index partition 7 requires a **cross-node write**. Keep it synchronous and you have bought a distributed transaction — two-phase commit on every base-table write (Chapter 10). Make it asynchronous and the index is maintained eventually, meaning it can return stale or phantom results.

```

flowchart TD
    subgraph LOCAL["local secondary index – pain on reads"]
        QL["query: email = x"] --> R["router"]
        R --> S1["shard 1  
rows + local index"]
        R --> S2["shard 2  
rows + local index"]
        R --> S3["shard 3  
rows + local index"]
        S1 --> MG["merge results  
scatter-gather, tail latency"]
        S2 --> MG
        S3 --> MG
    end
    subgraph GLOBAL["global secondary index – pain on writes"]
        W["write row to shard 2"] --> S2b["shard 2  
base row"]
        S2b --> GP["index partition 7  
keyed by email"]
        GP --> QG["query: email = x"]
        QG --> ONE["single-partition read  
possibly stale if async"]
    end

```

DynamoDB's GSIs are the canonical async design, and worth describing precisely. A DynamoDB global secondary index is a separate physical table with its own partition (and optional sort) key and its own provisioned throughput. Writes to the base table are replicated to each GSI *asynchronously*; consequently GSI queries support only eventually consistent reads — the strongly-consistent read option that exists for base tables and for *local* secondary indexes is not offered on GSIs, because there is no moment when the GSI is guaranteed current. A GSI can briefly miss a row just written, or return one just deleted (queries fetch non-projected attributes from the base item, so a dangling GSI entry resolves against the source of truth). And the coupling runs backward too: if a GSI has insufficient write capacity, *base-table writes are throttled* — the async index can backpressure the table it indexes. DynamoDB's LSIs make the opposite choice: same partition key as the base table, hence co-located, hence synchronously maintained and strongly-consistent-readable — at the price of only re-sorting within a partition and a 10 GB cap per partition-key value. The two features are the two branches of the design space, shipped side by side.

Two broader patterns complete the lens. First, **a search system is an externalized global secondary index**: Elasticsearch or OpenSearch alongside your OLTP database is exactly the async-GSI architecture with the index in a different engine — fed by CDC or an outbox (Volume 10, Chapter 6, for why dual-writing without one loses updates), eventually consistent by construction, and subject to the same read-your-own-writes surprises as any async index (Chapter 13 covers search engines themselves). Second, **index locality explains NoSQL's design constraints**. Cassandra's data-modeling doctrine — model tables per query, denormalize, repeat the data — is what you get when a system refuses global indexes and distrusts scatter-gather: every query must be answerable within one partition, so you build one hand-maintained "materialized index" (a table) per access path, and application code or materialized views take over the maintenance job the database's indexing subsystem does in a single-node SQL engine. The constraint is not arbitrary; it is the cost accounting of this chapter, applied at cluster scale.

Key takeaways

- An index is a **redundant, ordered, derived copy** whose consistency is maintained on every write. Its benefit accrues to specific queries; its cost — write amplification, space, buffer-pool dilution — accrues to every write and every cache miss, forever. Measure both sides.
- **Clustered (InnoDB) vs heap (Postgres) is the load-bearing distinction**. InnoDB: rows live in the PK B+ tree, secondaries store PK values (double descent, fat PKs inflate every index), PK insertion order determines table-wide split behavior. Postgres: all indexes point at heap TIDs; HOT makes non-indexed-column updates cheap and indexing a hot column disqualifies them; index-only scans depend on vacuum maintaining the visibility map.
- **Composite indexes serve leftmost prefixes**; a range predicate ends the seek; therefore order columns **equality, then range, then sort**. Skip scans soften the prefix rule only for tiny leading cardinalities.
- **Covering indexes eliminate the heap/clustered hop** — the biggest read lever — via `INCLUDE` or a wider key; every included column widens the write tax and narrows HOT eligibility.
- The optimizer uses an index only when the predicate is **selective enough** to beat a scan; low-cardinality columns belong in composites

or partial-index predicates, not standalone indexes. Physical **correlation** decides between B-tree range-scan efficiency and BRIN viability.

- Beyond B-trees: **hash** for pure equality (mostly in-memory), **partial** for hot subsets (soft deletes, queues), **expression** for function-wrapped predicates (which otherwise silently miss plain indexes), **GiST/R-tree** for spatial and ranges, **GIN** for containment via posting lists, **BRIN** for huge physically-ordered tables, **bitmap scans** to combine single-column indexes at query time.
- Maintenance is where indexes hurt: **random keys split pages everywhere** (UUIDv4 in a clustered PK is the classic self-inflicted wound; UUIDv7 fixes it), **MVCC leaves dead entries** that vacuum must prune and only REINDEX truly compacts, and **CREATE INDEX CONCURRENTLY can fail into an INVALID index** that taxes writes while serving nothing.
- **Design from queries, not tables**: extract real access patterns, build the minimal composite set, verify seeks with `EXPLAIN`, and audit `pg_stat_user_indexes` / `sys.schema_unused_indexes` for indexes that lost the bargain.
- Under sharding, secondary indexes force a choice: **local indexes make reads scatter-gather; global indexes make writes distributed** — synchronous (2PC) or asynchronous (eventual consistency, as in DynamoDB GSIs, which can also backpressure base writes). Search clusters are externalized global indexes; Cassandra's query-driven modeling is this trade-off adopted as doctrine.

Further reading

- PostgreSQL documentation, Chapter 11 "Indexes" — index types, partial and expression indexes, index-only scans and the visibility map, `CREATE INDEX CONCURRENTLY` and its recovery notes. <https://www.postgresql.org/docs/current/indexes.html>
- MySQL 8.4 Reference Manual, "InnoDB Index Types" and "How MySQL Uses Indexes" — clustered index behavior, secondary indexes storing PK values, multiple-column indexes and leftmost prefixes. <https://dev.mysql.com/doc/refman/8.4/en/innodb-index-types.html>
- Winand, M., *SQL Performance Explained / Use The Index, Luke!* — the best sustained treatment of index design from the query's point

of view, across Oracle, SQL Server, Postgres, and MySQL. <https://use-the-index-luke.com/>

- Graefe, G., "Modern B-Tree Techniques," *Foundations and Trends in Databases* 3(4), 2011 — the comprehensive survey: page structure, splits, fence keys, online index building, maintenance.
- Amazon DynamoDB Developer Guide, "Using Global Secondary Indexes" — the authoritative statement of GSI eventual consistency, projections, and throughput coupling with the base table. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html>
- PostgreSQL documentation, "GIN Indexes" and "BRIN Indexes" internals chapters — posting trees, the pending list, block-range summarization. <https://www.postgresql.org/docs/current/gin.html>
- Davoudian, A., Chen, L., and Liu, M., "A Survey on NoSQL Stores," *ACM Computing Surveys* 50(2), 2018 — context for secondary-index strategies across distributed stores.
- RFC 9562, "Universally Unique IDentifiers" (2024) — UUIDv7 and the time-ordered layout that makes UUIDs index-friendly. <https://www.rfc-editor.org/rfc/rfc9562>
- Chapter 2 — Storage Engines — B+ tree and LSM mechanics beneath everything here; Chapter 4 — Query Processing and Optimization — how the planner costs and chooses among these indexes; Chapter 6 — Isolation and MVCC — the versioning machinery behind HOT, vacuum, and the visibility map; Chapter 9 — Partitioning and Sharding — the placement decisions that make secondary indexes hard.

Chapter 4 — Query Processing and Optimization

What this chapter covers. Chapters 1 through 3 gave you the declarative surface (the relational model and SQL), the physical substrate (B-trees and LSM-trees), and the access structures (indexes). This chapter covers the machinery in between: how a database turns the sentence you wrote into the program it actually runs. That machinery is a compiler with an unusual property — it must choose among semantically

equivalent programs whose runtimes differ by factors of a thousand or more, and it must choose using *guesses* about your data. We walk the pipeline from parse through rewrite to plan search, dissect the physical operators (scans, the three join algorithms, aggregation, sorting) with their cost profiles, and then confront the honest truth of the field: cost-based optimization works remarkably well most of the time and fails catastrophically some of the time, and the failures are almost always cardinality-estimation failures. Because of that, the most valuable skill this chapter teaches is not theory — it is reading `EXPLAIN ANALYZE` output, finding the node where estimated rows diverged from actual rows, and knowing the ranked list of fixes. Examples are PostgreSQL-flavored because its planner is open, well-documented, and representative; the concepts transfer directly to MySQL, SQL Server, and Oracle, and — as the closing section shows — to distributed engines, where the same estimation errors get amplified by network data movement.

Learning goals — after this chapter you should be able to:

- Trace a query through parse → rewrite → optimize → execute, and say what each stage is responsible for and what can go wrong in it.
- Explain why a sequential scan beats an index scan above a surprisingly low selectivity fraction, in terms of random versus sequential I/O.
- Describe the mechanics and cost profiles of nested-loop, hash, and merge joins, and predict which one an optimizer should pick for a given input shape.
- Explain the Volcano iterator model, identify blocking operators, and say why vectorized and JIT-compiled execution exist.
- Describe what statistics an optimizer keeps (histograms, NDV, MCVs, correlation), how the independence assumption breaks cardinality estimates on correlated predicates, and how errors compound up a plan tree.
- Diagnose a bad plan from `EXPLAIN ANALYZE` by comparing estimated to actual rows, and apply fixes in the right order: statistics first, indexes second, rewrites third, hints last.
- Explain parameter sniffing, why plan caching makes it inevitable, and the standard mitigations.

- Extend all of the above to distributed engines: pushdown, broadcast versus shuffle joins, and why misestimates that cost milliseconds on one node cost minutes across a cluster.

The pipeline: from text to executable plan

Every mature SQL engine processes a query in the same four stages. The names vary by vendor; the structure does not.

```

flowchart LR
    SQL["SQL text"] --> P["Parse  
grammar check,  
build syntax tree"]
    P --> A["Analyze and bind  
resolve names and types  
against the catalog"]
    A --> R["Rewrite  
expand views  
flatten subqueries  
normalize predicates"]
    R --> O["Optimize  
search the plan space,  
cost each candidate"]
    O --> E["Execute  
run the physical plan,  
return rows"]
    STATS["Statistics  
histograms, NDV,  
MCV lists, correlation"] --> C["cardinality and  
cost estimates"]
    PC["Plan cache"] --> S["skip straight to execute  
for repeated statements"]
    E
  
```

Parse is ordinary compiler front-end work: tokenize, check the grammar, build a syntax tree. The only decisions here are syntactic. Binding (PostgreSQL calls this *analysis*) then resolves every name against the catalog — which table is `orders`, what type is `created_at`, does the user have permission — producing a fully typed query tree. Errors up to this point are cheap and deterministic; nothing interesting has happened yet.

Rewrite is where semantics-preserving transformations begin, and it matters more than its obscurity suggests. Three families dominate:

- *View expansion*. A reference to a view is replaced by the view's definition, inline. This is why a query over a view can be optimized as a whole — the planner sees one big tree, not an opaque call. It is also

why stacking five views on top of each other can produce a query tree the optimizer struggles with: each layer looked innocent alone.

- *Subquery flattening and decorrelation.* `WHERE x IN (SELECT ...)` becomes a semi-join; `EXISTS` becomes a semi-join; a scalar subquery in the select list may become an outer join. A **correlated** subquery — one that references the outer row — naively means re-executing the subquery once per outer row, which is a nested loop in disguise. Decorrelation transforms it into an ordinary join that the optimizer can then reorder and implement however it likes. When the rewriter *cannot* decorrelate (and every engine has constructs it gives up on), you get the classic per-row-subquery plan, and the fix is to decorrelate by hand — we return to this in the tuning section.
- *Predicate normalization.* Push predicates toward the tables they constrain (so filtering happens before joining, not after), derive implied predicates through equivalence classes (`a.id = b.id AND a.id = 5` implies `b.id = 5`), fold constants, simplify boolean structure.

The output of rewrite is still a *logical* query — relational algebra, in the sense of Chapter 1: a tree of selects, projects, joins, and aggregates that says *what* to compute with no commitment to *how*.

Optimize turns that logical tree into a physical plan: a specific join order, a specific algorithm for each join, a specific access path for each table, specific placements of sorts and aggregates. This is a search over a space that Chapter 1's algebra makes precise — joins commute and associate, so an n-way join has a number of orderings that grows factorially, and each ordering multiplies by the choices of join algorithm and access path. The optimizer navigates this space with cost estimates, and the bulk of this chapter is about how that goes right and wrong.

Execute runs the winning plan. The executor is where the physical operators live, so we start there — you cannot judge an optimizer's choices until you know what it is choosing between.

Physical operators

Access paths: how to read one table

For a single table with a filter, the executor has a small menu, and the interesting question is the crossover between the options.

A **sequential scan** reads every page of the table (its heap, in PostgreSQL terms) front to back, applying the filter to each row. It does the maximum number of page reads — but every one of them is sequential I/O, which the storage stack rewards heavily: read-ahead in the kernel and the drive, large request sizes, no seek or flash-translation overhead per page (Volume 1, Chapter 5). A sequential scan's cost is honest and flat: proportional to the table size, regardless of how many rows match.

An **index scan** walks a B-tree (Chapter 3) to the first matching key, then follows matching entries, fetching the corresponding heap page for each row to get the remaining columns and — in MVCC systems — to check visibility. Each of those heap fetches is potentially a *random* page read, and matching rows are scattered across the heap unless the table happens to be physically ordered by the indexed column. This is the crux: **an index scan does fewer page reads, but a worse kind.**

The crossover follows directly. If the filter matches 0.1% of a large table, the index scan touches a few thousand scattered pages while the sequential scan reads millions; the index wins enormously. If the filter matches 20% of the table, the index scan may touch nearly *every* heap page anyway — in random order, possibly more than once — while the sequential scan reads each page exactly once, in order, with read-ahead. The sequential scan wins, often by a large margin. The break-even point is workload- and hardware-dependent, but it is far lower than intuition suggests: on spinning disks the rule of thumb was that an index stops paying above roughly 1–5% selectivity for large tables; on SSDs the gap between random and sequential reads narrowed but did not close (and per-page CPU and buffer-management costs remain), so sequential scans still win at moderate selectivities. When you see a sequential scan in a plan, the correct first reaction is not "missing index" — it is "the planner believes a large fraction of this table qualifies." Sometimes that belief is wrong, which is a statistics problem; often it is right, and the seq scan is the fastest available plan.

Two refinements fill the middle ground:

- An **index-only scan** answers the query from the index alone, never visiting the heap, when every referenced column is in the index (a *covering* index, Chapter 3). In PostgreSQL this has a catch: index entries carry no visibility information, so the executor consults the *visibility map* and must still fetch heap pages for any page not marked all-visible. An index-only scan on a heavily updated, under-

vacuumed table silently degrades into an ordinary index scan — `EXPLAIN ANALYZE` exposes this as a high `Heap Fetches` count.

- A **bitmap scan** splits the difference. A bitmap *index* scan walks the index and collects matching row locations into an in-memory bitmap; a bitmap *heap* scan then sorts those locations and visits the heap pages **in physical order**, each page once. You pay index traversal plus one pass over just the relevant pages, in sequential-ish order — the right tool for the awkward 1-20% selectivity band. Bitmaps also compose: the executor can AND and OR the bitmaps of several single-column indexes to serve a multi-column predicate without a dedicated compound index. If the bitmap outgrows `work_mem` it degrades to one bit per *page* rather than per row ("lossy" in the plan output), and the filter is rechecked on each fetched page.

Access path	I/O pattern	Wins when
Sequential scan	All pages, sequential	Large fraction of rows qualifies; small tables
Index scan	Few index pages + scattered heap pages, random	Tiny fraction qualifies; correlated physical order
Index-only scan	Index pages only (if visibility map allows)	Covering index; well-vacuumed table
Bitmap scan	Index pages + relevant heap pages in physical order	Moderate selectivity; combining multiple indexes

Join algorithms: the big three

Every relational engine implements essentially three join algorithms. Each has a shape of input for which it is clearly best, which is precisely why the optimizer's choice — driven by row-count estimates — matters so much.


```

flowchart TB
    subgraph NL["Nested loop join"]
        direction TB
        NL1["For each OUTER row"] --> NL2["probe INNER"]
        naive: scan all M rows
        indexed: B-tree lookup"]
        NL2 --> NL3["Cost: N times M naive"]
        N times log M indexed
        wins: small outer,
        indexed inner"]
    end
    subgraph HJ["Hash join"]
        direction TB
        HJ1["BUILD: hash smaller input"]
        into in-memory table"] --> HJ2["PROBE: stream larger input, look up each row's key"]
        HJ2 --> HJ3["Cost: about N plus M"]
        needs memory for build side
        spills to disk partitions
        wins: large unsorted equijoins"]
    end
    subgraph MJ["Merge join"]
        direction TB
        MJ1["Both inputs sorted"]
        on the join key"] --> MJ2["Advance two cursors in lockstep, emit matches"]
        MJ2 --> MJ3["Cost: N plus M after sorting"]
        wins: order pre-exists via index or is needed anyway"]
    end
end

```

Nested loop join is the simplest and, in its naive form, the worst: for each of N outer rows, scan all M inner rows — $O(N \cdot M)$, which at $10^5 \times 10^5$ is 10^{10} comparisons and a query that never comes back. The redeemed form is the **indexed nested loop**: for each outer row, probe an index on the inner table's join key — $O(N \cdot \log M)$. For a *small* outer input this is unbeatable: five outer rows means five index lookups, microseconds of work, no setup cost, and the first output row appears almost immediately. The entire hazard of the nested loop is that its cost is linear in the outer cardinality, so it is exquisitely sensitive to the optimizer's estimate of N . A plan built on "N will be about 40" that meets $N = 800,000$ at runtime performs 800,000 index probes — the single most common way a misestimate becomes an incident, and the centerpiece of our `EXPLAIN` walk-through below. (PostgreSQL 14+ can interpose a **memoize** node that caches inner lookups by key, which

softens — but does not repair — this failure when the outer side has many duplicate keys.)

Hash join handles the case nested loops cannot: two large inputs, no useful indexes, an equality join condition. It runs in two phases. The **build** phase reads the smaller input once and builds an in-memory hash table keyed on the join columns. The **probe** phase streams the larger input once, hashing each row's key and checking the table. Total cost is roughly one pass over each input — $O(N + M)$ — which is asymptotically as good as a join can be. The costs are in the fine print. First, the build side must fit in the operator's memory budget (`work_mem` per node in PostgreSQL; a *memory grant* negotiated at plan time in SQL Server — itself computed from cardinality estimates, so a misestimate here means either a wasteful over-grant or a spilling under-grant). Second, when the build side does not fit, the join **spills**: both inputs are hash-partitioned into batches on disk such that each build partition fits in memory, and the join runs partition by partition. This is **Grace hash join** (from the GRACE database machine project, Kitsuregawa et al., 1983); the *hybrid* variant keeps the first partition in memory to save I/O. A spilling hash join still completes in near-linear time, but with an extra write and read of both inputs — visible in plans as `Batches: 16` (PostgreSQL) or hash spill warnings (SQL Server), and one of the first things to look for when a formerly fast query slows down as data grows. Third, hash joins require equality predicates; they cannot serve `<` or range conditions.

Merge join requires both inputs sorted on the join key, then advances two cursors in lockstep, emitting matches — one pass over each input, $O(N + M)$, with mark-and-restore rewinds when duplicate keys appear on both sides. Sorting two large inputs just to merge-join them rarely beats a hash join, so merge join wins when the order is *free or wanted*: an index scan already delivers rows in key order; the input is the output of a previous sort; or the query's `ORDER BY / GROUP BY` needs that order anyway, letting one sort serve two purposes. Merge join also handles some non-equality band conditions and degrades more gracefully than hash join at very large scale, which is why analytics engines love sort orders (Chapter 13).

	Nested loop (indexed)	Hash join	Merge join
Asymptotic cost	$O(N \log M)$	$O(N + M)$	$O(N + M)$ after sorting
Setup cost	None	Build hash table	Sort inputs (unless pre-sorted)
Memory	Tiny	Build side (spills via partitioning)	Sort space (spills to disk)
Join conditions	Any (index permitting)	Equality only	Equality and bands
First row latency	Immediate	After build completes	After sorts complete
Sweet spot	Small outer, indexed inner	Large unsorted equijoin	Order pre-exists or is needed
Failure mode	Outer misestimated small	Build side misestimated small → spill/OOM	Pointless sorts

Aggregation and sorting

Grouped aggregation mirrors the join dichotomy. **Hash aggregation** builds a hash table keyed on the grouping columns, updating each group's running state as rows stream in — one pass, no order required, but memory proportional to the number of *groups* (another quantity the optimizer must estimate: NDV again). **Sorted aggregation** (`GroupAggregate`) requires input ordered by the grouping columns and emits each group as it completes — trivial memory, and free if the order pre-exists from an index or a merge join. The estimation dependence has real teeth: before PostgreSQL 13, hash aggregation could not spill, so the planner refused it when it *estimated* the groups would exceed `work_mem` — and when it estimated wrong in the other direction, the executor blew through the budget and could take the process into out-of-memory territory. Modern PostgreSQL spills hash aggregation to disk like a hash join, converting the failure from an OOM into a slowdown.

Sorting itself is the executor's other memory-hungry operation. In-memory sorts use quicksort (or a top-N heapsort when a `LIMIT` bounds the output — the plan says `Sort Method: top-N heapsort`, and it is dramatically cheaper because it never materializes the full ordering). When the input exceeds `work_mem`, the executor switches to **external merge sort**: produce sorted runs the size of memory, write them to temporary files, then merge the runs — the classic external-memory algorithm, analyzed properly in Volume 14, Chapter 6. The plan output is explicit about it:

```
Sort Method: external merge  Disk: 138880kB
```

That line — a sort that wrote 135 MB of temporary file — is one of the highest-value strings to grep for in a slow query's plan. The fix is usually one of: raise `work_mem` (cautiously — it is a *per-operation* budget, and a busy server runs many operations), add an index that provides the order, or reduce the sorted row width by projecting fewer columns before the sort.

Pipelining, materialization, and the Volcano model

How do these operators compose into a running plan? The answer in nearly every row-store since the early 1990s is the **iterator model**, canonized by Goetz Graefe's Volcano system: every operator implements `open()` / `next()` / `close()`, and each `next()` call returns one tuple, pulled from the operator's children by calling *their* `next()`. The plan tree executes by the root pulling from its children, demand propagating down to the scans.

The model's virtue is composability and **pipelining**: a filter over a scan processes one row at a time with no intermediate storage; rows flow from disk to client touching each operator once. A `LIMIT 10` at the root pulls exactly as much work as ten output rows require, then stops.

But not every operator can pipeline. A sort cannot emit its first row until it has consumed its *last* input row; the build phase of a hash join must complete before probing begins; an explicit `Materialize` node buffers its input for rescanning. These **blocking** (pipeline-breaking) operators partition the plan into pipeline stages, and they are where memory is consumed and where spills happen — which is why they are where your eyes should go first in a plan.

The iterator model's vice is per-tuple overhead: one or more virtual function calls per row per operator, catastrophic branch misprediction, poor cache locality — tolerable when queries were I/O-bound, painful now that analytical scans run from memory. Two modern answers, both previewed here and developed in Chapter 13: **vectorized execution** (MonetDB/X100 lineage; DuckDB, ClickHouse, and most analytics engines) keeps the pull-based structure but passes *batches* of a few thousand column values per call, amortizing interpretation overhead and enabling SIMD (Volume 1, Chapter 7); and **JIT compilation** (HyPer lineage; Neumann's 2011 paper) compiles the plan — or, in PostgreSQL's more modest implementation, expression evaluation within it — to native code, eliminating interpretation entirely. For OLTP row stores the tuple-at-a-time model remains defensible; for scanning billions of rows it is not, which is a large part of why the OLTP/OLAP engine split (Chapter 13) exists.

Cost-based optimization

The executor gives the optimizer a menu; statistics and a cost model let it price the menu. The architecture every mainstream optimizer descends from is Selinger et al.'s 1979 System R paper: estimate the cardinality of each intermediate result from statistics, cost each physical alternative from those cardinalities, search join orders with dynamic programming, keep the cheapest. Forty-five years of refinement have improved every component — and left the weak point exactly where it was.

Statistics: what the optimizer knows

Statistics are collected by a background or manual sampling pass (`ANALYZE` in PostgreSQL, which samples `300 × default_statistics_target` rows — 30,000 at the default target of 100). Per column, the standard inventory is:

- **NDV** (number of distinct values, `n_distinct`): drives equality selectivity — with no better information, `col = ?` is assumed to match $1/\text{NDV}$ of the rows — and group-count estimates for aggregation. Estimating NDV from a sample is provably hard, and it is routinely wrong on skewed data; PostgreSQL lets you override it per column when you know better.

- **MCV list** (most common values): the top values with their measured frequencies. This is the defense against skew — `status = 'shipped'` uses the *measured* frequency of `'shipped'` if it is in the list, rather than the 1/NDV fiction.
- **Histogram**: equi-depth bucket boundaries over the non-MCV values, driving range selectivity (`created_at >= ...` matches the fraction of buckets past the cutoff).
- **Correlation**: the correlation between a column's values and the physical row order. This feeds the *cost* model, not cardinality: an index range scan over a column with correlation near 1.0 (say, an append-ordered timestamp) fetches heap pages nearly sequentially and is cheap; the same scan at correlation near 0 is a random-I/O storm.
- **Null fraction** and average width, for completeness of arithmetic and memory estimates.

Multiply these per-column numbers together up the tree and you get every cardinality in the plan. Which brings us to the load-bearing problem.

Cardinality estimation: the weak point

Here is the claim this chapter most wants you to retain: **when a plan is bad, it is almost always because a cardinality estimate was bad** — not the cost model, not the search algorithm. This is not folklore; it is the measured conclusion of Leis et al.'s "How Good Are Query Optimizers, Really?" (VLDB 2015), which evaluated the estimators of several production systems on a real-world dataset (IMDB) and found join-size estimates off by orders of magnitude as a routine occurrence, growing worse with each additional join — and found that feeding *true* cardinalities into the same optimizers fixed most bad plans, while improving the cost model without fixing cardinalities helped little.

The root cause is the **independence assumption**. Given `WHERE a = 1 AND b = 2`, the optimizer knows `sel(a = 1)` and `sel(b = 2)` individually and — knowing nothing about their joint distribution — multiplies them. If the predicates are correlated, the product can be wrong by the full strength of the correlation. The canonical example: `city = 'San Francisco' AND state = 'CA'`. Suppose each predicate individually matches 1% of an addresses table. Independence says the conjunction matches 0.01%. In reality every San Francisco row is a

California row, so the true answer is 1% — a 100× underestimate, from two innocent-looking filters. Real schemas are full of these: warehouse and region, product and category, status and terminal-timestamp (`status = 'completed' AND completed_at IS NOT NULL` — near-perfectly correlated, estimated as independent).

Joins make it worse, twice over. First, join selectivity estimation is intrinsically harder than filter estimation — the standard formula for an equijoin ($|N \times M| / \max(\text{NDV}_n, \text{NDV}_m)$) assumes uniformity and containment of key domains, both fragile. Second, and more damaging, **errors compound multiplicatively up the plan tree**: the output estimate of one join is the input estimate of the next, so a 20× error at the bottom becomes 400× two joins later. This is why optimizer misbehavior is disproportionately a many-join phenomenon, and why the estimates at the *leaves* of a plan deserve your scrutiny first — an error there poisons everything above it.

And the plan consequences are discontinuous. Recall the join table above: the indexed nested loop is optimal at $N = 40$ and catastrophic at $N = 800,000$. A cardinality error does not make the chosen plan proportionally slower — it makes the optimizer *choose a different plan*, on the wrong side of a cliff. This is the anatomy of the classic incident, "why did it pick a nested loop for 10 million rows": correlated predicates → underestimate at a scan → nested loop chosen for a "tiny" outer → millions of inner index probes → a query that ran in 200 ms yesterday runs in 20 minutes today, with no code change, because last night's data crossed a statistics boundary.

Partial fixes exist and you should use them. **Extended (multi-column) statistics** let you tell the collector which column combinations to measure jointly — PostgreSQL's `CREATE STATISTICS` supports functional-dependency coefficients, multi-column NDV, and multi-column MCV lists; SQL Server creates multi-column statistics with compound indexes and by hand; Oracle has column groups. These repair the specific correlations you name, at the cost of knowing in advance which ones matter. They do not generalize: the space of possible column combinations is combinatorial, join-crossing correlations (between columns of *different* tables) remain largely uncovered, and so estimation remains, honestly, the open wound of the field — which current research attacks with runtime feedback and learned models, none of it yet standard equipment.

The cost model

Given cardinalities, costing is arithmetic. PostgreSQL's model is representative and usefully transparent: a plan's cost is a unitless sum of page fetches and CPU work — `seq_page_cost` (1.0 by reference definition) per sequential page, `random_page_cost` per random page, `cpu_tuple_cost` (0.01) per row processed, plus smaller terms for index entries and operator evaluations. The single most consequential knob is `random_page_cost`, the ratio of random to sequential page cost. Its default of 4.0 encodes a disk-era compromise (raw spinning disks were more like 40× worse; caching pulled the effective ratio down). On SSD and NVMe storage, and on databases whose working set is mostly cached, the honest value is near 1.1 — and running the default 4.0 on flash systematically biases the planner *against* index scans and toward sequential scans. Lowering it on SSD-backed systems is one of the few configuration changes that reliably improves plan quality across a whole workload. The broader lesson from Leis et al. applies, though: cost-model tuning is second-order. A perfect cost model fed 400×-off cardinalities produces garbage with more decimal places.

Searching the plan space

Join ordering is the combinatorial heart of the search. The logical algebra of Chapter 1 says joins commute and associate; the number of orderings of an n-way join grows factorially (and the shape choice — left-deep chains versus bushy trees — multiplies it further). Selinger's answer, still the backbone: **dynamic programming** over subsets — compute the best plan for every pair of relations, then every triple reusing the pairs, upward until the full set is planned, pruning dominated alternatives at each level while retaining plans with useful "interesting orders" (sort orders a later merge join or ORDER BY could exploit). This finds the optimal order — with respect to the estimates — at exponential-in-n cost, fine for the 5-10 table joins of typical OLTP.

Past a threshold, exhaustive search is unaffordable, and engines switch to heuristics. PostgreSQL's cutover is explicit: at `geqo_threshold` (default 12) tables, the planner abandons dynamic programming for **GEQO**, a genetic algorithm that evolves a population of join orders — faster to plan, no optimality claim, and (unless seeded deterministically) capable of producing *different plans for the same query on different days*, which is exactly the kind of nondeterminism that makes 15-way-join reporting

queries erratically slow. Related knobs (`join_collapse_limit`, `from_collapse_limit`, default 8) bound how many relations the planner will even consider reordering — which, read from the other side, means explicit `JOIN` syntax order *becomes* the join order beyond that limit: a blunt but sometimes deliberate way to pin an order. The deeper point: a query joining 15 tables is not just linearly harder to plan than one joining 5 — it is exponentially harder, *and* its estimates are compoundedly worse. Very wide joins are a design smell worth fixing upstream.

Plan caching and parameter sniffing

Planning costs milliseconds; an OLTP system executing the same statement thousands of times per second cannot re-plan every execution. So plans are cached — per prepared statement in PostgreSQL, in a shared plan cache keyed by statement text in SQL Server and Oracle. And caching creates a new failure class with a memorable name.

A cached plan must serve *all* future parameter values, but it was chosen by looking at *some* value. **Parameter sniffing** (SQL Server's term; Oracle calls the mechanism bind peeking) is that look: optimize the statement using the first execution's actual parameters. Usually this is strictly better than optimizing blind. But consider `WHERE customer_id = $1` on a skewed distribution — most customers have a handful of orders, one aggregator account has four million. Sniff a small customer and you cache an indexed nested-loop plan that is perfect for millions of executions — until the aggregator arrives and the plan runs four million index probes. Sniff the aggregator first and you cache a hash-join-with-scan plan that makes every small-customer lookup do a table scan. Neither plan is wrong for the value it was built for; the *cache* is wrong to assume one plan fits all values. This is a certified production-incident classic, with a signature worth memorizing: a query is fast for weeks, then instantly and persistently slow after a restart, failover, or cache eviction re-sniffed it with an unlucky first parameter — "it got slow and `DBCC FREEPROCCACHE` / re-preparing fixed it" is parameter sniffing until proven otherwise.

PostgreSQL's variant is more polite but the same disease: a prepared statement's first five executions use custom per-parameter plans; the planner then compares their average cost to a **generic plan** (built with selectivity defaults instead of actual values) and, if the generic plan looks no worse *by estimated cost*, locks it in from execution six onward. When

the generic estimate is optimistic, the symptom is uncanny: the statement runs fast exactly five times, then degrades. `plan_cache_mode = force_custom_plan` is the escape hatch.

Mitigations, in rough order of preference: **re-plan the sensitive statements** (SQL Server `OPTION (RECOMPILE)`, PostgreSQL custom-plan mode — paying planning cost per execution to buy plan correctness); **plan invalidation and variability tooling** (Oracle's adaptive cursor sharing marks statements bind-sensitive and keeps multiple plans; SQL Server 2022's Parameter Sensitive Plan optimization caches per-band plans); **forced plans** (Query Store plan forcing, plan baselines) to pin the known-good plan; and schema-level fixes — sometimes the honest answer is that one logical query serves two different workloads and should be two queries.

Reading plans: EXPLAIN and EXPLAIN ANALYZE

Everything so far becomes practical through one skill: reading a plan, and specifically reading the *divergence between estimated and actual rows*. `EXPLAIN` shows the plan with estimates; `EXPLAIN ANALYZE` executes the query and annotates every node with what actually happened. The diagnostic method is one sentence: **walk the tree bottom-up, find the deepest node where estimated rows and actual rows diverge badly — that node is where the plan went wrong, and everything above it is collateral damage.**

A worked example. Schema: `orders` (50M rows), `customers` (2M rows), `order_items` (200M rows). The query: last 30 days of shipped orders from warehouse 9, revenue by customer region.

```
SELECT c.region,
       count(DISTINCT o.id)      AS orders,
       sum(oi.qty * oi.unit_price) AS revenue
FROM   orders o
JOIN   customers c ON c.id = o.customer_id
JOIN   order_items oi ON oi.order_id = o.id
WHERE  o.status = 'shipped'
       AND o.warehouse_id = 9
       AND o.created_at >= now() - interval '30 days'
GROUP BY c.region;
```

The trap is baked in: warehouse 9 is the high-volume fulfillment center, so `status = 'shipped'` and `warehouse_id = 9` are strongly correlated.

Individually, `shipped` matches ~20% of orders and `warehouse_id = 9` matches ~2%; independence predicts 0.4%, but nearly everything warehouse 9 touches gets shipped, so the true joint fraction is closer to 2%. The `EXPLAIN ANALYZE` output, trimmed to the fields that matter:

```
HashAggregate (cost=91427.18..91427.24 rows=6 width=48)
  (actual time=712140.63..712140.66 rows=6 loops=1)
  Group Key: c.region
  -> Nested Loop (cost=1.14..91302.55 rows=4982 width=26)
    (actual time=4.11..708341.02 rows=3945648 loops=1)
    -> Nested Loop (cost=0.57..38214.90 rows=1246 width=22)
      (actual time=0.35..14872.89 rows=986412
loops=1)
      -> Index Scan using orders_created_at_idx on orders o
        (cost=0.44..29655.17 rows=1246 width=18)
        (actual time=0.29..9531.44 rows=986412 loops=1)
        Index Cond: (created_at >= (now() - '30 days'::interval))
        Filter: ((status = 'shipped') AND (warehouse_id = 9))
        Rows Removed by Filter: 2113588
      -> Index Scan using customers_pkey on customers c
        (cost=0.13..6.87 rows=1 width=12)
        (actual time=0.004..0.004 rows=1 loops=986412)
    -> Index Scan using order_items_order_id_idx on order_items o_i
      (cost=0.57..41.12 rows=4 width=16)
      (actual time=0.42..0.70 rows=4 loops=986412)
Planning Time: 1.9 ms
Execution Time: 712302.51 ms
```

Read it bottom-up:

1. **The scan on `orders` is the crime scene.** Estimated `rows=1246`, actual `rows=986412` — a **792× underestimate**, and the `Filter` line tells you why: the correlated pair `status/warehouse_id` was multiplied as independent. Everything else in this plan is a *rational response to a wrong number*.
2. **The join choices were correct for the estimate.** For 1,246 outer rows, two indexed nested loops are exactly right — cheap probes, no setup. For 986,412 outer rows they mean roughly two million index probes (`loops=986412` on *both* inner scans — always multiply an inner node's cost by its `loops`). The per-probe times look innocent (0.004 ms, 0.70 ms); the multiplication is the disaster: $986,412 \times 0.70 \text{ ms} \approx 690 \text{ s}$, which is the query.

3. **The estimate error propagated upward:** the top join estimated 4,982 rows, actual 3.9 million — the leaf error times four items per order, compounding exactly as the theory said.

The fix is not an index — every scan here already uses one. The fix is the estimate:

```
CREATE STATISTICS orders_wh_status (dependencies, mcv)
ON status, warehouse_id FROM orders;
ANALYZE orders;
```

Re-planned with a truthful ~980k-row estimate, the planner does what you would do by hand:

```
Finalize HashAggregate (rows=6) (actual time=8912.44..8912.47 rows=6
loops=1)
-> Gather (workers launched: 4)
-> Partial HashAggregate
-> Parallel Hash Join (hash cond: oi.order_id = o.id)
    (est rows=1004890) (actual rows=789130 loops=5)
-> Parallel Seq Scan on order_items oi
-> Parallel Hash (build: orders join customers, 986412 r
ows)
Execution Time: 8944.07 ms
```

Hash joins, parallel scans, 712 seconds down to 9. Same data, same indexes, same SQL — one repaired statistic.

The ranked fix list

When a plan is wrong, apply remedies in this order — cheapest and most durable first:

1. **Refresh or strengthen statistics.** Run `ANALYZE` (autovacuum's analyze can lag on fast-growing or recently bulk-loaded tables — stale statistics after a big import is the most boring and most common cause of bad plans). Raise the per-column statistics target where histograms or MCV lists are too coarse for skew. Add extended statistics for correlated predicate pairs, as above.
2. **Fix the indexing.** Add the missing index the plan is compensating for; make a hot index covering to unlock index-only scans; and remove the self-inflicted wounds from Chapter 3 — a function wrapped around a column
(`WHERE date_trunc('day', created_at) = ...`, `WHERE lower(email) = ...`)

defeats both the index *and* the statistics on that column, so rewrite the predicate as a range or create the matching expression index (expression indexes get their own statistics, repairing estimation too).

3. **Rewrite the query.** Decorrelate the subqueries the rewriter could not: turn a per-row scalar subquery into a `LEFT JOIN` on a grouped derived table, or use a lateral join deliberately. Split `OR`s across different columns into a `UNION` of two indexable branches — `WHERE a = 1 OR b = 2` cannot use single-column indexes as one predicate, but the union of two single-predicate queries can (bitmap-OR handles some of these automatically; not all). Break a 15-way join into temp-table stages so estimation errors cannot compound end to end.
4. **Hints, last.** PostgreSQL core famously refuses them (`pg_hint_plan` exists as an extension); SQL Server, Oracle, and MySQL embrace them. Use a hint when a plan must be stabilized *now*; then treat it as technical debt. A hint encodes today's data shape into the query text — silently wrong when the data grows, invisible to the optimizer's improvements after an upgrade, and scattered across a codebase where nobody re-audits it. Prefer the engine's managed pinning machinery (next section) to ad-hoc hint sprinkling.

What the optimizer cannot see — and stability versus optimality

Two limits are worth stating plainly, because they shape production practice.

The optimizer's world ends at the statement boundary. It cannot see application semantics: that you will iterate the cursor and stop after the first row anyway (tell it — `LIMIT`), that this "one query" is actually executed in an N+1 loop by an ORM and should be a join, that two queries run back-to-back and the second could reuse the first's work, that the parameter is always the current tenant and never the aggregator. It optimizes each statement in isolation against a statistical sketch of the data. Every fix in the previous section is, at bottom, a way of telling the optimizer something true that it could not know.

Optimality and stability are different goals, and production often wants the second. The optimizer re-decides on every planning event: after an `ANALYZE` shifts a boundary, after an upgrade changes costing,

after a failover empties the cache. Each re-decision is a chance to find a better plan — and a chance to regress a critical query at 3 a.m. with no code change. A plan that is 20% worse than optimal but *always the same* is, for a hot OLTP path, often the better engineering choice; the cliff-edge discontinuity of plan choices means "slightly different estimate" can mean "50× slower plan". Hence **plan pinning** as a production discipline: SQL Server's Query Store records plan history per query and lets you force a known-good plan; Oracle's SQL Plan Baselines only admit *verified* new plans; and even in PostgreSQL — philosophically committed to re-planning — prepared-statement generic plans and `plan_cache_mode` are a crude form of the same trade. The cost of pinning is symmetrical: a pinned plan is a hint with better bookkeeping, and it too rots as data grows. Pin deliberately, record why, and revisit.

The distributed-systems lens: optimization when data has a location

Everything above assumed the data is on one machine. Distributed SQL engines — the NewSQL systems of Chapter 12, the analytics engines of Chapter 13 (Trino, Spark SQL, BigQuery), sharded PostgreSQL via Citus — keep the entire pipeline and add one new decision dimension: **where**. Every operator now has a location, every edge between operators potentially crosses the network, and data movement joins I/O and CPU as a first-class cost term — usually the *dominant* one.

The new decisions:

- **Pushdown.** Execute predicates, projections, and partial aggregations on the nodes that hold the data, so what crosses the network is the filtered, projected, pre-aggregated residue rather than raw rows. A `count(*) GROUP BY region` over 64 shards should move 64 small partial-result sets, not 200 million rows; the coordinator merges partial aggregates (note the `Partial / Finalize` aggregate split in the parallel plan above — single-node parallelism and distribution use the same decomposition, a **scatter-gather aggregation tree**).
- **Distribution strategy for joins.** When the join's two inputs are not partitioned on the join key, rows must move, and there are two ways to move them:

```

flowchart TB
    subgraph BC["Broadcast join – replicate the small side"]
        direction TB
        BS["Small table S  
1 copy sent to EVERY node"] --> BN1["Node 1  
local join: S with T shard 1"]
        BS --> BN2["Node 2  
local join: S with T shard 2"]
        BS --> BN3["Node 3  
local join: S with T shard 3"]
        BNOTE["Network cost: size of S times node count"]
        LTNM["Large table T never moves"]
        WSGS["Wins when S is genuinely small"]
    end
    subgraph SH["Shuffle join – repartition both sides"]
        direction TB
        SA["Table A shards"] --> SN1["Node 1  
on the join key | key range 1"]
        SA --> SN2["Node 2  
key range 2"]
        SB["Table B shards"] --> SN1
        SB --> SN2
        SNOTE["Network cost: size of A plus size of B  
one full pass of both over the network"]
        WBSL["Wins when both sides are large"]
    end

```

A **broadcast** join replicates the smaller input to every node holding the larger one; cost is $|S| \times \text{number of nodes}$, and the big table never moves. A **shuffle** (repartition) join re-hashes *both* inputs on the join key so matching rows land on the same node; cost is $|A| + |B|$ crossing the network once. The choice is a cardinality-driven threshold — and that should make you nervous, because you now know what cardinality estimates are worth.

- **Misestimate amplification.** On one node, underestimating a "small" input by $100\times$ costs you a spilling hash join — minutes. In a distributed plan, the same underestimate makes the engine *broadcast* a table that is not actually small: $100\times$ the expected bytes, multiplied by the node count, pushed through the network and into every worker's memory simultaneously — the classic cluster-wide incident where one query's bad broadcast decision inflicts memory pressure and network saturation on every other query's workers. (This failure is common enough that engines grew guardrails:

broadcast size caps, and Spark's adaptive query execution, which replans the join strategy mid-query using the *observed* size of the completed stage — runtime feedback standing in for estimates, exactly the research direction single-node optimizers are slower to adopt.) Join-order errors amplify the same way: a wrong order that materializes a huge intermediate result now materializes it *across the network*.

- **The same reading skill applies.** Distributed `EXPLAIN` output arrives as a tree of **fragments** or **stages** (Trino fragments, Spark stages, Citus per-shard subplans) connected by exchange operators — and the method is unchanged: find the exchange moving orders of magnitude more bytes than estimated, find the leaf estimate that caused it, fix the estimate or the partitioning. The vocabulary is new; the diagnosis is this chapter's.

The deep continuity: distribution does not change what query optimization *is* — it raises the stakes on the same weak point. The cost cliff between good and bad plans gets taller (network bytes dwarf local I/O the way random I/O dwarfs CPU), while the estimates driving the choice get no better. That is why every serious distributed engine invests in pushdown (shrink data before it moves), co-located partitioning (Chapter 9 — make the shuffle unnecessary), and runtime adaptivity (stop trusting estimates once real numbers exist).

Key takeaways

- **The pipeline is parse → rewrite → optimize → execute.** Rewrite does semantics-preserving surgery (view expansion, subquery decorrelation, predicate pushdown); optimization searches the physical plan space of the relational algebra from Chapter 1; execution runs Volcano-style iterators.
- **Sequential scans win more often than intuition says.** Index scans buy fewer page reads at the price of random I/O; above single-digit-percent selectivity on large tables, reading everything sequentially is often cheaper. Bitmap scans and index-only scans fill the middle.
- **Three join algorithms, three sweet spots.** Indexed nested loop for a small outer with an indexed inner; hash join for large unsorted equijoins (build/probe, spilling by Grace partitioning when the build

side exceeds memory)); merge join when sort order pre-exists or is needed anyway. The optimizer's choice among them sits on cost cliffs driven entirely by row estimates.

- **Blocking operators — sorts, hash builds — are where memory lives and spills happen.** Grep plans for external sorts and multi-batch hashes. Vectorized execution and JIT compilation are the modern answers to the iterator model's per-tuple overhead.
- **Cardinality estimation is the weak point — the Leis et al. result.** The independence assumption multiplies selectivities of correlated predicates into order-of-magnitude errors, and errors compound multiplicatively up the join tree. Bad plans are almost never cost-model failures; they are estimate failures. Extended statistics repair the correlations you name.
- **Read plans bottom-up, hunting estimate-versus-actual divergence.** The deepest badly wrong node is the cause; everything above is consequence. Multiply inner-node times by `loops`. The 792×-underestimated scan that chose a million-probe nested loop is the canonical incident.
- **Fixes in order: statistics, indexes, rewrites, hints.** ANALYZE and extended statistics first; covering/expression indexes and unwrapping function-mangled predicates second; decorrelation and OR→UNION rewrites third; hints last, as recorded technical debt.
- **Plan caching trades correctness-per-parameter for planning cost — parameter sniffing is the bill.** A plan sniffed for one parameter can be catastrophic for another; mitigate with recompilation, adaptive plan machinery, or managed plan forcing. In production, plan *stability* is often worth more than plan optimality.
- **Distribution adds "where" to every decision and amplifies every misestimate.** Pushdown shrinks data before it moves; broadcast versus shuffle is a cardinality-driven choice whose failure mode — broadcasting a "small" table that isn't — takes down clusters, not queries. The same EXPLAIN-reading skill applies to fragments and stages.

Further reading

- Selinger, P. G., Astrahan, M. M., Chamberlin, D. D., Lorie, R. A., and Price, T. G., "Access Path Selection in a Relational Database

Management System," *SIGMOD*, 1979 — the System R paper: statistics, cost estimation, and dynamic-programming join enumeration; the ancestor of every optimizer discussed here. <https://dl.acm.org/doi/10.1145/582095.582099>

- Graefe, G., "Volcano — An Extensible and Parallel Query Evaluation System," *IEEE Transactions on Knowledge and Data Engineering* 6(1), 1994 — the iterator model canonized. His survey "Query Evaluation Techniques for Large Databases," *ACM Computing Surveys* 25(2), 1993, remains the most complete treatment of the physical operators in this chapter.
- Leis, V., Gubichev, A., Mirchev, A., Boncz, P., Kemper, A., and Neumann, T., "How Good Are Query Optimizers, Really?," *PVLDB* 9(3), 2015 — the empirical demonstration that cardinality estimation, not cost modeling or search, is where plans go wrong. <https://www.vldb.org/pvldb/vol9/p204-leis.pdf>
- Neumann, T., "Efficiently Compiling Efficient Query Plans for Modern Hardware," *PVLDB* 4(9), 2011 — the HyPer JIT-compilation paper. Boncz, P., Zukowski, M., and Nes, N., "MonetDB/X100: Hyper-Pipelining Query Execution," *CIDR*, 2005 — the vectorized-execution counterpart.
- Kitsuregawa, M., Tanaka, H., and Moto-Oka, T., "Application of Hash to Data Base Machine and Its Architecture," *New Generation Computing* 1(1), 1983 — Grace hash join.
- PostgreSQL documentation: "Using EXPLAIN," "Planner Statistics" (including extended statistics via `CREATE STATISTICS`), "Genetic Query Optimizer," and the `runtime-config-query` planner cost constants. <https://www.postgresql.org/docs/current/using-explain.html>
- Lahdenmäki, T. and Leach, M., *Relational Database Index Design and the Optimizers* (Wiley, 2005) — the practitioner's treatment of access-path economics, complementing Chapter 3.
- Volume 5, Chapter 2 — Storage Engines — the random-versus-sequential I/O economics beneath the access-path crossover; Chapter 3 — Indexing — covering indexes, expression indexes, and the predicates that defeat them; Chapter 12 — NewSQL — distributed planning in full.
- Volume 14, Chapter 6 — external merge sort, analyzed properly.

- Volume 1, Chapter 5 — Storage Hardware — why `random_page_cost` exists and how SSDs changed it.

Chapter 5 — Transactions and ACID

What this chapter covers. The previous four chapters treated the database as a machine for storing and finding data. This chapter is about the machine's central promise: the **transaction**, the unit within which your application may pretend the world is simple. Inside a transaction, your code reasons as if it were the only client and as if failure did not exist; the engine absorbs the interleaving of hundreds of concurrent clients and the possibility of a crash at any instruction. This is the same bargain Volume 4, Chapter 3 called DRF-SC — adopt a discipline, and you may reason with the naive sequential model — offered one layer up, and it is arguably the single most valuable abstraction in systems software.

We define ACID precisely, and spend as much effort un-teaching folklore as teaching the substance: atomicity is about failure, not concurrency; consistency is not a database property at all in the sense the other three are; isolation's gold standard — serializability — has a precise definition that most engineers misremember. We then make serializability theory practical: schedules, conflicts, and the precedence-graph test you can run on paper. We tour the anomaly zoo that weaker isolation admits, survey the three concurrency-control families (two-phase locking, optimistic concurrency, and multiversion concurrency), and then get pragmatic: how to scope transactions, how to retry them, how to avoid the read-modify-write trap, and how long-running transactions quietly poison an MVCC system. Chapter 6 takes isolation levels and MVCC internals to full depth; Chapter 7 does the same for logging and recovery. This chapter is the frame both of those hang on.

Learning goals — after this chapter you should be able to:

- Explain what the transaction abstraction actually buys an application, and state its parallel to the DRF-SC bargain of Volume 4, Chapter 3.
- Define each ACID property precisely, including what atomicity is *not* about, why consistency is the odd one out, and what the durability spectrum really promises at each setting.

- Define serializability correctly — equivalence to *some* serial order — and test a schedule for conflict-serializability with a precedence graph.
- Recognize the standard anomalies — dirty read, dirty write, non-repeatable read, phantom, lost update, write skew — from symptoms in production.
- Compare strict 2PL, OCC, and MVCC: what each blocks, what each aborts, and where each wins.
- Write application code that uses transactions well: tight scope, correct retry on 40001/40P01, `FOR UPDATE` where it is needed, savepoints where they help.
- Explain why the transaction boundary becomes a design decision the moment data spans services, and what the outbox pattern and sagas are compensating for.

The transaction: the abstraction that lets you reason sequentially

A transaction is a group of reads and writes that the database promises to execute as a unit: either all of its effects become durable, or none of them do, and while it runs, other transactions do not interfere with it. The application states *what* must happen together; the engine handles *everything else* — the interleaving of concurrent clients, the buffer pool page that was half flushed when the power failed, the replica that was mid-apply, the operating system that lied about `write()` reaching the platter.

Consider what your code would have to do without this. A transfer between two account rows is two `UPDATE` s. Without transactions, a crash between them leaves money destroyed or created; a concurrent reader between them observes a state that never logically existed; two concurrent transfers touching the same account can interleave their read-modify-write cycles and silently lose one. Every application would carry its own crash-recovery protocol and its own concurrency-control protocol, and Volume 4 spent ten chapters showing how reliably engineers get the second of those wrong even with the full toolbox in hand.

The transaction removes both problems by contract. Volume 4, Chapter 3 closed with the observation that **serializable transactions are the**

database's DRF-SC, and it is worth spelling the parallel out, because it is exact:

DRF-SC (shared memory)	Transactions (databases)
Discipline: order all conflicting accesses with synchronization	Discipline: put related reads/writes inside one transaction
Reward: every execution is sequentially consistent	Reward: every execution is equivalent to some serial order
You may reason with naive interleaving semantics	You may reason as if transactions ran one at a time
Violate it: reordering, stale reads, "impossible" states	Weaken it: the anomaly zoo of this chapter
The compiler/CPU may reorder freely underneath	The engine may interleave freely underneath

Both bargains have the same fine print, too. DRF-SC holds only if you actually synchronize; the transaction bargain holds at full strength only at the **serializable** isolation level, and — as Chapter 6 will show in detail — most databases do not run there by default. PostgreSQL defaults to read committed; MySQL/InnoDB to repeatable read; both admit anomalies that serializability forbids. The engineer who "uses transactions" and assumes serial reasoning is in exactly the position of the engineer who uses threads and assumes sequential consistency: right most of the time, wrong under load, and wrong in ways that look like impossible states.

One more framing point before ACID. The transaction is the database's unit of *three* different things at once: the unit of **atomicity** (all-or-nothing under failure), the unit of **isolation** (the granularity at which concurrency is hidden), and — as we will argue in the application sections — the natural unit of **retry and idempotency**. Keeping those three roles aligned is most of what "using transactions well" means.

ACID, precisely

The acronym comes from Theo Härder and Andreas Reuter's 1983 survey "Principles of Transaction-Oriented Database Recovery," which packaged concepts Jim Gray and colleagues had developed at IBM System R

through the 1970s. The four letters are not four parallel guarantees — two are about failure, one is about concurrency, and one is a statement about your application — and collapsing them into a vague aura of "safety" is the root of most transaction folklore. Take them one at a time.

Atomicity: all or nothing — under *failure*

Atomicity says: either every write of the transaction takes effect, or none does. If the transaction aborts — because your code issued `ROLLBACK`, because a constraint fired, because the engine chose it as a deadlock victim, or because the server lost power mid-flight — the database ends up as if the transaction had never started.

The most common misreading is that atomicity is about concurrency — that "atomic" means other transactions can't observe the intermediate state. It does not. Whether others can see your uncommitted writes is an **isolation** question. Atomicity in the ACID sense is entirely about **abort and crash**: the guarantee that partial effects can always be erased. A better name, as Kleppmann and others have suggested, would be *abortability*.

The mechanism is **undo**. The engine must be able to reverse every write of an uncommitted transaction: via a rollback log or undo segments (InnoDB, Oracle), or — the MVCC variant Postgres uses — by never updating in place at all, so that "undo" is simply marking the new row versions as belonging to an aborted transaction and letting visibility rules ignore them. After a crash, the recovery procedure must finish the job for every transaction that was in flight: redo the committed work, undo the uncommitted. Chapter 7 — WAL and Crash Recovery — is the full treatment of that machinery (write-ahead logging, ARIES, and Postgres's redo-only variant); here it is enough to know that atomicity is a *recovery* property with a *logging* mechanism.

Atomicity is also what makes abort a **feature** rather than a failure mode. Because the engine can always erase a transaction cleanly, it is free to abort transactions as a concurrency-control tactic — deadlock victims, serialization-failure losers, OCC validation failures all lean on it. And your application gets the same power: a transaction that throws an exception halfway through leaves nothing behind to clean up. That guarantee is precisely what makes the retry loops later in this chapter safe to write.

Consistency: the odd one out

Härder and Reuter's C says: a transaction, run alone against a consistent database, produces a consistent database — where "consistent" means "all integrity constraints hold." Read that carefully: it is a property of the *transaction's program logic*, i.e., of your application. If your transfer procedure debits 30 and credits 50, the database will atomically, durably, in isolation, execute your bug. The engine cannot know that money should be conserved unless you tell it.

So consistency is not a guarantee the engine makes the way it makes A, I, and D. It is the *conclusion* of an inductive argument in which A, I, and D are the machinery: **if** each transaction individually preserves the invariants (your obligation), **then** atomicity ensures no half-applied transaction breaks them, isolation ensures no interleaving breaks them (serial equivalence means you only need to reason about transactions running alone — exactly the precondition of the C definition), and durability ensures a crash does not silently revert to a state your logic never produced. A + I + D turn per-transaction correctness into whole-database correctness. That is the actual content of the C, and it is why several authors have observed, only half-jokingly, that it is in the acronym mostly to make the acronym work.

The database's genuine contribution to consistency is **declared constraints**: primary keys, foreign keys, `UNIQUE`, `NOT NULL`, `CHECK`, exclusion constraints. These move a class of invariants from "every code path must remember" to "the engine refuses to violate," and a senior engineer should push as many invariants as possible into that form — a constraint is an invariant enforced against every writer, including tomorrow's batch job and next year's second service. But most business invariants ("an account's balance never goes below its credit limit minus pending holds," "every published article has at least one reviewer approval") are not expressible as row- or table-level constraints, and for those the C is entirely on you. Note the term collision, and keep the concepts apart: this C has nothing to do with *replica consistency* in the CAP sense (Volume 6) or *memory consistency* (Volume 4). Three fields, three unrelated meanings, one overloaded word.

Isolation: the freedom to ignore concurrency

Isolation says: concurrent transactions do not interfere — each behaves as if it were alone. The precise gold standard is **serializability**:

An execution of a set of transactions is *serializable* if its outcome — the final database state and the values every read returned — is equal to the outcome of *some* serial execution of those same transactions, one at a time in some order.

Two details in that definition do the work. First, **some** order, not a specific one: the engine may pick any serial order it likes, including one that differs from wall-clock arrival order. If that surprises you, you are thinking of *strict serializability* (serializability plus real-time ordering), which single-node databases running serializable mode do in practice give you, and which becomes an expensive, separate guarantee in distributed databases (Volume 6). Second, serializable does **not** mean serial. The engine's entire job is to interleave transactions as aggressively as possible while preserving equivalence to a serial order — concurrency is the point; serializability is the constraint on it. The next section makes "equivalent" precise.

Serializability at full strength costs performance, so the SQL standard defines weaker isolation levels — read uncommitted, read committed, repeatable read — that permit specific anomalies in exchange for less blocking or aborting, and real engines add snapshot isolation, which the standard never anticipated. That entire topic — what each level actually guarantees in each engine, which is emphatically not what the standard's table implies — is Chapter 6. This chapter needs only the frame: isolation is a *spectrum*, serializability is its top, and everything below the top is a negotiated exposure to the anomalies cataloged below.

Durability: committed means survives

Durability says: once the database acknowledges `COMMIT`, the transaction's effects survive a crash — process kill, kernel panic, power loss. The mechanism is the **write-ahead log** plus `fsync`: before acknowledging, the engine appends the transaction's log records to the WAL and forces them to stable storage; the data pages themselves can be flushed lazily, because after a crash the log replays the lost work. Chapter 7 covers the mechanics (and the ugly corners, like `fsync` error reporting and disks that lie about their write caches). What belongs here is the **durability spectrum**, because it is a knob you will actually be asked to set, and each position should be described honestly:

Setting	On commit	Crash risk	Honest description
<code>fsync -per-commit</code> (PG <code>synchronous_commit=on</code> , InnoDB <code>innodb_flush_log_at_trx_commit=1</code>)	WAL forced to disk before ack	None (single node)	The default. Commit latency includes a device flush.
Group commit	One flush amortized over many commits	None	Concurrent commits share a single <code>fsync</code> ; throughput scales without weakening the guarantee. Free lunch — this is why default durability is affordable under load.

Setting	On commit	Crash risk	Honest description
PG <code>synchronous_commit=off</code>	Ack immediately; WAL flushed in background (~every 200 ms)	Lose up to a few hundred ms of <i>acknowledged</i> commits	No corruption, no partial transactions — atomicity and ordering survive; recently acknowledged work may vanish as a unit. Legitimate for data you can afford to lose (metrics, sessions); indefensible for money. Settable per-transaction, which is the right way to use it.
InnoDB <code>innodb_flush_log_at_trx_commit=2</code>	Write to OS cache; flush per second	Lose ~1 s on OS/power crash; nothing on process crash	Similar honest trade, coarser knob.
<code>fsync=off</code> (PG)	Never force	Corruption on crash	Not a durability trade — a correctness forfeit. The recovery protocol's assumptions are void. Test rigs only.

The spectrum extends upward, too: `synchronous_commit=remote_write/on/remote_apply` in Postgres makes commit wait for replication, buying durability against whole-node loss at the price of a network round trip — Chapter 8's territory. The engineering point is that "durable" is not a boolean; it is a question — *durable against what, acknowledged when* — and the defaults answer it sensibly while the knobs let you trade honestly when a workload justifies it.

The transaction's life

The classical state machine ties A and D together and is worth having in your head, because every error your driver surfaces corresponds to an edge in it:

```
stateDiagram-v2
    [*] --> Active: BEGIN
    Active --> Active: reads and writes
    Active --> PartiallyCommitted: COMMIT issued
    Active --> Failed: error, ROLLBACK,
    deadlock victim, crash
    PartiallyCommitted --> Committed: log records forced
    to stable storage
    PartiallyCommitted --> Failed: crash before
    log flush
    Failed --> Aborted: undo applied,
    effects erased
    Committed --> [*]: effects permanent
    Aborted --> [*]: as if never run
```

The interesting state is *partially committed*: `COMMIT` has been issued but the log is not yet on disk. A crash there aborts the transaction — which is correct, because the client never received the acknowledgment. The commit *point* is the instant the commit record reaches stable storage; everything before it is revocable, everything after it is not. (A client that sent `COMMIT` and lost its connection before the reply genuinely cannot know which side of the point it landed on — a small preview of Chapter 10, where that ambiguity becomes the central problem.)

Serializability made practical

"Equivalent to some serial order" needs a workable definition of *equivalent*. The theory — developed in the 1970s and covered

exhaustively in Bernstein, Hadzilacos, and Goodman — gives one that is both checkable and the actual basis of real schedulers.

A **schedule** is the interleaved sequence of operations from a set of transactions, as the engine actually executed them: `r1(x)` is transaction T1 reading item x, `w2(y)` is T2 writing y. Two operations **conflict** if they belong to different transactions, touch the same item, and at least one is a write. That yields exactly three conflict shapes, and they are the atoms everything else in this chapter is built from:

- **RW** (read then write): T1 reads x, then T2 writes x. T1 read the *old* value — so in any equivalent serial order, T1 must come before T2.
- **WR** (write then read): T1 writes x, then T2 reads x. T2 saw T1's value — T1 must come first.
- **WW** (write then write): both write x; the final value is T2's — T1 must come first.

Two schedules are **conflict-equivalent** if they order every conflicting pair the same way; non-conflicting operations may be reordered freely, exactly as independent memory operations could be in Volume 4, Chapter 3. A schedule is **conflict-serializable** if it is conflict-equivalent to some serial schedule. And there is a clean test: build the **precedence graph** — one node per committed transaction, an edge $T_i \rightarrow T_j$ for every conflict in which T_i 's operation came first.

The theorem: a schedule is conflict-serializable **iff its precedence graph is acyclic**. If it is, any topological sort of the graph is an equivalent serial order.

Intuition: each edge says "Ti must precede Tj in any serial equivalent." A cycle says T1 must precede T2 *and* T2 must precede T1 — no serial order can satisfy both, so none exists.

Worked example

Two transactions each increment the same counter: Ti reads x, computes, writes x.

Schedule A — T2 slots in after T1 is done with x, while T1 continues on y:

```
r1(x) w1(x) r2(x) w2(x) r1(y) w1(y)
```

Conflicts on x: $r1(x) \rightarrow w2(x)$ (RW), $w1(x) \rightarrow r2(x)$ (WR), $w1(x) \rightarrow w2(x)$ (WW) — every edge is $T1 \rightarrow T2$. $T2$ never touches y . The graph is acyclic; the schedule is interleaved ($T2$ ran in the middle of $T1$) yet equivalent to serial $T1, T2$. This is the "serializable $\neq$ serial" point made concrete: the interleaving is harmless because every conflict points the same way.

Schedule B — the classic lost-update interleaving:

```
r1(x) r2(x) w1(x) w2(x)
```

Now $r1(x) \rightarrow w2(x)$ gives $T1 \rightarrow T2$, but $r2(x) \rightarrow w1(x)$ gives $T2 \rightarrow T1$. Cycle. In serial order $T1, T2$, $T2$ would have read $T1$'s write; in serial order $T2, T1$, vice versa; in this schedule *each read the initial value*, which no serial order produces. Two increments, counter goes up by one.

```
flowchart LR
    subgraph SA["Schedule A – acyclic, serializable as T1 then T2"]
        A1(("T1")) -->|"RW, WR, WW on x"| A2(("T2"))
    end
    all point one way
    subgraph SB["Schedule B – cycle, NOT serializable"]
        B1(("T1")) -->|"r1 of x before w2 of x"| B2(("T2"))
        B2 -->|"r2 of x before w1 of x"| B1
    end
```

Two caveats to keep you honest. Conflict-serializability is *sufficient*, not *necessary*: a strictly larger class (view-serializability) exists but is NP-hard to test, so every practical scheduler enforces the conflict flavor and cheerfully rejects a few schedules that were actually fine. And the precedence graph is an *after-the-fact* test on a complete schedule; a live scheduler must prevent or detect cycles as they form. The three families below are three strategies for exactly that — and Postgres's serializable implementation (SSI, Chapter 6) is quite literally a runtime that watches for dangerous edge structures in this graph and aborts a transaction to break them. When you see error 40001, an engine just told you: "committing you would have closed a cycle."

The anomaly zoo

Below serializability, specific cycles and specific reads-of-wrong-values are permitted, and they have names. Chapter 6 maps each anomaly to the isolation levels that allow it in each engine; here is the field guide — one

crisp example each, because you will meet these in incident reviews, not textbooks.

Dirty read — reading uncommitted data. T1 debits an account; T2 reads the debited balance and sends a low-balance alert; T1 aborts. T2 acted on a state that, per atomicity, *never existed*. Essentially no production engine permits this by default (even "read uncommitted" in Postgres behaves as read committed).

Dirty write — overwriting uncommitted data. T1 writes `order.buyer = A`; before T1 commits, T2 writes `order.buyer = B`; each then writes the matching `invoice` row. Interleaved, order and invoice can end up naming *different* buyers. Dirty writes also make rollback ill-defined (undo of T1 clobbers T2's committed value), which is why every serious engine excludes them at every level, typically by having writers hold row write-locks until commit even when everything else is MVCC.

Non-repeatable read — T1 reads a row, T2 commits an update to it, T1 reads it again and gets a different value. A report that sums balances twice for a consistency check fails its own check with no bug anywhere. Permitted at read committed; this is the anomaly snapshot reads exist to kill.

Phantom — the predicate version. T1 runs `SELECT count(*) FROM bookings WHERE room = 7 AND day = ...`, sees zero, and inserts a booking; concurrently T2 does the same. Neither's read saw the other's *new row* — locking the rows you read cannot help, because the conflict is with a row that did not exist yet. Phantoms are why serializable engines need predicate-based techniques — next-key/gap locking in InnoDB, predicate read tracking in Postgres SSI (Chapter 6) — not just row locks. The problem was identified, along with 2PL itself, in Eswaran et al. 1976.

Lost update — Schedule B above, in application clothing: two clients read-modify-write the same row and the second write silently swallows the first. Two `+1`s become one; two inventory decrements become one. This is the anomaly you personally will cause; a full section below is devoted to it.

Write skew — the subtle one. Invariant: at least one doctor on call. T1 and T2 each run `SELECT count(*) FROM oncall WHERE ward = 3` — both see 2 — and each takes a *different* doctor off call. Both reads were of a consistent snapshot; the two writes touch *disjoint rows*, so there is no WW conflict for a lock or first-writer-wins rule to catch; and yet the

combined outcome (zero on call) satisfies neither serial order. Write skew is the signature anomaly of snapshot isolation — the level that many engineers believe "is basically serializable" — and it is the standing proof that it is not. Chapter 6 dissects it; Postgres's SSI exists to catch precisely this shape.

Note the pattern across the zoo: every anomaly is a precedence-graph cycle that the isolation level in force failed to prevent, viewed from the application's side. The names are just the recurring shapes.

Concurrency control: three families

An engine that wants (conflict-)serializability must keep cycles out of the precedence graph. The three strategies map exactly onto the pessimistic/optimistic axis of Volume 4, Chapters 2 and 4 — block conflicts before they happen, or let them happen and abort on detection.

Pessimistic: two-phase locking

2PL attaches locks to data items: shared locks for reads (compatible with each other), exclusive locks for writes (compatible with nothing). The protocol's rule is the *two phases*: a **growing phase** in which the transaction acquires locks and a **shrinking phase** in which it releases them — and once it has released any lock, it may never acquire another. The theorem (Eswaran, Gray, Lorie, Traiger, 1976): all-2PL transactions produce only serializable schedules, with the serial order given by the order of *lock points* (each transaction's moment of maximum lock holding). The proof is a two-line induction on the precedence graph: every conflict edge $T_i \rightarrow T_j$ means T_j acquired a lock T_i released, so T_i 's lock point precedes T_j 's; a cycle would make a lock point precede itself.

Pure 2PL, though, allows a transaction to release locks before committing, and that breaks something *other* than serializability: **recoverability**. If T_2 reads data T_1 wrote and released early, and T_1 then aborts, T_2 must abort too — and anything that read from T_2 , transitively: a **cascading abort**. Worse, if T_2 already committed, durability and atomicity are now in direct contradiction. So real systems use **strict 2PL**: hold exclusive locks until commit or abort (in practice, engines hold *all* locks to the end — "rigorous" 2PL, which also makes the serialization order match commit order). Nobody reads uncommitted data, aborts never cascade, and undo is always safe. Every lock-based engine you will

meet — InnoDB's write path, SQL Server's default engine, DB2 — is strict 2PL at heart.

The price is **deadlock**, and it is not an edge case but a structural consequence: transactions acquire locks incrementally in data-driven order, so cyclic waits *will* occur. This is the same phenomenon as Volume 4, Chapter 5, with the same theory (a cycle in the waits-for graph) and one big difference: the database can do what a pthreads program cannot — detect the cycle and **abort a victim**, leaning on atomicity to erase it cleanly. InnoDB detects immediately on lock wait; Postgres checks after `deadlock_timeout` (1 s default); the victim's client sees an error (`40P01` in Postgres) and is expected to retry. Application-side, the Volume 4, Chapter 5 disciplines still pay: touch rows in a consistent order (e.g., always lock the lower account id first) and you starve the cycle of its raw material.

Optimistic: OCC

Kung and Robinson's 1981 alternative bets that conflicts are rare and locking is therefore mostly wasted overhead. A transaction runs in three phases: a **read phase** — execute fully, reads tracked in a read set, writes buffered privately, no locks; a **validation phase** — at commit, check whether any concurrently committed transaction's write set intersects this transaction's read set; a **write phase** — if validation passes, install the buffered writes and commit; if not, abort and rerun. This is Volume 4, Chapter 4's CAS loop promoted to transaction granularity: "proceed assuming no interference; at the last instant, verify; on failure, retry" — validation plays exactly the role of the compare in compare-and-swap, and the retry loop moves from your code into the transaction machinery.

The trade is the mirror image of 2PL's. No blocking, no deadlock, negligible overhead when transactions do not collide — and under contention, an **abort storm**: work is done in full and then thrown away, repeatedly, precisely when the system is busiest. Pessimism pays a constant tax and degrades by queueing; optimism is free until it degrades by burning CPU on discarded work. You also run OCC yourself, at application level, whenever you use a version-column (`UPDATE ... WHERE id = ? AND version = ?`, checking the row count) — the "optimistic locking" of ORMs — which is OCC with the database's own update atomicity as the validator.


```

flowchart LR
    subgraph PL["Strict 2PL – pay before, block on conflict"]
        direction LR
        P1["growing phase:  
acquire lock at first touch  
of each item"] --> P2["work under locks  
conflicting txns WAIT"] --> P3["commit:  
log forced"] --> P4["release all locks"]
        P2 -->| "cyclic wait" | P5["deadlock detected:  
victim aborted, 40P01"]
    end
    subgraph OC["OCC – pay after, abort on conflict"]
        direction LR
        O1["read phase:  
run fully, no locks,  
writes buffered privately"] --> O2["validation:  
read set vs concurrent  
committed write sets"]
        O2 -->| "clean" | O3["write phase:  
install writes, commit"]
        O3 -->| "overlap" | O4["abort:  
all work discarded, retry"]
    end
end

```

Multiversion: MVCC

The dominant modern design — Postgres, InnoDB, Oracle, and most newcomers — refuses the premise that readers and writers must fight at all. Writers never overwrite: each write creates a new **version** of the row, stamped with its creator transaction. Each transaction reads from a **snapshot** — the set of versions committed as of some instant — so **readers never block writers and writers never block readers**. A long analytics query and a stream of updates coexist without a single read lock; the pure-2PL cost of reads simply vanishes. Writes still conflict with writes, so every MVCC engine pairs versioning with something for its write path — row write locks (InnoDB, Postgres), first-committer-wins aborts, or SSI's validation — which is why MVCC is best understood not as a third family but as a substrate that makes reads free and lets the engine choose pessimism or optimism for writes only.

The catch, and it is the theme of two later sections: old versions must be kept as long as *any* snapshot might need them, and reclaimed afterward — Postgres's VACUUM, InnoDB's purge. That garbage collection is gated by the *oldest live snapshot*, which hands every long-running transaction a lever over system-wide storage health. Snapshot construction, visibility

rules, and what snapshot isolation does and does not guarantee are Chapter 6's core material.

Using transactions well

Everything above is the engine's side of the contract. Your side is scope, retry, and a few disciplines that separate codebases that hum from codebases that page you.

Scope discipline: short, and never across the outside world

A transaction, while open, holds: a pooled connection; every lock it has taken; and — in MVCC — a pin on its snapshot, which blocks version garbage-collection for the *entire cluster*, not just the rows it touched. All three costs scale with wall-clock duration, so the rule is the one from Volume 4, Chapter 2 — **no I/O under a lock** — applied at the next layer up, where it becomes:

Never hold a transaction across user think-time or a call to another system.

The classic sins: opening a transaction when a form is displayed and committing when the user submits (your lock hold time is now a coffee break); calling a payment gateway, an email API, or another microservice between `BEGIN` and `COMMIT` (your lock hold time is now their p99, and their outage is now your database outage). Run the arithmetic that Volume 4 ran for locks: a pool of 50 connections and transactions held open 2 s caps you at 25 transactions/s regardless of how fast the database is; row locks held for seconds turn hot-row workloads into convoys; and a snapshot pinned for minutes forces every MVCC table in the system to retain minutes of dead versions. The correct shape is always: gather inputs first, then one short transaction that does all the reads and writes back-to-back, then act on the outside world *after* commit (and if that outside action must be reliable, see the outbox pattern below — that ordering problem has a name and a solution).

Retry: serialization failures and deadlocks are not errors, they are the protocol

Run at serializable (or hit a deadlock at any level) and the engine will sometimes abort you by design: `SQLSTATE 40001`

(`serialization_failure`) means SSI or first-committer-wins broke a would-be cycle through you; `40P01` (`deadlock_detected`) means you were the victim. Neither means your code is wrong. They mean the concurrency control worked, and the contract obliges you to **retry the whole transaction**. Three rules make retry correct:

1. **Retry the transaction, not the statement.** The failed transaction's snapshot and partial logic are void; re-execute from the top so all reads are fresh.
2. **The transaction is the idempotency unit.** Atomicity guarantees an aborted attempt left *nothing* behind — so re-running it is safe *if and only if* every effect lives inside the transaction. Any side effect that escaped it (an HTTP call, a file write, a published message, mutated application state) will be duplicated by the retry. This is the scope discipline again, now load-bearing for correctness rather than just performance.
3. **Bound and back off.** Cap attempts, back off exponentially with jitter — the same herd logic as Volume 4, Chapter 5 — because serialization failures correlate: the conflicting workload that aborted you is still running.

```

import random
import time

import psycopg
from psycopg import errors

RETRYABLE = (
    errors.SerializationFailure, # SQLSTATE 40001
    errors.DeadlockDetected,    # SQLSTATE 40P01
)

def transfer(pool, src: int, dst: int, amount: int, max_attempts: int
= 5):
    for attempt in range(max_attempts):
        try:
            with pool.connection() as conn:
                conn.execute(
                    "SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")
                with conn.transaction(): # BEGIN; commit/rollback on
                    exit
                    row = conn.execute(
                        "UPDATE accounts SET balance = balance - %s "
                        "WHERE id = %s AND balance >= %s "
                        "RETURNING balance",
                        (amount, src, amount)).fetchone()
                    if row is None:
                        raise InsufficientFunds(src) # clean abort,
no retry
                    conn.execute(
                        "UPDATE accounts SET balance = balance + %s "
                        "WHERE id = %s", (amount, dst))
                    return # committed durably; NOW do external effects
(email, events)
        except RETRYABLE:

# Whole transaction rolled back; atomicity makes re-running safe.
        time.sleep(0.05 * (2 ** attempt) + random.uniform(0, 0.05))
        raise TransferContention(src, dst)

```

Note the shape: no external calls inside the `with conn.transaction():` block; business-rule failures (insufficient funds) raise *distinct* exceptions that abort without retry; the retryable set is exactly the two SQLSTATEs the engine defines for "try again."

The read-modify-write trap

The most common transaction bug in application code is Schedule B, written by hand:

```

-- Session A
BEGIN;
SELECT balance FROM accounts
  WHERE id = 7;    -- sees 100
100
-- app computes 100 - 30
UPDATE accounts SET balance = 70
  WHERE id = 7;
A's row lock...
COMMIT;

-- Session B
BEGIN;
SELECT balance FROM accounts
  WHERE id = 7;    -- also sees
100
-- app computes 100 - 50
UPDATE accounts SET balance = 50
  WHERE id = 7;    -- blocks on
-- ...unblocks, overwrites
COMMIT;

-- Final balance: 50.  A's withdrawal of 30 has vanished.

```

At read committed — the Postgres default — this is not a bug in the database. Both `SELECT` s took no locks; B's `UPDATE` waited for A's row lock, then re-read the *current* row and applied a value computed from a stale one. "I used a transaction" did not help, because the transaction faithfully executed a non-serializable interleaving that read committed permits. The fixes, in order of preference:

1. **Push the computation into the write.**

`UPDATE accounts SET balance = balance - 30 WHERE id = 7 AND balance >= 30` is a single atomic read-modify-write inside the engine; the row lock covers the read and the write together. When the new value is a function of the old, this is the whole fix, at any isolation level, in one statement.

2. **`SELECT ... FOR UPDATE`** when the application genuinely needs the value in hand before deciding (multi-row logic, branching): take the exclusive row lock at *read* time, so the concurrent reader blocks *before* seeing the stale value — converting the optimistic interleaving into strict-2PL behavior for exactly these rows. (`FOR NO KEY UPDATE` is the politer Postgres variant when you won't touch key columns; `FOR UPDATE SKIP LOCKED` turns the same primitive into a work queue — Chapter 14.)

3. **Raise isolation.** At `REPEATABLE READ` in Postgres, B's `UPDATE` fails with `40001` instead of proceeding on a stale snapshot (first-updater-wins); at `SERIALIZABLE`, all such patterns are caught. The retry loop above becomes mandatory equipment.

4. **Version-column OCC** when the read and write are separated by user think-time — the one case where you *cannot* hold a transaction (or its locks) across the gap, so you detect staleness at write time

instead:

```
UPDATE ... SET ..., version = version + 1 WHERE id = ? AND version = ? and treat zero rows updated as "someone else got there first."
```

```
sequenceDiagram
    participant A as Session A
    participant DB as Database row id 7
    participant B as Session B
    Note over A,B: Broken: plain SELECT then UPDATE at read committed
    A->>DB: SELECT balance
    DB-->>A: 100
    B->>DB: SELECT balance
    DB-->>B: 100
    A->>DB: UPDATE balance = 70
    A->>DB: COMMIT
    B->>DB: UPDATE balance = 50
    B->>DB: COMMIT
    Note over A,B: Balance 50 – the withdrawal of 30 is LOST
    Note over A,B: Fixed: SELECT FOR UPDATE serializes the read-modify-
write
    A->>DB: SELECT balance FOR UPDATE
    DB-->>A: 100 – exclusive row lock held
    B->>DB: SELECT balance FOR UPDATE
    Note over B,DB: B BLOCKS at the read
    A->>DB: UPDATE balance = 70
    A->>DB: COMMIT
    DB-->>B: 70 – fresh value after A commits
    B->>DB: UPDATE balance = 20
    B->>DB: COMMIT
    Note over A,B: Balance 20 – equivalent to serial A then B
```

Savepoints: partial rollback inside the unit

Atomicity is all-or-nothing at transaction granularity, but **savepoints** let you mark points within a transaction and roll back to them without abandoning the whole:

```

BEGIN;
INSERT INTO orders (id, customer_id, total) VALUES (9001, 42, 129.00);

SAVEPOINT before_promo;
INSERT INTO promo_redemptions (order_id, code) VALUES (9001,
'SUMMER25');
-- unique violation: code already redeemed by this customer
ROLLBACK TO SAVEPOINT before_promo;
-- the order INSERT is intact; only the redemption attempt is undone

UPDATE orders SET total = 129.00 WHERE id = 9001; -- price without
promo
COMMIT;

```

In Postgres, savepoints have a second, more important role: after *any* statement error, the whole transaction enters an aborted state and rejects every subsequent statement until rollback — so a savepoint before a statement that may legitimately fail (a speculative insert, a best-effort constraint probe) is the only way to absorb the error and continue. That is precisely what drivers and ORMs are doing when you see `SAVEPOINT`s in your query logs. Use them deliberately, not reflexively: each savepoint in Postgres opens a subtransaction, and thousands of subtransactions per transaction (a savepoint-per-statement ORM habit) degrade performance measurably — a Chapter 14 operational story.

Autocommit and transaction hygiene

Every mainstream driver defaults to **autocommit**: each statement is its own transaction. Both directions of confusion hurt. Engineers who don't notice autocommit write two `UPDATE`s in a row and believe they are atomic — they are not; a crash or a concurrent reader can fall between them. Engineers whose driver *disables* autocommit (psycopg2's historical behavior, Java with `setAutoCommit(false)`) get the opposite failure: the driver silently opened a transaction at the first statement, nobody ever commits, and the connection sits **idle in transaction** — holding its snapshot and locks — for hours. That state is the classic silent killer of MVCC systems, which is why Postgres grew `idle_in_transaction_session_timeout`; set it in every production config. The hygiene rules are short: know your driver's autocommit mode; make transaction boundaries explicit and visible in the code (a `with`-block or a decorator, not connection state mutated at a distance); and never let a

transaction's lifetime be controlled by anything other than the code inside it.

The long transaction as a systemic hazard

Combine the MVCC section with the scope section and you get the most under-appreciated fact in this chapter: in an MVCC engine, a long-running transaction damages the *whole system*, not just its own latency. Its snapshot defines the horizon behind which no dead version may be reclaimed. One forgotten `BEGIN` on a `psql` prompt, one analytics query wedged for six hours, one idle-in-transaction connection — and `VACUUM` (or InnoDB purge) silently stops reclaiming *every* table's dead versions cluster-wide. Update-heavy tables bloat; queries slow as they wade through dead versions; indexes swell; in Postgres, the transaction-ID wraparound clock ticks toward forced shutdown. The failure is insidious because the symptom (bloat, slow queries, autovacuum warnings) appears far from the cause (one old snapshot) and hours later. Chapter 6 explains the mechanism precisely; Chapter 14 covers monitoring and the timeouts that bound the damage. The design consequence belongs here: batch jobs should chunk their work into many short transactions, and anything that must read a consistent view for a long time should do so on a replica or an export, not on the primary's MVCC horizon.

The distributed-systems lens

Everything in this chapter shares one silent assumption: **one node**. One WAL to force, one lock table to check, one commit point that is a single atomic disk write. That assumption is what makes ACID cheap, and senior engineers should be clear-eyed that single-node ACID is the *easy case* — the moment data spans shards or services, each guarantee has to be re-earned against partial failure, and the price changes qualitatively.

The canonical failure is the **dual write**. Your service commits an order to its database and publishes an `OrderPlaced` event to Kafka — two systems, two writes, no shared transaction. Crash between them and the two systems permanently disagree; no retry policy fixes it, because each retry can itself half-fail. Notice this is just the transaction-scope discipline from the previous section, promoted to an architectural rule: "don't call another system inside a transaction" was a performance rule at one node; across systems it becomes the recognition that *there is no transaction that spans both*, and pretending otherwise is the bug.

You have two honest ways out. The first is to **buy distributed atomicity**: two-phase commit, in which a coordinator collects prepare votes and then broadcasts the decision. It works, and XA, distributed SQL engines, and every NewSQL system (Chapter 12) rest on it — but it converts the commit point from a local disk write into a multi-round-trip consensus-adjacent protocol that *blocks* while a coordinator is unreachable, with participants holding locks the whole time. Chapter 10 is the full accounting. The second is to **restructure so that every transaction is local** and cross-system effects flow asynchronously. The **outbox pattern** (Volume 10, Chapter 6) solves the dual write with tools from this chapter: write the business rows *and* a message row into the same local transaction — atomicity now covers both — and let a relay publish from the outbox table afterward, at-least-once. **Sagas** (Chapter 10) extend the idea to multi-step workflows: a sequence of local transactions with *compensating* transactions for rollback — manually re-implementing atomicity's undo at business granularity, with the crucial weakness that intermediate states are visible (there is no distributed isolation) and compensations must be written and tested like the safety-critical code they are. Choosing where the local-transaction boundaries fall — which invariants get real ACID and which get eventual convergence — is one of the highest-leverage design decisions in a service architecture, and it should be made explicitly, invariant by invariant, not inherited from whatever the first schema happened to look like.

Which brings us to **BASE** — "Basically Available, Soft state, Eventually consistent," the 2008-era banner (Pritchett's ACM Queue article is the standard citation) under which first-generation NoSQL systems dropped multi-object transactions. Stated honestly: BASE is not an alternative guarantee, it is the *absence* of one, renamed. The anomalies in this chapter's zoo do not disappear in a BASE system — they are re-delivered to your application code, which must now detect lost updates, merge divergent writes, and reason about read skew, using tools (vector clocks, CRDTs, reconciliation jobs — Volume 6) that are strictly harder to wield than `BEGIN / COMMIT`. Sometimes that trade is right: availability under partition and horizontal write scale are real requirements the single-node transaction cannot meet, and Chapters 10–12 take them seriously. But the trade should be *purchased*, deliberately, for named invariants — never accepted as the default because transactions were assumed slow or old-fashioned. The industry's own trajectory is the tell: a decade of NewSQL (Spanner, CockroachDB — Chapter 12) has been one long,

expensive effort to give the transaction abstraction *back* to engineers at distributed scale. The abstraction was never the problem. It was, and remains, the thing worth paying for.

Key takeaways

- **The transaction is the database's DRF-SC bargain:** put related operations inside one, run at a strong isolation level, and reason as if transactions execute one at a time; the engine owns interleaving and failure. The bargain is at full strength only at serializable — defaults are weaker, and the difference is the anomaly zoo.
- **Atomicity is about failure, not concurrency:** all-or-nothing under abort and crash, mechanized by undo/rollback (Chapter 7). It is what makes abort-and-retry a safe, load-bearing tool.
- **Consistency is the odd one out:** an application property — each transaction preserves the invariants — which A, I, and D lift inductively to the whole database. Declared constraints are the engine's contribution; declare every invariant you can express.
- **Serializability = equivalent to *some* serial order**, and serializable $\neq$ serial: interleaving is fine when conflict-free. Test with the precedence graph — RW/WR/WW conflict edges, cycle means non-serializable — and read 40001 as "the engine broke a cycle through you."
- **Durability is a spectrum, not a boolean:** `fsync-per-commit` with group commit is the sane default; `synchronous_commit=off` trades a bounded window of acknowledged commits for latency, honestly and per-transaction; `fsync=off` trades away correctness and is never a durability setting.
- **Three concurrency-control families:** strict 2PL blocks conflicts and pays with deadlocks (victim abort, 40P01); OCC runs first and validates at commit, paying with abort storms under contention — CAS at transaction granularity; MVCC makes reads free via snapshots and is the modern substrate, at the price of version garbage-collection (Chapter 6).
- **Scope discipline is "no I/O under a lock," one layer up:** transactions hold a connection, locks, and an MVCC snapshot — never span user think-time or external calls; do outside-world effects after commit.

- **Retry the transaction, not the statement**, on 40001/40P01, with bounded exponential backoff — and keep every effect inside the transaction, because the transaction is the idempotency unit.
- **SELECT then UPDATE without FOR UPDATE is a lost update waiting to happen** at default isolation. Prefer a single atomic `UPDATE` ; else `FOR UPDATE` ; else serializable-plus-retry; use version columns across user think-time.
- **Long transactions are a cluster-wide hazard in MVCC**: one old snapshot blocks dead-version reclamation everywhere. Chunk batch work; set `idle_in_transaction_session_timeout`.
- **Across services there is no transaction unless you build one**: 2PC buys distributed atomicity at blocking cost (Chapter 10); the outbox and sagas restructure around local transactions. BASE is the honest name for "the anomalies are now your application's job" — sometimes the right purchase, never a free one.

Further reading

- Härder, T. and Reuter, A., "Principles of Transaction-Oriented Database Recovery," *ACM Computing Surveys* 15(4), 1983 — the paper that coined ACID; the definitions of A, C, I, D in their original form. <https://dl.acm.org/doi/10.1145/289.291>
- Gray, J., "The Transaction Concept: Virtues and Limitations," *VLDB*, 1981 — Gray's own framing of the transaction as an abstraction, including its limits, a decade before it was folklore.
- Gray, J. and Reuter, A., *Transaction Processing: Concepts and Techniques*, Morgan Kaufmann, 1993 — the encyclopedic reference for everything in this chapter and Chapters 6–7.
- Eswaran, K., Gray, J., Lorie, R., and Traiger, I., "The Notions of Consistency and Predicate Locks in a Database System," *CACM* 19(11), 1976 — two-phase locking, the 2PL theorem, and the first identification of phantoms. <https://dl.acm.org/doi/10.1145/360363.360369>
- Kung, H.T. and Robinson, J., "On Optimistic Methods for Concurrency Control," *ACM TODS* 6(2), 1981 — the original OCC read/validate/write design.
- Bernstein, P., Hadzilacos, V., and Goodman, N., *Concurrency Control and Recovery in Database Systems*, Addison-Wesley, 1987 — the

rigorous treatment of serializability theory; freely available. <https://www.microsoft.com/en-us/research/people/philbe/book/>

- Berenson, H. et al., "A Critique of ANSI SQL Isolation Levels," *SIGMOD*, 1995 — the anomaly definitions used here (including write skew), and the bridge into Chapter 6.
- PostgreSQL documentation: *Transaction Isolation* (<https://www.postgresql.org/docs/current/transaction-iso.html>), *Explicit Locking* (<https://www.postgresql.org/docs/current/explicit-locking.html>), and the `synchronous_commit` discussion in *Write-Ahead Log* configuration (<https://www.postgresql.org/docs/current/wal-async-commit.html>) — precise, honest, and the ground truth for the Postgres behaviors cited in this chapter.
- Pritchett, D., "BASE: An ACID Alternative," *ACM Queue* 6(3), 2008 — the origin of the BASE framing, worth reading in the original before judging it. <https://queue.acm.org/detail.cfm?id=1394128>
- Volume 4, Chapters 2-5 — locks and I/O discipline, memory models and DRF-SC, optimistic concurrency and CAS, deadlock theory — the shared-memory versions of every idea here.
- Chapter 6 — Isolation Levels and MVCC, and Chapter 7 — WAL and Crash Recovery — the two deep dives this chapter frames; Chapter 10 — Distributed Transactions — where the single-node assumption is finally dropped.

Chapter 6 — Isolation Levels and MVCC

What this chapter covers. Chapter 5 introduced transactions and the ACID contract; this chapter goes deep on the I, which is the letter engineers misunderstand most expensively. The central fact is that **almost no production database runs serializable transactions by default, and the levels they do run permit specific, nameable anomalies that will corrupt your invariants while every individual statement returns correct-looking results.** Isolation levels are the database's weak memory models: if you read Volume 4, Chapter 3 — Memory Models and Happens-Before, this chapter will feel like déjà vu, deliberately. The same bargain reappears one layer up — strong is easy to

reason about and slow; weak is fast and treacherous; and the vendor's default is weak.

We start by making that parallel explicit, then examine the ANSI SQL isolation levels and the famous 1995 Berkeley critique that demonstrated the standard's definitions were broken — they missed entire anomalies and could not even describe snapshot isolation, the level half the industry actually runs. We work each anomaly with a concrete two-transaction example, then survey the real landscape engine by engine, where the trap is that **the same level name means different guarantees in different engines**. The second half opens the hood on MVCC — versioned tuples, snapshots, visibility rules, and garbage collection, contrasting PostgreSQL's copy-on-write heap with InnoDB's undo-log architecture — because you cannot operate these systems well without understanding why old versions accumulate and what makes them go away. We close with locking reads, a practical decision framework, and the distributed-systems lens, where this vocabulary reappears yet again across replicas.

Learning goals — after this chapter you should be able to:

- Map isolation levels onto the memory-model framework from Volume 4, Chapter 3: serializable as the DRF-SC analogue, weaker levels as weak memory models with named litmus tests.
- Define each anomaly — dirty read, dirty write, non-repeatable read, read skew, phantom, lost update, write skew — with a two-transaction example, and state which levels permit which.
- Explain why the ANSI SQL definitions were inadequate, per Berenson et al. 1995.
- Define snapshot isolation precisely, including first-committer-wins, and explain why write skew survives it.
- State what READ COMMITTED, REPEATABLE READ, and SERIALIZABLE actually guarantee in PostgreSQL versus MySQL/InnoDB, and why the shared names are misleading.
- Walk the MVCC mechanics: xmin/xmax versioning and snapshot visibility in PostgreSQL; undo logs, roll pointers, and ReadView in InnoDB; and the garbage-collection consequences of each.
- Diagnose the long-running-transaction disease in both engines: VACUUM's xmin horizon and bloat; InnoDB's history list growth.

- Choose an isolation level or an explicit locking strategy for a given invariant, and write correct retry logic for serialization failures.

Isolation levels are the database's memory models

Volume 4, Chapter 3 established a pattern you should now recognize on sight. Programmers assume sequential consistency — one global interleaving of operations — and no mainstream processor provides it, because it is too expensive. Instead, hardware and languages provide *weak* models that permit specific reorderings, document them via litmus tests, and offer tools (fences, volatile, atomics) to buy back ordering where you need it. The bargain is DRF-SC: follow the discipline and you may reason with the simple model; skip it and you are exposed to the machine.

Databases made exactly the same trade, a decade earlier, for exactly the same reason.

Programmers assume **serializability**: the result of running transactions concurrently is the same as *some* serial, one-at-a-time execution. That is the transactional analogue of sequential consistency — a single global order that explains everything you observed. And almost no database gives it to you by default, because enforcing it costs throughput: either pessimistic locking that serializes conflicting work and invites deadlock, or optimistic validation that aborts transactions and demands retries. So the vendors ship weaker levels — READ COMMITTED, REPEATABLE READ — that permit specific, documented anomalies, exactly as x86-TSO permits store-load reordering and documents it.

The correspondence is close enough to use as a study aid:

Memory models (Vol 4, Ch 3)	Isolation levels (this chapter)
Sequential consistency	Serializability
x86-TSO, ARM/POWER weak models	READ COMMITTED, snapshot isolation
Litmus tests: store buffering, message passing	Anomalies: write skew, lost update, read skew
Reordering invisible single-threaded	Anomalies invisible to a lone transaction

Memory models (Vol 4, Ch 3)	Isolation levels (this chapter)
Fences, volatile, atomics	<code>SELECT ... FOR UPDATE</code> , explicit locks
DRF-SC: synchronize and reason serially	Run serializable and reason serially
"It worked on x86"	"It worked at READ COMMITTED under low load"

Every row transfers. An anomaly, like a reordering, is invisible to a transaction running alone — it appears only when a second transaction observes overlapping state, and only in a narrow timing window, which is why these bugs pass every test and surface in production under load. The named anomalies are litmus tests: minimal two-transaction programs that distinguish levels, exactly as store-buffering distinguishes TSO from sequential consistency. And the practical discipline is the same: know precisely which model your engine gives you, reason from the documented contract rather than from observed behavior, and buy back strength surgically where an invariant demands it.

One difference is worth flagging early. Memory models are about *visibility ordering* of individual loads and stores; isolation levels are about *entire multi-statement transactions* whose reads and writes are supposed to behave as an atomic unit. That makes the database problem in one way harder — the units are bigger and hold state longer — and in one way easier: the database is a single arbiter that sees every access, so it can detect conflicts and abort, an option shared-memory hardware does not have. Optimistic concurrency control (Volume 4, Chapter 4) is exactly this abort-and-retry option, and serializable-via-SSI below is its most sophisticated production form.

The ANSI levels and why their definitions failed

The SQL standard (SQL-92 onward) defines four isolation levels by way of three *phenomena* each level must exclude:

- **P1 — dirty read:** transaction T1 reads data written by a concurrent, uncommitted T2. If T2 aborts, T1 has read data that never existed.

- **P2 — non-repeatable read (fuzzy read):** T1 reads a row; T2 modifies or deletes it and commits; T1 reads the row again and sees a different value, or nothing.
- **P3 — phantom:** T1 reads a *set* of rows matching a predicate; T2 inserts or modifies rows so that new rows match; T1 re-runs the query and the set has changed.

The levels are then defined subtractively: READ UNCOMMITTED permits all three; READ COMMITTED excludes P1; REPEATABLE READ excludes P1 and P2; SERIALIZABLE excludes all three.

This framework is broken, and the definitive demolition is **Berenson, Bernstein, Gray, Melton, O'Neil, and O'Neil, "A Critique of ANSI SQL Isolation Levels" (SIGMOD 1995)** — one of the most consequential systems papers of the decade, and short enough that you should actually read it. Its arguments:

The definitions are ambiguous. Read strictly (the "anomaly interpretation"), the English phenomena describe only specific completed histories, and excluding those exact histories still admits closely related bad ones. The paper shows the phenomena must be read broadly — as constraints on the *possibility* of interleavings, not on particular outcomes — or the levels they define are weaker than everyone intended.

The list of anomalies is incomplete. The standard never mentions **dirty writes** (T1 overwrites T2's uncommitted write — every real level excludes this, and the standard forgot to require it), **lost updates**, **read skew**, or **write skew**. A level can exclude P1-P3 as literally written and still permit all of these.

Serializable is defined wrong. ANSI says SERIALIZABLE is the level that excludes P1-P3. But excluding those three phenomena does not imply serializability — write skew slips through. This is not a pedantic gap: Oracle read the standard exactly this way, implemented snapshot isolation, observed that it excludes P1-P3, and to this day sells it under the name `SERIALIZABLE`. An Oracle transaction at the highest available level can exhibit write skew, which no serial execution could produce. The standard's own definition licensed this.

A real, important level is missing entirely. The paper introduced and named **snapshot isolation** — the level that DB2's competitors, Oracle, and later PostgreSQL and countless MVCC engines actually implemented — and showed it is *incomparable* with ANSI REPEATABLE READ: SI

excludes phantoms-on-read (a snapshot never changes) but permits write skew; classic lock-based REPEATABLE READ prevents write skew on read rows but permits phantoms. Neither is strictly stronger. The ANSI ladder is not a ladder; it is a partial order, and the standard drew it as a line.

The lesson generalizes beyond SQL: **defining consistency by listing prohibited anomalies is fragile, because the list is never complete.** The modern approach — Adya's 1999 thesis, and the formal definitions used by Jepsen — defines levels by constraints on the dependency graph of committed transactions, from which the anomalies fall out as consequences. You do not need that formalism day to day, but you do need the corrected anomaly vocabulary, so we build it next.

The anomaly bestiary, worked

Each anomaly below is a two-transaction litmus test. Timelines run downward; both transactions begin before either commits.

Dirty read and dirty write

Dirty read: T2 writes `balance = 900` and has not committed; T1 reads 900 and acts on it; T2 rolls back. T1 has consumed data that never existed. Dirty write: T1 and T2 both update the same row before either commits, interleaving their writes; whatever commit order follows, one transaction's update is entangled with the other's, and rollback becomes incoherent — which is why even READ UNCOMMITTED excludes dirty writes in every real engine (first writer takes a row lock; the second blocks). You will essentially never observe either anomaly in a modern MVCC database; they matter as the floor of the hierarchy, not as practical risks.

Non-repeatable read and read skew

Non-repeatable read is the single-row form: T1 reads a row, T2 updates it and commits, T1 reads it again and gets a different answer within one transaction. **Read skew** is the multi-row form the ANSI standard missed, and it is the one that actually burns people: T1 reads row X, then T2 updates X and Y together (preserving an invariant between them) and commits, then T1 reads Y. T1 has seen X's old value and Y's new value — a combination that never existed at any instant.

Concretely: accounts A and B each hold 500, invariant `A + B = 1000`. T1 reads `A = 500`. T2 transfers 100 (`A := 400, B := 600`) and commits. T1 reads `B = 600` and concludes the pair sums to 1100. Every read returned genuinely committed data; the *combination* is fictional. This is precisely why a long analytics query or a backup taken at READ COMMITTED can observe a state that violates invariants the application never violated — and why per-transaction snapshots exist.

Lost update

T1 and T2 both execute read-modify-write on the same row: read counter = 10, compute 11, write 11. Both commit; one increment vanished. No dirty read occurred — both read committed data — and no dirty write — the writes were serialized by row locking. The *interleaving* is the problem: T2's read happened before T1's write, so T2's write clobbers it. Any pattern that reads a value into the application, computes, and writes back is exposed: counters, "append to a JSON column," read-check-update workflows. The fixes, previewed here and detailed below: an atomic single statement (`UPDATE t SET n = n + 1`), a locking read (`SELECT ... FOR UPDATE`), an engine that detects the conflict (PostgreSQL REPEATABLE READ and above), or compare-and-set (`UPDATE ... WHERE n = 10`) with application retry — the same optimistic pattern as Volume 4, Chapter 4.

Write skew — the one that survives snapshot isolation

Write skew is the aristocrat of anomalies: it defeats snapshot isolation, it is the reason "REPEATABLE READ" is not enough for constraint-like invariants, and the classic example (from the literature on SI, popularized by Cahill's and Kleppmann's treatments) is the on-call roster.

Hospital rule: at least one doctor must be on call. Alice and Bob are both on call. Each, simultaneously, requests to go off call. Each transaction runs the same logic:

```
BEGIN;
SELECT count(*) FROM doctors WHERE on_call;  -- returns 2: safe to
leave
UPDATE doctors SET on_call = false WHERE name = 'alice';  -- (Bob
updates his own row)
COMMIT;
```

Both transactions read from a snapshot in which the count is 2, both conclude the invariant survives their own departure, and both commit — **their write sets are disjoint** (Alice's row versus Bob's row), so no write-write conflict detection fires. The roster is now empty. No serial execution could produce this: whichever transaction ran second would have counted 1 and refused.

```
sequenceDiagram
    participant A as "Txn A - Alice"
    participant DB as "Database"
    participant B as "Txn B - Bob"

    Note over DB: "on_call: Alice true, Bob true"
    A->>DB: "BEGIN - snapshot taken"
    B->>DB: "BEGIN - snapshot taken"
    A->>DB: "SELECT count WHERE on_call"
    DB-->>A: "2 - safe to leave"
    B->>DB: "SELECT count WHERE on_call"
    DB-->>B: "2 - safe to leave"
    A->>DB: "UPDATE alice SET on_call = false"
    B->>DB: "UPDATE bob SET on_call = false"
    A->>DB: "COMMIT - ok, write sets disjoint"
    B->>DB: "COMMIT - ok, write sets disjoint"
    Note over DB: "on_call count is 0"
    invariant broken
    no serial order explains this"
```

The general shape: **read a predicate, decide based on it, write something the *other* transaction's predicate depends on**. Each transaction's write invalidates the other's read, but neither writes what the other wrote. Once you know the shape you see it everywhere: enforcing "username unique" by select-then-insert; booking the last seat by counting bookings; spending against a shared budget by summing spends; two workers each claiming a task after checking no one has. Phantoms compound it — when the decision depends on the *absence* of rows, there is not even a row for conflict detection to notice.

Phantoms

A phantom is a predicate read invalidated by insertion: T1 counts rows `WHERE on_call`, T2 *inserts* a new matching row and commits, T1's re-read (or its decision) is stale. Under a per-transaction snapshot the re-read within T1 is stable — SI has no phantoms *within* the transaction's view — but the write-skew structure above shows the decision can still be

invalidated at commit time. Lock-based engines attack phantoms differently, by locking the *predicate's key range* rather than existing rows; that is InnoDB's next-key locking, below.

The real level landscape

With the vocabulary fixed, here is what the levels actually mean in the engines you run — and the running theme is that **the same name buys different guarantees in different engines**.

READ UNCOMMITTED — vestigial

Permits dirty reads. In the locking-scheduler world of the 1980s this was a real performance tier: readers skipped shared locks. In an MVCC engine it buys nothing, because readers never block on writers anyway. **PostgreSQL accepts the syntax and silently gives you READ COMMITTED** — it does not implement a distinct level, and the documentation says so plainly. InnoDB does implement it (reads skip the ReadView and read the newest uncommitted version); there is almost no defensible modern use. Treat it as historical.

READ COMMITTED — the default you are probably running

The default in PostgreSQL, Oracle, and SQL Server (in its read-committed-snapshot form on Azure SQL). The MVCC implementation: **every statement gets a fresh snapshot** of the database as of that statement's start. Each individual statement sees only committed data — no dirty reads — but two statements in the same transaction can see different committed states.

What it therefore permits: non-repeatable reads and read skew (statement 2 runs against a newer snapshot than statement 1); lost updates via application-level read-modify-write (your SELECT and your UPDATE are different statements with different snapshots, and nothing checks that the row you read is the row-version you overwrite); write skew, trivially. One subtlety worth knowing in PostgreSQL: when an `UPDATE` at READ COMMITTED finds its target row locked by a concurrent transaction, it waits; if that transaction commits, PostgreSQL re-evaluates the `WHERE` clause against the *new* row version and proceeds. That re-check prevents blind overwrites within a single UPDATE

statement, but it does nothing for values your application read in an earlier statement — the classic lost-update hole remains open.

READ COMMITTED is a reasonable default for workloads whose writes are single-statement and atomic (`UPDATE ... SET n = n + 1`), whose invariants live in constraints the database checks (`UNIQUE`, `CHECK`, foreign keys — these are enforced regardless of isolation level), and whose multi-statement reads tolerate mild inconsistency. It is the wrong level for any read-decide-write sequence, and most application code is full of them.

REPEATABLE READ — one name, two different beasts

PostgreSQL REPEATABLE READ is snapshot isolation. One snapshot is taken at the transaction's first query and every statement reads from it: no non-repeatable reads, no read skew, and no phantoms within your view — the snapshot is frozen. Additionally, PostgreSQL enforces a write-conflict rule: if you attempt to `UPDATE` or `DELETE` a row that a concurrent transaction has modified since your snapshot, you block until that transaction resolves; if it commits, your transaction aborts with `ERROR: could not serialize access due to concurrent update (SQLSTATE 40001)`. This is **first-updater-wins**, the blocking variant of SI's classic first-committer-wins, and it means lost updates are *detected* at this level — the second read-modify-write aborts instead of clobbering. Write skew, of course, is still permitted: disjoint write sets never trigger the check.

MySQL/InnoDB REPEATABLE READ — the default level in MySQL — is a different animal. Non-locking `SELECT`s use a consistent snapshot established at the first read, so plain reads behave like SI. But there is no first-committer/updater check: a concurrent transaction can modify a row after your snapshot, and your subsequent `UPDATE` of that row simply proceeds against the *current* committed version — **lost updates via read-modify-write are permitted at InnoDB REPEATABLE READ**, where PostgreSQL's level of the same name aborts them. Furthermore, your `UPDATE` reads and writes the current version, not your snapshot's, so a transaction can act on data newer than what its `SELECT`s showed — a mind-bending mix of snapshot reads and current writes. On the other hand, InnoDB's *locking* reads and writes take **next-key locks** — a lock on the index record plus the gap before it — so a `SELECT ... FOR UPDATE` over a range prevents concurrent inserts into that range: phantoms are

blocked for locking reads, which is a guarantee PostgreSQL SI does not express in locking terms at all.

Same words on the wire — `SET TRANSACTION ISOLATION LEVEL REPEATABLE READ` — and materially different anomaly profiles. **Never port an application between engines, or reason about one from experience with the other, on the assumption that the level names carry the semantics.** They carry the syntax.

Snapshot isolation, precisely

Since SI is the de facto standard level, define it exactly. A transaction T at snapshot isolation:

1. Reads from a **snapshot**: the committed state as of T's start (its *start timestamp*). T also sees its own uncommitted writes. Nothing that commits after T's start is visible.
2. On commit, passes **first-committer-wins** validation: if any transaction that was concurrent with T (committed between T's start and T's commit) wrote to an item T also wrote, T aborts. Write-write conflicts on overlapping write sets are thus impossible; one of the pair dies. (Implementations often use the blocking *first-updater-wins* variant — detect at write time rather than commit time — with the same net guarantee.)

What follows from the definition: no dirty reads, no non-repeatable reads, no read skew, no phantoms in the transaction's view, no lost updates. And one precise hole: **write skew**, plus its phantom-flavored variants, because validation examines only *write-write* overlap. Two transactions whose reads overlap but whose writes are disjoint sail through — the doctors example. The read that justified the write is never validated. That asymmetry — writes checked, reads not — is the entire gap between SI and serializability, and closing it is exactly what SSI does.

SERIALIZABLE — two implementations, two cost models

PostgreSQL: Serializable Snapshot Isolation (SSI). Based on Michael Cahill's work (Cahill, Röhm, Fekete, SIGMOD 2008) and brought to PostgreSQL 9.1 by Dan Ports and Kevin Grittner (VLDB 2012). The theory: every SI anomaly requires a cycle in the transaction dependency graph containing two consecutive **rw-antidependency** edges — T1 reads something T2 then overwrites ("T2 invalidates T1's read"), and T2's read

is in turn invalidated by T3 (possibly T1 itself). In the doctors example, A's count is invalidated by B's update and B's count by A's update: two rw edges forming the dangerous structure. SSI runs transactions optimistically under plain SI, tracks reads with in-memory **SIREAD locks** (including predicate/range information), watches for pairs of consecutive rw-antidependencies, and aborts one transaction of any dangerous structure with SQLSTATE 40001. The detection is conservative — dangerous structures do not always complete a cycle, so **false-positive aborts happen by design** — but it never admits a non-serializable execution. Costs: memory and CPU for read tracking, reduced concurrency from aborts, and a hard requirement that **every serializable transaction be wrapped in a retry loop**, because any of them can fail through no fault of its own. Readers never block writers and writers never block readers — the MVCC property survives; conflict resolution happens by abort, not by waiting. Two operational notes: SIREAD tracking can escalate from row to page to relation granularity under memory pressure, raising false-positive rates; and `SELECT ... FOR UPDATE` inside serializable transactions adds nothing to correctness — SSI already covers reads.

MySQL/InnoDB: two-phase locking. `SERIALIZABLE` in InnoDB converts plain `SELECT s` into `SELECT ... FOR SHARE` — every read takes shared next-key locks, held to commit. This is classical strict two-phase locking with range locking: correct, pessimistic, and blocking. Readers block writers and writers block readers; the failure mode is deadlock (error 1213 `ER_LOCK_DEADLOCK`, victim chosen and rolled back) rather than optimistic abort — so you need a retry loop here too, just triggered by a different error. Throughput under contention degrades by queueing rather than by aborting. Neither cost model is free; they fail differently, and which is cheaper depends on whether your conflicts are real (pessimism wastes less work) or rare (optimism wastes less waiting) — the same trade-off as Volume 4, Chapter 4.

Oracle: caveat emptor. `SERIALIZABLE` is snapshot isolation. Write skew is possible at Oracle's maximum level; if you carry an invariant of that shape, you need `SELECT ... FOR UPDATE` or a materialized conflict, below.

```

flowchart BT
    RU["READ UNCOMMITTED  
permits dirty reads  
vestigial - PG aliases to RC"]
    RC["READ COMMITTED  
per-statement snapshots  
permits: read skew, lost update,  
write skew, phantoms"]
    SI["SNAPSHOT ISOLATION  
PG REPEATABLE READ, Oracle SERIALIZABLE  
per-transaction snapshot,  
first-committer-wins  
permits: write skew"]
    IRR["InnoDB REPEATABLE READ  
snapshot reads, current-version writes,  
next-key locks on locking reads  
permits: lost update, write skew"]
    SER["SERIALIZABLE  
PG SSI - optimistic, retry on 40001  
InnoDB 2PL - blocking, retry on deadlock  
permits: nothing"]

    RU -->|"stronger"| RC
    RC -->|"stronger"| SI
    RC -->|"stronger"| IRR
    SI -->|"stronger"| SER
    IRR -->|"stronger"| SER
    SI -.->|"incomparable siblings  
different anomaly profiles"| IRR

```

MVCC mechanics

Every level above READ UNCOMMITTED in a modern engine is built on **multi-version concurrency control**: instead of updating data in place and making readers wait, the engine keeps multiple versions of each row and shows each transaction the versions consistent with its snapshot. Readers never block writers; writers never block readers; writers still block writers on the same row. The design questions are where old versions live, how a reader decides which version to see, and who cleans up — and PostgreSQL and InnoDB answer all three differently.

Two architectures: copy-on-write heap versus in-place plus undo

PostgreSQL: versions live in the table. Every heap tuple carries system columns, chiefly `xmin` — the transaction ID (XID) that created

this version — and `xmax` — the XID that deleted or superseded it (0 if live). An `UPDATE` is physically a delete-plus-insert: the old tuple gets its `xmax` stamped, and a complete new tuple is written elsewhere in the heap with `xmin` set to the updater. Old versions sit in the table proper, interleaved with live data, until `VACUUM` removes them. Consequences: updates write a whole new row (mitigated by `HOT` — heap-only tuples — which avoids new index entries when no indexed column changed and the new version fits on the same page); rollback is trivially cheap (the new version simply remains invisible because its creator aborted — nothing to undo); and dead versions inflate the table and its indexes until vacuumed — **bloat**.

InnoDB: versions live in the undo log. The clustered index holds only the newest version, updated in place. Each record carries `DB_TRX_ID` (last writer) and `DB_ROLL_PTR`, a pointer into the **undo log**, where the pre-image needed to reconstruct the previous version was written before the update. Old versions form a chain through undo: a reader needing an older version follows roll pointers backward, applying undo records until it reaches a version visible to its snapshot. Consequences: reads of hot current data are clean — the table contains exactly the live rows; reads of *old* data pay per-version reconstruction cost that grows with the chain length; rollback is expensive (undo records must actually be applied to restore the pre-image); and the garbage problem moves from the table to the undo/history subsystem. Oracle pioneered this architecture (undo segments); SQL Server's snapshot modes use a version store in tempdb, a cousin.

```

flowchart TB
    subgraph PG["PostgreSQL - copy-on-write heap"]
        P1["tuple v1 in heap  
xmin 100 xmax 205  
dead - awaiting VACUUM"]
        P2["tuple v2 in heap  
xmin 205 xmax 310  
dead - awaiting VACUUM"]
        P3["tuple v3 in heap  
xmin 310 xmax 0  
live version"]
        P1 -->|"ctid chain forward"| P2 -->|"ctid chain forward"| P3
        PIDX["index entries may point  
at each version  
HOT avoids some"] --> P3
    end
    subgraph INNO["InnoDB - in-place update plus undo log"]
        C["clustered index record  
newest version only  
DB_TRX_ID 310"]
        U1["undo record  
pre-image for v2"]
        U2["undo record  
pre-image for v1"]
        C -->|"DB_ROLL_PTR"| U1 -->|"roll pointer"| U2
        N["old readers walk the chain  
reconstructing versions  
purge trims from the tail"] -.-> U1
    end
end

```

Neither architecture dominates. PostgreSQL pays on the write path (full-row versions, index amplification, VACUUM) and gets cheap aborts and simple reads; InnoDB pays on rollback and on old-snapshot reads and keeps its tables compact. The operational failure modes below are the same disease expressed in different organs.

Snapshots and visibility

A snapshot answers one question: *which transactions' effects should I see?* PostgreSQL represents it explicitly as three pieces: `xmin` — the oldest XID still running when the snapshot was taken (everything before it is definitely resolved); `xmax` — the next XID to be assigned (everything at or after it is definitely invisible, it started after us); and `xip[]`, the list of XIDs in progress at snapshot time (in the gap between the two boundaries, these are the concurrent transactions whose effects must be

hidden even if they commit later). You can inspect this live: `SELECT pg_current_snapshot();` returns something like `748:757:751,754`.

A tuple is visible to a snapshot roughly when: its creator (`xmin`) committed, was not in progress at snapshot time, and is not from the future — and its deleter (`xmax`), if any, does *not* pass those same tests. Commit status comes from the commit log (`pg_xact`), consulted once and then cached on the tuple itself as **hint bits** so subsequent readers skip the lookup — one reason the first scan after a bulk load is slow and writes pages. In full detail the rules run to hundreds of lines (`heapam_visibility.c` repays reading), handling subtransactions, your own uncommitted work, and in-progress edge cases, but the skeleton is this:

```
flowchart TB
    S["tuple: xmin, xmax  
snapshot: xmin, xmax, xip list"] --> A["tuple xmin  
aborted?"]
    A -->|"yes"| DEAD["invisible - creator rolled back"]
    A -->|"no"| B["tuple xmin is my own  
uncommitted txn?"]
    B -->|"yes"| MINE["visible - I see my own writes"]
    B -->|"no"| C["tuple xmin at or after  
snapshot xmax, or in xip list?"]
    C -->|"yes"| INV["invisible - creator concurrent  
or started after snapshot"]
    C -->|"no"| D["tuple xmax set and that txn  
committed and visible  
to this snapshot?"]
    D -->|"yes"| DEL["invisible - deleted before  
my snapshot"]
    D -->|"no"| VIS["visible"]
```

InnoDB's **ReadView** is the same idea with the same fields under different names: a low-water mark (`up_limit_id`), a high-water mark (`low_limit_id`), and the list of active transaction IDs at creation. The mechanical difference is what happens on a miss: PostgreSQL simply skips an invisible heap tuple (the visible one is elsewhere in the heap), while InnoDB takes the current record and walks its undo chain, applying pre-images until a version's `DB_TRX_ID` passes the ReadView test. Same visibility logic; the storage architecture decides who does the work.

The levels now reduce to snapshot lifetime: READ COMMITTED takes a fresh snapshot per statement; REPEATABLE READ/SI takes one per transaction (PostgreSQL at the first query, InnoDB at the first consistent

read, or at `START TRANSACTION WITH CONSISTENT SNAPSHOT`); `SERIALIZABLE` in PostgreSQL is per-transaction snapshots plus SSI's conflict tracking. Commit itself is what flips visibility atomically: writing the commit record makes every one of the transaction's tuples visible to later snapshots at once — in PostgreSQL the XID's status changes in `pg_xact`; engines with explicit commit timestamps (and every distributed SQL engine, Chapter 12) stamp the version with commit time and compare timestamps instead of consulting XID sets.

Garbage collection: the unavoidable tax

MVCC's contract is "keep old versions as long as someone might need them." The corollary defines the operational pain of both engines: **the oldest snapshot in the system determines what can be reclaimed, so one long-running transaction blocks cleanup for everyone.**

PostgreSQL: VACUUM. A dead tuple — superseded or deleted, and invisible to every current and future snapshot — is reclaimable. `VACUUM` (normally `autovacuum`) scans for dead tuples, removes them, updates indexes, and makes space reusable (it does not usually shrink files; that is `VACUUM FULL`, which rewrites the table under an exclusive lock). The horizon rule: `VACUUM` may only remove tuples deleted before the oldest `xmin` any active snapshot might use — the minimum across all running transactions' snapshots (visible in `pg_stat_activity.backend_xmin`), held prepared transactions, replication slots, and, with `hot_standby_feedback`, the standbys' readers too. A single idle-in-transaction session holding a snapshot from six hours ago pins six hours of dead versions *in every table in the database* — cleanup is blocked fleet-wide, tables and indexes bloat, scans slow down as they wade through dead tuples, and the damage persists after the culprit exits until vacuum catches up. Defenses: `idle_in_transaction_session_timeout`, monitoring for old `backend_xmin` / `xact_start`, and treating long transactions as incidents. Separately, XIDs are 32-bit and comparison is circular, so tuples must eventually be **frozen** (marked as "committed in the infinite past") before the counter wraps; `autovacuum` does this on schedule, and a cluster that cannot vacuum — often *because* of the same horizon problems — marches toward wraparound protection, where PostgreSQL first warns and ultimately refuses new writes. Every serious PostgreSQL operator eventually learns this the hard way; monitor `datfrozenxid` age.

InnoDB: purge and the history list. Same disease, different organ. Undo records for committed transactions cannot be purged while any ReadView might still need them; the pending backlog is the **history list**, whose length is visible in `SHOW ENGINE INNODB STATUS` and `information_schema.innodb_metrics` (`trx_rseg_history_len`). A long-running consistent-read transaction pins the tail of the undo log: history list length climbs into the millions, undo tablespaces grow (before MySQL 8.0's separate truncatable undo tablespaces, this permanently grew the system tablespace), and — the InnoDB-specific symptom — reads get *slower*, because every read of a hot row by an old snapshot walks an ever-longer version chain. Where PostgreSQL bloat slows scans by volume, InnoDB history slows point reads by chain length. The operational posture is identical: alert on history list length, kill long transactions, keep OLTP transactions short and move analytics to a replica (where, note, long queries create the analogous problem via replication conflict or feedback — Chapter 8).

The design lesson: **MVCC converts blocking into garbage.** You pay either way; MVCC's genius is deferring the payment off the critical path, and its trap is that deferred payments compound.

Locking reads: buying strength surgically

Inside an MVCC engine, plain reads are optimistic and lock-free. Sometimes you want one read to be pessimistic — to say "I am reading this row *in order to write based on it*; hold it still." That is `SELECT ... FOR UPDATE`: it reads the current committed version (not your snapshot's — at READ COMMITTED in PostgreSQL it locks the newest version, waiting out concurrent writers) and takes a row lock that conflicts with other locking reads and with writes, held to commit.

The lost-update fix is the canonical use:

```
BEGIN;
SELECT balance FROM accounts WHERE id = 42 FOR UPDATE;  -- row locked;
concurrent RMs queue here
-- application computes new balance
UPDATE accounts SET balance = 900 WHERE id = 42;
COMMIT;
```

And the write-skew fix, by forcing the disjoint-write-set structure into a real conflict — lock the rows your *decision* depends on, not just the row you write:

```
BEGIN;
SELECT * FROM doctors WHERE on_call FOR UPDATE; -- both txns try to
lock BOTH rows
-- count them in the application; if > 1, proceed
UPDATE doctors SET on_call = false WHERE name = 'alice';
COMMIT;
```

Now Alice's and Bob's transactions collide on the locked rows; one waits, re-reads a count of 1, and refuses. The limits: this only works when the rows embodying the invariant *exist to be locked* — a select-then-insert uniqueness check has nothing to `FOR UPDATE` (the row that matters is the one that does not exist yet), which is where you need a unique constraint, InnoDB's gap locks, or **materializing the conflict**: keeping a row whose only job is to be locked (a per-shift roster row, a per-bucket counter row) so the phantom becomes a row conflict.

The gradations, in PostgreSQL: `FOR UPDATE` (exclusive, "I will modify or delete"); `FOR NO KEY UPDATE` (slightly weaker — "I will modify non-key columns" — and crucially does not conflict with `FOR KEY SHARE`, the lock foreign-key checks take on referenced rows, so concurrent inserts of child rows are not blocked; prefer it when you are not deleting or changing keys); `FOR SHARE` (shared — "hold this still but other readers may too"); and `FOR KEY SHARE`. Modifiers `NOWAIT` (error instead of queueing) and `SKIP LOCKED` (skip contended rows — the standard idiom for job-queue workers pulling tasks) complete the toolkit.

Locking reads versus serializable is a scoping decision. Explicit locks are targeted: you pay contention only on the invariant you protect, you get blocking rather than aborts, and you carry the burden of *finding every read-decide-write site and locking correctly* — miss one and it is silently broken, which is exactly the "buy back ordering with fences by hand" discipline of Volume 4, with the same failure mode. Serializable is global: every invariant expressible in transactions is protected without you identifying it, at the price of tracking overhead, false-positive aborts, and mandatory retry loops. A defensible default: serializable for workloads whose correctness is hard to enumerate; explicit locking for a small number of known hot invariants inside an otherwise READ COMMITTED application.

Reproducing and fixing anomalies: real SQL

Write skew, live, in PostgreSQL at REPEATABLE READ (snapshot isolation). Two `psql` sessions:

```
-- setup
CREATE TABLE doctors (name text PRIMARY KEY, on_call boolean NOT NULL);
INSERT INTO doctors VALUES ('alice', true), ('bob', true);

-- session 1                                -- session 2
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT count(*) FROM doctors WHERE on_call;
--> 2

EATABLE READ;                                BEGIN ISOLATION LEVEL REP
ors WHERE on_call;                            SELECT count(*) FROM doct
--> 2                                          --> 2

UPDATE doctors SET on_call = false
  WHERE name = 'alice';

l = false                                    UPDATE doctors SET on_cal
                                          WHERE name = 'bob';

COMMIT;  -- succeeds                        COMMIT;  -- succeeds.

Write sets disjoint.
SELECT count(*) FROM doctors WHERE on_call;
--> 0    -- invariant destroyed at "repeatable read"
```

Re-run the same script with `BEGIN ISOLATION LEVEL SERIALIZABLE;` and the second `COMMIT` fails:

```
ERROR:  could not serialize access due to read/write dependencies
among transactions
DETAIL:  Reason code: Canceled on identification as a pivot, during com
mit attempt.
HINT:  The transaction might succeed if retried.
SQLSTATE: 40001
```

SSI tracked both transactions' predicate reads via SIREAD locks, saw each read invalidated by the other's write — the dangerous pair of rw-antidependencies — and aborted the pivot. The `HINT` is literal: retry it and it succeeds, because the conflict partner is gone. Which is why serializable code must be structured as a retry loop; `SQLSTATE 40001` (`serialization_failure`) and `40P01` (`deadlock_detected`) are *expected outcomes*, not errors to page on:

```

import random, time
import psycopg

RETRYABLE = {"40001", "40P01"}

def go_off_call(conn_str: str, doctor: str, max_attempts: int = 5) -> bool:
    for attempt in range(max_attempts):
        try:
            with psycopg.connect(conn_str) as conn:
                conn.execute("SET default_transaction_isolation =
'serializable'")
                with conn.transaction():
                    n = conn.execute(
                        "SELECT count(*) FROM doctors WHERE on_call"
                    ).fetchone()[0]
                    if n <= 1:
                        return False # invariant
                would break; refuse
                conn.execute(
                    "UPDATE doctors SET on_call = false WHERE name
= %s",
                    (doctor,),
                )
                return True # committed
        except psycopg.errors.Error as e:
            if e.sqlstate in RETRYABLE and attempt < max_attempts - 1:
                time.sleep(random.uniform(0, 0.05 * 2**attempt)) #
jittered backoff
            continue
        raise

```

The loop's obligations, per Chapter 5: the transaction body must be **safe to re-execute** (no side effects outside the database before commit — no emails sent, no HTTP calls made mid- transaction), retries must be bounded with backoff and jitter (contention plus tight retries is a retry storm), and the whole unit — not individual statements — is what retries. The equivalent loop for InnoDB at SERIALIZABLE catches MySQL error 1213 (deadlock) and 1205 (lock wait timeout).

Choosing a level in practice

Step one: audit your defaults. Most teams have never explicitly chosen an isolation level, which means the vendor chose: PostgreSQL and Oracle run READ COMMITTED, MySQL runs InnoDB REPEATABLE READ, SQL Server runs locking READ COMMITTED (or RCSI where enabled). Write

down what your engine's default *permits* — for PostgreSQL READ COMMITTED: read skew, lost updates on read-modify-write, write skew — and grep the codebase for read-decide-write sequences. Every one is a potential anomaly site. This audit regularly finds real bugs; the anomalies are not theoretical, they are merely rare, which is worse.

Step two: classify your invariants. Constraints the database enforces (unique, FK, check) are safe at any level. Single-statement atomic writes (`UPDATE ... SET n = n + 1`, `INSERT ... ON CONFLICT`) are safe at READ COMMITTED. Invariants that span a read and a write — balances never negative, at least one doctor on call, at most N seats sold, no overlapping bookings — are write-skew-shaped and are *not* protected below serializable; they need serializable, an explicit locking read over the rows the decision depends on, or a materialized conflict/constraint that turns the invariant into something the database checks.

Step three: match the concurrency-control style to the conflict rate — the Volume 4, Chapter 4 rule. Rare conflicts favor optimism (serializable-SSI, or compare-and-set at lower levels): almost every transaction commits without waiting. Frequent conflicts on hot rows favor pessimism (`FOR UPDATE`, or InnoDB's locking serializable): waiting wastes less than repeated aborted work. High-contention hot spots defeat both, and the fix is schema-level: shard the counter, batch the updates, or queue the writes (Volume 10).

Step four: if you adopt serializable, adopt the whole discipline. Retry loops everywhere (ideally in one shared transaction wrapper, not scattered), no external side effects inside transactions, transactions kept short (long ones both aggravate SSI's false-positive rate and pin the MVCC horizon), and monitoring for serialization-failure rates the way you monitor error budgets. Serializable without retries is an availability bug; serializable with retries and long transactions is a throughput bug.

The distributed-systems lens

The isolation vocabulary is the distributed-consistency vocabulary one layer up — not by analogy but by construction, and this volume keeps meeting it.

A snapshot is a consistent cut. The MVCC snapshot — "the committed state as of timestamp *t*" — is exactly the consistent-snapshot concept of distributed systems: a cut of the global history closed under happens-

before. Within one node, PostgreSQL builds it from an XID array; across nodes there is no shared XID counter, so distributed SQL engines must manufacture globally comparable commit timestamps — Spanner's TrueTime intervals, or hybrid logical clocks in CockroachDB and YugabyteDB — to give a cross-shard transaction one snapshot that is consistent on every node (Chapter 12; Volume 6, Chapter 2). Same abstraction, harder clock.

Replication adds an isolation dimension your level does not govern. A perfectly serializable primary with an asynchronous read replica gives readers of the replica stale data — effects of committed transactions that have not yet applied. That is read skew *across replicas*, and no `SET TRANSACTION ISOLATION LEVEL` fixes it, because the isolation level is a contract about transactions on one consistent store, and the replica is a second store lagging the first. The store-buffer-as-replication-lag identity from Volume 4, Chapter 3 lands here with money attached: "I committed the write and my next request read the replica and it wasn't there" is the read-your-writes problem, solved by session guarantees, primary-pinned reads after writes, or causal tokens — Chapter 8 for the replication mechanics, Volume 6, Chapter 3 for the consistency taxonomy. When you evaluate a system's guarantees, ask two separate questions: what isolation between concurrent transactions, and what consistency between replicas. Jepsen analyses score both axes, and real systems fail both independently.

Write skew across services has no database fix. The doctors anomaly assumed one database noticing both transactions. Now let two *services*, each with its own database, each check an invariant over data the other writes — the inventory service checks capacity while the booking service checks inventory, each then writing its own store. This is write skew where no arbiter sees both write sets, so no isolation level anywhere can detect the rw-antidependency. The distributed answers are structural: put the invariant's data under one transactional authority; serialize the decisions through a single writer or a log (Volume 10); or accept the anomaly and compensate — sagas with compensating actions, reservation patterns, and the outbox pattern for atomically publishing what you committed (Chapter 10; Volume 10). Choosing among these is most of the craft of transactional microservice design, and it starts with recognizing the anomaly shape you learned in this chapter.

And the memory-model bargain closes the loop. DRF-SC said: follow the synchronization discipline and you may reason as if sequentially consistent. Serializable transactions say: follow the transaction discipline and you may reason as if serial. Distributed strict serializability (Spanner's guarantee) says it again across datacenters, paying with commit-wait what SSI pays with aborts and 2PL pays with blocking. It is the same bargain at every scale: **strength is bought, never free, and the only real choices are which currency and where to spend it.**

Key takeaways

- **Isolation levels are weak memory models for transactions.** Serializability is the sequential consistency you assume; your engine's default is weaker, its anomalies are litmus tests, and the discipline is to reason from the documented contract, not from behavior observed in testing.
- **The ANSI definitions are broken** (Berenson et al. 1995): ambiguous phenomena, missing anomalies (dirty write, lost update, read skew, write skew), a serializable definition that admits non-serializable executions, and no place for snapshot isolation — which is incomparable with REPEATABLE READ, not below it. Oracle's `SERIALIZABLE` is SI to this day.
- **Write skew is the anomaly to internalize:** read a predicate, decide, write disjoint rows; first-committer-wins validation never fires because write sets do not overlap. Any "check an aggregate condition, then write" invariant is exposed below serializable.
- **The same level name means different things per engine.** PostgreSQL REPEATABLE READ is snapshot isolation with first-updater-wins (lost updates abort); InnoDB REPEATABLE READ has snapshot reads but current-version writes (lost updates succeed) plus next-key locking on locking reads. Never port assumptions across engines by level name.
- **Serializable comes in two cost models:** PostgreSQL SSI is optimistic — rw-antidependency tracking, false-positive aborts, mandatory retry loops on SQLSTATE 40001; InnoDB is pessimistic 2PL — blocking, deadlock victims, retry on error 1213. Both require retry logic; they differ in when and how you pay.

- **MVCC has two architectures:** PostgreSQL's copy-on-write heap (versions in the table, cheap aborts, VACUUM and bloat) versus InnoDB's in-place-plus-undo (compact tables, expensive rollback, version-chain walks and purge). Visibility is the same idea in both: a snapshot is a set of transaction IDs whose effects you may see.
- **Garbage collection is the tax, and long transactions are the disease.** One old snapshot pins dead versions everywhere: PostgreSQL bloat and, unchecked, wraparound pressure; InnoDB history- list growth and slowing version-chain reads. Monitor for it; treat long-running transactions as incidents.
- **Buy strength surgically with locking reads** — `FOR UPDATE` on the rows a decision depends on fixes lost update and write skew locally; materialize the conflict when the critical row does not exist to be locked; prefer `FOR NO KEY UPDATE` when not touching keys; `SKIP LOCKED` for queues. Or buy it globally with serializable and the retry discipline.
- **Replicas add an isolation dimension no level governs:** a serializable primary plus an async replica still serves stale reads. Isolation between transactions and consistency between replicas are separate axes — ask about both.

Further reading

- Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., O'Neil, P., "A Critique of ANSI SQL Isolation Levels," *SIGMOD*, 1995 — the paper that fixed the vocabulary; still the best single read on this chapter's first half. <https://dl.acm.org/doi/10.1145/223784.223785>
- Adya, A., *Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions*, MIT PhD thesis, 1999 — the dependency-graph formalization of isolation levels that replaced the ANSI phenomena.
- Cahill, M., Röhm, U., Fekete, A., "Serializable Isolation for Snapshot Databases," *SIGMOD*, 2008 — the SSI algorithm. <https://dl.acm.org/doi/10.1145/1376616.1376690>
- Ports, D. R. K., and Grittnier, K., "Serializable Snapshot Isolation in PostgreSQL," *VLDB*, 2012 — the production implementation, including SIREAD locks, granularity promotion, and safe snapshots. <https://arxiv.org/abs/1208.4179>

- Kleppmann, M., *Designing Data-Intensive Applications*, O'Reilly, 2017, Chapter 7 — the best book-length treatment of anomalies and levels for practitioners, including the doctors-on-call example this chapter uses.
- Jepsen, *Consistency Models* — <https://jepsen.io/consistency> — the clickable map of isolation and distributed-consistency levels and their relationships, with Adya-style definitions.
- PostgreSQL documentation: *Transaction Isolation* — <https://www.postgresql.org/docs/current/transaction-iso.html> — precise per-level semantics, including the READ UNCOMMITTED aliasing and SSI behavior; and *Routine Vacuuming* — <https://www.postgresql.org/docs/current/routine-vacuuming.html> — the horizon, bloat, and wraparound mechanics.
- MySQL 8.0 Reference Manual: *InnoDB Transaction Isolation Levels* — <https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-isolation-levels.html> — and *InnoDB Multi-Versioning* — <https://dev.mysql.com/doc/refman/8.0/en/innodb-multi-versioning.html> — undo logs, ReadViews, and purge.
- Fekete, A., Liarokapis, D., O'Neil, E., O'Neil, P., Shasha, D., "Making Snapshot Isolation Serializable," *ACM TODS* 30(2), 2005 — the static-analysis approach to finding write-skew- prone transaction pairs that SSI later automated at runtime.
- Volume 4, Chapter 3 — Memory Models and Happens-Before — the sibling chapter; and Volume 4, Chapter 4 — optimistic concurrency, the abort-and-retry pattern SSI industrializes.
- This volume: Chapter 5 — Transactions and ACID; Chapter 7 — WAL and crash recovery (where commit becomes durable); Chapter 8 — Replication (the second isolation dimension); Chapter 10 — Distributed Transactions; Chapter 12 — NewSQL (global snapshots, TrueTime, HLC).

Chapter 7 — Write-Ahead Logging and Crash Recovery

What this chapter covers. Chapter 5 promised that a committed transaction survives a crash — the D in ACID — and deferred the

mechanism. This chapter is the mechanism. The problem is created by an optimization no serious engine can give up: data pages are modified in the buffer pool and written back to disk lazily, so at any instant the on-disk state is a torn, partially-updated mixture of old and new. The solution is the write-ahead log, and its central rule is small enough to fit on an index card: *before* any change reaches a data page on disk, a log record describing that change must already be durable, and *commit* means the log is flushed — not that any data page has been written at all. Everything else in this chapter is the working-out of that rule. We make the rule precise, examine what actually goes into log records (physical, logical, and the physiological compromise that won), and descend into the fsync discipline — what `fsync` really guarantees, the history of hardware that lied about it, and group commit, the batching trick that makes one disk flush pay for many transactions. We then work through ARIES, the canonical recovery algorithm, whose three passes — analysis, redo, undo — rest on one deep insight: on restart, first *repeat history exactly*, including the work of transactions that will be rolled back. We cover torn pages and the two production defenses (Postgres full-page writes, InnoDB's doublewrite buffer), checkpointing as the operational dial between runtime overhead and recovery time, and then widen the lens: the WAL is not just a recovery mechanism but an *interface* — replication ships it (Chapter 8), change-data-capture decodes it, point-in-time recovery replays it. The distributed-systems lens completes the thought: the log is the abstraction that distributed systems generalize, from Raft to Kafka to event sourcing.

Learning goals — after this chapter you should be able to:

- State the WAL rule precisely — both halves of it — and explain why commit is a log flush, not a page write.
- Distinguish physical, logical, and physiological logging, and explain why real systems log page-oriented redo and logical undo.
- Explain what `fsync` guarantees, what it historically did not, and how group commit converts a per-commit flush into an amortized one.
- Walk through ARIES's three passes over a concrete log, explain why redo repeats uncommitted work, and explain how CLRs make undo safe against crashes during recovery.
- Explain the torn-page problem and the mechanics of full-page writes and the doublewrite buffer.

- Tune checkpointing consciously: what `checkpoint_timeout` and `max_wal_size` trade against recovery time and WAL volume.
- Describe how physical replication, logical decoding, and PITR are all consumers of the same log, and connect the WAL to the log abstraction underlying distributed systems.

The problem: a buffer pool full of unwritten truth

Chapter 2 established the buffer pool: the engine keeps hot data pages in memory, applies every modification there first, and writes dirty pages back to disk lazily — on eviction, or in the background, on the checkpointer's schedule. This is not an optional optimization. A single `UPDATE` may touch a heap page and three index pages; forcing all four to disk, at random offsets, before acknowledging the statement would tie every transaction to several random writes and their latencies (Volume 1, Chapter 6). Write-back caching turns that into sequential background I/O and lets one page absorb hundreds of modifications between flushes.

The price is that the disk is perpetually stale and — worse — *inconsistently* stale. At the moment of a crash, some dirty pages have been written back and some have not, in no meaningful order. The on-disk image is a state that never existed logically: page A reflects a transaction whose change to page B never made it out; half of a B-tree split is on disk and the other half evaporated with the buffer pool. Two distinct disasters are lurking here, and it pays to name them separately, because recovery must fix both:

1. **Committed work may be missing.** A transaction committed — the client was told so — but its dirty pages were still in memory when the power failed. Durability is violated unless something other than the data pages recorded the change.
2. **Uncommitted work may be present.** A transaction was mid-flight, and some of its dirty pages were flushed (perhaps evicted under memory pressure) before it aborted or before the crash. Atomicity is violated unless something can undo changes that are already on disk.

You could imagine solving both by brute force: never write a dirty page before its transaction commits (avoiding disaster 2 — a *no-steal* policy), and force all of a transaction's pages to disk at commit (avoiding disaster 1 — a *force* policy). A force/no-steal engine needs no recovery log at all. It is also unusable: force reintroduces the random-write storm at every

commit, and no-steal means a large transaction's dirty pages are pinned in memory for its whole lifetime, capping transaction size at RAM. Every serious engine therefore runs **steal/no-force** — dirty pages may be written early *and* late — which is exactly the combination that makes both disasters possible. The write-ahead log is what makes steal/no-force safe.

The WAL principle, precisely

The idea: keep a separate, append-only file — the log — and record every change there *before* the change reaches the data files. Appending to one sequential file is the cheapest write pattern storage offers (Volume 1, Chapter 6), so we pay one cheap sequential write up front to earn the right to do the expensive random writes lazily, or never before a crash. The discipline has two halves, and both are load-bearing:

- **The WAL rule (undo half).** Before a dirty page is written to disk, all log records describing changes to that page must already be durable in the log. This guarantees that anything on disk can be undone: if an uncommitted change made it to a data page, the log record that describes it — and identifies the transaction responsible — is on disk too.
- **The commit rule (redo half).** A transaction is committed when its commit log record, and all records before it, are durably in the log. Not when its data pages are written — they may not be written for minutes. This guarantees that anything committed can be redone: the pages can be reconstructed from the log even if none of them were flushed.

Read the second half again, because it inverts the naive picture of what commit means. `COMMIT` returns when a few hundred bytes have been appended to a sequential file and that file has been flushed. The heap pages, the index pages, the TOAST pages — all of it may exist only in memory. The database's authoritative state at any instant is *the log*; the data files are a lazily materialized cache of it. This framing sounds like rhetoric but is operationally literal: replicas are built by replaying the log (Chapter 8), backups are restored by replaying the log, and after a crash the primary itself rebuilds current state by replaying the log. The log is the truth; pages are an optimization for reading it.

The enforcement mechanism is bookkeeping on both objects. Every log record has a **log sequence number (LSN)** — in practice a byte offset into the log, hence totally ordered and monotonic. Every page carries a **pageLSN**: the LSN of the last record that modified it. The buffer manager enforces the WAL rule with one comparison: a page may be written to disk only if `pageLSN ≤ flushedLSN`, the durable frontier of the log. If not, flush the log first. This single comparison is the entire coupling between the buffer pool and the log, and the same pageLSN will reappear below as the key to making redo idempotent.

What goes in a log record: physical, logical, physiological

What should a log record actually *say*? The design space has two pure poles and a hybrid, and the hybrid won so decisively that its name — coined in Gray and Reuter's *Transaction Processing* — is worth knowing.

Physical logging records bytes: "page 4711, offset 200, length 16, old value X, new value Y" — or, at the extreme, before- and after-images of the whole page. Redo and undo are trivial and blindly idempotent: overwrite the bytes. The costs are volume — an insert that shifts a slot array physically changes half the page and must log everything it moved — and rigidity: the log records where bytes landed, so replay must reproduce byte-identical layouts.

Logical logging records operations: "insert tuple T into table R." Compact and layout-independent — one record covers the heap change and every index update. But redo becomes treacherous. A logical record can only be replayed against a state where the operation makes sense, and after a crash the disk holds a mixture: the heap page received the tuple before the crash, the index page did not. Replaying "insert T" wholesale double-inserts in one place while fixing the other. Logical redo forces recovery to determine, per structure, whether each operation happened — exactly the ambiguity a recovery algorithm exists to avoid.

Physiological logging — *physical to a page, logical within it* — is the compromise every major engine landed on. A record names one page physically ("heap page 4711") but describes the change logically within that page ("insert tuple at slot 3"), letting the page reorganize its internals freely. Crucially, redo becomes decidable with the pageLSN: because a record targets exactly one page, and the page atomically

carries the LSN of its last applied change, recovery compares `pageLSN < recordLSN` and knows — per page, with certainty — whether to apply or skip. A multi-structure operation becomes several physiological records, one per page touched.

Undo is the mirror image, and real systems make it *logical* at the operation level. The reason is concurrency: after transaction A inserts a key into a B-tree page and aborts, that page may have been split by transaction B. A physical undo — restore the old page image — would destroy B's committed work. A logical undo — "delete the key you inserted, wherever it now lives" — navigates the current tree and removes exactly A's effect. This pairing, **page-oriented physiological redo plus logical undo**, is what ARIES specifies and what DB2, SQL Server, and InnoDB implement (InnoDB logs physiological redo in its redo log via mini-transactions and keeps undo as logical before-images in undo segments — which, as Chapter 6 showed, double as its MVCC version store). PostgreSQL sits slightly apart in a way we will use below: its WAL carries physiological redo, but it performs no undo at all — MVCC's old row versions and the commit-status log make uncommitted data invisible rather than removed.

The fsync discipline

The WAL rule says "durable in the log," and durability bottoms out in one syscall. Everything about commit latency follows from what that syscall costs and how often you issue it.

What fsync actually guarantees — and the lying-disk history

`fsync(fd)` requests that the file's modified data and metadata reach *stable storage* — not merely the kernel's page cache — before returning (Volume 2, Chapter 6 covers the writeback machinery). On a modern Linux with a mainstream filesystem, that includes issuing a cache-flush command (and where useful, FUA — force-unit-access — writes) to the device, so that data sitting in the drive's volatile write cache is pushed to media. Two honest caveats belong next to that sentence:

- **It was not always so.** For years, consumer drives acknowledged writes on arrival in their volatile caches, and parts of the stack failed to send cache-flush barriers — ext3 shipped with barriers off by default for a long time. The result was databases that called `fsync`,

were told yes, and lost acknowledged commits on power failure. The modern stack is trustworthy when configured plainly, but the same class of bug reappears wherever a layer buffers and acknowledges: virtualized disks with unsafe cache modes, and RAID controllers whose write-back caches are only safe if the battery that protects them is actually healthy. Enterprise NVMe devices with power-loss-protected caches can honestly acknowledge flushes from cache — that is the legitimate version of the trick.

- **Error semantics bite.** In 2018 the PostgreSQL community discovered ("fsyncgate") that on Linux, a writeback error is reported to *one* `fsync` call and then cleared — pages are marked clean, and a retried `fsync` succeeds while the data was never written. Postgres had been retrying; it now PANICs on `fsync` failure and recovers from WAL, and the episode is the sharpest available lesson that durability code must treat a failed flush as a crash, not a retryable hiccup.

One more precision point that generalizes: durability of a *file* and durability of its *directory entry* are separate. Creating a new WAL segment durably requires an `fsync` on the directory as well. Every database and every write-ahead-logging library has a commit history containing this lesson.

For the data files, engines differ on whether to route through the OS page cache at all. InnoDB defaults to `O_DIRECT` for data files — the buffer pool is already a cache, and double-buffering wastes memory and muddles writeback timing. PostgreSQL uses buffered I/O and consciously cooperates with the page cache. Volume 2, Chapter 6 makes the general case; the point here is that the WAL itself is almost always written sequentially and flushed explicitly (`fsync` / `fdatasync` or `O_DSYNC`), because the log's durability, unlike the data files', is on the commit path.

The log buffer, and when it flushes

Log records are not written to disk one at a time. They are appended to an in-memory **log buffer** (`wal_buffers` in Postgres, `innodb_log_buffer_size` in InnoDB), which reaches disk on four triggers:

1. **Commit.** A committing transaction flushes the buffer through its commit record — this is the commit rule, and it is the only trigger on the latency-critical path.

2. **Buffer full.** A background writer (Postgres's WAL writer) and foreground backends flush as the buffer fills.
3. **Timeout.** The background writer flushes on a short period regardless (`wal_writer_delay`), bounding how long records linger.
4. **The WAL rule itself.** Evicting a dirty page whose pageLSN exceeds the flushed LSN forces a log flush first, as above.

Trigger 1 is negotiable, and Postgres names the negotiation honestly: `synchronous_commit = off` acknowledges commit *before* the flush; the WAL writer flushes within roughly three times `wal_writer_delay`. What you risk is precisely bounded and worth stating without euphemism: a crash can lose the last few hundred milliseconds of *acknowledged* commits, but — because the WAL rule (trigger 4) still holds — the database recovers to a consistent state. You lose recent transactions, never integrity. That trade is honest for high-volume telemetry and wrong for payments, and it is per-transaction settable, which is the right granularity: `SET LOCAL synchronous_commit = off` inside the sessions that can afford it. InnoDB's `innodb_flush_log_at_trx_commit = 0|2` are the same dial with coarser semantics.

Group commit: one flush, many transactions

An `fsync` costs on the order of tens of microseconds on a good local NVMe, hundreds of microseconds to milliseconds on network-replicated cloud block storage. If every commit paid its own flush, commits per second would be capped at flushes per second — a few hundred per second on cloud disks — regardless of CPU count.

Group commit breaks the cap by amortization. When multiple transactions reach commit concurrently, one flush suffices for all of them: the log is sequential, so flushing through the *latest* commit record durabilizes every earlier one for free. Implementations elect a leader — whichever backend initiates the flush — while followers whose commit records are already in the buffer simply wait for the flushed LSN to pass their record, then acknowledge. Under load the batching is self-reinforcing: while one flush is in flight, more committers queue behind it, so the next flush covers a larger group. Throughput scales with concurrency while each transaction still pays roughly one flush latency. Postgres does this implicitly (with `commit_delay` as a rarely-needed knob to hold the leader briefly and grow the group); MySQL's binary log implements a staged leader-follower group commit for the same reason.

Recognize the shape, because this book keeps meeting it: batching converts a fixed per-operation cost into a shared per-batch cost, buying throughput with (at most bounded) latency. Nagle's algorithm on the network (Volume 3), writeback coalescing in the OS (Volume 2), and — the direct descendant — Kafka producer batching (Volume 10, Chapter 3) are the same trade wearing different uniforms. A senior engineer who has internalized group commit already understands `linger.ms`.

One commit, end to end

The pieces assemble into a sequence worth having fully explicit in your head. An `UPDATE` followed by `COMMIT`, on a healthy single-node Postgres with default settings:

```
sequenceDiagram
    participant C as Client
    participant B as Backend process
    participant BP as Buffer pool
    participant LB as WAL buffer
    participant WD as WAL on disk
    participant CK as Checkpointer

    C->>B: UPDATE accounts SET ...
    B->>BP: modify heap page in memory
    Note over BP: page is dirty, pageLSN = LSN of record
    B->>LB: append WAL record for the change
    C->>B: COMMIT
    B->>LB: append commit record
    B->>WD: flush WAL through commit LSN
    Note over WD: fsync – the only durable write
    on the commit path. Group commit
    batches concurrent committers here
    WD-->>B: flushed
    B-->>C: COMMIT acknowledged
    Note over BP,WD: dirty data page still only in memory
    CK->>BP: minutes later, checkpoint
    BP->>CK: write dirty page to data file
    Note over CK: legal because pageLSN is
    already behind flushed WAL
```

Three observations. First, the only synchronous disk write is the sequential log flush; the random data-page write happens minutes later, off the critical path, and may cover hundreds of accumulated changes to that page. Second, the WAL record was appended *before* commit but only *flushed* at commit — the WAL rule constrains ordering relative to *page*

writes, not wall-clock eagerness. Third, if the machine dies between the acknowledgment and the checkpoint, the committed change exists only in the log — which is exactly the situation recovery is built for.

ARIES: recovery as three passes over the log

The canonical recovery algorithm is ARIES — *Algorithms for Recovery and Isolation Exploiting Semantics* — published by C. Mohan and colleagues at IBM in 1992 after years inside DB2. Real engines vary in detail, but ARIES is the reference frame in which every variation is explained, including the engines that deviate from it. It assumes exactly the regime we have built: steal/no-force buffering, physiological redo, logical undo, LSNs on every page.

Two in-memory tables and one humility about checkpoints complete the setup. The **active transaction table (ATT)** tracks in-flight transactions and the LSN of each one's latest record. The **dirty page table (DPT)** tracks pages that are dirty in the buffer pool; each entry carries a **recLSN** — the LSN of the *first* record that dirtied the page since it was last clean, i.e., the earliest log record whose effect might not be on disk for that page. And checkpoints are **fuzzy**: a checkpoint cannot stop the world — quiescing all transactions and flushing every dirty page in a large buffer pool would pause service for seconds to minutes, an availability cost no production engine will pay. So an ARIES checkpoint flushes *nothing* and forbids *nothing*; it merely writes the current ATT and DPT into the log between begin- and end-checkpoint records, and records the checkpoint's location where recovery can find it. A checkpoint is not a promise that the disk is clean; it is a photograph of exactly *how dirty* things were, so recovery can bound how far back it must look.

Recovery after a crash makes three passes:

```

flowchart TD
    subgraph LOG["The log as a timeline of LSNs"]
        direction LR
        CP["checkpoint  
ATT and DPT snapshot"] --> R1["updates by T1, T2  
T1 commits"] --> R2["updates by T2, T3  
T3 commits"] --> CRASH["CRASH  
T2 still active"]
    end
    end
    LOG --> A["PASS 1 – ANALYSIS  
forward from checkpoint  
rebuild ATT and DPT  
losers: T2. redo start: min recLSN"]
    A --> RD["PASS 2 – REDO  
forward from min recLSN  
repeat history: reapply EVERY change  
including T2's, gated by pageLSN"]
    RD --> U["PASS 3 – UNDO  
backward over losers only  
roll back T2, writing a CLR  
for each undone record"]
    U --> DONE["database consistent  
equals committed work exactly"]

```

Analysis scans forward from the last checkpoint, replaying the *bookkeeping*: the checkpoint's ATT and DPT snapshots are loaded, then updated per record — new transactions enter the ATT, commit and abort records retire them, updates to pages absent from the DPT add entries. At the crash point, analysis has reconstructed what the engine knew in memory at the moment of death: which transactions were in flight (the **losers**) and which pages might be dirty. The minimum recLSN in the reconstructed DPT is where redo must start.

Redo scans forward from that minimum recLSN and *repeats history*: it reapplies **every** update — committed transactions, losers, everything — restoring the buffer pool and pages to exactly the state at the instant of the crash. Idempotency comes from the pageLSN gate: for each record, if the target page's pageLSN is already $\geq$ the record's LSN, the effect is on disk — skip; otherwise apply and set the pageLSN. (The DPT provides a cheaper first-level skip: a page not in the DPT, or with recLSN beyond this record, needs no read at all.)

Repeating history — including work that is about to be undone — is *the* ARIES insight, and it is worth dwelling on why. The tempting shortcut is to redo only committed work. But loser transactions interacted with the

same pages as winners under fine-grained locking: a loser's insert may have split a B-tree page that a winner then also modified. Selective redo would have to reconstruct committed effects against page states those effects never actually occurred in — the logical-redo swamp again. Repeating history sidesteps every such case: first return to the exact pre-crash state, where every page is in a condition the logged operations were actually applied to; *then* undo the losers as ordinary transaction rollbacks. Recovery-time undo becomes the same code path as a normal runtime `ROLLBACK`, exercised on every abort rather than only in disasters — a reliability property as valuable as the correctness one.

Undo rolls back the losers, walking each one's chain of log records backward (records carry a `prevLSN` linking each transaction's records) and applying logical undo for each. And here ARIES adds its second signature device: every undo action is itself logged, as a **compensation log record (CLR)**. A CLR says "I undid record X," and carries an **undoNextLSN** pointing at the next record *before* X still awaiting undo. CLRs are redo-only — they are never themselves undone — and that closes the last failure window: a crash *during recovery*. On the next restart, redo repeats history again, which now includes replaying the CLRs — so the partial undo is restored, not lost — and the new undo pass resumes exactly where the last one died, following `undoNextLSN` past everything already compensated. Undo never repeats an undo and never misses one; recovery is idempotent no matter how many times it is interrupted. Progress made is progress kept — the log records it, like everything else.

When undo completes, the database state equals: all committed work, no uncommitted work. Durability and atomicity, delivered.

What real engines actually do

InnoDB is close to the ARIES scheme: physiological redo in the redo log, logical undo from undo segments, fuzzy checkpointing driven by the flushed-LSN horizon. PostgreSQL is the instructive deviation: **redo only, no undo pass**. Analysis and redo proceed as above (from the last checkpoint's redo pointer), but losers are simply *left in place* — their heap tuples remain on disk, marked with their transaction IDs, and since `pg_xact` records those transactions as never committed, MVCC visibility (Chapter 6) makes the tuples invisible to everyone forever. `VACUUM` reclaims the space eventually. Postgres traded the undo machinery for

MVCC bloat and vacuum debt — a trade whose costs Chapter 6 already itemized, but which buys a dramatically simpler recovery path. The lesson generalizes: *how much recovery you need depends on how destructive your writes are*. Postgres never overwrites a live tuple, so it never needs to un-overwrite one. The LSM engines below push the same idea to its endpoint.

Torn pages, and two defenses

There is a hole in the story so far. Redo assumes the pageLSN is trustworthy — that a page on disk is *some* consistent version of itself. But a database page (8 KB in Postgres, 16 KB in InnoDB) is larger than the unit the hardware writes atomically — traditionally a 512-byte or 4 KB sector. A crash mid-write can leave a **torn page**: the first sectors new, the rest old. Such a page is not an old version — it is garbage with a plausible header. Its pageLSN might read as current while half its contents are stale; per-record redo, applied to garbage, produces garbage. Checksums (`data_checksums` in Postgres, mandatory page checksums in InnoDB) *detect* the tear; something else must repair it. The two production answers bracket the design space:

```
flowchart TD
    subgraph PG["Postgres – full-page writes"]
        P1["checkpoint completes"] --> P2["first modification of page P  
after the checkpoint"]
        P2 --> P3["WAL record carries a full image  
of page P, not just the change"]
        P3 --> P4["later changes to P until next  
checkpoint log only deltas"]
        P5["crash tears P during write-back"] --> P6["redo restores P from  
the full image  
then applies deltas – tear irrelevant"]
    end
    subgraph IN["InnoDB – doublewrite buffer"]
        I1["flush a batch of dirty pages"] --> I2["write pages to the  
doublewrite  
area sequentially, then sync"]
        I2 --> I3["write pages to their real  
locations, then sync"]
        I4["crash tears a page in step two"] --> I5["recovery finds intact  
copy in  
doublewrite area, repairs, then redo"]
    end
```

Postgres: full-page writes. With `full_page_writes = on` (the default, and effectively mandatory on ordinary storage), the *first* modification of each page after a checkpoint logs a complete image of the page instead of a delta. Redo then never needs the on-disk page's prior contents for that first touch — it overwrites wholesale from the image, and subsequent delta records apply on top of a known-good base. The tear is simply never read. The cost is WAL volume, and it has a characteristic signature every Postgres operator learns: **WAL output spikes sharply just after each checkpoint**, when the working set's pages take their first post-checkpoint touches, then decays as pages accumulate deltas instead. Shorter checkpoint intervals mean more frequent spikes — the first concrete reason checkpoint frequency is a real cost dial. `wal_compression = on` blunts the volume, spending CPU to compress the images.

InnoDB: the doublewrite buffer. InnoDB writes every flushed page *twice*: first into a small dedicated doublewrite area (its own files since MySQL 8.0.20; a region of the system tablespace before that) as a batch of sequential writes plus a sync, then to the pages' real locations. A tear can now only exist in one of the two copies. On recovery, InnoDB scans the doublewrite area; any page whose home copy fails its checksum is repaired from the doublewrite copy, and only then does redo run. The cost is on the page-flush path — every data-page write is doubled — rather than in the log, but the first copy is sequential and batched, so the practical overhead is modest (and it is off the commit path entirely, since page flushing is background work). On storage that guarantees atomic 16 KB writes, `innodb_doublewrite` can be disabled — the same class of hardware assumption that lets Postgres operators consider `full_page_writes = off` on filesystems with copy-on-write semantics like ZFS, and it should be believed only with the vendor's guarantee in writing.

Checkpoints as an operational dial

A checkpoint's job, in both engines, is to bound recovery: redo starts at (roughly) the checkpoint's position, so the interval between checkpoints is the amount of log recovery may need to replay. That makes checkpoint

frequency a genuine two-sided dial, one of the few in a database where the trade is clean:

- **Checkpoint more often** → less log to replay → faster crash recovery; but more full-page-write spikes (Postgres), more repeated flushing of the same hot pages, more background I/O competing with foreground work.
- **Checkpoint less often** → less runtime overhead and less WAL volume; but recovery replays more, and after a crash the database is down for the duration of that replay. Recovery time is availability: replaying twenty minutes of log is a twenty-minute outage appended to every crash.

Postgres exposes the dial directly, and the honest reading of each knob:

```

# postgresql.conf – WAL and checkpoint settings that matter, honestly
annotated

wal_level = replica
# minimal: enough WAL for crash recovery only.
# replica: adds what physical standbys
and PITR need
# (the default, and the floor for any
serious setup).
# logical: adds tuple metadata for
logical decoding/CDC.
# Each step costs WAL volume; step up
only for the
# consumers below.

synchronous_commit = on
# on: commit waits for local WAL flush – the durability
# contract this chapter is about.
# off: ack before flush; can lose the
last ~600ms of

# acknowledged commits on crash, never consistency.
# remote_write / on / remote_apply also
gate on a
# synchronous standby – Chapter 8's
territory.

checkpoint_timeout = 15min # upper bound between checkpoints. The
default 5min is           # conservative; longer intervals cut
                           # volume at the price of longer crash
full-page-write           # recovery.

max_wal_size = 4GB         # soft cap on WAL between checkpoints;
crossing it forces         # an early checkpoint. Size it so
                           # by the timeout, not by this – a WAL-
checkpoints are driven    # checkpoint arriving early is the
size-triggered            # surprise I/O storms under write
                           # classic cause of
classic cause of          # bursts. Watch for
bursts. Watch for         # "checkpoints are occurring too
frequently" in the log.

checkpoint_completion_target = 0.9 # spread the checkpoint's page
write-back over 90%          # of the interval instead of

```

```

slamming it out at
                                # once – a steady trickle, not a
periodic cliff.

full_page_writes = on
# torn-page defense; see above. Leave on unless your
                                # storage atomically writes 8KB pages
and you can prove it.
wal_compression = on
# compress full-page images; CPU for WAL volume.

```

The spreading knob deserves the extra sentence: early Postgres wrote each checkpoint's dirty pages as fast as possible, producing a periodic latency cliff as the burst saturated the device and fsync stalls rippled into commits. `checkpoint_completion_target` schedules the same writes as a paced trickle across the interval. The pattern — replace periodic bursts with continuous paced background work — recurs in LSM compaction scheduling (Chapter 2) and in every system that ever had a "stop-the-world flush" phase it learned to regret.

What the log's contents actually look like is worth seeing once. `pg_waldump` decodes WAL segments into one line per record — LSN, transaction, resource manager, and the physiological description:

```

$ pg_waldump 0000000100000000000000016 | head -8
rmgr: Heap  len (rec/tot):  54/  54, tx: 771, lsn: 0/16A2A18, prev
0/16A29D0,
  desc: INSERT off: 3, flags: 0x00, blkref #0: rel 1663/16384/16402 blk
9
rmgr: Btree  len (rec/tot):  53/  53, tx: 771, lsn: 0/16A2A50, prev
0/16A2A18,
  desc: INSERT_LEAF off: 42, blkref #0: rel 1663/16384/16408 blk 4
rmgr: Heap  len (rec/tot):  66/ 8250, tx: 772, lsn: 0/16A2A88, prev
0/16A2A50,
  desc: UPDATE off: 7 xmax: 772 flags: 0x10, blkref #0: rel
1663/16384/16402 blk 12 FPW
rmgr: Transaction len (rec/tot): 34/ 34, tx: 771, lsn: 0/16A4AD0, prev
0/16A2A88,
  desc: COMMIT 2026-08-14 09:41:07.114 UTC

```

Every concept in this chapter is on display: one logical `INSERT` became two physiological records (heap page, then B-tree leaf page), each naming exactly one block; LSNs are byte positions with `prev` back-links forming each transaction's chain; the `UPDATE` at `0/16A2A88` happened to be the first touch of block 12 after a checkpoint, so its record carries a

full-page image — note the `FPW` flag and the total length jumping to 8250 bytes for a 66-byte logical change; and the commit record for transaction 771 is just another entry in the stream, whose flush was the moment 771's durability became fact.

The log as an interface

Everything so far treats the WAL as a crash-recovery mechanism. But look at what the engine has been forced to build in order to recover: a totally ordered, durable, replayable stream of every change to the database. That artifact is too useful to leave to crashes, and mature systems expose it in four directions:

```
flowchart TD
    WAL["The write-ahead log  
ordered, durable, replayable  
stream of every change"]
    WAL --> REC["Crash recovery  
replay into the local buffer pool  
this chapter"]
    WAL --> REP["Physical replication  
stream segments to standbys that  
run redo continuously – Ch 8"]
    WAL --> CDC["Logical decoding – CDC  
decode records back into row events  
pgoutput, wal2json, Debezium"]
    WAL --> PITR["Archiving and PITR  
base backup plus archived WAL  
replayed to any moment"]
```

Physical replication is recovery, running forever. A streaming standby is a server that starts in recovery mode and never leaves it: the primary ships WAL as it is generated, and the standby's apply process is the same redo loop this chapter described, running continuously against a live buffer pool. Synchronous replication reuses the commit rule with a wider definition of "durable": `synchronous_commit = remote_write` or `on` makes the commit flush wait for a standby's acknowledgment too. Chapter 8 takes this up properly; note here only that no new machinery was invented — replication is the recovery subsystem pointed at another machine.

Logical decoding turns the WAL into an event stream. The physiological records were written for redo, but with `wal_level = logical` they carry enough metadata to be decoded *backward* into the

row-level changes that produced them — "insert into `orders`, columns (...)", grouped by transaction, in commit order. Postgres exposes this through replication slots and output plugins (`pgoutput` feeds built-in logical replication; `wal2json` emits JSON), and change-data-capture platforms — Debezium is the canonical one — sit on this interface to publish every committed change into Kafka. This is the honest solution to the dual-write problem that Chapter 5 flagged and Volume 10, Chapter 6 develops as the outbox/CDC pattern: instead of writing to the database *and* separately to a message bus (two non-atomic writes that will eventually disagree), write only the transaction, and let the log — which already totally orders and durably records every commit — be the bus's source of truth. One caution from operations: a replication slot pins WAL until its consumer confirms receipt, so a dead consumer causes unbounded WAL retention; monitor slot lag or learn this during a disk-full incident.

PITR: the backup that is actually a log replay. Archive every completed WAL segment (`archive_command`, or streaming via `pg_receivewal`), take periodic *base backups* — filesystem copies of the data directory, taken online with no quiesce needed — and you can reconstruct the database as of *any* moment between the base backup and now: restore the base backup, then replay archived WAL up to a target. The base backup's internal inconsistency (pages copied at different times while writes continued) is exactly the inconsistency crash recovery already fixes — replay handles it, which is why online backups work at all. The restore, in sketch:

```
# 1. Restore the base backup into an empty data directory
tar -C /var/lib/postgresql/17/main -xf /backups/base/
2026-08-13T02:00.tar

# 2. Tell recovery where archived WAL lives, and when to stop
cat >> /var/lib/postgresql/17/main/postgresql.auto.conf <<'EOF'
restore_command = 'cp /wal-archive/%f %p'
recovery_target_time = '2026-08-14 09:37:00+00'    # just before the bad
deploy
recovery_target_action = 'promote'
EOF
touch /var/lib/postgresql/17/main/recovery.signal

# 3. Start postgres; it replays WAL to the target time, then promotes
```

This is the tool for the failure replication cannot fix: a bad `DELETE`, a botched migration, a compromised credential. Replication faithfully replicates the disaster within seconds; PITR winds time back to just before it. And the operational discipline matters more than the mechanism: **an untested backup is a hope, not a backup**. Restore paths fail silently — an `archive_command` that started erroring months ago, a base backup missing a tablespace, a target time in the wrong zone — and the only way to know your recovery time and your recovery *works* is to run the restore on a schedule, automatically, and fail loudly when it breaks. Volume 11 returns to this as a pillar of operational readiness; the engineers who skip it find out during the incident.

LSM engines: the same rule, a shorter story

Chapter 2's LSM-tree engines — RocksDB, Cassandra's storage layer, and *kin* — obey the identical WAL principle with a strictly simpler recovery, and the contrast illuminates *why* ARIES is as elaborate as it is. An LSM write goes to the memtable (in memory) and, first, to a WAL — same rule: log durable before acknowledgment, commit is a log flush, group commit amortizes it.

But recovery is redo-only and trivial: replay the WAL into a fresh memtable, done. No analysis pass reconstructing dirty-page state, no undo, no CLRs, no torn-page defense for data files. Every complication in ARIES traces back to one property of B-tree engines: they **update pages in place**, so the disk can hold half-applied, torn, or uncommitted states of shared structures. LSM engines never update in place — SSTables are written once, immutably, and an incomplete SSTable from a crash mid-flush is simply discarded and rebuilt from the WAL that still covers its contents. Once flushed, an SSTable's WAL can be dropped; compaction (Chapter 2) rewrites data into new immutable files rather than modifying old ones. The design bet is visible in the recovery code: ARIES-class recovery is thousands of lines of subtle bookkeeping; LSM recovery is "replay the tail of the log." Immutability didn't eliminate the WAL — nothing eliminates the WAL — but it eliminated almost everything recovery had to be clever about.

The distributed-systems lens: the log is the abstraction

Jay Kreps's 2013 essay "The Log: What every software engineer should know about real-time data's unifying abstraction" made an argument this chapter has been quietly assembling: the write-ahead log is not a database implementation detail — it is the fundamental primitive, and distributed systems are largely the WAL, generalized. The correspondences are exact enough to be load-bearing:

State-machine replication is log shipping. The oldest theorem in distributed systems: if deterministic replicas apply the same operations in the same order, they hold the same state. "The same operations in the same order" is a log. Primary-replica database replication ships the WAL (Chapter 8). Raft and Paxos-based systems (Volume 6, Chapter 6) are, at their core, protocols for agreeing on the contents of a replicated log — Raft's central data structure is literally named the log, with its own LSNs (term and index) and its own commit rule (an entry is committed when replicated to a majority). Consensus is the WAL rule negotiated among peers who can fail independently.

Commit generalizes from one fsync to a quorum of logs. On a single node, durable means "my log is flushed." In a replicated system, durable means "the log entry exists in enough logs" — a quorum — so that any surviving majority contains it. Postgres's `synchronous_commit = remote_write` sits exactly on the boundary between the two worlds: one commit rule, progressively widened. Volume 6 makes this the definition of durability in distributed storage.

Kafka is a WAL as a service. Strip the buffer pool, the B-trees, and the query engine from this chapter, keep the append-only ordered durable log with offsets for LSNs and consumers running redo at their own pace, and you have described a Kafka partition (Volume 10, Chapter 3). Consumer offset tracking is the flushed-LSN frontier; log compaction is checkpointing's space-reclamation role; producer batching (`linger.ms`, `batch.size`) is group commit re-invented at the client. Kreps was a Kafka author; the essay is the design document.

Event sourcing is the application-level WAL (Volume 10, Chapter 5): persist the ordered log of domain events as the source of truth and materialize current state as views derived by replay. Every property this chapter established reappears — replay is recovery, snapshots are

checkpoints bounding replay time, and the CDC pipeline above is the bridge by which a conventional database's WAL feeds such architectures without dual writes.

Recovery-by-replay is the universal pattern. A crashed Postgres redoes its WAL tail; a rebooted Raft follower replays its log and fetches what it missed from the leader; a restarted stream processor rewinds its Kafka offsets and reprocesses; a new cache node warms from the event stream. In every case the durable log is authoritative and the queryable state is a disposable materialization — which is precisely the "pages are a cache of the log" inversion from the start of this chapter. Internalize it once, at the level of a single database's WAL, and you have the mental model for most of Volumes 6 and 10: the systems differ in how many machines hold the log and how they agree on its contents; they do not differ in what the log *is*.

Key takeaways

- Buffer pools make on-disk state perpetually inconsistent; **steal/no-force** buffering is mandatory for performance and is exactly what makes crashes dangerous. The WAL makes it safe.
- **The WAL rule:** log records durable before the pages they describe (enables undo); **commit = log flushed**, not pages written (enables redo). The log is the authoritative state; data files are a materialized cache of it — literally, not rhetorically.
- Real systems log **physiological redo** (physical to a page, logical within it) because the pageLSN comparison makes replay decidable and idempotent, and **logical undo** because pages are shared with committed work that physical undo would destroy.
- `fsync` is the durability primitive: know its history (volatile caches, missing barriers), its error semantics (`fsyncgate`: a failed `fsync` is a crash, not a retry), and its cost — which **group commit** amortizes across concurrent transactions, the same batching trade as producer batching everywhere else. `synchronous_commit = off` risks the last moments of acknowledged commits, never consistency.
- **ARIES:** analysis rebuilds the transaction and dirty-page tables from a fuzzy checkpoint; redo **repeats history** from the minimum `recLSN` — including losers, so undo always runs against real states; undo rolls back losers writing **CLRs**, which make recovery idempotent under

repeated crashes. Postgres skips undo entirely by letting MVCC visibility hide losers.

- Pages exceed the atomic write unit, so crashes **tear pages**; Postgres logs **full-page images** on first post-checkpoint touch (hence WAL spikes after checkpoints), InnoDB writes pages twice through the **doublewrite buffer**.
- **Checkpoint frequency is a dial**: recovery time (availability after a crash) versus runtime I/O and full-page-write volume. Spread checkpoints; size `max_wal_size` so the timeout, not the size cap, drives them.
- The WAL is an **interface**, not just a mechanism: physical replication is recovery running forever on another machine; logical decoding/CDC turns it into an event stream and solves the dual-write problem; PITR is base backup plus replay-to-timestamp — and **an untested backup is a hope, not a backup**.
- LSM engines keep the WAL rule but get **redo-only, trivial recovery** because immutable SSTables remove in-place updates — the source of everything ARIES has to be clever about.
- Distributed systems generalize the WAL: Raft replicates a log, quorum commit widens the fsync, Kafka serves the log as a product, event sourcing applies it at the application layer. Recovery-by-replay is the universal pattern.

Further reading

- Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H., and Schwarz, P., "ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging," *ACM Transactions on Database Systems* 17(1), 1992 — the canonical recovery paper; long, and worth the effort at least through the repeating-history and CLR sections.
- Gray, J. and Reuter, A., *Transaction Processing: Concepts and Techniques* (Morgan Kaufmann, 1992) — the definitive treatment of logging, recovery, and the physiological-logging vocabulary used in this chapter.
- Kreps, J., "The Log: What every software engineer should know about real-time data's unifying abstraction" (LinkedIn Engineering, 2013) — the essay behind this chapter's distributed-systems lens. <https://>

engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying

- PostgreSQL documentation, "Reliability and the Write-Ahead Log" and "WAL Configuration" — the authoritative source for `wal_level`, `synchronous_commit`, checkpoint behavior, and full-page writes. <https://www.postgresql.org/docs/current/wal.html>
- PostgreSQL documentation, "Continuous Archiving and Point-in-Time Recovery" — the normative PITR procedure sketched above. <https://www.postgresql.org/docs/current/continuous-archiving.html>
- MySQL 8.x Reference Manual, "InnoDB On-Disk Structures: Doublewrite Buffer" and "InnoDB Recovery" — the doublewrite mechanics and InnoDB's redo/undo split.
- Hellerstein, J., Stonebraker, M., and Hamilton, J., "Architecture of a Database System," *Foundations and Trends in Databases* 1(2), 2007 — a compact survey placing logging and recovery within whole-engine architecture.
- The PostgreSQL "fsyncgate" mailing-list thread and follow-up conference talks (2018–2019) on Linux fsync error semantics — the modern cautionary tale about what durability code may assume.
- Volume 2, Chapter 6 — the page cache, writeback, `fsync`, and `O_DIRECT` from the OS side.
- Chapter 2 of this volume — buffer pools and LSM engines; Chapter 8 — replication as WAL shipping; Volume 6, Chapter 6 — consensus and the replicated log; Volume 10 — Kafka and event sourcing, the log as a first-class system.

Chapter 8 — Replication: Physical, Logical, Sync, and Async

What this chapter covers. Chapter 7 established that the write-ahead log is the database's source of truth: the heap and indexes are just a cache of what the log says. This chapter follows the log out of the machine. Replication is the practice of shipping a database's changes to other machines so that more than one copy of the data exists, and every replication design answers two independent questions: **what travels on the wire** (physical bytes of WAL, decoded logical row changes, or —

historically and disastrously — the SQL statements themselves) and **when the primary acknowledges a commit** (before the copy exists anywhere else, or after). The first question determines what your replicas can do; the second determines how much data you lose when the primary dies and how much latency every commit pays while it lives.

The central fact of the chapter is that **an asynchronous replica is a store buffer one layer up**. If you worked through Volume 4, Chapter 3 — Memory Models and Happens-Before, you already know the shape of every anomaly here: a write acknowledged to its author but not yet visible to another observer, and the whole family of "impossible" readings that follow. The mitigations will look familiar too, because they are the same mitigations — ordering tokens, session guarantees, and paying latency to buy back visibility. We cover the mechanics of physical and logical replication in PostgreSQL and MySQL with real configuration, the sync/async/semi-sync spectrum and its exact failure modes, replication lag as a production discipline (causes, honest monitoring, and the anomaly catalogue), failover without the vendor gloss — including split brain and why serious tooling delegates leader election to a consensus store — and a clear-eyed look at multi-primary. Volume 6 generalizes all of it; this chapter is the single-system foundation.

Learning goals — after this chapter you should be able to:

- Separate the two axes of every replication design — what ships and when acks happen — and place any system you meet on both.
- Explain physical WAL shipping versus logical decoding: what each replica can and cannot do, the version and architecture constraints, and the DDL and replica-identity caveats of logical.
- Explain why statement-based replication lost to row-based, with the actual nondeterminism cases.
- Describe PostgreSQL `synchronous_commit` levels and `synchronous_standby_names` quorum syntax, and MySQL semi-synchronous replication including `AFTER_SYNC` versus `AFTER_COMMIT`, accurately.
- Diagnose replication lag: name its causes, monitor it without being lied to, and enumerate the read anomalies it produces.
- Implement read-your-writes routing with LSN waits (PostgreSQL) and GTID waits (MySQL).

- Run a failover honestly: detection false positives, choosing the most-advanced replica, `pg_rewind`, fencing, and why Patroni keeps its leader key in a consensus store.
- Argue for or against multi-primary for a given workload, and say what LWW actually costs.

Why replicate, and what it costs

Four reasons justify nearly every replication deployment:

1. **Availability.** One machine is a single point of failure. A warm copy turns "restore from last night's backup" into "promote and repoint," shrinking recovery from hours to minutes.
2. **Read scaling.** Most OLTP workloads are read-heavy. Replicas multiply read capacity without touching the write path.
3. **Locality.** A replica in Frankfurt serves Frankfurt reads at 2 ms instead of 90 ms trans-Atlantic. (Multi-region architecture is Volume 7, Chapter 10; the replication substrate is this chapter.)
4. **Workload isolation.** Backups, analytics scans, and ad-hoc queries run against a replica so the primary's buffer pool and I/O budget serve production traffic.

The bill arrives in four currencies. **Lag:** a replica is always somewhat behind, and "somewhat" has a heavy tail. **Anomalies:** readers of a lagging replica observe states of the database that the primary's history never contained side by side — you post a comment, refresh, and it is gone. **Failover complexity:** promotion is the most dangerous routine operation in database administration, because doing it wrong loses data or splits the brain. **Operational surface:** replication slots that fill disks, sync standbys that block commits, replicas that silently drift. None of these are reasons not to replicate — running a serious system on one copy is malpractice — but each one must be engineered for, not discovered.

Keep the two axes separate as you read. *What ships* — physical, logical, statement — is about fidelity and flexibility. *When acks happen* — async, sync, semi-sync — is about durability and latency. They compose freely: PostgreSQL physical replication can be async or quorum-sync; MySQL's logical binlog stream can be async or semi-sync. Conflating the axes is the most common source of confused replication discussions.

What ships: physical, logical, statement

Physical replication: shipping the WAL itself

Chapter 7's punchline was that the WAL fully determines the database state: replay the log from a consistent base and you reconstruct every page. Physical replication weaponizes that. The primary streams the **byte-identical WAL** to replicas, and each replica runs continuous crash recovery, applying page-level redo records forever. A physical replica is not "a copy of your tables" — it is a copy of your *cluster*, bit for bit: same pages, same bloat, same indexes, same everything.

In PostgreSQL this is **streaming replication**. A `walsender` process on the primary ships WAL to a `walreceiver` on the standby; the standby's startup process applies it. Setup is a base backup plus a handful of parameters:

```
# On the standby: clone the primary and configure streaming in one step
pg_basebackup -h primary.db.internal -U replicator -D /var/lib/
postgresql/17/main \
    --wal-method=stream --write-recovery-conf --checkpoint=fast
# --write-recovery-conf creates standby.signal and sets
primary_conninfo
```

```
# postgresql.conf on the primary
wal_level = replica           # the default since PostgreSQL 10
max_wal_senders = 10
# Replication slots make the primary retain WAL until this standby
# confirms it.
# Cap the retention or a dead standby fills the primary's disk:
max_slot_wal_keep_size = 100GB
```

```
-- On the primary: a slot per standby, so WAL is never recycled out
# from under it
SELECT pg_create_physical_replication_slot('standby_fra_1');
```

The properties all follow from "byte-identical":

- **Low overhead.** The primary was writing this WAL anyway (Chapter 7); replication adds a network send. No decoding, no per-row work.
- **Complete fidelity.** DDL, sequences, all of it — because none of it is special. It is all just pages.

- **Everything replicates, so nothing is optional.** You cannot replicate one database or one table; the unit is the whole cluster.
- **Same major version, same architecture.** WAL records address pages in an on-disk format that changes across major versions and differs by platform. A 16 primary cannot stream to a 17 standby, which is why major-version upgrades under physical replication require logical methods or downtime.
- **Replicas are strictly read-only.** The standby's state is the replayed WAL; a local write would diverge from it. PostgreSQL's **hot standby** mode lets the standby serve read-only queries during replay — with a conflict problem we return to under lag.

MySQL has no equivalent of streaming physical replication in the server itself; its native replication is logical (below). Physical copies in the MySQL world come from backup tooling (Percona XtraBackup, MySQL Enterprise Backup) used for provisioning, not for continuous replication.

Logical replication: shipping decoded changes

Logical replication ships **row changes as data**, not page images: "in table `orders`, insert this tuple; in `accounts`, update the row whose key is 42 from these values to those." The stream is a sequence of typed change events, which decouples the replica's physical layout, version, and even engine from the primary's.

PostgreSQL implements this as **logical decoding**: the same WAL is read, but a decoding plugin reassembles page-level records into transactions of row changes. The built-in plugin, `pgoutput`, drives the publication/subscription system (PostgreSQL 10+):

```
-- On the publisher
CREATE PUBLICATION billing_pub FOR TABLE invoices, payments;

-- On the subscriber (which may be a different major version)
CREATE SUBSCRIPTION billing_sub
  CONNECTION 'host=primary.db.internal dbname=billing
  user=replicator'
  PUBLICATION billing_pub;
-- Performs an initial table copy, then streams changes via a logical
slot.
```

The flexibility is real: per-table granularity, cross-major-version streaming (the standard low-downtime upgrade path), subscribers that also hold

their own writable tables, fan-in from multiple publishers, heterogeneous targets via the same decoding interface (Debezium reads `pgoutput` into Kafka). The caveats are equally real and bite in production:

- **DDL is not replicated.** `ALTER TABLE` on the publisher is not sent; the subscriber's apply worker errors when row images stop matching the schema. Schema changes must be coordinated — applied on the subscriber first for additive changes, orchestrated carefully otherwise.
- **Replica identity.** To ship an `UPDATE` or `DELETE`, the publisher must know how to identify the row on the subscriber. By default that is the primary key (`REPLICA IDENTITY DEFAULT`); a published table with no key must be set to `REPLICA IDENTITY FULL` (whole old row in every change — expensive) or updates and deletes on it simply fail on the publisher.
- **Sequences do not replicate**, so after a failover-by-logical-replica, sequences must be advanced manually past the maximum used values. Large objects are likewise excluded.

MySQL's native replication has always been logical: the primary writes a **binary log** of changes, replicas' I/O threads pull it into a relay log, and applier threads execute it. With `binlog_format = ROW` (the default since 5.7.7), events carry before/after row images — the same species of stream as `pgoutput`, and the reason the MySQL ecosystem grew change-data-capture tooling a decade before PostgreSQL's became mainstream.

```

flowchart TB
    subgraph P["Primary"]
        EX["Executor: commit T1"] --> WAL["WAL / binlog  
page redo or row events"]
    end
    end
    subgraph PHY["Physical path – PostgreSQL streaming"]
        WS["walsender: raw WAL bytes"] --> WR["walreceiver on standby"]
        WR --> REDO["startup process:  
page-level redo, forever"]
        REDO --> C1["Byte-identical cluster  
read-only, same major version"]
    end
    end
    subgraph LOG["Logical path – pgoutput / MySQL binlog ROW"]
        DEC["decoder: WAL to row changes  
INSERT / UPDATE / DELETE + old key"] --> APP["apply worker:  
executes row changes as SQL"]
        APP --> C2["Independent database  
own layout, own version,  
per-table, DDL NOT shipped"]
    end
    end
    WAL --> WS
    WAL --> DEC

```

Statement-based replication: the cautionary tale

The historically obvious design — replay the SQL text itself on each replica — was MySQL's original and, until 5.7.7, default mode. It is compact and human-readable, and it is wrong, because SQL statements are not deterministic functions of database state.

The famous examples are `NOW()` and `RAND()`, and the history is instructive: MySQL actually *patched* both, shipping the statement's timestamp and the random seed alongside the event, so the textbook examples eventually worked. What killed the approach was the unwinnable tail: `UUID()`, `SYSDATE()`, user-defined and stored functions with side effects, `LIMIT` without a deterministic `ORDER BY` (which rows did the primary delete?), concurrent transactions whose apply order on the replica differs from commit order on the primary under anything weaker than serializable. Each case produced *silent divergence*: no error, just a replica whose contents drift from the primary's, discovered months later by a checksum run (`pt-table-checksum` exists precisely because of this era). Row-based replication won because it replicates *effects*, which are deterministic by construction. `binlog_format = STATEMENT` survives as a compatibility setting; treat it as a historical artifact, and treat its lesson as general: **replicate decisions, not the process that made them**,

whenever the process can be nondeterministic. (Consensus-based state machine replication makes the same choice for the same reason — Volume 6, Chapter 5.)

When acks happen: async, sync, and the honest middle

Everything above describes a pipeline; this axis decides where in the pipeline `COMMIT` returns.

Asynchronous: fast, and lossy on the worst day

The default everywhere. The primary flushes its own WAL (Chapter 7's durability contract), acknowledges the client, and ships the WAL to replicas *afterward*, on whatever schedule the network allows. Commit latency is unchanged from a standalone server, and replicas can be slow, distant, or down without affecting writes.

The cost is stated in one acronym: **RPO — recovery point objective — is greater than zero**. If the primary's disk dies right now, every transaction acknowledged but not yet shipped is gone. That window is exactly the replication lag: milliseconds in the happy case, minutes or hours under the pathologies covered below. And the loss is invisible until failover, at which point "we promoted the replica" quietly means "we deleted the last N seconds of acknowledged writes." The companion term is **RTO — recovery time objective** — how long until service is restored; async replication buys good RTO while leaving RPO exposed.

Synchronous: durable, and coupled

Synchronous replication moves the acknowledgment after the replica confirms. PostgreSQL's `synchronous_commit` setting names the exact confirmation point, and the levels are worth knowing precisely because they are distinct promises:

<code>synchronous_commit</code>	Commit returns when	Guarantee
<code>off</code>	WAL handed to the walwriter; not yet flushed locally	Can lose recent async-committed transactions on a <i>primary</i> crash (no corruption — Chapter 7)

synchronous_commit	Commit returns when	Guarantee
local	Local WAL flushed to disk	Standalone durability; nothing about replicas
remote_write	Sync standby has <i>written</i> WAL to its OS	Survives standby postgres crash, not standby OS crash
on (default)	Sync standby has <i>flushed</i> WAL to disk	RPO = 0 for that standby
remote_apply	Sync standby has <i>applied</i> WAL	The commit is also <i>visible</i> to reads on the standby

`remote_apply` is the interesting outlier: it is not about durability but visibility — after commit, a read routed to that standby will see the write, which makes it a blunt (and latency-expensive) read-your-writes mechanism.

The costs: every commit pays a network round trip to the slowest required standby, and — the part people forget — **availability couples to the standby**. With a single named synchronous standby, that standby going down does not merely raise latency; commits *hang*, waiting for an acknowledgment that will never come. Your durability guarantee has converted a replica failure into a primary outage. (A canceled hanging commit is a special trap: the transaction is committed locally — cancel only stops the waiting — so the client's "error" does not mean rollback.)

Hence quorum syntax. PostgreSQL's `synchronous_standby_names` accepts:

```
# Priority-based: s1 and s2 are sync; s3 promotes to sync if one dies
synchronous_standby_names = 'FIRST 2 (s1, s2, s3)'
# Quorum-based: any one ack from the set suffices – survives any single
standby failure
synchronous_standby_names = 'ANY 1 (s1, s2, s3)'
```

`ANY k (n)` is the sane production shape: `ANY 1 (s1, s2)` gives RPO = 0 as long as *some* standby is alive, at the cost of the fastest standby's RTT rather than the slowest's. Note what it does not give you: on failover you must promote a standby that actually holds the acknowledged WAL — with `ANY 1` of two standbys, one of them may be behind — so promotion logic must compare positions, not pick arbitrarily. This is a two-line

preview of quorum intersection reasoning, which Volume 6, Chapter 7 develops properly.

Semi-synchronous: MySQL's negotiated settlement

MySQL's semi-synchronous plugin waits for a configurable number of replica acknowledgments (`rpl_semi_sync_source_wait_for_replica_count`, default 1) that the transaction's binlog events have been *received and flushed to the replica's relay log* — not applied. Two details matter.

First, the wait point. `rpl_semi_sync_source_wait_point = AFTER_SYNC` (the default since 5.7, "lossless semi-sync") waits after the primary syncs its binlog but **before** the storage-engine commit makes the transaction visible to other clients. If the primary dies during the wait, no client ever observed the transaction, so failing over to the replica that lacks it loses nothing anyone saw. The older `AFTER_COMMIT` waited after the transaction became visible, opening a window where other sessions could read a transaction that then vanished in failover — a committed read of data that never survived. The rename from "semi-sync" to "lossless semi-sync" was this one-line change of wait point.

Second, the escape hatch: if no replica acknowledges within `rpl_semi_sync_source_timeout` (default 10 000 ms), the primary **silently degrades to asynchronous** and keeps committing. This is the deliberate opposite of PostgreSQL's hang: MySQL chooses availability, PostgreSQL chooses durability, and both are defensible — but you must know which contract you hold. A semi-sync deployment is only RPO = 0 *while the light is green*, so monitoring `Rpl_semi_sync_source_status` is part of the durability story, not an optional nicety.

```
sequenceDiagram
    participant C as Client
    participant P as Primary
    participant R as Replica

    Note over C,R: ASYNC – ack before ship
    C->>P: COMMIT
    P->>P: flush local WAL
    P-->>C: OK
    Note over P: X dies here: acknowledged
    commit lost – RPO > 0
    P->>R: WAL stream (later)

    Note over C,R: SYNC – ack after standby flush
    C->>P: COMMIT
    P->>P: flush local WAL
    P->>R: WAL
    R->>R: flush
    R-->>P: ack
    P-->>C: OK (latency += RTT)
    Note over R: X replica dead: commits HANG
    availability now coupled

    Note over C,R: SEMI-SYNC AFTER_SYNC – ack or timeout
    C->>P: COMMIT
    P->>P: sync binlog (not yet visible)
    P->>R: binlog events
    alt ack within timeout
        R-->>P: relay log flushed
        P-->>C: OK – lossless
    else timeout
        P-->>C: OK – degraded to async, silently
    end
end
```

Replication lag: the production discipline

Lag is not an edge case; it is the steady-state property that defines what an async replica is. The question is never "is there lag" but "how much, how variable, and what does your application do about it."

Where lag comes from

Single-threaded apply. The primary executed transactions on dozens of cores; historically, replicas replayed them on one. MySQL's SQL thread was strictly serial until 5.6, and useful parallelism arrived with `replica_parallel_type = LOGICAL_CLOCK` (transactions that were in the primary's group-commit window together are provably non-conflicting

and can apply in parallel) and, better, `binlog_transaction_dependency_tracking = WRITESET`, which tracks actual row-set overlap and extracts far more parallelism from single-threaded primaries. PostgreSQL physical replay is still a single startup process — mitigated by the fact that page-level redo is much cheaper than re-executing SQL — and logical replication applies serially per subscription, with PostgreSQL 16 adding parallel apply only for large in-progress ("streamed") transactions. The structural rule: a replica must respect commit-order constraints that the primary discovered dynamically, so the primary's parallelism is always an upper bound the replica approaches from below.

Long and large transactions. A one-hour batch `UPDATE` on the primary commits as a wall of log the replica must chew through; logical replication is worse because most decoders begin shipping at commit, so the replica starts an hour behind by construction.

Recovery conflicts on hot standby — the PostgreSQL-specific classic. A long report query on the standby holds a snapshot needing tuple versions that `VACUUM` on the primary has meanwhile removed (Chapter 6's xmin-horizon disease, now spanning two machines). The standby's replay reaches the `VACUUM` record and must choose: apply it and break the query, or wait and fall behind. `max_standby_streaming_delay` (default 30 s) sets the choice — replay pauses up to this long, then cancels the query with `ERROR: canceling statement due to conflict with recovery`. Raise it and your analytics queries survive at the cost of unbounded lag; lower it and lag stays tight while long queries become unrunnable. The third option, `hot_standby_feedback = on`, has the standby report its oldest snapshot to the primary so `VACUUM` spares those tuples — exporting the problem back to the primary as table bloat. There is no fourth option that costs nothing; pick which resource pays.

Monitoring without being lied to

PostgreSQL exposes per-standby positions in `pg_stat_replication` as LSNs — WAL byte positions — which make lag a subtraction:

```

SELECT application_name,
       pg_wal_lsn_diff(pg_current_wal_lsn(), sent_lsn) AS send_lag_bytes,
       pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS replay_lag_bytes,
       write_lag, flush_lag, replay_lag
-- time-based, PG 10+
FROM pg_stat_replication;

```

Byte lag is the honest metric: it measures work outstanding. The standby-side time metric, `now() - pg_last_xact_replay_timestamp()`, carries a classic lie — on an idle primary no new transactions arrive, the last-replayed timestamp goes stale, and "lag" grows forever while the replica is perfectly caught up.

MySQL's famous liar is `Seconds_Behind_Source`. It is computed from the timestamp of the event the applier is currently executing versus the replica's clock, which means: it reads **0** when the applier has caught up with the *relay log* even if the I/O thread has been unable to fetch from the primary for ten minutes (a network partition shows as zero lag); it reads **NULL**, not "very large," when a thread has stopped; and it jumps discontinuously across large events. The industry answer is a **heartbeat table** — `pt-heartbeat` writes a wall-clock timestamp row on the primary every interval, and the replica's lag is `now()` minus the replicated value: an end-to-end measurement through the entire pipeline that cannot be fooled by any component's private bookkeeping. The generalizable lesson: **measure lag as data through the pipe, not as a component's self-report.**

The anomalies lag causes

Route reads to a lagging replica and you have re-created Volume 4, Chapter 3 at datacenter scale. The parallel is exact, not poetic: a commit applied on the primary but not on the replica is a store sitting in a store buffer — acknowledged, durable, and invisible to another observer. The anomalies even map one-to-one onto the session guarantees of Terry et al. (1994), which Volume 6, Chapter 3 treats formally:

- **Read-your-writes violation.** You update your profile (write hits the primary), the confirmation page's read hits a replica 400 ms behind, your change is absent. Users interpret this as data loss and retry, sometimes creating duplicates.

- **Non-monotonic reads.** Two successive page loads hit two replicas with different lag; the second shows an *older* state than the first. Time runs backward. A comment appears, then vanishes, then reappears.
- **Causal violations.** User A posts a question (primary), user B reads it from a fresh replica and posts an answer (primary); user C reads from a stale replica and sees the answer to a question that does not exist. The write dependency — B read A's post before writing — is invisible to per-object replication.

```
sequenceDiagram
    participant U as User session
    participant P as Primary
    participant R as Lagging replica

    U->>P: UPDATE profile SET bio = 'new' - COMMIT
    P-->>U: OK (durable on primary)
    Note over P,R: WAL for the update still in flight
    U->>R: GET /profile - read routed to replica
    R-->>U: bio = 'old'
    Note over U: "My save was lost" - retry, support ticket
    P->>R: WAL arrives, replica applies
    U->>R: later read
    R-->>U: bio = 'new' - write "reappears"
```

The mitigation toolkit

In escalating order of machinery:

Route by semantics. The oldest fix: reads that follow the user's own write, or that gate a decision, go to the primary; browse traffic goes to replicas. Cheap, effective, and erodes as every code path eventually acquires a "must be fresh" case.

Session stickiness. Pin a session to one replica to get monotonic reads (one replica's state only advances), and pin to the primary for N seconds after any write to approximate read-your-writes. Crude — the timer is a guess racing actual lag — but a large fraction of production systems run exactly this.

Causal tokens: wait for a position, not a duration. The correct mechanism. The primary's log position after your commit *names* the state your next read requires; carry it in the session and make the replica wait until it has applied that far.

PostgreSQL, with LSNs:

```
-- After the write, on the primary:
SELECT pg_current_wal_insert_lsn(); -- e.g. '7D3/48A1B2F0' – the
session token

-- Before the read, on the replica: proceed only when replay has passed
the token
SELECT pg_last_wal_replay_lsn() >= '7D3/48A1B2F0':pg_lsn; -- poll,
or:
CALL pg_wal_replay_wait('7D3/48A1B2F0', 200); --
PostgreSQL 17+: block up to 200 ms
```

MySQL, with GTIDs (`gtid_mode = ON` gives every transaction a globally unique `server_uuid:seqno` identifier):

```
-- After the write, on the primary:
SELECT @@global.gtid_executed; -- token, e.g. '3e11fa47-...:1-77'

-- Before the read, on the replica:
SELECT WAIT_FOR_EXECUTED_GTID_SET('3e11fa47-...:1-77', 0.2);
-- returns 0 when the replica has applied the set, 1 on the 200 ms
timeout
```

On timeout, fall back to the primary. This is precisely a happens-before edge established by waiting: the write happens-before the read because the read refused to execute until the replica's state included the write. The token plays the role the volatile write/read pair played in Volume 4, Chapter 3 — and generalized tokens of this kind become session-guarantee vectors and causal consistency in Volume 6, Chapter 3. `remote_apply` (above) achieves the same edge by pushing the wait to commit time on the writer instead of read time on the reader — same guarantee, opposite party pays.

Failover, honestly

The availability story of leader-based replication rests on a maneuver — detect primary death, promote a replica, repoint clients — in which every step can go wrong in a data-losing way.

Detection is guessing. You cannot distinguish "primary is dead" from "primary is slow" from "the network between the monitor and the primary is partitioned" — this is the fundamental limit of failure detection over asynchronous networks (Volume 6, Chapter 10). A timeout is a

wager. Declare death too eagerly and you promote while the old primary is alive and accepting writes from clients on its side of the partition; too lazily and your RTO balloons. There is no correct timeout, only a chosen trade.

Promotion must choose the most-advanced replica. Under async or ANY k sync replication, replicas hold different prefixes of the log. Promote any but the furthest-ahead acknowledged one and you discard committed work unnecessarily. Tooling compares `pg_last_wal_receive_lsn()` across standbys, or GTID sets in MySQL — where GTIDs also give the elegant follow-up: point the other replicas at the new primary with `SOURCE_AUTO_POSITION = 1` and each one negotiates, from its own executed-GTID set, exactly where to resume. No manual coordinate arithmetic, which is what failing over on legacy binlog file/position coordinates required, at 3 a.m., correctly.

The old primary is now poison. When it comes back, it likely holds transactions the new primary never received — acknowledged writes now on the losing timeline. It cannot simply re-join as a standby: its history *diverged*. PostgreSQL's `pg_rewind` handles this mechanically — it finds the point where the timelines forked, rewinds the old primary's data directory to it by copying back the blocks changed since (requiring `wal_log_hints = on` or data checksums), and lets it re-sync as a standby. Note what that is: a tool for *discarding* the diverged committed transactions. With async replication, some data loss at failover is not a bug in the tooling; it is the contract you signed.

Split brain is the catastrophic version. If clients can still reach the undead old primary while others write to the new one, you have two databases accepting conflicting writes under one name. Recovery from a split brain is manual, application-specific, and sometimes impossible; the discipline is prevention by **fencing** — making it *impossible*, not unlikely, for the old primary to accept writes. STONITH ("shoot the other node in the head") powers off or network-isolates the old primary before promotion; softer fences revoke its VIP, its credentials, or its storage lease. This is the same reasoning as fencing tokens in Volume 4, Chapter 2: a node that believes it holds a lease must be unable to act on that belief once the lease has moved, because its belief cannot be corrected remotely.

Why serious tooling uses a consensus store. The failover decision — *who is primary now* — is itself a distributed agreement problem, and

solving it with the same ad-hoc timeouts that caused the false detection just moves the split brain into the control plane (two monitors, each promoting its own candidate). Patroni therefore keeps a leader key with a TTL in etcd, ZooKeeper, Consul, or the Kubernetes API: a store that solves agreement *properly*, with Raft or ZAB underneath. The primary holds the lease by refreshing the key; a candidate can promote only by winning an atomic compare-and-set on it; the consensus store's own quorum guarantees at most one winner even under arbitrary partitions. Orchestrator, on the MySQL side, runs Raft among its own nodes for the same reason. The pattern to internalize: **the data plane can remain plain leader-follower replication, but the decision of who leads is delegated to a consensus system** — a small, rarely-written, strongly-consistent kernel governing a large, fast, weakly-replicated body. Volume 6, Chapter 8 covers these coordination services in depth; Chapters 5-6 cover the consensus protocols inside them.

```
sequenceDiagram
    participant OP as Old primary
    participant DCS as Consensus store etcd
    participant A as Patroni on replica A
    participant CL as Clients

    OP->>DCS: refresh leader key (TTL 30s)
    Note over OP,DCS: partition: refreshes stop
    DCS->>DCS: leader key expires
    A->>DCS: compare-and-set leader = A
    DCS-->>A: won – at most one winner, by quorum
    Note over OP: demotion watchdog: cannot see DCS,
    cannot refresh – steps down, read-only
    A->>A: verify highest LSN, promote
    CL->>DCS: discover new leader
    CL->>A: writes resume
    Note over OP: partition heals: key holder is A –
    pg_rewind, rejoin as standby
```

One number closes the topic: with async replication, failover RPO is the lag at the moment of death. Your steady-state p99 lag is your data-loss budget, which is why the lag graphs from the previous section belong on the same dashboard as your durability SLO (Volume 11).

Multi-primary, honestly

Accepting writes on more than one node at once is perennially attractive — write locality, no failover — and the systems exist: MySQL Group Replication in multi-primary mode, Galera, PostgreSQL's pglogical/BDR lineage, and every multi-leader geo-replication design. The physics is unattractive: two primaries accepting writes concurrently *will* produce conflicting histories, and someone must merge them.

The conflict classes: **write-write** (both sides update row 42's balance; whose update survives?), **uniqueness** (both sides insert `alice@example.com`; the constraint held locally on each and is violated globally), and **update-delete** (one side updates a row the other has deleted — apply the update to what?). Certification-based systems like Galera detect the first class at commit and abort one side; asynchronous multi-primary detects conflicts *after* both sides have acknowledged, which is where resolution strategies come in:

- **Last-writer-wins**: pick the higher timestamp, discard the other. Simple, convergent, and a data-loss machine — the "loser" was an acknowledged, committed transaction, silently deleted, with clock skew choosing the victim. LWW is defensible only where any single value is acceptable (a heartbeat field, a cache); as a general policy it is the quiet destruction of writes.
- **Application-defined merge**: surface both versions to code that knows the semantics — add the two counter increments, union the two sets. Correct where the data type admits a merge; a per-type engineering effort forever after.
- **CRDTs** formalize exactly that: data types whose merge is commutative, associative, and idempotent, so all replicas converge without coordination regardless of delivery order. Volume 6, Chapter 11 develops them; note here only that they solve convergence, not invariants — no CRDT enforces "balance never negative" or global uniqueness, because those require the coordination CRDTs exist to avoid.

Hence the usual sane answer: **single writer per shard**. Partition the keyspace (Chapter 9), give each partition one primary, and place partitions near their writers. You keep write locality and horizontal write scaling while every individual key retains a single ordered write history — no merges, no LWW, ordinary failover per shard. Cross-shard

operations then need distributed transactions, which is Chapter 10's problem, but that trade is at least explicit. Reach for true multi-primary when disconnected operation is a requirement (offline-first sync, multi-datacenter survival of full partitions with writes continuing on both sides) and the data model has been designed for mergeability — not as a bolt-on availability upgrade for a schema full of uniqueness constraints and balances.

Topology variations worth knowing

Cascading replication. Replicas can feed replicas: PostgreSQL standbys accept downstream `primary_conninfo` connections, MySQL intermediates run `log_replica_updates`. The win is fan-out relief — a primary serving one cross-region stream while the remote intermediate feeds five local replicas — at the cost of an added lag hop and a dependency: the intermediate's failure orphans its subtree, and failover tooling must repair the tree, not just the head.

Delayed replicas. A replica configured to apply the stream on a fixed delay:

```
# PostgreSQL standby
recovery_min_apply_delay = '4h'
```

```
-- MySQL replica
CHANGE REPLICATION SOURCE TO SOURCE_DELAY = 14400;
```

This is an operator-error time machine. Replication faithfully propagates `DROP TABLE users` to every up-to-date replica in milliseconds — redundancy protects against machine failure, never against commands. A four-hour-delayed replica holds a pre-mistake state you can read the lost rows from immediately, or roll forward to the second before the mistake — hours faster than a backup restore (Chapter 7's PITR), at the price of one replica's hardware. Cheap insurance for the most common cause of real data loss, which is not disk failure but a human with production credentials.

The distributed-systems lens

This chapter deliberately stayed inside one system: one database product, one primary at a time, replication as a feature you configure. Volume 6 re-derives everything here from first principles, and the mapping is worth previewing because you now hold the concrete half of it.

Leader-based replication is the general pattern; election is the missing piece. What Patroni does with an etcd leader key, Raft (Volume 6, Chapter 6) integrates into the replication protocol itself: leader election, log replication, and commit acknowledgment become one mechanism, and "promote the most-advanced replica" stops being a script and becomes a safety property the protocol proves. Your PostgreSQL failover stack is a hand-assembled consensus system; Raft is the same machine with the invariants welded shut.

Quorums dissolve the sync/async dichotomy. This chapter treated "how many replicas ack" as a binary with a patched middle. Quorum systems (Volume 6, Chapter 7) make it a dial: W-of-N acks per write, R-of-N per read, with $R + W > N$ guaranteeing read-write intersection. PostgreSQL's `ANY 1 (s1, s2)` is a write quorum of 2-of-3 (primary plus one standby) with reads served at $R = 1$ — which is exactly why a read from an arbitrary standby can miss an acknowledged write, and why the promotion logic must find the intersection replica at failover.

The anomaly catalogue becomes the consistency-model menagerie. Read-your-writes, monotonic reads, and causal ordering — patched here with stickiness and LSN waits — are named consistency models with formal definitions in Volume 6, Chapter 3, arranged in the same strong-slow-versus-weak-fast hierarchy as Chapter 6's isolation levels and Volume 4, Chapter 3's memory models. This is the third time this book series has shown you the same picture, and it is the same picture: acknowledged-but-not-visible writes, a partial order of what observers may see, and a toolkit that trades latency for ordering edges.

Every topology is a point on one triangle. Latency, durability, availability: async is fast and lossy; sync is durable and coupled; semi-sync buys durability until the timeout, then quietly trades it back; quorums price the continuum per operation. There is no configuration that maximizes all three, because the constraint is physical — information cannot be durable in two places faster than the wire between them, and a guarantee that requires a peer is hostage to that peer. Choose the vertex

your business actually needs, write the choice down, and make sure the failover runbook and the SLO document (Volume 11) describe the same triangle point your configuration actually implements.

Key takeaways

- Replication has **two independent axes**: what ships (physical WAL bytes, logical row changes, statements) and when the commit is acknowledged (async, sync, semi-sync). Keep them separate.
- **Physical replication** streams byte-identical WAL: cheap, complete, whole-cluster, read-only replicas, same major version and architecture. **Logical replication** ships decoded row changes: portable, cross-version, flexible — but DDL does not replicate in PostgreSQL, and updates/deletes need a replica identity. **Statement-based** replication died of nondeterminism and silent divergence; replicate effects, not processes.
- **Async** means $RPO > 0$ and your steady-state lag is your data-loss budget at failover. **Sync** means $RPO = 0$ for the acked standby, commit latency $\approx$ RTT, and a dead sync standby blocks writes — hence quorum forms: `synchronous_standby_names = 'ANY k (...)'`, and MySQL semi-sync with `AFTER_SYNC` — which silently degrades to async on timeout, so its durability guarantee is only as good as the monitoring on `Rpl_semi_sync_source_status`.
- **Lag** comes from single-threaded apply, large transactions, and (PostgreSQL) recovery conflicts, where `max_standby_streaming_delay` and `hot_standby_feedback` choose which resource pays. Monitor lag as bytes of outstanding WAL (`pg_stat_replication` LSN deltas) or end-to-end heartbeats; `Seconds_Behind_Source` reads 0 during a partition and NULL when broken.
- Lag reproduces the **store-buffer anomalies** one layer up: read-your-writes violations, non-monotonic reads, causal violations. The principled fix is a **position token** — LSN or GTID captured at commit, waited on before the read (`pg_wal_replay_wait`, `WAIT_FOR_EXECUTED_GTID_SET`) — a manufactured happens-before edge.
- **Failover**: detection is a wager against an asynchronous network; promotion must select the most-advanced replica; the old primary's diverged history must be discarded (`pg_rewrite`) and the node

fenced before anything else trusts the new primary. Auto-failover tooling puts the leader decision in a consensus store because "who is primary" is itself an agreement problem — solve it with a system that actually solves agreement.

- **Multi-primary** means conflicts: LWW silently destroys acknowledged writes; merges and CRDTs work where the data type admits them and enforce no global invariants. **Single writer per shard** keeps ordered per-key history and is the usual sane answer.
- A **delayed replica** is cheap insurance against the leading cause of data loss — humans — because normal replication propagates the mistake faithfully everywhere else.

Further reading

- PostgreSQL documentation, *High Availability, Load Balancing, and Replication* — streaming replication, `synchronous_commit`, `synchronous_standby_names`, hot standby conflicts. <https://www.postgresql.org/docs/current/high-availability.html>
- PostgreSQL documentation, *Logical Replication* — publications, subscriptions, replica identity, and the restrictions list (DDL, sequences). <https://www.postgresql.org/docs/current/logical-replication.html>
- MySQL 8.4 Reference Manual, *Replication* — binlog formats, GTIDs and auto-positioning, semi-synchronous replication and `AFTER_SYNC`. <https://dev.mysql.com/doc/refman/8.4/en/replication.html>
- Patroni documentation — the leader-key/DCS architecture for PostgreSQL auto-failover. <https://patroni.readthedocs.io/>
- Kleppmann, M., *Designing Data-Intensive Applications*, O'Reilly, 2017 — Chapter 5, "Replication": the best single treatment of leader-based, multi-leader, and leaderless replication and their anomalies.
- Terry, D. et al., "Session Guarantees for Weakly Consistent Replicated Data," *PDIS*, 1994 — the origin of read-your-writes, monotonic reads, and the other session guarantees, from the Bayou project. <https://dl.acm.org/doi/10.1109/PDIS.1994.331722>
- Percona Toolkit documentation, `pt-heartbeat` and `pt-table-checksum` — honest lag measurement and replica-drift detection, born of the statement-based era. <https://docs.percona.com/percona-toolkit/>

- Chapter 7 — WAL and Crash Recovery — the log that this chapter ships; Chapter 9 — Partitioning — the single-writer-per-shard follow-through; Chapter 10 — Distributed Transactions — what cross-shard writes cost.
- Volume 4, Chapter 3 — Memory Models and Happens-Before — the store-buffer parallel; Volume 6, Chapters 3, 5–8, 10, 11 — consistency models, consensus, quorums, coordination services, failure detectors, and CRDTs: this chapter generalized.

Chapter 9 — Partitioning and Sharding

What this chapter covers. Chapter 8 ended at a wall: no matter how you replicate, a single-primary system funnels every write through one machine, and replication does nothing to increase the total amount of data one machine can hold. The answer to both limits is to split the data itself. Unfortunately the industry uses one word — “partitioning” — for two different things, and this chapter treats them separately and in order. The first is **single-node table partitioning**: splitting one table into many physical child tables inside one database instance, a technique whose biggest payoff is usually not query speed but data lifecycle — dropping a partition is an $O(1)$ alternative to a `DELETE` that would grind through vacuum and WAL for hours (Chapters 2 and 7). The second is **distributed sharding**: splitting a dataset across machines, which buys write scaling and blast-radius isolation at the cost of everything that quietly assumed all the data lived in one place. For sharding, the shard-key decision is *the* decision, so we work through hash, range, directory, and compound schemes and the failure mode each one invites. We then confront skew and hot shards, the mechanics of rebalancing — why modulo- N is a catastrophe, why every serious system moves fixed virtual shards instead, and how an online resharding actually runs — and an honest catalogue of what sharding breaks: cross-shard queries, transactions, secondary indexes, and uniqueness. The distributed-systems lens closes the loop opened in Volume 4, Chapter 2: sharding is lock striping at planetary scale, with the same hot-key failure and the same rebalancing pain.

Learning goals — after this chapter you should be able to:

- Distinguish single-node partitioning from distributed sharding, and know which problem each solves.
- Design a Postgres or MySQL partitioned table for a time-series workload, verify partition pruning with `EXPLAIN`, and use partition drop/attach for retention and bulk load.
- State the real limits of single-node partitioning: unique constraints that must include the partition key, and practical ceilings on partition count.
- Choose a shard key deliberately: hash for uniformity, range for locality, directory for control, compound keys for multi-tenant systems — and predict the failure mode of each.
- Detect and mitigate hot shards with per-shard metrics, key salting, request coalescing, caching, and whale isolation.
- Explain why fixed virtual shards make rebalancing tractable, and run the online resharding playbook: backfill, verify, cut over, retire.
- Enumerate what sharding breaks — scatter-gather queries, cross-shard joins and transactions, global secondary indexes, global uniqueness — and the standard design responses to each.
- Argue the build-vs-buy decision among application-level sharding, middleware such as Vitess and Citus, and natively distributed databases (Chapter 12), and defend the "design keys early, shard late" discipline.

Two things sharing a word

The vocabulary is genuinely confused, so let us fix terms for this chapter. **Partitioning** will mean splitting one logical table into multiple physical tables *within a single database instance*, managed by the database itself, invisible to queries. **Sharding** will mean splitting a dataset *across database instances on different machines*, where each instance holds a subset and something — the application, a proxy, or the database's own distribution layer — routes each request to the right one.

The confusion is not arbitrary: the underlying idea is the same (divide data by a key so that operations touch only their division), and several systems blur the line — Cassandra calls its unit of distribution a "partition," Kafka distributes "partitions" across brokers, and Citus turns

Postgres table partitioning machinery into a sharding layer. But the engineering consequences are utterly different. Single-node partitioning changes almost nothing about your application: SQL still works, transactions still work, unique constraints mostly still work, joins still work. It is a storage-layout and lifecycle optimization. Sharding changes almost everything: it is an architectural commitment that removes capabilities you have relied on since Chapter 1, and most of this chapter is about the removals. Conflating the two leads teams to fear partitioning (which is nearly free and they probably need) because they have heard horror stories about sharding (which is expensive and they probably do not need yet).

Single-node table partitioning

Declarative partitioning in Postgres and MySQL

Modern relational databases support **declarative partitioning**: you state a partitioning method and key in the DDL, and the database routes rows, maintains the children, and rewrites queries. Postgres has had this since version 10 (range and list; hash arrived in 11); MySQL has supported `PARTITION BY` on InnoDB tables for much longer. Three methods cover practice:

- **Range partitioning** assigns each row by which interval its key falls into. This is the workhorse, and time is the workhorse key: one partition per day, week, or month of a `created_at` column. Time-series and event data are range partitioning *par excellence*, because both the queries ("last 7 days") and the lifecycle ("retain 13 months") are expressed in the same dimension the table is cut on.
- **List partitioning** assigns by explicit value: one partition per region, per product line, per tenant tier. Useful when the divisions are few, named, and operationally meaningful — for instance when EU rows must live in a different tablespace than US rows.
- **Hash partitioning** assigns by hash of the key, modulo the partition count. It buys uniform spread and smaller per-partition indexes when there is no natural range or list dimension, at the price of making per-partition operations meaningless — no hash partition ever corresponds to "old data" or "tenant 7," so you lose the lifecycle payoff described below.

Here is the canonical time-series shape in Postgres:

```
CREATE TABLE events (
    event_id    bigint          NOT NULL,
    tenant_id   bigint          NOT NULL,
    created_at  timestampz      NOT NULL,
    kind        text            NOT NULL,
    payload     jsonb,
    PRIMARY KEY (event_id, created_at)    -- must include the
partition key; see below
) PARTITION BY RANGE (created_at);

CREATE TABLE events_2026_07 PARTITION OF events
    FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
CREATE TABLE events_2026_08 PARTITION OF events
    FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');

CREATE INDEX ON events (tenant_id, created_at);    -- cascades to every
partition
```

Each partition is a real table with its own files, its own indexes, its own statistics, its own vacuum schedule. That physical independence is the source of every benefit that follows.

Partition pruning

The query-side payoff is **pruning**: the planner proves from the `WHERE` clause that most partitions cannot contain matching rows and never touches them.

```
=# EXPLAIN (COSTS OFF)
SELECT count(*) FROM events
WHERE created_at >= '2026-08-01' AND created_at < '2026-08-15';

                        QUERY PLAN
-----
Aggregate
-> Seq Scan on events_2026_08 events
    Filter: ((created_at >= '2026-08-01 00:00:00+00'::timestampz)
        AND (created_at < '2026-08-15 00:00:00+00'::timestampz))
)
```

One partition scanned, the rest never opened — with a year of monthly partitions, roughly 92% of the table statically excluded. Pruning happens at **plan time** when the predicate compares the partition key to constants, and since Postgres 11 also at **execution time**, when the comparison

value is a parameter, a subquery result, or the inner side of a join. Execution-time pruning shows up in `EXPLAIN ANALYZE` as an `Append` node annotated `Subplans Removed: 11` — the plan carried all partitions but skipped the impossible ones once the parameter value was known.

Two things defeat pruning, and both are common in the wild. First, any predicate that is not on the partition key — pruning is not magic; a query filtered only on `tenant_id` scans an index in *every* partition of the range-partitioned table above. Second, wrapping the key in a function or a mismatched type (`created_at::date = '2026-08-01'`) can make the predicate opaque to the pruning logic, exactly as it makes an index unusable (Chapter 3). A **default partition** — the catch-all Postgres allows for rows that fit no declared range — is a third quiet hazard: for open-ended predicates the planner often cannot exclude it, so it gets scanned every time. Verify pruning with `EXPLAIN` the way you verify index usage: assume nothing.

Briefly on **partition-wise operations**: when two tables are partitioned identically, Postgres can join them partition-by-partition and aggregate partition-by-partition (`enable_partitionwise_join` and `enable_partitionwise_aggregate`, both off by default because plan-time cost grows with partition count). These are worth knowing mostly because they are the single-node rehearsal of a distributed idea — co-partitioned joins are precisely what makes some cross-shard joins feasible later in this chapter.

The payoff that actually matters: $O(1)$ retention

Ask teams who run partitioned tables in production why, and the most common answer is not query speed. It is `DROP`.

Consider deleting one month of data from a 2 TB unpartitioned events table. `DELETE FROM events WHERE created_at < '2025-08-01'` must find every matching row, write a WAL record for every row it kills (Chapter 7), and mark every row dead-but-present. The table does not get smaller; it fills with dead tuples that vacuum must reclaim, generating a second wave of I/O, and the indexes bloat identically (Chapters 2 and 3). Replicas replay all of that WAL, so the delete storm propagates down the replication tree (Chapter 8). A month-sized delete on a large table is routinely an hours-long, I/O-saturating event — and then vacuum runs.

Now the partitioned version:

```
ALTER TABLE events DETACH PARTITION events_2025_07; -- CONCURRENTLY
since PG 14
DROP TABLE events_2025_07;
```

This is a catalog update and a file unlink. Milliseconds, a few kilobytes of WAL, no dead tuples, no vacuum, no index maintenance, no replica lag spike. The same asymmetry works in reverse for bulk load: build tomorrow's partition offline — load it with `COPY`, index it, `ANALYZE` it, validate it — and then `ATTACH PARTITION` makes it visible in one metadata operation. Detach and re-attach is also the sane path for archiving a partition to cold storage, or for rewriting one partition (compressing, re-clustering) without touching the rest of the table.

If your table has time-based retention and you are running periodic mass deletes against an unpartitioned table, partitioning is not an optimization, it is the correction of a mistake. Tooling like `pg_partman` (or TimescaleDB's automation, Chapter 13) exists to create future partitions and drop expired ones on schedule, which is the part people forget until an insert lands with no partition to accept it.

Limits, stated honestly

Single-node partitioning has two limits you must design around rather than discover.

Unique constraints must include the partition key. A unique index in Postgres is a per-table B-tree, and a partition is a table; there is no global index spanning partitions. Postgres therefore refuses any `UNIQUE` or `PRIMARY KEY` constraint on a partitioned table that does not include all partition-key columns — that is why the DDL above declares `PRIMARY KEY (event_id, created_at)` rather than `event_id` alone. Uniqueness of `event_id` by itself is now *your* problem: enforce it by construction (generate IDs from a sequence or an ID allocator, so duplicates cannot arise) rather than by constraint. MySQL has the same rule in stricter form: every unique key on a partitioned table, including the primary key, must contain every column in the partitioning expression. This is the first appearance of a theme that dominates the sharding half of this chapter: **partitioning by one key destroys global enforcement on every other key.**

Partition counts have practical ceilings. Every partition adds catalog entries, statistics, relation-cache footprint, and planning work; operations

that cannot be pruned touch every partition and take a lock on each. Postgres handles thousands of partitions reasonably in current versions — planning cost with good pruning is now sublinear in partition count, and the documentation's guidance is that up to a few thousand is workable when queries prune well — but tens of thousands is asking for pain in planning time, lock counts per query, and autovacuum scheduling. MySQL enforces a hard limit of 8,192 partitions per table and gets uncomfortable well before it. The practical rule: partition at the granularity your retention and queries need and no finer. Daily partitions with 13-month retention is ~400 partitions and entirely comfortable; hourly partitions "to be safe" is 9,500 and is not.

Distributed sharding: why

Three pressures push a dataset off one machine, and it is worth being precise about them because they have different remedies and sharding is only mandatory for the first two.

Write throughput. Chapter 8's conclusion stands: read replicas scale reads, but every write still traverses the single primary — its WAL, its buffer pool, its vacuum, its CPU. When the write workload exceeds what the biggest affordable machine can absorb, the only remaining move is to have *multiple primaries, each owning a disjoint slice of the data*. That is sharding. (Multi-leader replication of the *same* data is the other theoretical option, and Chapter 8 explained why its conflict-resolution burden makes it the wrong default.)

Dataset size. When the working set outgrows one machine's storage or — more often and more subtly — its RAM, so that the cache-hit economics of Chapter 2 collapse, splitting the data restores locality: each shard's working set fits its machine again.

Blast radius. A single database is a single failure domain. Shard it twenty ways and a corrupt index, a runaway query, a botched migration, or a hardware failure degrades 5% of customers instead of 100%. For multi-tenant SaaS this isolation argument sometimes justifies sharding even when raw capacity does not; it also gives you per-shard maintenance windows and canary shards for risky schema changes (Chapter 14).

Be equally clear about what sharding does *not* buy: it does not reduce latency for single-row operations (a lookup on one shard is the same B-

tree descent it always was, now plus a routing hop), and it does not improve availability by itself — each shard still needs the replication of Chapter 8 underneath it. Sharding and replication are orthogonal and virtually always composed: N shards $\times$ a primary and replicas per shard.

The shard key: the decision

Everything about a sharded system follows from one choice: the **shard key**, the attribute of each row that determines which shard owns it. Queries that include the key route to one shard and stay cheap. Queries that do not include it must ask every shard. Transactions confined to one key value stay ACID; transactions spanning key values become distributed transactions (Chapter 10). The shard key is load-bearing for performance, correctness, and operations simultaneously, and it is expensive to change — so it deserves more design scrutiny than any other schema decision you make. There are four basic schemes.

```

flowchart TD
    subgraph HASH["Hash sharding"]
        HK["key = user_4711"] --> HF["hash to slot  
slot to shard map"]
        HF --> HS1["shard A  
slots 0-5460"]
        HF --> HS2["shard B  
slots 5461-10922"]
        HF --> HS3["shard C  
slots 10923-16383"]
    end
    subgraph RANGE["Range sharding"]
        RK["key = 2026-08-14/evt123"] --> RM["range map  
split points"]
        RM --> RS1["shard A  
min .. g"]
        RM --> RS2["shard B  
g .. p"]
        RM --> RS3["shard C  
p .. max"]
    end
    subgraph DIR["Directory sharding"]
        DK["key = tenant_42"] --> DL["lookup service  
tenant to shard table"]
        DL --> DS1["shard A  
tenants 1,7,42"]
        DL --> DS2["shard B  
tenants 3,9,55"]
        DL --> DS3["shard C  
whale tenant 12"]
    end

```

Hash sharding

Route each key by a hash of its value. The virtue is statistical: a decent hash spreads any key population — sequential IDs, timestamps, hot prefixes — uniformly across shards, so you get even data volume and, for most workloads, even load, with zero placement management. The cost is that hashing destroys order. Adjacent keys land on unrelated shards, so **range queries over the key become scatter-gather**: "events between Tuesday and Thursday" must ask every shard and merge. Hash sharding is the right default when access is point-lookup dominated and the key population is adversarially skewable — user IDs, session IDs, device IDs.

Two refinements matter. First, never hash straight into `mod N` where `N` is the live shard count — the rebalancing section explains why this is a catastrophe. Hash into a **fixed, large slot space** and map slots to shards,

as Redis Cluster does with its 16,384 slots. Second, **consistent hashing** arranges hashes on a ring so that adding a node moves only the keys adjacent to it, and **virtual nodes** give each physical machine many ring positions to smooth the statistical imbalance and spread a departed node's load across all survivors. Those two sentences are the concepts; the algorithm, its variants (rendezvous hashing, jump hash), and its load-balance mathematics are treated properly in Volume 14, Chapter 5, and Dynamo-style systems built on it in Volume 6, Chapter 7.

Range sharding

Divide the key space into contiguous ranges at chosen split points, one range per shard. This is the Bigtable lineage — Bigtable range-partitions rows into tablets by key, and its descendants and cousins (HBase, Spanner, CockroachDB, TiDB) kept the design. Range sharding preserves order, so range scans touch one or a few shards, and it gives you **locality**: keys that sort together live together, which composes beautifully with the clustered storage of Chapters 2 and 3.

Its characteristic failure is the **hot tail**. If the key is monotonic — an auto-increment ID, a timestamp, a ULID — every insert lands in the *last* range, and one shard absorbs 100% of the write load while the rest hold cold history. This is the same pathology as the rightmost-leaf B-tree contention of Chapter 3, promoted from a page latch to an entire machine. The Bigtable paper and every Bigtable-descendant's documentation warn about it, and it remains the most common self-inflicted wound in range-sharded systems: schemas keyed by `(timestamp, ...)` hammer one tablet forever, however aggressively the system splits it. The standard remedies are to lead the key with something distributive — `(user_id, timestamp)` rather than `(timestamp, user_id)` — or to explicitly prefix a hash bucket, which is the salting pattern discussed under hot spots.

Directory sharding

Keep an explicit lookup table: key (or key group) → shard. A small metadata store — replicated, cached, consistency-managed (Volume 6, Chapter 8) — answers routing queries. This trades the elegance of computed placement for **control**: you can place tenant 12 alone on its own shard because it is enormous, pin EU tenants to EU shards for residency, and migrate one tenant at a time by updating one row. Multi-

tenant SaaS platforms converge on directory sharding at the tenant grain almost universally, precisely because tenant sizes are wildly skewed and law constrains placement. The costs: the directory is a dependency on every request path (cache it hard), a consistency problem during migrations (the playbook below), and a thing that can itself get large if you foolishly track individual keys rather than key groups.

Compound keys: the SaaS workhorse

Real shard keys are usually compound: a **routing component** that picks the shard and an **ordering component** that arranges rows within it. The canonical example is `(tenant_id, entity_id)`: shard by `tenant_id` — every query in a B2B product carries it, and a tenant's data stays transactionally co-located — then order by entity or time within the tenant.

Cassandra makes this structure explicit and mandatory, and its terminology is worth learning even if you never run Cassandra because it names the two roles precisely. The **partition key** determines placement by hash; the **clustering key** determines sort order within the partition; queries must supply the partition key and may range over clustering columns:

```
-- CQL: one partition per tenant; rows ordered newest-first within it.
CREATE TABLE orders_by_tenant (
    tenant_id    bigint,
    created_at   timeuuid,
    order_id     bigint,
    status       text,
    total_cents  bigint,
    PRIMARY KEY ((tenant_id), created_at, order_id)
) WITH CLUSTERING ORDER BY (created_at DESC, order_id ASC);

-- Served by one partition, one sequential read:
SELECT * FROM orders_by_tenant
WHERE tenant_id = 42 AND created_at > minTimeuuid('2026-08-01');
```

This is **query-driven design**: the table exists to serve this access path, and other access paths get other tables (Chapter 11 develops the philosophy). The same discipline applies verbatim in a sharded relational system — every table carries `tenant_id`, every index leads with it, every query filters on it — and a schema with that shape is what this chapter's final section calls *shardable*, whether or not it is sharded yet.

Hot spots and skew

Uniform hashing gives you uniform *keys*, not uniform *load*. Skew arrives in two forms. **Celebrity keys**: one key value receives orders of magnitude more traffic than the median — the viral post, the account with forty million followers, the tenant that is 30% of your revenue running a batch import. Whatever the scheme, that key maps to one shard, and that shard melts. **Temporal skew**: load migrates across the key space over time — monotonic keys hammering the range tail, Monday-morning tenants in one timezone, a marketing campaign lighting up one region's shard.

Detection is a measurement discipline, not an algorithm: export throughput, latency, CPU, and queue depth **per shard**, not just in aggregate, and alert on dispersion (max/median) rather than on totals — a fleet at 40% average CPU with one shard at 98% is an outage in progress that aggregate dashboards will hide. Per-key top-K sampling (a count-min sketch or the database's own statistics) tells you *which* key is responsible. Volume 11 treats the observability machinery; the point here is that per-shard metrics are not optional in a sharded system, because the shard is the unit at which the system fails.

Mitigations, in escalating order of invasiveness:

- **Cache the celebrity.** A read-hot key is the textbook case for a cache in front of the shard (Volume 7, Chapter 3); a handful of cache servers can absorb read traffic no database shard can.
- **Coalesce duplicate requests.** When ten thousand concurrent requests want the same hot key, let one fetch and the rest await its result — request coalescing / singleflight, the pattern from Volume 4, Chapter 9. This collapses read stampedes at the source.
- **Salt or split the key.** For write-hot keys, append a small random or derived suffix — `key#0 ... key#7` — so the hot key's writes spread across 8 shards. The price is paid at read time: reading the key now fans out to all 8 sub-keys and merges. Salting is therefore a targeted tool for identified hot keys (apply it to the celebrity, not the population), and the fan-out factor is a dial between write relief and read amplification.
- **Isolate the whale.** In directory-sharded systems, move the enormous tenant to dedicated hardware. This is skew management by placement rather than by key design, and it is often the operationally sanest answer: the whale gets its own capacity, its own

SLO, and its own blast radius, and the shared shards return to statistics that hashing can handle.

```
flowchart TD
    subgraph BEFORE["Before: celebrity key melts shard B"]
        W1["writers"] --> KB["key = star_99"]
        KB --> SB["shard B - 98% CPU"]
        SA["shard A - 12%"]
        SC["shard C - 11%"]
    end
    subgraph AFTER["After: salted into 3 sub-keys"]
        W2["writers"] --> S0["star_99#0"] --> TA["shard A - 40%"]
        W2 --> S1["star_99#1"] --> TB["shard B - 41%"]
        W2 --> S2["star_99#2"] --> TC["shard C - 39%"]
        RD["reader"] -->|"fan-out and merge"|
    end
    all 3 sub-keys | TA
    RD --> TB
    RD --> TC
    BEFORE -->|"salt the hot key only"| AFTER
```

If this section feels familiar, it should: it is the hot-stripe problem from Volume 4, Chapter 2, where a striped lock degenerates to a global lock when one key dominates. The failure and every mitigation have exact in-process ancestors.

Rebalancing and resharding

Shards fill up; traffic grows; machines are added. Data must move. How gracefully it moves is decided by a choice made on day one.

Modulo-N is a catastrophe

The naive scheme — `shard = hash(key) mod N` — hard-codes the shard count into the placement of every key. Grow from 4 shards to 5 and `hash(k) mod 5` disagrees with `hash(k) mod 4` for 80% of keys: **nearly the entire dataset must move**, all at once, to add 25% capacity. During the move, every key is potentially in transit, so routing is ambiguous everywhere and there is no incremental path — it is a big-bang migration of the whole system, under load, or a maintenance window sized in days. Any design whose placement function takes the live shard count as an input has this defect. Never ship it.

Fixed virtual shards: move whole units

The fix is a level of indirection: hash keys into a **fixed, permanent number of virtual shards** (slots, chunks, tablets — the name varies), chosen once and generously, and separately maintain a small **assignment map** from virtual shard to physical node. Rebalancing then never rehashes anything: it picks whole virtual shards and moves them, updating one map entry per move. Adding a node means transferring it a fair share of vshards from the others — each move is independent, incremental, throttleable, and abortable, and routing stays unambiguous for every key not currently in a moving vshard.

```
flowchart TD
    subgraph B4["Before: 12 vshards on 3 nodes"]
        N1["node A  
v0 v1 v2 v3"]
        N2["node B  
v4 v5 v6 v7"]
        N3["node C  
v8 v9 v10 v11"]
    end
    subgraph AF["After: node D added – only 3 vshards moved"]
        M1["node A  
v0 v1 v2"]
        M2["node B  
v4 v5 v6"]
        M3["node C  
v8 v9 v10"]
        M4["node D  
v3 v7 v11"]
    end
    B4 -->|"move v3, v7, v11"| AF
    AF -->|"update assignment map  
no key rehashed"| AF
```

This design is ubiquitous because it is right:

- **Redis Cluster** fixes the slot space at **16,384 slots** ($\text{CRC16}(\text{key}) \bmod 16384$); every node owns a set of slots, resharding is slot migration, and the `{hash tag}` syntax lets related keys share a slot for multi-key operations. The Redis Cluster specification is a compact, readable description of the whole pattern, including how clients follow `MOVED / ASK` redirections during a live slot migration.
- **Bigtable/HBase-style tablets and CockroachDB/TiDB ranges** are the dynamic variant: instead of a fixed count, contiguous key

ranges **split when they grow or run hot and merge when they shrink**, and a placement layer moves whole tablets between nodes (CockroachDB targets ranges of 512 MiB by default; TiKV regions are ~96 MiB). Same indirection, with the vshard boundaries themselves adaptive — which is what finally tames range sharding's hot tail, by splitting the hot range and shipping half of it elsewhere.

- **Vitess** reshard MySQL by the same logic at coarser grain: shards own ranges of a keyspace-ID space, and a resharding workflow splits a shard by streaming its rows and then its binlog into new shards before atomically switching traffic — which is exactly the playbook we now make explicit.

Sizing note: pick the fixed count generously (it bounds your maximum node count and your rebalancing granularity) but not absurdly, since each vshard carries metadata and per-unit overhead — the same "ceilings, honestly" argument as single-node partition counts.

The online resharding playbook

Whether you are splitting one shard into four or migrating a tenant between shards, every serious online data migration is the same five-phase pipeline. The enabling technology is **change data capture** — tailing the source's WAL or binlog as a stream of row changes (Chapter 7 built the log; Volume 10, Chapter 6 treats CDC as infrastructure) — or, lacking that, application-level dual-writes.


```
sequenceDiagram
    participant App as Application
    participant Src as Source shard
    participant Pipe as CDC pipeline
    participant Dst as New shards
    participant Ver as Verifier

    Note over Src,Dst: Phase 1 – bulk backfill
    Pipe->>Src: snapshot rows, consistent cut
    Pipe->>Dst: copy, throttled
    Note over Src,Dst: Phase 2 – tail the log
    App->>Src: writes continue
    Src->>Pipe: WAL or binlog changes
    Pipe->>Dst: apply until lag near zero
    Note over Ver: Phase 3 – verify
    Ver->>Src: checksum row ranges
    Ver->>Dst: checksum row ranges
    Ver->>Ver: must match, continuously
    Note over App,Dst: Phase 4 – cut over
    App->>Dst: reads shift, canary first
    App->>Dst: writes switch, brief write pause covers lag
    Note over Src: Phase 5 – retire
    App->>Src: kept as rollback target, then dropped
```

1. **Backfill.** Snapshot the source at a consistent point (an MVCC snapshot, Chapter 6) and bulk- copy rows to their new homes, throttled so the source's live traffic is not degraded.
2. **Tail.** Apply the change stream from the snapshot's log position forward, continuously, until replication lag to the new shards is near zero. The new shards are now a live, trailing copy.
3. **Verify.** Compare source and destination — row counts first, then range checksums, then sampled row-level diffs — and keep verifying while tailing continues. Skipping this phase is how silent row loss ships; every mature tool (Vitess's VDiff, for one) makes it a first-class step.
4. **Cut over.** Shift reads first (canary a fraction, compare results, watch error rates), then writes — typically with a seconds-long write pause or reject-and-retry window that lets the tail fully drain, so no write lands on the source after the destination takes over. Update the routing map (directory entry, slot assignment) atomically; this is where the coordination machinery of Volume 6, Chapter 8 earns its keep, because two routers disagreeing about ownership means split-brain writes.

5. **Retire.** Keep the source frozen as an instant-rollback target until confidence is earned — reverse-tailing destination changes back to it if you want true reversibility — then reclaim it.

A runbook checklist, compressed from the scars of teams who have done this under load:

```
RESHARDING RUNBOOK – shard S -> shards S1..Sn
[ ] Capacity: destinations provisioned, replicated, backed up, in
monitoring
[ ] Throttle plan: backfill rate cap agreed; source p99 SLO alarmed
during copy
[ ] Lag budget: max acceptable tail lag at cutover defined (e.g. < 2 s)
[ ] Verification: checksum job green for N consecutive hours before
cutover
[ ] Cutover drill: rehearsed on a staging shard, timed, rollback
rehearsed too
[ ] Freeze window: schema migrations and other DDL frozen for the
duration
[ ] Cutover: reads canaried -> reads switched -> writes paused -> lag
drained
    -> routing map flipped -> writes resumed; each step has an owner
and an abort
[ ] Rollback trigger: explicit error-rate/latency thresholds that
reverse the flip
[ ] Retire: source kept read-only for X days, then decommissioned
```

The operational scar tissue behind that list: backfills that saturated the source's disks and caused the outage they were meant to prevent (throttle, and copy from a replica); cutovers attempted while lag was "only a few seconds" that lost the writes in flight (drain fully, pause writes); verification skipped "because the pipeline is battle-tested" (it was not, for the one table with a type the copier mishandled); DDL applied to the source mid-migration, which the pipeline did not replicate, corrupting the copy silently. Resharding under load is routine at organizations that have industrialized this playbook and a near-death experience everywhere else; the difference is the checklist, not the tooling.

What sharding breaks

Sharding is subtraction. Here is what you give up, catalogued honestly, with the standard responses.

Cross-shard queries become scatter-gather. Any query not filtered by the shard key fans out to all N shards and merges the results. Latency is

the **maximum** of N shard latencies, not the mean — with 100 shards, a query hits a shard's p99 with probability $1 - 0.99^{100} \approx 63\%$, so "every query experiences somebody's bad moment." This is tail-latency amplification, the fan-out pathology of Volume 4, Chapter 9, and it is intrinsic to the shape, mitigable only by hedged requests, partial results, or not fanning out. Aggregations can push work down (sum per shard, combine centrally — partition-wise aggregation again), but top-K, pagination, and joins across a fan-out are genuinely painful.

Cross-shard joins are mostly impossible. A join between rows on different machines means shipping data at query time, and OLTP systems do not do it (analytic engines do, Chapter 13, at analytic latencies). The design responses, in order of preference: **co-partition** tables that join (same shard key → join is local — the distributed twin of partition-wise join); **denormalize** — store the joined shape you need, accepting redundancy, per the data-modeling discipline of Volume 7, Chapter 5; **replicate small reference tables** to every shard so joins against them are always local (Citius calls these reference tables).

Cross-shard transactions stop being free. Within one shard you retain everything Chapters 5–7 built: ACID, MVCC, fast commits. A transaction spanning shards needs two-phase commit or an equivalent protocol, with its blocking window and coordinator failure modes — Chapter 10's subject. The design response is to not need them: **choose the shard key so that the invariants that must be transactional live inside one shard.** Order and its line items share the order's shard; account and its ledger entries share the account's. If two entities must be atomically consistent with each other and routinely land on different shards, the shard key is wrong, not the transaction layer.

Secondary indexes go local or go asynchronous. Chapter 3 set up this dichotomy. A **local** index lives on each shard, indexing only that shard's rows: cheap to maintain (same-shard, same transaction) but useless for routing, so a secondary-key lookup is a scatter-gather. A **global** index is itself sharded by the indexed value, so lookups route directly — but now an index update is a cross-shard write, and systems that refuse to pay 2PC per index entry maintain it **asynchronously**, buying performance with a window where the index lies. DynamoDB's global secondary indexes are the honest, documented version: eventually consistent, full stop. Design consequence: reads through async global

indexes must tolerate staleness and verify against the base row when it matters.

Global uniqueness needs an allocator. "Unique across the system" can no longer be a constraint the database checks, because no single node sees all values — single-node partitioning already previewed this. The answers: derive uniqueness from placement (unique within tenant, tenant in the key); allocate IDs from a service handing out disjoint blocks to each node; or generate **Snowflake-style IDs** — 64 bits composed of a millisecond timestamp, a worker ID, and a per-worker sequence — unique by construction, roughly time-ordered, coordination-free at generation time. The trade-offs (clock skew handling, the hot-tail interaction of time-ordered IDs with range sharding) are real but well understood; what you must not do is pretend a unique index still protects you.

Application, middleware, or natively distributed

Who does the routing? Three architectures, in ascending order of what you delegate.

Application-level sharding. Your code computes the shard, holds a connection pool per shard, and owns the directory, the fan-outs, and the migration tooling. Maximum control and no new infrastructure, but the sharding logic smears into every service that touches the data, cross-shard queries are hand-written, and resharding is a bespoke engineering project each time. Reasonable at small shard counts with a clean tenant key; increasingly expensive as either grows.

Sharding middleware presents one logical database over many real ones. **Vitess** does this for MySQL — a query proxy (vtgate) plans and routes against a sharding-aware schema, and resharding workflows are built in; it ran YouTube and is now CNCF-graduated infrastructure behind PlanetScale, Slack, and others. **Citus** does it as a Postgres extension: distributed tables declared by shard key, reference tables, co-located joins, parallel cross-shard queries. Middleware buys you the accumulated playbooks — the hard 20% (resharding, verification, cutover) is product, not project — at the cost of a new critical-path component and a query surface that is *almost* all of SQL, where the missing sliver is discovered by your application at runtime.

Natively distributed databases — Spanner, CockroachDB, TiDB, and kin — internalize sharding entirely: automatic range splitting, placement,

rebalancing, and distributed transactions behind a SQL interface. That is Chapter 12's subject, including the prices (consensus latency per write, different tail behaviors, operational novelty).

The guidance this volume stands behind: **shard late, but design keys early**. Sharding before the single-node ceiling — after partitioning, read replicas, caching, and buying a bigger machine are exhausted — is premature complexity with permanent costs. But the *schema* must be shardable from early on, because retrofitting a tenant ID into two hundred tables and every query is the multi-year project that makes sharding horror stories. The shardable-schema discipline is cheap and concrete: every tenant-scoped table carries `tenant_id`; primary keys are `(tenant_id, ...)` compounds or globally unique by construction, never bare auto-increments; no query joins across tenants; no invariant spans tenants. A schema with those properties can adopt any of the three architectures later. A schema without them cannot adopt any of them without a rewrite.

The distributed-systems lens

Sharding is lock striping at planetary scale. Volume 4, Chapter 2 partitioned a hot map into N stripes so threads contend only within a stripe; this chapter partitioned a hot dataset into N shards so requests contend only within a shard. It is the same move with the same contract and the same failure modes: the win depends entirely on key distribution; a hot key collapses either design back to a serial bottleneck (the celebrity shard *is* the hot stripe); operations spanning all divisions — `size()` across stripes, scatter-gather across shards — are disproportionately expensive and weakly consistent; and changing the division count is the painful operation both designs must plan for (resizing a striped map, resharding a fleet). If you understood why `ConcurrentHashMap` stripes, and what its `size()` gives up, you already hold the mental model for everything in this chapter — sharding adds partial failure and a network, which is why the playbooks are longer, but subtracts no ideas.

The partition is the unit of scaling and the unit of failure. You add capacity one shard at a time, and you fail one shard at a time. Take that seriously in your operational model: SLOs, alerts, dashboards, capacity plans, and incident response should all be per-shard, because "the database is up" is meaningless when shard 7 is down and 4% of tenants are hard-out while aggregate success rate reads 96%. The blast-radius

benefit of sharding is only realized if your observability and routing can see shards; otherwise you have the failure isolation without the operational isolation, which is the worst quadrant.

The shard key is the physical embodiment of your consistency boundary. Entities that must be transactionally consistent with each other must hash together — which means the shard-key decision is secretly a domain-modeling decision. Volume 15, Chapter 3 introduces the DDD *aggregate*: a cluster of entities with an invariant, modified through one root, consistent within itself, eventually consistent with everything outside. A well-chosen shard key and a well-drawn aggregate boundary are the same boundary seen by an infrastructure engineer and a domain modeler; when the two disagree, the database wins and the domain model discovers its "atomic" operation is now a saga (Chapter 10). Draw the boundary once, deliberately, from the invariants.

Placement is a coordination problem, and the pattern recurs everywhere. The assignment map — which node owns which vshard — is small, hot, and must be agreed upon: two routers with different maps produce split-brain writes during migration. That is why serious systems keep it in a consensus-backed store and fence ownership transfers (Volume 6, Chapter 8). And once you see the pattern — fixed partitions, an assignment map, rebalancing by moving whole partitions, hot-partition pathologies — you will recognize it far from databases: Kafka distributes topic partitions across brokers and rebalances consumer groups with exactly these mechanics and exactly these failure modes (Volume 10), and every stateful service you ever shard by hand will rebuild it again. It is one idea, from `ConcurrentHashMap` to `Spanner`. This chapter's job was to make sure you never mistake it for two.

Key takeaways

- **Partitioning and sharding are different things.** Single-node partitioning is a nearly-free storage-layout and lifecycle tool; sharding is an architectural commitment that removes capabilities. Do not let fear of the second deprive you of the first.
- **Partition time-series tables by range on time, and verify pruning with EXPLAIN .** Pruning fails silently for non-key predicates, type mismatches, and default partitions.

- **The biggest partitioning payoff is $O(1)$ retention:** `DETACH / DROP` replaces a WAL-storm, vacuum-debt `DELETE` with a metadata operation; `ATTACH` does the same for bulk load.
- **Partitioning by one key forfeits global enforcement on others** — unique constraints must include the partition key — and partition counts have honest ceilings (thousands, not tens of thousands; MySQL caps at 8,192).
- **Shard for write throughput past one primary, dataset size past one machine, or blast-radius isolation** — and for nothing else. Sharding does not reduce single-row latency or, by itself, improve availability.
- **The shard key is the decision.** Hash for uniformity (losing range queries), range for locality (risking the hot tail — never lead with a monotonic key), directory for control, compound `tenant + entity` keys for SaaS. Choose it from your queries and your invariants.
- **Skew is when sharding fails.** Watch per-shard dispersion, not aggregates; mitigate with caching, request coalescing, targeted key salting, and whale isolation.
- **Never place with `mod N`.** Hash into fixed virtual shards and rebalance by moving whole vshards with an assignment map — Redis slots, Bigtable tablets, CockroachDB/TiDB ranges, Vitess shards all embody this.
- **Online resharding is a playbook:** throttled backfill, CDC tail, continuous verification, canaried cutover with a drained write pause, rollback-ready retirement. The checklist is the difference between routine and near-death.
- **Sharding breaks** cross-shard queries (scatter-gather = slowest shard), joins (co-partition, denormalize, replicate reference data), transactions (keep invariants inside one shard; else Chapter 10), global indexes (local scatter-gather vs async global), and uniqueness (allocators, Snowflake IDs).
- **Shard late; design keys early.** A shardable schema — tenant ID everywhere, compound keys, no cross-tenant invariants — is cheap insurance; retrofitting one is the multi-year horror story.

Further reading

- Kleppmann, M., *Designing Data-Intensive Applications* (O'Reilly, 2017), Chapter 6 "Partitioning" — the best single treatment of shard-key schemes, skew, rebalancing, and secondary indexes under partitioning.
- Chang, F. et al., "Bigtable: A Distributed Storage System for Structured Data," *OSDI*, 2006 — the range-sharded tablet design, including the row-key locality model this chapter's range section descends from. <https://research.google/pubs/pub27898/>
- Corbett, J. C. et al., "Spanner: Google's Globally-Distributed Database," *OSDI*, 2012 — range sharding plus placement plus distributed transactions, previewing Chapters 10 and 12. <https://research.google/pubs/pub39966/>
- PostgreSQL documentation, "Table Partitioning" — declarative partitioning, pruning, the unique-constraint rule, and the project's own guidance on partition counts. <https://www.postgresql.org/docs/current/ddl-partitioning.html>
- MySQL 8.4 Reference Manual, "Partitioning" — including "Partitioning Keys, Primary Keys, and Unique Keys" for the every-unique-key rule. <https://dev.mysql.com/doc/refman/8.4/en/partitioning.html>
- Redis Cluster specification — the 16,384-slot design, hash tags, and live slot migration with `MOVED` / `ASK` . https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/
- Vitess documentation, "Resharding" — the split-backfill-verify-switch workflow (VReplication, VDiff, SwitchTraffic) as productized playbook. <https://vitess.io/docs/reference/vreplication/>
- Citus documentation — distributed tables, reference tables, and co-located joins on Postgres. <https://docs.citusdata.com/>
- DynamoDB Developer Guide, "Global Secondary Indexes" — the canonical honest documentation of asynchronously-maintained global indexes and their consistency. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html>
- Twitter Engineering, "Announcing Snowflake" (2010) — the original coordination-free ID-allocator design this chapter sketches.

- Volume 4, Chapter 2 — Threads, Mutual Exclusion, and Locks — lock striping, the in-process ancestor of everything here.
- Volume 14, Chapter 5 — Consistent Hashing — the ring, virtual nodes, and the algorithmic depth deferred from this chapter.

Chapter 10 — Distributed Transactions: 2PC, Sagas, and Alternatives

What this chapter covers. Chapters 5 through 7 built up the machinery of the local transaction: atomicity from the commit protocol, durability from the WAL, isolation from locking or MVCC. All of that machinery shares one silent assumption — that there is a single log, in a single engine, on a single machine, whose one forced write is the commit point. This chapter is about what happens when that assumption breaks: when one *logical* operation must update two databases, or a database and a message broker, or three services each owning their own store. The central fact is that **local ACID does not compose**. Two systems that are each perfectly atomic do not add up to one atomic system, and the gap between them is where a crash writes half of your operation into permanent, durable existence.

We start with the canonical trap — the dual write — and show precisely why no ordering of "write the database" and "publish the event" is safe. Then we treat two-phase commit rigorously: the protocol, the durability obligations that make it correct, the recovery matrix, and the fundamental flaw — blocking — that explains why an algorithm proven correct in 1978 is used so sparingly today. We give sagas the same honest treatment: what they guarantee and, more importantly, what they conspicuously do not (isolation), along with the design discipline that makes compensation workable. We introduce the transactional outbox as the standard answer to the dual write specifically, sketch Try-Confirm-Cancel, and close with a decision ladder whose first rung is the one most engineers skip: redesign so you do not need a distributed transaction at all.

Learning goals — after this chapter you should be able to:

- State the dual-write problem precisely and demonstrate the failure interleaving for *both* orderings of database write and event publish.
- Walk the two-phase commit protocol including every forced log write, and explain why a YES vote must be durable and binding before it is sent (Chapter 7 — WAL).
- Reconstruct the recovery matrix: what coordinator and participant each do after a crash at any point in the protocol, and why the coordinator's decision record is the ground truth.
- Explain blocking as 2PC's fundamental flaw — in-doubt transactions, held locks, heuristic outcomes — and why 3PC does not fix it under real network assumptions.
- State precisely how atomic commitment differs from consensus, and why making 2PC non-blocking requires consensus underneath it (Volume 6, Chapters 5-6; Spanner, Chapter 12).
- Define a saga per the 1987 paper, identify the pivot transaction, and apply countermeasures for the isolation a saga does not provide.
- Choose between orchestration and choreography with a defensible rule, and design compensations that are idempotent and semantically achievable.
- Apply the decision ladder: single-shard redesign, outbox plus idempotency, saga, and 2PC — in that order — and justify the ordering.

The problem: local ACID does not compose

Everything in Chapter 5 rested on a single moment: the forced write of the commit record to the WAL. Before that write, a crash means the transaction never happened; after it, recovery replays it to completion. Atomicity is real because there is exactly one bit, in one place, whose durable transition *is* the commit.

Now put a second transactional resource into the operation. An order service writes its PostgreSQL database and publishes an `OrderPaid` event to Kafka. A payment flow debits one bank's ledger and credits another's. A checkout updates the orders shard and the inventory shard — which, after Chapter 9, may be two PostgreSQL clusters that merely look like one logical database. Each resource still has its own commit point, its

own log, its own recovery. But there are now *two* bits in *two* places, and no machine instruction flips both at once. Any protocol you build must issue two separate commits, and between them the process can crash, the network can partition, or the second resource can simply say no.

This is not an edge case; it is the default condition of service-oriented architecture. The moment you split data across services — each owning its store — every business operation that touches two of them has this problem. The transaction machinery of Chapters 5 through 7 stops at the edge of each database, and nothing outside it composes the pieces back together.

The dual write: the canonical trap

The most common instance deserves to be worked in full, because nearly every backend system contains it: **update the database and publish an event, such that both happen or neither does**. The naive implementation writes one, then the other. There are only two orderings, and both are broken.

Ordering A – commit the database first, then publish:

```
t1 BEGIN; UPDATE orders SET status = 'PAID' WHERE id = 42;
COMMIT; -- durable
t2 *** process crashes: OOM-kill, deploy, kernel panic ***
t3 producer.send(OrderPaid{order:
42}) -- never executes
```

Outcome: the database says PAID forever. No event is ever published. Shipping, notifications, analytics – every consumer – never learns the order was paid.

Ordering B – publish first, then commit the database:

```
t1 producer.send(OrderPaid{order:
42}) -- delivered, fanned out
t2 BEGIN; UPDATE orders SET status = 'PAID' WHERE id = 42;
COMMIT; -- fails: crash before commit, serialization failure,
constraint violation
```

Outcome: consumers act on an order the system of record never marked paid.

The warehouse ships goods for a payment that, as far as the database knows, did not happen.

Notice what does *not* fix this. A try/catch cannot help: in ordering A the process is dead before `t3`, and in ordering B the event is already irrevocably fanned out before the failure at `t2` — you cannot catch your way into un-publishing. Retries do not help ordering A, because the process that knew a publish was owed no longer exists; the knowledge died with it. Doing the publish "inside" the database transaction does not help either — the broker is not a participant in the database's commit protocol, so the publish is just a side effect that commits and aborts independently of the transaction wrapping it.

What the dual write actually needs is atomic commitment across two resources — precisely the problem two-phase commit solves, and precisely the machinery your database and your broker do not share. Kafka's transactions, for instance, are real but internal: they make writes to Kafka partitions and consume-transform-produce cycles atomic *within Kafka*; they cannot enroll your PostgreSQL commit. Keep the dual write in mind; the transactional outbox later in this chapter is the standard resolution, and it works not by achieving atomic commitment but by cleverly refusing to need it.

Two-phase commit

Two-phase commit (2PC) is the classic atomic commitment protocol, described by Jim Gray in his 1978 "Notes on Data Base Operating Systems" and independently by Lamson and Sturgis. It is worth learning rigorously because its guarantees and its flaw define the entire design space: every alternative in this chapter is an answer to "2PC, but without the blocking."

The cast: one **coordinator** and N **participants** (resource managers), each participant being a transactional resource that has done work on behalf of the distributed transaction and can locally commit or abort it.

The protocol

Phase 1 — voting. The coordinator sends `PREPARE` to every participant. Each participant decides whether it can commit: it finishes all work, force-writes a *prepare record* to its WAL — containing everything needed to either commit or roll back after a crash — and only then replies `YES`. A participant that cannot commit (constraint violation, deadlock victim, disk

full, or simply too slow) replies `NO`, or the coordinator times out waiting, which counts as `NO`.

Phase 2 — decision. If and only if every participant voted `YES`, the coordinator force-writes a *commit record* to its own log — **this write is the commit point of the entire distributed transaction** — and then sends `COMMIT` to all participants. Any `NO` or timeout, and it writes and sends `ABORT` instead. Each participant applies the decision locally, writes its own outcome record, releases locks, and acknowledges. When all acknowledgments arrive, the coordinator writes an *end* record and forgets the transaction.

```
sequenceDiagram
    participant C as Coordinator
    participant A as Participant A
    participant B as Participant B

    Note over C: phase 1 - voting
    C->>A: PREPARE
    C->>B: PREPARE
    Note over A: force-write prepare record to WAL
    vote is now durable and binding
    A-->>C: VOTE YES
    Note over B: force-write prepare record to WAL
    B-->>C: VOTE YES
    Note over C: force-write COMMIT record
    THE commit point of the
    whole distributed transaction
    Note over C: phase 2 - decision
    C->>A: COMMIT
    C->>B: COMMIT
    Note over A,B: write commit record
    apply, release locks
    A-->>C: ACK
    B-->>C: ACK
    Note over C: write end record, forget transaction
```

Prepared means promised: the durability obligations

The protocol's correctness hangs entirely on two forced writes, and it is worth dwelling on each because both are Chapter 7 in action.

The participant's prepare record. A `YES` vote is not an opinion; it is a binding, permanent promise: *I will commit this transaction if told to, no matter what happens to me in between.* That promise must survive a crash, so before voting the participant must have the transaction's redo

and undo information durably on disk — force-written, `fsync` and all, exactly the WAL discipline of Chapter 7 — such that after a restart it can still go either way. A prepared transaction is a strange object in recovery: unlike ordinary in-flight transactions, which recovery simply rolls back, it must be *reinstated* — locks re-acquired, state held — because the participant gave away the right to decide its fate when it voted. The same logic binds a live participant whose commit message is slow: unilaterally aborting would break the promise, and unilaterally committing might contradict a coordinator that decided `ABORT` because some *other* participant voted `NO`.

The coordinator's decision record. The coordinator force-writes `COMMIT` or `ABORT` *before* sending any phase-2 message. That log record is the ground truth of the transaction's outcome — the single bit we lacked at the start of the chapter, relocated into the coordinator's WAL. If the coordinator crashes after writing it, recovery reads the log and re-sends the decision; participants that already applied it treat the duplicate as a no-op. If it crashes before writing it, no participant can have received a decision, so recovery may safely abort. That asymmetry is the whole design: every possible crash lands on a well-defined side of one durable write. (A standard refinement, *presumed abort*, exploits it: if recovery finds no record of a transaction at all, the answer is abort — so abort decisions and their acknowledgments need not be logged. Real implementations run *presumed abort*.)

State machines and recovery

Both roles are small state machines, and writing them down makes the recovery rules mechanical rather than mysterious.

Coordinator state	Entered by	Durable record	On timeout	On crash + recovery
<code>INIT</code>	transaction begins	none	—	abort silently (presumed abort)
<code>WAITING</code>	sent <code>PREPARE</code>	none required	decide <code>ABORT</code>	abort: no decision was logged

Coordinator state	Entered by	Durable record	On timeout	On crash + recovery
COMMITTED	all voted YES	commit record, forced	re-send COMMIT	re-send COMMIT until all ack
ABORTED	any NO / timeout	abort record (lazy under presumed abort)	re-send ABORT	nothing owed; participants will ask
DONE	all acks received	end record	—	forget

Participant state	Entered by	Durable record	May unilaterally abort?	On crash + recovery
WORKING	doing the transaction's work	ordinary WAL	yes	roll back, as any in-flight txn
PREPARED	force-wrote prepare, voted YES	prepare record, forced	no	reinstate: hold locks, ask coordinator for outcome
COMMITTED	received COMMIT	commit record	—	nothing; ack if asked again
ABORTED	received ABORT or voted NO	abort record	—	nothing

The recovery matrix falls straight out of the tables. Participant crashes before writing its prepare record: recovery rolls the transaction back like any other in-flight work, and its missing vote becomes a coordinator timeout, hence a global abort — safe, because it never promised anything. Participant crashes after preparing: recovery reinstates the prepared transaction and asks the coordinator how it ended. Coordinator crashes before logging a decision: recovery aborts; no participant can have heard otherwise. Coordinator crashes after logging the decision: recovery re-drives phase 2 from the log. Every cell is covered — as long

as the coordinator eventually comes back. That proviso is the next section.

The fundamental flaw: blocking

Look again at the participant table, at the `PREPARED` row, at the word **no**. A prepared participant has surrendered the right to decide. Now let the coordinator crash — or become unreachable, which is indistinguishable — after the votes are in but before the decision reaches anyone.

```
sequenceDiagram
    participant C as Coordinator
    participant A as Participant A
    participant B as Participant B

    C->>A: PREPARE
    C->>B: PREPARE
    A-->>C: VOTE YES
    B-->>C: VOTE YES
    Note over C: CRASHES - possibly before,
    possibly after logging a decision
    Note over A: in doubt
    cannot abort: C may have logged COMMIT
    cannot commit: another vote may have been NO
    A->>B: cooperative termination:
    do you know the outcome?
    B-->>A: no - I am in doubt too
    Note over A,B: both hold every lock the prepared
    transaction took, indefinitely,
    until the coordinator returns
```

The prepared participants are **in doubt**. Each one reasons: "I voted YES, so the coordinator *may* have logged COMMIT — I cannot abort. But some other participant *may* have voted NO — I cannot commit." Both inferences are sound, so the participant can do nothing but wait. Cooperative termination — asking the other participants — helps only if one of them actually received the decision; if all are in doubt, the protocol is stuck. And "stuck" is not idle: a prepared transaction holds its write locks (Chapter 6), so every ordinary local transaction touching those rows queues behind a distributed transaction that cannot finish. In PostgreSQL an orphaned prepared transaction (visible in `pg_prepared_xacts`) additionally pins the xmin horizon, so VACUUM reclaims nothing newer — Chapter 6's long-running-transaction disease, inflicted by a *dead* coordinator. A crashed coordinator does not just delay one transaction; it degrades the participants for unrelated work.

This is not an implementation defect to be patched. It is the essential property of the protocol: **2PC buys atomicity across resources by making every participant hostage to the coordinator's availability during the prepare window.** Gray knew it in 1978; the literature calls 2PC a *blocking* protocol for exactly this reason.

Practice could not always wait, so XA gave operators a hatch: **heuristic resolution.** A human — or a resource manager configured with a timeout — unilaterally commits or rolls back an in-doubt branch (`XA COMMIT / XA ROLLBACK` on a recovered branch in MySQL, `COMMIT PREPARED / ROLLBACK PREPARED` in PostgreSQL). The word "heuristic" means *guess*. When the coordinator finally returns with the real decision and it disagrees, the outcome is heuristic-mixed — some branches committed, some rolled back — and atomicity, the one property the protocol existed to provide, is silently gone. XA even has error codes for this (`XA_HEURCOM` , `XA_HEURRB` , `XA_HEURMIX`), which tells you how routine it was. The damage is the worst kind: discovered later, by reconciliation jobs or customers.

What 2PC costs even when nothing fails

Set the crashes aside; the sunny-day protocol is expensive too.

Latency. Two sequential network round trips, plus at least two forced-write points on the critical path: every participant's prepare record, then the coordinator's decision record — each an `fsync` (Chapter 7 taught you what those cost). Each phase waits for the *slowest* participant, so the distributed commit's p99 is roughly the max of the participants' p99s, plus the coordinator's write, plus the round trips.

Availability. A 2PC transaction can commit only when the coordinator *and every participant* are simultaneously up and reachable. Availability multiplies: five participants at 99.5% each, with a 99.5% coordinator, yields roughly $0.995^6 \approx 0.970$ — about 2.6 days of unavailability a year for the composed operation, an order of magnitude worse than any single component. This is the precise sense in which distributed transactions "don't scale": each added participant multiplies the failure probability and takes a max over latencies, while the blocking window puts every participant at the mercy of any one of them.

XA, and why app-server XA fell out of favor

X/Open XA (1991) standardized the interface between a *transaction manager* (the coordinator) and *resource managers* (databases, queues), so that heterogeneous products could participate in one 2PC. The Java mapping (JTA) put the transaction manager inside the application server — WebLogic, WebSphere, JBoss — and for a decade enterprise architecture assumed a container coordinating XA across Oracle and MQ Series.

It fell out of favor for reasons that follow directly from the analysis above, not from fashion. The coordinator lived in the application server: a stateful, singular process embedded in the tier that gets redeployed most often — the worst possible home for the one component whose crash blocks everyone and whose log is needed for recovery. That log had to survive restarts and be reattached to a recovering instance, which fit badly with stateless horizontal scaling and, later, with containers that vanish rather than restart in place. Drivers' XA code paths were less exercised and buggier than their local-transaction paths. Heuristic outcomes turned "atomic" into "atomic, usually." And the newer resources people wanted to enroll — HTTP APIs, Kafka, Redis, object stores — never spoke XA at all. None of this means 2PC is dead: it remains alive and correct *inside* single administrative domains with purpose-built infrastructure — cross-shard transactions in distributed SQL engines (Chapter 12), MySQL's internal 2PC between InnoDB and the binlog, Kafka's transaction coordinator. What died was 2PC as *application architecture* across heterogeneous, independently operated systems.

3PC, and the road that leads to consensus

The obvious question — can atomic commitment be made non-blocking? — was tackled by Dale Skeen in 1981. Three-phase commit inserts a *pre-commit* round between voting and commit, so that no participant ever commits while another could still be in a state that permits abort; surviving participants can then always decide among themselves after a coordinator crash. It is a genuine theoretical contribution, and it essentially never shipped, because its non-blocking guarantee holds only under assumptions reality declines to provide: crash-stop failures, a synchronous network with bounded delay, and reliable failure detection — no partitions. Under a real network partition, 3PC can split into two groups that each "safely" terminate the protocol with opposite decisions,

which is strictly worse than blocking. A protocol that trades *stuck* for *wrong* is not an upgrade.

The insight hiding in 3PC's failure is the segue to the modern answer: what 3PC was groping for — a set of nodes reaching one durable decision that survives the failure of any minority — *is consensus*, which is solvable under realistic assumptions with Paxos or Raft (Volume 6, Chapters 5 and 6). Gray and Lamport made the connection exact in "Consensus on Transaction Commit" (2006): 2PC is the degenerate case of Paxos Commit with a single acceptor — and that single acceptor is exactly why it blocks. Replace the singular coordinator's decision with a consensus decision among $2F+1$ nodes and the protocol tolerates F failures without blocking. This is precisely Spanner's architecture (Corbett et al., 2012; Chapter 12): each participant *and the coordinator* is a Paxos-replicated group rather than a machine, so 2PC still runs between groups, but "the coordinator crashed" stops being a singular event — the group elects a new leader that reads the decision state from the replicated log and drives the protocol onward. The blocking flaw was never about 2PC's message pattern; it was about unreplicated decision state. Fix the replication and 2PC is rehabilitated — at the price of running consensus, which is Volume 6's subject.

Sagas

If 2PC's price is unacceptable and the operation still spans resources, the alternative is to stop pretending the operation is one transaction and engineer it as several. Hector Garcia-Molina and Kenneth Salem named this pattern **sagas** in 1987 — originally for *long-lived transactions* inside a single database, where holding locks for a multi-hour batch job was intolerable. The microservices world rediscovered the paper because its problem is structurally identical: a multi-step business operation where holding cross-resource locks for the duration is impossible.

A saga is a sequence of local transactions $T_1, T_2, \dots, T_n$, each atomic in its own resource, where each step T_i (up to the last) has a **compensating transaction** C_i that semantically undoes it. The guarantee: either the saga runs $T_1 \dots T_n$ to completion, or it runs $T_1 \dots T_j$ followed by $C_j, C_{j-1}, \dots, C_1$ — forward to the end, or backward to the start, with every step's effect eventually cancelled. (The paper also defines *forward recovery* — persist enough state to retry onward from the

failure point rather than unwinding — which is what modern workflow engines call "just retry the activity.")

```
flowchart TB
    T1["T1 create order  
state PENDING - semantic lock"] --> T2["T2 reserve inventory"]
    T2 --> T3["T3 charge payment  
PIVOT - last compensatable step"]
    T3 --> T4["T4 arrange shipment  
retriable - after the pivot  
only forward, retry until success"]
    T4 --> OK["T5 mark order CONFIRMED"]

    T2 -->|"reservation fails"| C1["C1 cancel order  
state REJECTED"]
    T3 -->|"payment declined"| C2["C2 release inventory"]
    C2 --> C1
    T4 -->|"failure here does NOT compensate  
money is taken - roll FORWARD"| T4
```

What a saga is not: the missing I

Here is the sentence to tattoo somewhere visible: **a saga is not a transaction**. It preserves a weak form of atomicity — eventually, all-or-compensated — and each step is individually durable. It provides **no isolation whatsoever**. Between **T1** and the saga's end, every intermediate state is committed and visible to the entire world: other transactions, other sagas, users. Other work reads and *writes against* your intermediate states, and these are not even dirty reads you could blame on an isolation level (Chapter 6) — every one of them is a committed local transaction.

The anomalies are concrete. *Lost updates between steps*: the inventory your saga reserved in **T2** is released by a concurrent cancellation saga that read stale order state, and your **T4** ships inventory that no longer exists. *Reads of doomed state*: an analytics query counts the order that **C1** will cancel two seconds later. *Interleaved sagas*: two sagas each complete their first steps, then each fail on the other's intermediate state, then each compensate.

Since the database will not give a saga isolation, the application must buy back what it needs with **countermeasures** (the term is Chris Richardson's, in *Microservices Patterns*):

- **Semantic locks / pending states.** `T1` writes `status = PENDING`, not `status = CONFIRMED`, and every other piece of code treats `PENDING` as "hands off, in flux" — declining to ship, count, or modify it. This is an application-level lock encoded as data, unlocked by the saga's completion or compensation. It is the single most important saga discipline, and it is why well-designed saga systems are full of statuses like `RESERVED` and `PENDING_SHIPMENT`.
- **Commutative updates.** Where possible, make steps commute so interleaving cannot corrupt: `balance = balance - 40` commutes with a concurrent `- 25`; `balance = 960` does not. Design for arithmetic, not assignment.
- **Re-read and verify.** A later step re-checks the invariant it depends on — optimistic concurrency at the workflow level — rather than trusting a value read three steps and four seconds ago.
- **Order the steps by risk — and know your pivot.** The **pivot transaction** is the step after which the saga can no longer compensate, because a step has no meaningful undo or a business commitment has escaped (money captured, goods shipped, email sent). Steps before the pivot are *compensatable*; steps after it must be **retriable** — designed to succeed eventually, retried forever if need be — because the only direction left is forward. Put the steps most likely to fail earliest and the pivot as late as possible: validate, reserve, and authorize before you capture, ship, or notify. A saga that puts "send confirmation email" before "charge card" has chosen its regret in advance.

Designing compensations

Compensation is where saga designs are won or lost, and three properties are non-negotiable.

Semantic undo, not restore. `Ci` does not reset rows to their prior bytes — other transactions have long since seen and built on `Ti`'s output, and a byte-level restore would be a lost update against *them*. `Ci` is a new forward-moving business action that cancels `Ti`'s *meaning*: a refund, not a deleted charge row; a reservation release, not a restored quantity snapshot. And some actions have no semantic undo at all — you cannot

un-send an email, un-ship a package, or un-call a third-party API with no cancellation endpoint. Every such action is a forced pivot: it must sit after the real pivot in the ordering, or the saga must accept apology-based compensation ("we emailed you in error"), which is a business decision, not a technical one.

Idempotent. The orchestrator will crash mid-compensation and re-run `Ci`; the network will deliver its request twice. `Ci` must therefore be safe to apply repeatedly — refund *this charge id* (a no-op the second time), not "refund \$50." The same is true of every forward step `Ti`, for the same reason: sagas live on retries, and retries demand idempotency keys end to end (Volume 6, Chapter 9 treats idempotency fully).

Must eventually succeed. A compensation has no compensation. If `C2` fails, the saga cannot "roll back the rollback" — it can only retry `C2` until it succeeds, which is why compensations should be simple, dependency-light, and free of business-rule rejections (a refund that can be *declined* is not a compensation, it is another step that can fail). When a compensation fails persistently — the refund API is down for a day — the saga parks in a dead-letter state and **escalates to a human**, with enough persisted context to resume. Every production saga system has this drawer of stuck sagas; the only question is whether you built it deliberately, with alerting, or are discovering it in an incident.

Orchestration versus choreography

Someone has to know the saga's definition — the sequence, the compensations, the current position. The two answers differ in where that knowledge lives.

Orchestration makes it explicit: a coordinator — a persistent state machine — invokes each step, records the result durably, and decides what happens next. The saga's definition is code you can read in one place, and the saga's *state* is a row you can query. This is conceptually what Temporal and AWS Step Functions provide as infrastructure: the engine persists the state machine's position (Temporal by durably logging the workflow's event history and replaying it to reconstruct state; Step Functions as a managed state-machine execution), so the orchestrator is not a singular process whose crash loses the saga — the pattern's own coordinator-death problem, solved by making coordinator state durable and resumable rather than by consensus.

```

# Orchestrated saga as a persisted state machine (conceptual pseudo-
code).
# The framework guarantees: state transitions are durable before
effects are
# visible, every step call carries an idempotency key, and a crashed
# orchestrator resumes from the last recorded transition.

class CreateOrderSaga:
    steps = [
        #      forward action          compensation
        Step(create_order,              cancel_order),      # writes
status=PENDING
        Step(reserve_inventory,         release_inventory),
        Step(charge_payment,            None),              # PIVOT:
capture; no undo past here
        Step(arrange_shipment,          None),              #
retriable: retry until success
        Step(confirm_order,              None),              # writes
status=CONFIRMED
    ]

    def run(self, ctx):
        for i, step in enumerate(self.steps):
            self.record(ctx, position=i, phase="attempting")      #
durable BEFORE acting
            try:
                step.action(ctx, idempotency_key=(ctx.saga_id, i))
            except RetriableError:
                raise Retry(backoff=True)                          #
same step, same key
            except BusinessFailure:
                if self.past_pivot(i):
                    raise Retry(backoff=True)  # no way back: forward-
only, escalate on budget
                return self.compensate(ctx, from_step=i - 1)
            self.record(ctx, phase="completed")

    def compensate(self, ctx, from_step):
        for i in range(from_step, -1, -1):
            if self.steps[i].compensation is None:
                continue
            self.record(ctx, position=i, phase="compensating")
            retry_forever_then_escalate(
                self.steps[i].compensation, ctx,
                idempotency_key=(ctx.saga_id, i, "comp"),
                escalation="page team; park saga in NEEDS_ATTENTION")
            self.record(ctx, phase="compensated")

```

Choreography distributes the knowledge: there is no coordinator, and each service reacts to events. Order service emits `OrderCreated`; inventory hears it, reserves, emits `InventoryReserved`; payment hears that, charges, emits `PaymentCompleted` or `PaymentFailed`; inventory hears `PaymentFailed` and releases. The saga *emerges* from the subscriptions. For a short linear flow this is attractively decoupled. The costs arrive with scale and time. The saga's definition exists nowhere; to learn what the checkout flow actually does, you grep event subscriptions across a dozen repositories and reconstruct the graph by hand — archaeology, not architecture. Failure paths are the worst of it: *who compensates*? Each service must subscribe to the failure events of every step *after* its own and know to undo itself — a web of reverse-direction couplings that is combinatorially worse than the forward flow, rarely tested, and silently wrong when a new step is inserted and nobody updates the predecessors' subscriptions. And there is no single place to observe a stuck saga, because no component knows the saga exists.

The guidance follows from the failure modes, not from taste: **choreograph short, linear, stable sagas — two or three steps whose compensation story is trivial. Orchestrate anything long, branching, or evolving.** The moment you catch yourself drawing the event graph on a whiteboard to answer "what happens if payment fails after partial shipment," the diagram is telling you it wants to be code in an orchestrator.

Timeouts, and the saga that never ends

A saga has no lock manager and no deadlock detector to kill it (Volume 4, Chapter 5's detector has no distributed analogue here), and no coordinator timeout aborts it automatically. A saga stalls silently — a step's request lost, a consumer down — and the order sits `PENDING` forever, holding its semantic locks exactly as a prepared 2PC participant holds real ones. The in-flight-forever saga is the pattern's version of the in-doubt transaction, softened only in that the locks are advisory. Two disciplines close the gap: every saga carries a **deadline**, after which the orchestrator (or a watchdog, for choreography) forces a decision — compensate if before the pivot, escalate if after — and saga state is **observable**: a queryable table of live sagas, their positions, and their ages, with an alert on age. If you cannot list your in-flight sagas, you already have some you do not know about.

The transactional outbox

Return now to the dual write, because the standard fix is elegant enough to state in one sentence: **if you cannot atomically write two systems, write one system twice — atomically — and let an asynchronous process convey the second write to the second system.**

The service writes the business row *and* a row describing the event into an `outbox` table **in the same local ACID transaction** — one database, one commit point, Chapter 5 machinery, nothing distributed. A separate **relay** then reads committed outbox rows and publishes them to the broker, marking them as sent. If the process crashes after commit, the event is not lost — it is sitting durably in the outbox, and the relay publishes it after recovery. If the transaction aborts, the outbox row aborts with it and no event escapes. Both dual-write interleavings are dead: the event and the state change share a single commit.

```
flowchart LR
    SVC["order service"] --> TXN["one local ACID transaction"]
    TXN --> UPDATE["UPDATE orders SET status = PAID"]
    TXN --> INSERT["INSERT INTO outbox VALUES ..."]
    TXN --> DB["PostgreSQL"]
    DB --> WAL["orders + outbox + WAL"]
    WAL --> DB
    DB --> POLL["poll: SELECT unpublished"]
    POLL --> CDC["CDC: read the WAL"]
    CDC --> RELAY["RELAY"]
    RELAY --> PUBLISH["publish, then mark sent"]
    PUBLISH --> BROKER["BROKER"]
    BROKER --> CON["consumer"]
    CON --> IDEMPOTENT["idempotent - dedup on event_id"]
```

```

CREATE TABLE outbox (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_id    uuid      NOT NULL DEFAULT gen_random_uuid(),
-- consumer dedup key
  aggregate_type text    NOT NULL,          -- 'order'
  aggregate_id text    NOT NULL,          -- '42' -> broker
partition key
  event_type  text      NOT NULL,          -- 'OrderPaid'
  payload     jsonb     NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  published_at timestamptz          -- NULL = not yet
  relayed
);

-- Relay loop (polling variant), safe to run in one instance per
aggregate hash:
BEGIN;
SELECT id, event_id, aggregate_id, event_type, payload
FROM   outbox
WHERE  published_at IS NULL
ORDER BY id
LIMIT 100
FOR UPDATE SKIP LOCKED;          -- competing relays skip, not block
(Chapter 6)
-- ... publish each row to the broker, keyed by aggregate_id ...
UPDATE outbox SET published_at = now() WHERE id = ANY (:published_ids);
COMMIT;

```

Two consequences are load-bearing. First, delivery is **at-least-once**, irreducibly: the relay can crash after publishing and before marking `published_at`, and will re-publish on restart. The outbox therefore *requires* its other half — **idempotent consumers** that deduplicate on `event_id` (Volume 6, Chapter 9); deploying the outbox without consumer idempotency merely trades missing events for duplicate ones. Second, ordering is per aggregate at best: publish keyed by `aggregate_id` so one entity's events land in one partition in order, and note that `SKIP LOCKED` with competing relays can reorder across batches — if per-aggregate order matters, shard relays by aggregate rather than racing them.

The polling relay's tail latency and load have a sharper alternative: a **CDC-based relay** (Debezium is the canonical one) tails the database's WAL through the replication protocol — literally the logical-decoding interface of Chapters 7 and 8 — and streams committed outbox inserts to the broker with no polling at all. The commit record in the WAL is the event's release: the same forced write that made the transaction durable

makes the event eligible for publication. Relay operations, topic routing, and outbox hygiene get their full treatment in Volume 10, Chapter 6; event sourcing — where the log stops being a side-channel and becomes the store — is Volume 10, Chapter 5.

Try-Confirm-Cancel, briefly

TCC recurs in payments architectures and deserves a short honest note. It is 2PC lifted out of the database and into the business layer: **Try** executes a *reservation* — an application-level prepare that sets aside resources in a business-visible pending state (an authorization hold on a card, a seat hold with a 10-minute expiry) — then **Confirm** makes all reservations final, or **Cancel** releases them. The improvement over XA is that the "prepared" state is not a database lock but a first-class business object with a **timeout**: an unconfirmed hold expires on its own, so a dead coordinator inconveniences rather than blocks. The costs: every participating service must design and expose all three operations, Confirm and Cancel must be idempotent (the coordinator retries them), and expiry races confirmation — the hold that expires just as Confirm arrives must be handled explicitly. TCC is best understood as a saga whose first phase is deliberately shaped like 2PC's prepare — reservations instead of locks, expiry instead of blocking.

Choosing honestly: the decision ladder

Given a business operation that appears to need a distributed transaction, work this ladder from the top; each rung is strictly cheaper to operate than the one below it.

Rung 1 — redesign so the invariant lives in one shard. Most "distributed transaction problems" are Chapter 9 shard-key problems wearing a disguise. If checkout must atomically touch the order and its line items, the answer is not a cross-shard protocol; it is sharding by `order_id` so they colocate. If a transfer must atomically touch two accounts, ask whether the *invariant* is really per-pair, or whether an append-only ledger with balances derived asynchronously serves the business — banks themselves settle asynchronously with reconciliation, the world's oldest saga. Aggregate boundaries in the domain-driven-design sense are exactly this: the unit within which invariants must hold transactionally, and therefore the unit that must not be split across

stores. The cheapest distributed transaction is the one your data model made unnecessary.

Rung 2 — outbox plus idempotent consumers, for database-plus-messaging. If the second "resource" is a broker — which covers the majority of real cases — the dual write is the actual problem, and the outbox is its complete solution. No new infrastructure beyond a table and a relay; failure modes are duplicates, not inconsistency.

Rung 3 — a saga, for genuine multi-service workflows. When distinct services must each commit local state, model the flow as a saga — orchestrated unless it is trivially short — with pending states, a consciously placed pivot, idempotent compensations, deadlines, and an escalation drawer. Accept, explicitly and in writing, that intermediate states are visible and that the anomaly countermeasures are now part of your domain model.

Rung 4 — 2PC, within one administrative domain, on infrastructure built for it. Cross-shard transactions inside a distributed SQL engine whose commit protocol runs over replicated coordinators (Chapter 12), or a mature XA deployment you operate yourself with tested recovery runbooks. What earns 2PC its place on the ladder at all is strength: it *does* provide atomicity across resources, which no rung above it does. What puts it last is everything this chapter established — blocking, availability multiplication, heuristics — costs manageable inside one team's blast radius and unmanageable across organizational boundaries.

The distributed-systems lens

Atomic commitment and consensus are different problems — be precise about which one 2PC solves. 2PC solves atomic commitment: *every* participant must agree, because any single participant's NO is a veto — its local integrity constraint failed, and committing over a veto would corrupt it. Consensus (Volume 6, Chapters 5–6) requires only a *majority*, and therefore keeps deciding while a minority is down. The requirements shape the failure behavior: a protocol that needs everyone can be stalled by anyone, which is 2PC's blocking; a protocol that needs a majority tolerates a minority's silence, which is Paxos's liveness. They compose rather than compete — Spanner runs atomic commitment *between* groups and consensus *within* them. The slogan version: 2PC gets

unanimity about a veto-able decision; consensus gets a fault-tolerant decision. Confusing them is how 3PC happened.

A saga is structured concurrency with compensation, at workflow scale. Volume 4, Chapter 8's discipline — every task has an owner; a scope does not exit until its children resolve; failure of one child triggers deliberate cancellation of its siblings; nothing leaks — is exactly the orchestrated saga's discipline with "task" replaced by "local transaction" and "cancel" replaced by "compensate," because committed work cannot be cancelled, only counteracted. The orchestrator is the scope; the in-flight-forever saga is the leaked goroutine. Choreography, in this frame, is unstructured concurrency — fire-and-forget with the ownership question unanswered — and it earns the same verdict for the same reasons.

Exactly-once delivery is an end-to-end fiction; at-least-once plus idempotency is the real contract. Every hop in this chapter — 2PC's phase-2 retries, the saga's step retries, the outbox relay's re-publish after crash — delivers duplicates under precisely the failures it is designed to survive, because "did my effect land?" is unanswerable at the moment of timeout and retrying is the only safe response. Systems that advertise exactly-once (Kafka transactions, Temporal) achieve exactly-once *processing* within a boundary they control, by — look inside — deduplicating at-least-once delivery with persistent identifiers. The contract worth writing down: every effect carries an idempotency key, every consumer deduplicates, and delivery is assumed repeated (Volume 6, Chapter 9).

Atomicity across a partition boundary costs availability — so design the boundaries. The availability product and the blocking window are not implementation warts; they are the price any protocol pays for refusing to complete until multiple failure domains agree. You choose where to pay it when you choose where transactional boundaries fall: inside a shard, atomicity is a forced log write; across shards, it is a protocol with hostages. This is Pat Helland's 2007 argument in "Life beyond Distributed Transactions": scale-out systems end up with *entities* — single-shard units of atomicity — and *activities* between them that manage uncertainty with messages, retries, and apologies. That is this chapter's ladder, derived from first principles: atomicity is cheap inside a boundary and expensive across one, so put the boundaries where the invariants are.

Key takeaways

- **Local ACID does not compose.** Two resources means two commit points, and no instruction flips both. The dual write is the canonical trap: commit-then-publish loses events on crash; publish-then-commit emits events for state that never committed. Neither ordering is safe.
- **2PC relocates the commit point** into the coordinator's log: participants force-write prepare records that make their YES votes durable and binding (Chapter 7), and the coordinator's forced decision record is the ground truth. Every crash lands on a defined side of that one write.
- **2PC's fundamental flaw is blocking.** A prepared participant can neither commit nor abort unilaterally; a dead coordinator leaves participants in doubt, holding locks that stall unrelated work. XA's heuristic outcomes trade the stall for silently broken atomicity. Even sunny-day 2PC costs two round trips plus forced writes, and availability multiplies across participants.
- **3PC never shipped because it needs a synchronous, partition-free world.** Making atomic commitment non-blocking under real assumptions requires consensus — Gray and Lamport's Paxos Commit, realized in Spanner's 2PC-over-Paxos-groups (Volume 6, Chapters 5-6; Chapter 12).
- **A saga is a sequence of local transactions with compensations — and no isolation.** Intermediate states are committed and visible; buy back safety with pending states, commutative updates, re-verification, and a late, consciously placed pivot, after which the saga only rolls forward.
- **Compensations must be semantic, idempotent, and unable to be refused** — and some actions (sent email, shipped goods) have no undo, which makes them forced pivots. Failed compensations retry, then escalate to a human; build that drawer before the incident does.
- **Orchestrate long or branching sagas; choreograph only short, stable, linear ones.** Every saga needs a deadline and an observable state table.
- **The transactional outbox solves the dual write** by making event and state share one local commit, with a polling or CDC relay publishing afterward. Delivery becomes at-least-once by

construction, so idempotent consumers are the other half of the pattern, not an option.

- **Work the ladder in order:** colocate the invariant in one shard (most distributed-transaction problems are shard-key problems); outbox plus idempotency for DB-plus-messaging; saga for real multi-service workflows; 2PC only inside one administrative domain on infrastructure built for it.
- **Atomic commitment is not consensus:** unanimity with vetoes versus majority with fault tolerance — different problems that compose. And exactly-once is a fiction at the boundary: the real contract everywhere is at-least-once delivery plus idempotent effects.

Further reading

- Gray, J., "Notes on Data Base Operating Systems," in *Operating Systems: An Advanced Course*, Springer LNCS 60, 1978 — the original systematic treatment of transactions, recovery, and two-phase commit.
- Garcia-Molina, H. and Salem, K., "Sagas," *SIGMOD*, 1987 — the source, including compensation and forward/backward recovery; short and very readable. <https://dl.acm.org/doi/10.1145/38713.38742>
- Skeen, D., "Nonblocking Commit Protocols," *SIGMOD*, 1981 — three-phase commit and the formal analysis of blocking.
- Gray, J. and Lamport, L., "Consensus on Transaction Commit," *ACM TODS* 31(1), 2006 — the precise bridge between atomic commitment and consensus; 2PC as degenerate Paxos Commit. <https://dl.acm.org/doi/10.1145/1132863.1132867>
- Corbett, J. et al., "Spanner: Google's Globally-Distributed Database," *OSDI*, 2012 — 2PC layered over Paxos groups in production. <https://research.google/pubs/spanner-googles-globally-distributed-database-2/>
- Kleppmann, M., *Designing Data-Intensive Applications*, O'Reilly, 2017 — Chapter 9's treatment of 2PC, XA, and the commit/consensus relationship is the best short published account.
- Helland, P., "Life beyond Distributed Transactions: an Apostate's Opinion," *CIDR*, 2007 — entities, activities, and why scale-out systems abandon cross-entity transactions.

- Richardson, C., *Microservices Patterns*, Manning, 2018, and <https://microservices.io/patterns/data/saga.html> — sagas for services, the countermeasure vocabulary, orchestration versus choreography.
- X/Open, *Distributed Transaction Processing: The XA Specification*, 1991 — the interface underneath JTA and XA PREPARE / XA COMMIT, heuristic outcomes included.
- Temporal documentation — <https://docs.temporal.io/> — durable-execution workflow orchestration; see the concepts section on workflows, activities, and retries for the modern orchestrator in practice.
- Debezium documentation, "Outbox Event Router" — <https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html> — the CDC-based outbox relay in deployable form (depth in Volume 10, Chapter 6).
- Volume 5: Chapter 7 — WAL (the forced writes this chapter leans on); Chapter 9 — Partitioning and Sharding (rung 1); Chapter 12 — NewSQL (Spanner and friends in full).
- Volume 6: Chapters 5-6 — consensus; Chapter 9 — idempotency and delivery semantics.
- Volume 4: Chapter 8 — structured concurrency, the single-process shape of the orchestrated saga.

Chapter 11 — NoSQL: Key-Value, Document, Wide-Column, and Graph

What this chapter covers. Ten chapters into this volume you have seen how a database stores (Chapter 2), indexes (Chapter 3), isolates (Chapter 6), logs (Chapter 7), replicates (Chapter 8), partitions (Chapter 9), and coordinates across partitions (Chapter 10). This chapter is about the family of systems that, starting in the mid-2000s, deliberately gave up parts of that machinery to buy scale, availability, or modeling flexibility — and about what two decades of production experience have taught us about which of those trades were worth making. We take the history seriously, because the history explains the designs: two papers, Google's Bigtable (2006) and Amazon's Dynamo (2007), founded nearly every lineage in this space, and each system's behavior is best predicted by

knowing which parent it descends from. We then work through the four data models — key-value, document, wide-column, graph — with real mechanics and real exemplars, not category slogans. The stance throughout is the one worth internalizing: **these categories describe data models and trade-off points, not tribes**. The interesting question about any store is never "is it NoSQL?" but "what does it make cheap, what does it make expensive, and what does it hand back to my application to solve?"

The distributed-systems lens is central rather than appended: the deepest thing NoSQL did was surface the distributed trade-offs that the single-node RDBMS had hidden, and hand partition-key design — which is distributed-systems design — to the application developer. Quorum mechanics get their full treatment in Volume 6, Chapter 7; this chapter previews them only far enough to read a Cassandra consistency level honestly.

Learning goals — after this chapter you should be able to:

- Explain *why* NoSQL happened — the operational misery of hand-sharded RDBMS at 2000s web scale — and trace the two founding lineages from the Bigtable and Dynamo papers to today's systems.
- Describe the mechanics of each of the four models, including Redis's persistence trade-offs, DynamoDB's capacity and partition model, Cassandra's partition/clustering layout, MongoDB's atomicity contract, and index-free adjacency in graph stores.
- Practice query-first modeling: design a Cassandra table or a DynamoDB key schema *from* an access pattern, and make the embed-versus-reference decision in a document store deliberately.
- State honestly what each model gives up — secondary indexing, ad-hoc aggregation, cross-entity invariants — and where those costs reappear in application code.
- Apply a decision framework that starts from access patterns and consistency invariants, treats "just use Postgres" as the default to be argued *against*, and prices polyglot persistence at its true integration cost.

Why NoSQL happened

The misery that motivated it

By the mid-2000s, the largest web properties had driven the single-node relational database past its limits, and the standard escape — sharding by hand, exactly the technique dissected in Chapter 9 — was operationally brutal. The application picked a shard key (usually user ID), a routing layer mapped keys to one of *N* MySQL primaries, and from that moment the team owned every problem the database used to own: cross-shard queries became application-level scatter-gather, cross-shard transactions became either two-phase commit (Chapter 10) or a prayer, resharding meant weeks of dual-write migration choreography, and every JOIN that crossed a shard boundary simply died. Facebook, LiveJournal, Flickr, and YouTube all ran variants of this architecture, all built substantial in-house tooling to survive it, and none would have described it as pleasant. Add the era's second friction — `ALTER TABLE` on a multi-hundred-gigabyte MySQL table could lock writes for hours, making schema evolution a scheduled outage — and the appetite for something else is easy to understand.

Two papers turned that appetite into an industry.

Bigtable (Chang et al., OSDI 2006) described Google's storage system for crawl data, Google Earth, and analytics: a sparse, distributed, persistent multi-dimensional sorted map, partitioned into tablets, each tablet served by exactly one tablet server, with metadata coordinated through Chubby (Google's lock service, a Paxos consumer). Bigtable chose consistency: a single master per tablet means reads and writes for a row have one authoritative home, and a tablet is briefly unavailable while it is reassigned after a failure. HBase is its open-source descendant almost component-for-component; Cassandra took its *data model* (the wide, sparse, sorted row) while rejecting its single-master architecture.

Dynamo (DeCandia et al., SOSP 2007) described Amazon's shopping-cart store and chose the opposite corner: an always-writable key-value store with no master at all. Consistent hashing spread keys across a ring; every write went to *N* replicas; sloppy quorums and hinted handoff kept writes flowing during node failures; vector clocks tracked causality so that concurrent writes produced *siblings* the application had to reconcile ("merge the carts") rather than a silently lost update. Dynamo chose availability and pushed conflict resolution up the stack. Cassandra took

its *distribution* architecture, Riak implemented it nearly verbatim, and Amazon's later DynamoDB service inherited the name and the partitioning philosophy while quietly walking back some of the radicalism (DynamoDB offers strongly consistent reads and does not expose vector clocks).

Nearly everything in this chapter descends from one paper or the other, and the lineage predicts behavior: Dynamo-lineage systems are leaderless, always-writable, and make you think about read-repair and conflicts; Bigtable-lineage systems have a per-range leader, stronger ordering, and a failover story. Keep the family tree in mind and few of these systems will surprise you.

The wave, the retreat, and the honest frame

What followed the papers was a genuine engineering movement wrapped in a marketing wave. Roughly 2009–2013, "NoSQL" was pitched as a generational replacement — schemas were legacy, JOINS didn't scale, SQL itself was the problem. The term's own history records the correction: it was quickly rebranded from "no SQL" to "not only SQL" as it became clear that most adopters were not actually Google, and that the things sacrificed — transactions, secondary indexes, ad-hoc queries, decades of operational tooling — had been paying rent all along. Early MongoDB shipped with defaults tuned for benchmark optics (unacknowledged writes) and a first-generation sharding and failover implementation that taught a cohort of engineers hard lessons about what "eventually consistent" means at 3 a.m.

The retreat was not a rout but a *re-convergence*, and it is still running in both directions. Document stores grew up: MongoDB added multi-document ACID transactions (4.0 in 2018 on replica sets, 4.2 across shards), a serious aggregation pipeline, and tunable read/write concerns that map directly onto the replication vocabulary of Chapter 8. Cassandra grew CQL, a deliberately SQL-shaped dialect. Meanwhile the relational world absorbed the other side's virtues: Postgres's `jsonb` with GIN indexing (Chapter 3) gives you schema-on-read *inside* a transactional store, and the NewSQL systems of Chapter 12 exist precisely to offer NoSQL's horizontal scale without surrendering SQL or serializability. The SQL:2016 standard even added JSON operators.

So the honest frame, and the one this chapter uses: the categories that follow are **data models** — what shape your data takes and what

operations are native to that shape — bundled with **trade-off points** — what the exemplar systems chose about consistency, distribution, and indexing. They are not ideologies, and choosing among them is engineering, not tribal affiliation.

```
flowchart TB
    subgraph KV["Key-value"]
        K1["key: user:412:session"] --> V1["value: opaque bytes"]
        store cannot see inside
    end
    subgraph DOC["Document"]
        D1["_id: 412"] --> D2["nested JSON document"]
        name, addresses array,
        orders array
        store indexes inside it
    end
    subgraph WC["Wide-column"]
        W1["partition key: sensor_17"] --> W2["clustering key 09:00 - cols: temp, hum"]
        W1 --> W3["clustering key 09:05 - cols: temp"]
        W1 --> W4["sparse sorted rows within the partition"]
    end
    subgraph GR["Graph"]
        G1["node: Person Alice"] -- "KNOWS since 2019" --> G2["node: Person Bob"]
        G2 -- "OWNS" --> G3["node: Account 991"]
    end
    KV -. NOTE["Left to right: the store understands progressively more structure, and native queries get richer"] .-> DOC
    DOC -. NOTE .-> WC
    WC -. NOTE .-> GR
```

The ordering in the diagram is the useful mental model: from key-value to graph, the store understands progressively more of your data's structure, and in exchange for that understanding it can execute progressively richer queries natively — at the cost of progressively harder distribution stories. A key-value store scales almost embarrassingly well precisely because it understands nothing; a graph partitions badly precisely because its whole value is in the connections that partitioning must cut.

Key-value stores

The key-value model is the minimal contract: `get(key)`, `put(key, value)`, `delete(key)`, where the value is opaque bytes. The store cannot query inside values, which sounds like poverty and is actually the source of the model's power — with no structure to maintain, every operation touches exactly one key, keys hash cleanly across partitions (Chapter 9), and there are no cross-key invariants for the store to enforce or betray. Everything in Chapters 5 and 10 about transactional coordination is simply out of scope by construction. The application inherits whatever coordination it actually needed.

Redis: the data-structure server

Redis is the most instructive key-value system because it deviates from the model in one direction — values are not opaque but *typed data structures* — while staying radically simple in another: command execution is single-threaded. One thread runs an event loop (the same `epoll`-based architecture as Volume 4, Chapter 6) and executes commands strictly one at a time. Since Redis 6.0 network I/O can be spread across threads, but the command itself still executes serially, and this is a feature, not a limitation: every command, and every Lua/MULTI block, is atomic with no locking anywhere, because there is nothing to lock against. The costs follow just as directly: a single slow command — `KEYS *` on a large keyspace, `SMEMBERS` on a ten-million-member set, a long Lua script — stalls every client, which is Volume 4's "never block the event loop" rule wearing a database costume. And one instance uses roughly one core for command work, so scaling compute means Redis Cluster and hash-slot partitioning, at which point multi-key operations only work when all keys share a slot (the `{hashtag}` convention) — Chapter 9's cross-partition limitation, in miniature.

The typed values are why Redis absorbed so many use cases. Strings with `INCR` give you counters and rate limiters; hashes give you compact objects; lists give you queues; sets give you tags and deduplication; sorted sets — a hash table plus skiplist, ranked by score — give you leaderboards and sliding-window rate limiting in one command family; streams (5.0+) give you an append-only log with consumer groups, a lightweight Kafka-shaped tool. Each structure ships with operations that would be a transaction elsewhere (`ZINCRBY`, `LPOS`, `SETNX`) and are atomic here for free.

Persistence is where honesty matters most, because Redis is memory-first and every durability option is a compromise you must choose with open eyes:

```
# redis.conf – persistence, honestly annotated

# --- RDB: point-in-time snapshots ---
# Fork a child; child writes a compact binary snapshot via copy-on-
write.
# "3600 1" = snapshot if >=1 change in 3600s; tighter rules for busier
periods.
save 3600 1 300 100 60 10000
dbfilename dump.rdb
# What RDB loses: EVERYTHING since the last snapshot. With the rules
above,
# a crash can discard minutes of acknowledged writes. Forking also
costs:
# on a large heap under heavy writes, copy-on-write can briefly double
RSS.

# --- AOF: append-only command log (Chapter 7's WAL idea, applied here)
---
appendonly yes
# fsync policy – the actual durability dial:
#   always   : fsync every command. ~No loss, but throughput drops to
disk speed.
#   everysec  : fsync once per second (default). Crash loses up to ~1s
of
#               acknowledged writes. The pragmatic choice for most
workloads.
#   no       : let the OS decide (typically ~30s exposure). Fastest,
weakest.
appendfsync everysec
# AOF rewrite compacts the log in the background (fork again, same CoW
cost).
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb
```

Note what even `appendfsync always` does *not* buy: replication to a Redis replica is asynchronous, so a failover can lose writes the primary acknowledged — Chapter 8's async-replication window, and Redis documentation says so plainly. Which forces the question every team using Redis must answer explicitly: **is this a cache or a store?** As a cache, eviction (`maxmemory` with an LRU/LFU policy) is correct behavior, loss is harmless by definition, and the source of truth lives elsewhere. As a store — session state, queues, counters that matter — eviction is data loss, and you must run with `maxmemory-policy noeviction`, AOF on, and a

tested failover story. The recurring production disaster is the slow blur from the first mode into the second: a "cache" quietly becomes the only home of session or queue state, nobody revisits the eviction policy or the durability config, and a routine restart becomes an incident. Decide which one each Redis deployment is, write it down, and configure accordingly.

Memcached deserves its paragraph as the contrast case: it is the key-value cache with *no* ambitions — opaque values, multithreaded (unlike Redis) over a slab allocator with LRU eviction, no persistence, no replication, no data structures. All distribution is client-side consistent hashing. Its narrowness is the point: as a pure look-aside cache it is simple, fast, and has decades of production mileage (Facebook's fleet, described in their 2013 NSDI paper "Scaling Memcache at Facebook," remains the canonical study). If you need only a cache, its inability to be anything else is protection.

DynamoDB: partition-key design as a service contract

Amazon DynamoDB (the 2012 service, distinct from the 2007 paper) is the clearest illustration of this chapter's central claim, because it makes the distributed-systems contract *explicit and priced*. Every table has a **partition key**, hashed to spread items across physical partitions (each partition capped at roughly 10 GB and a fixed throughput ceiling — on the order of 3,000 read units and 1,000 write units per second), and optionally a **sort key** that orders items within a partition, enabling **Query** to fetch ranges. That is the entire native query model: **get** by key, or **range-scan** within one partition. **Scan** exists and is priced to discourage you.

Capacity is metered in units that force you to think in the store's terms: one **WCU** writes one item up to 1 KB per second; one **RCU** performs one strongly consistent read up to 4 KB (an eventually consistent read costs half). Because per-partition throughput is capped, a **hot key** — one tenant, one celebrity, one "today" bucket receiving disproportionate traffic — throttles even when the table's aggregate capacity is mostly idle: Chapter 9's skew problem, now with an invoice. DynamoDB's *adaptive capacity* mitigates this by shifting throughput toward hot partitions and isolating hot items, and it has improved markedly since launch, but it cannot repeal the ceiling — a single item's traffic cannot exceed one partition's limit, ever. The fixes remain the ones from Chapter

9: choose higher-cardinality keys, or shard the hot key yourself with a suffix (`date#2026-08-14#7`) and fan-in on read.

Secondary access patterns come from **global secondary indexes**: each GSI is effectively a shadow table with its own key schema and its own capacity, populated by *asynchronous* replication from the base table. GSI reads are therefore always eventually consistent — the write you just made may not be visible via the index for a moment — which is Chapter 3's index-maintenance cost restructured as replication lag. (Local secondary indexes offer strong consistency but must be declared at table creation and share the partition's 10 GB cap; they are used far less.)

The design culture that grew around these constraints is **single-table design**: because there are no JOINS, you pre-join by *interleaving* related entities in one table with composite keys, so each access pattern is one Query :

Table: app_data attributes	PK	SK
-----	-----	-----
Customer profile name, email, tier	CUST#412	PROFILE
Customer's orders total, status	CUST#412	ORDER#2026-08-01#9871
	CUST#412	ORDER#2026-08-11#9902
total, status		
Order's line items sku, qty, price	ORDER#9902	ITEM#1
	ORDER#9902	ITEM#2
sku, qty, price		
GSI1 (by status): -> open orders, newest first	GSI1PK=STATUS#pending	GSI1SK=2026-08-11...
Access patterns served:		
profile + recent orders : Query	PK=CUST#412, SK begins_with ORDER#	
(one request)		
one order's items	: Query PK=ORDER#9902	
all pending orders	: Query GSI1PK=STATUS#pending (eventually	
consistent)		

The trade is stark and should be stated as such: you get single-digit-millisecond reads at any scale for the access patterns you designed, and near-total rigidity for the ones you didn't. A new access pattern means a new GSI (with backfill) or a data migration. Single-table design is the purest expression of query-first modeling — powerful when access

patterns are stable and known, punishing when the product is still discovering what it is. That conditionality, not the technique itself, is the lesson.

Document stores

The document model stores self-describing hierarchical records — JSON, or MongoDB's binary BSON — retrievable and *queryable* by their contents, not just by key. The historical contract that defined the category: **atomicity ends at the document boundary**. A write to one document — however deeply nested, however many fields — is atomic; a write spanning two documents was, classically, two writes with no isolation between them. That single line explains most document-database design: if your invariant lives inside one document, you get Chapter 5's guarantees for free; if it spans documents, you have left the contract and must either restructure until it fits or reach for the newer, costlier machinery below.

MongoDB is the exemplar, and its architecture maps cleanly onto this volume's earlier chapters. A **replica set** is Chapter 8's single-leader replication: one primary accepts writes and appends them to the oplog (a logical WAL — Chapter 7), secondaries replay it, and an election (Raft-derived since the 3.2-era protocol) chooses a new primary on failure. Durability and read semantics are tunable per operation: `writeConcern: majority` waits for a majority of replicas to make the write durable before acknowledging (survives failover; slower), `w: 1` acknowledges from the primary alone (fast; a failover can lose it); `readConcern` similarly ranges from `local` to `majority` to `linearizable`. **Sharding** is Chapter 9 with a router: collections are range- or hash-partitioned on a shard key across replica sets, `mongos` routes queries, and a config-server replica set holds the routing table. Every shard-key caution from Chapter 9 applies verbatim, including the classic mistake of sharding on a monotonically increasing field and pinning all inserts to one shard.

Multi-document ACID transactions exist — since 4.0 (2018) within a replica set, 4.2 across shards — and the accurate statement is that they are real but not free. They use snapshot isolation, hold locks over their lifetime (with a default expiry of about 60 seconds), and their cross-shard form is two-phase commit with everything Chapter 10 taught you about its latency and failure windows. MongoDB's own guidance is honest here: model so that single-document atomicity covers your invariants, and treat

multi-document transactions as an escape hatch, not the default — the opposite of relational habit, and the correct instinct for the architecture.

Schema-on-read, and where the schema actually went

"Schemaless" was the marketing word; **schema-on-read** is the accurate one. The store enforces almost nothing at write time, so records of different shapes coexist in one collection. The schema did not disappear — schemas never disappear — it *moved into the application code that reads the data*, which now must tolerate every shape ever written. This is genuinely valuable when shapes vary intrinsically (product catalogs, event payloads, third-party documents) and during rapid iteration, and it is genuinely dangerous without discipline, because the write path evolves faster than old data does: five years of shape drift becomes five years of `if (doc.v === undefined)` archaeology in every reader.

The working discipline is a version field and explicit migration policy: stamp every document with `schemaVersion`, keep readers able to handle version N and $N-1$, and upgrade documents either lazily (rewrite on next touch) or eagerly (background migration), retiring old-version code only when the data says no old versions remain. MongoDB's optional JSON-schema validation can enforce a floor at write time. Teams that skip this discipline have not escaped schema migrations; they have distributed one migration across every reader, indefinitely.

Embed versus reference: the decision

Modeling in a document store reduces to one recurring decision — embed the related data inside the parent document, or reference it by ID and fetch separately — and it is the mirror image of Chapter 1's normalization question. Embedding is denormalization: one read fetches everything, and one atomic write updates it all, at the cost of duplication and growth. Referencing is normalization: one copy of shared data, at the cost of application-side joins (`$lookup` exists but is deliberately limited, especially across shards) and multi-document update surfaces.

```

flowchart TD
    Q1["Is the child data always  
read with the parent?"] -->|"no"| REF1["Reference"]
    Q1 -->|"yes"| Q2["Is the relationship bounded?  
A few, not thousands"]
    Q2 -->|"no - unbounded growth"| REF2["Reference  
unbounded arrays grow toward the  
16 MB document cap and make  
every rewrite more expensive"]
    Q2 -->|"yes"| Q3["Is the child shared  
by many parents?"]
    Q3 -->|"yes - shared"| REF3["Reference  
embedding shared data means  
fan-out updates to every copy"]
    Q3 -->|"no - exclusive"| Q4["Must an invariant hold  
across parent and child?"]
    Q4 -->|"yes"| EMB1["Embed  
single-document atomicity  
covers the invariant for free"]
    Q4 -->|"no"| EMB2["Embed by default  
one read, no join"]

```

The rules of thumb the tree encodes: data read together should live together; unbounded arrays (comments on a viral post, events on a long-lived entity) must be referenced or bucketed, both because of MongoDB's 16 MB document cap and because rewriting an ever-growing document gets steadily more expensive; data shared across parents should be referenced unless it is immutable, because embedded copies mean fan-out updates; and an invariant that must hold atomically is the strongest argument *for* embedding, since it buys you Chapter 5's guarantees without Chapter 10's machinery. Where the relational modeler asks "what is the normalized form?", the document modeler asks "what does the application read and change together?" — which is query-first modeling again, in milder form than Cassandra will demand next.

Wide-column stores

The Bigtable data model, which Cassandra, HBase, and ScyllaDB share, is best described not as "columns" but as a **sparse, sorted, two-level map**: a row key locates a partition; within it, column families group columns; and a row stores only the columns it actually has — sparseness is free, and two rows in one table can have entirely different columns. Bigtable kept versioned cells (timestamps as a third dimension); Cassandra's CQL flattened the surface into something deliberately table-

shaped, and it is Cassandra's version of the model that most backend engineers meet.

The two-part primary key is the entire mental model, and it is worth being precise. The **partition key** is hashed (Murmur3, by default) to place the partition on the ring — Chapter 9's hash partitioning, so partitions have no meaningful order across the cluster. The **clustering columns** define the physical sort order of rows *within* the partition on disk. Consequently: any query that supplies the partition key and a prefix or range of the clustering columns is a sequential read of one contiguous region on a handful of replicas — this is the fast path, and it is very fast. Any query that does not supply the partition key is a cluster-wide scatter (ALLOW FILTERING — the clause whose name is a warning). The table is the index, and there is exactly one of it per table.

```
flowchart TB
    subgraph RING["Cluster - partition key hashed onto the ring"]
        N1["node A"] --- N2["node B"] --- N3["node C"]
    end
    end
    RING --> P["partition: sensor_17  
lives on its replica set of nodes"]
    subgraph PART["Inside the partition - rows physically sorted by  
clustering key"]
        direction TB
        R1["day 2026-08-14, ts 09:00 - temp 21.4, hum 40"]
        R2["day 2026-08-14, ts 08:55 - temp 21.3, hum 41"]
        R3["day 2026-08-14, ts 08:50 - temp 21.1"]
        R4["... contiguous on disk, DESC order ..."]
        R1 --> R2 --> R3 --> R4
    end
    P --> PART
    PART --> QQ["Query with partition key + clustering range  
= one sequential read on one replica set.  
Query without partition key  
= scatter to every node."]
```

This makes **query-first modeling** not a style preference but a discipline the architecture enforces. You do not model your entities and then write queries; you enumerate your queries and then create one table *per query*, denormalizing the same data into as many layouts as you have access patterns, with the application writing every copy (or BATCH doing so with best-effort, not isolated, semantics). Storage is spent to buy read locality; consistency between the copies is the application's problem. Concretely:

```

-- Access pattern: "latest N readings for a sensor, newest first,
--                   optionally bounded by time range"
CREATE TABLE readings_by_sensor (
    sensor_id text,
    day       date,          -- part of the partition key: buckets the
    ts        timestamp,     -- time series so one partition cannot
grow forever
    temp      double,
    humidity  double,
    PRIMARY KEY ((sensor_id, day), ts)
) WITH CLUSTERING ORDER BY (ts DESC);

-- Served natively, as one contiguous read:
SELECT ts, temp FROM readings_by_sensor
WHERE sensor_id = 'sensor_17' AND day = '2026-08-14'
AND ts > '2026-08-14 08:00:00'
LIMIT 100;

-- A different access pattern ("all sensors in a region, latest
reading")
-- is a DIFFERENT TABLE, written in parallel by the application.

```

Note the `(sensor_id, day)` composite partition key: bucketing by day is the standard defense against the unbounded-partition antipattern, because a partition that grows without limit concentrates load and data on one replica set — the hot-partition problem of Chapter 9, plus operational pain (repair, compaction, and read latency all degrade on multi-gigabyte partitions).

Underneath, Cassandra is Chapter 2's LSM tree at each replica: writes hit a commit log and memtable, flush to immutable SSTables, and compaction merges them. Two consequences deserve emphasis. First, writes are extraordinarily cheap — sequential appends, no read-before-write — which is what makes tables-per-query denormalization affordable. Second, **deletes are writes**: a delete inserts a tombstone that must persist for `gc_grace_seconds` (default ten days) so that repair can propagate it to any replica that missed it — drop the tombstone too early and a lagging replica resurrects the deleted data. The pathology at scale is the queue-shaped workload: insert and delete at high volume in one partition, and reads must scan thousands of tombstones to find live rows. Cassandra ships thresholds that warn and then abort such reads; the real fix is modeling (TTL-and-bucket rather than delete, or a different store for queues). Using Cassandra as a queue is a known antipattern for exactly this reason.

Cassandra's **tunable consistency** is the Dynamo inheritance and gets one honest paragraph here, with the quorum mathematics deferred to Volume 6, Chapter 7. Every read and write names a consistency level — `ONE`, `QUORUM`, `LOCAL_QUORUM`, `ALL` — which is the number of replicas (of the replication factor N) that must respond. Writes at `QUORUM` and reads at `QUORUM` with $R + W > N$ give you reads that see the latest acknowledged write under normal operation; `ONE` on both gives you the lowest latency and the real possibility of reading stale data until read repair and anti-entropy catch up. The point to carry forward is that this dial exists *per operation* — consistency became a request parameter, and the RDBMS's hidden constant became your explicit, per-query choice.

HBase completes the contrast: same data model, opposite lineage — a faithful Bigtable descendant with single-master RegionServers over HDFS, giving strongly consistent row operations and genuine range scans over the *global* row-key order (Cassandra's hashing forfeits this), at the cost of a region being briefly unavailable during failover: CP where Cassandra chose AP. ScyllaDB is Cassandra's protocol-compatible C++ reimplementation on a thread-per-core, shard-per-core architecture (Volume 4's ideas applied ruthlessly), delivering substantially better and more predictable per-node performance with the same model and the same modeling discipline.

Graph databases

The property-graph model makes relationships first-class: **nodes** and **edges**, both bearing labels/types and arbitrary key-value properties, with edges directed and traversable in either direction. Its claim to existence is a workload shape, not a scale story: queries where the answer is defined by *traversal* — follow edges of these types, three or five or unknown-many hops out, filtering as you go. Friends-of-friends recommendation, fraud-ring detection (accounts linked through shared devices, addresses, and payment instruments), dependency and blast-radius analysis, permission inheritance, network topology. What these share is that the join depth is high, variable, or unknown at design time, and the touched set is a tiny neighborhood of a huge graph.

Neo4j is the reference implementation, and Cypher its query language — a pattern-matching syntax in which you draw the traversal:

```
// Fraud check: is this new account connected, within 4 hops through
// shared identifiers, to any account already flagged for fraud?
MATCH (a:Account {id: $newAccountId})
MATCH p = (a)-[:USED_DEVICE|USED_CARD|SHARES_ADDRESS*1..4]-(b:Account)
WHERE b.flagged = true
RETURN b.id, length(p) AS hops
ORDER BY hops
LIMIT 10;
```

The `*1..4` is the point: a variable-length traversal over three edge types, unspeakable in one SQL query without recursive CTEs, natural here. (Cypher was opened as openCypher and is the main input to ISO GQL, the graph query language standard published in 2024 — the first new ISO database language since SQL.)

The performance claim behind native graph stores is **index-free adjacency**: each node holds direct physical references to its edges, so following an edge is a pointer dereference — $O(1)$ per hop, independent of graph size — rather than an index lookup. Examined honestly, the claim is real but narrower than the marketing. It is a statement about *traversal* cost only: finding your *starting* nodes still requires ordinary property indexes (Chapter 3 lives here too); a pointer chase over a disk-resident graph is random I/O with poor locality, so the $O(1)$ constant varies enormously with cache residency; and dense hubs — the celebrity with two million followers — make "per hop" costs explode combinatorially regardless of representation. Index-free adjacency wins where the workload is deep traversal over warm neighborhoods; it is not a general claim that graph databases are faster.

Equally honest is the negative space, because graph databases are among the most over-adopted categories. Data being "connected" is not the criterion — *all* relational data is connected; that is what foreign keys are. If your traversals are one or two hops of known shape — user to orders, order to items — that is a JOIN, the RDBMS executes it superbly with Chapter 4's machinery, and a graph database buys you an unfamiliar query language and a thinner operational ecosystem for nothing. The relational approach only genuinely breaks down as depth grows: an adjacency-list table (`edges(src, dst)`) traversed k hops deep is k self-joins — with recursive CTEs it is expressible, but each level is another join whose intermediate result can grow multiplicatively, and there is no adjacency locality to exploit; at unknown depth the plans degrade badly.

That $O(\text{depth})$ join explosion, not connectedness, is the honest trigger for a graph store.

The model's structural weakness is distribution. Chapter 9's methods assume data that partitions into independent shards; a graph's value is its edges, and any partitioning of a well-connected graph cuts many of them, turning single-machine pointer chases into network hops mid-traversal. Balanced graph partitioning that minimizes cut edges is NP-hard, and real social graphs resist even good heuristics. This is why Neo4j's classic scaling story is replication for reads with the full graph on each machine, why sharded deployments (Neo4j Fabric and kin) push partitioning decisions back to you, and why you should treat "distributed graph database" claims with the specific question: *what happens to a traversal that crosses a shard boundary?*

Cross-cutting: what you actually give up

Three capabilities the relational world bundles are worth auditing across all four models, because their absence is where budgets and pager rotations go.

Secondary indexes (Chapter 3). Postgres will index any column, expression, or `jsonb` path, consistently, on the same node as the data. Across the NoSQL models the story ranges from "extra tables you maintain" (key-value; Redis sorted sets as indexes), to "genuinely good but watch write amplification" (MongoDB's B-tree secondary indexes, including on nested fields — the strongest story in the group), to "asynchronous shadow tables" (DynamoDB GSIs, eventually consistent), to "exists but scatter-gathers the cluster, prefer another denormalized table" (Cassandra's secondary indexes and their SAI successor). The pattern: in distributed stores, a global secondary index is itself a distributed-systems problem — either co-partitioned with the data (scatter on read) or partitioned by the indexed value (coordinate on write) — and each system has simply picked its poison.

Aggregation. MongoDB's aggregation pipeline is a real query engine — multi-stage, indexed, spillable — and the honest exception. Cassandra and DynamoDB offer nearly nothing meaningful server-side at scale; the intended pattern is maintaining running aggregates at write time (counters, summary rows) or shipping data to an analytical store (Chapter 13, and Volume 10's pipelines). Budget for that pipeline on day

one, because "we can't answer sum-by-month without a Spark job" arrives sooner than teams expect.

Backup and consistency of backups. A single-node RDBMS backup is a consistent snapshot almost by accident (Chapter 7's WAL makes point-in-time recovery routine). A distributed store's backup is per-node snapshots taken at *not-quite-the-same* instant — Cassandra snapshots are per-replica SSTable hardlinks; a cluster-wide restore is only as consistent as your repair discipline. Managed services (DynamoDB PITR, MongoDB Atlas continuous backup) have largely solved this for you, which is a real and underweighted argument for them; self-hosted, the consistent-restore drill is part of the total cost of ownership and must actually be rehearsed.

Choosing: a decision framework

The framework, in the order the questions should be asked:

1. Access patterns first. Enumerate the queries — reads and writes, with rough rates and latency needs — *before* choosing a store, because every system in this chapter is a bet on a query shape: point lookups (key-value), retrieve-and-update an aggregate (document), range-within-partition at high write volume (wide-column), deep traversal (graph), ad-hoc and evolving (relational — this is precisely what the relational model's physical independence buys, and none of the alternatives replicate it). If you cannot enumerate access patterns yet, that is itself the answer: choose the model that tolerates not knowing, which is relational.

2. Invariants second. List what must be true — balances non-negative, usernames unique, order-and-items consistent — and where each invariant lives. Invariants within one key or document are cheap everywhere. Invariants *across* keys, documents, or partitions put you back into Chapters 5 and 10: you will either pay for coordination (RDBMS transactions, MongoDB multi-document transactions, DynamoDB's transactional API at double the capacity cost), restructure the data until the invariant fits in one atom, or relax the invariant and reconcile — a business decision wearing a technical costume, which Volume 6 treats properly.

3. Scale, honestly assessed. State it plainly: **most workloads fit in Postgres**, and fit further than intuition suggests — a few terabytes and

tens of thousands of transactions per second on one well-tuned primary with read replicas is unremarkable on 2020s hardware. The honest threshold for this chapter's systems is when a *specific, measured* pattern outgrows that — write throughput beyond one primary, a working set beyond one machine, or multi-region active-active writes (Chapter 8 explains why single-leader replication cannot give you that). Dan McKinley's "Choose Boring Technology" argument formalizes the prior: every technology you operate consumes a limited innovation budget in monitoring, backup, upgrade, and 3-a.m.-debugging competence; spend those tokens on problems you actually have. Adopting Cassandra for a workload Postgres handles converts a solved problem into an unsolved one.

4. Operational maturity. The steady-state cost of these systems is operational, not intellectual: compactions and repairs to schedule, consistency levels to reason about during incidents, capacity models to maintain. A managed service (DynamoDB, Atlas, managed Cassandra) moves much of that cost onto the provider's ledger — often the decisive factor for a small team, and a legitimate one.

5. Polyglot persistence, priced honestly. Real systems end up with several stores — Postgres for transactional truth, Redis for caching, Elasticsearch for search, a warehouse for analytics — and the architecture is sound *if* you price the integration tax: every pair of stores holding overlapping data is a consistency problem you now own, with no transaction spanning them. The disciplined pattern designates one store as the system of record and derives the rest asynchronously via change data capture — tail the WAL (Chapter 7) with Debezium or equivalent, publish to a log, and let each derived store consume at its own pace (Volume 10, Chapter 6 covers the pipeline mechanics). This gives ordering, replayability, and a bounded-staleness story. The undisciplined pattern — application code dual-writing to two stores — reliably produces silent divergence, because the second write fails independently of the first and nothing reconciles them. Cache invalidation is the same problem in miniature, and even the CDC-driven form only bounds the inconsistency window; it never removes it. Every derived read is a stale read of some age, and the age must be acceptable *by design*, not by luck.

```

flowchart LR
    APP["Application services"] -->|"transactional writes"| PG["Postgres  
system of record"]
    PG -->|"WAL tail - CDC  
Debezium"| LOG["Event log  
Kafka"]
    LOG --> ES["Elasticsearch  
search index"]
    LOG --> RS["Redis  
cache warm and invalidate"]
    LOG --> DW["Warehouse  
analytics"]
    APP -->|"reads, seconds stale"| ES
    APP -->|"reads, ms to s stale"| RS
    APP -.->|"NO dual writes from the app  
derived stores are downstream only"| LOG

```

The distributed-systems lens

The deepest legacy of the NoSQL era is not any database; it is that **the trade-offs were always there, and NoSQL made them visible**. A single-node RDBMS is a machine for hiding distributed problems: one WAL provides total order, one buffer pool provides coherence, one lock manager provides mutual exclusion, and the application sees the serene fiction of Chapter 5. The moment data outgrows one node those problems return — Chapters 8 through 10 were the story of the relational world fighting them — and the NoSQL systems, born distributed, simply stopped hiding them. Cassandra's per-request consistency level is quorum arithmetic as an API parameter (the mathematics in Volume 6, Chapter 7). The Dynamo lineage embraced availability under partition and made the consequences explicit machinery: vector clocks to detect concurrent writes, sloppy quorums and hinted handoff to keep accepting writes while nodes are down, read repair and anti-entropy to converge afterward — each one a named, inspectable answer to a question the RDBMS never let you ask. The Bigtable lineage answered the same question the other way: one master per tablet or region preserves strong per-range consistency and accepts a window of unavailability at failover. That is the CAP trade-off in the flesh — under a network partition, a system chooses availability or consistency for the affected data, not both — stated here as preview and treated with the care it needs (including what CAP does *not* say) in Volume 6, Chapter 4.

The second theme deserves its own paragraph, because it names who now owns the problem: **partition-key design is distributed-systems design, delegated to the application developer**. When you choose a DynamoDB partition key or a Cassandra partition key, you are choosing the unit of data placement, the unit of load distribution, the boundary of cheap atomicity, and the boundary beyond which Chapter 10's coordination costs begin — decisions a DBA and a query planner once made behind your back, now made in your schema, permanently, and visible in your latency histograms when made badly. Everything Chapter 9 taught about skew, hot keys, and resharding, and everything Chapter 10 taught about cross-partition invariants, applies to a keyspace you now design by hand. The teams that succeed with these systems are the ones that treat that design with the seriousness the database formerly applied on their behalf; the incident reviews that mention "hot partition" are written by the teams that did not. Volume 6 builds the theory — consistency models, quorum mathematics, CRDTs (Chapter 11 there) for the conflicts the Dynamo lineage hands you — and Volume 7, Chapter 5 returns to modeling with the full toolbox in hand.

Key takeaways

- **NoSQL was a reaction to real misery:** hand-sharded RDBMS at 2000s web scale and lock-the-table schema migrations. Two papers founded the space — Bigtable (2006, CP lineage: single master per tablet, ordered rows) and Dynamo (2007, AP lineage: leaderless, always writable, conflicts surfaced to the application) — and a system's lineage predicts its behavior.
- **The categories are data models and trade-off points, not tribes.** The wave overclaimed, the retreat re-converged: document stores grew transactions and query engines; relational grew `jsonb`; NewSQL (Chapter 12) attacks the scale problem without abandoning SQL.
- **Key-value:** opaque values, perfect partitioning, no cross-key anything. Redis is a single-threaded data-structure server whose durability is a dial (RDB loses minutes; AOF `everysec` loses about a second; async replication can lose acknowledged writes on failover) — decide explicitly whether each deployment is a cache or a store, and configure accordingly.

- **DynamoDB prices the distributed contract:** per-partition throughput ceilings make hot keys a throttling event, GSIs are eventually consistent shadow tables, and single-table design buys guaranteed latency for known access patterns at the cost of rigidity toward new ones.
- **Document stores' contract is per-document atomicity.** Model so invariants fit inside one document; multi-document transactions (MongoDB 4.0/4.2+) are real but are the escape hatch, not the default. Schema-on-read moves the schema into reader code — version your documents or pay in archaeology. Embed versus reference is normalization's mirror: embed what is read and changed together, reference what is shared or unbounded.
- **Wide-column enforces query-first modeling:** partition key locates, clustering columns sort physically, one table per query, denormalization as discipline. LSM underneath makes writes cheap and deletes into tombstones — whose accumulation is the signature pathology. Tunable consistency makes the consistency/latency trade a per-request parameter (mathematics in Volume 6, Chapter 7).
- **Graph databases earn their place only when traversal dominates** — deep, variable-depth relationship queries where relational self-joins explode with depth. Index-free adjacency is a real but narrow claim, and graphs partition badly because cutting edges is the one thing a graph cannot afford.
- **Choose by access patterns, then invariants, then honestly-assessed scale.** Most workloads fit Postgres, and boring technology is an argument, not an insult. Polyglot persistence works when one store is the system of record and the rest derive via CDC; dual writes from the application are how stores silently diverge.
- **Partition-key design is distributed-systems design handed to you.** Everything from Chapters 9 and 10 — skew, hot keys, cross-partition invariants — now lives in your schema.

Further reading

- Chang, F., et al., "Bigtable: A Distributed Storage System for Structured Data," *OSDI*, 2006 — the CP-lineage founding paper. <https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/>

- DeCandia, G., et al., "Dynamo: Amazon's Highly Available Key-value Store," *SOSP*, 2007 — the AP-lineage founding paper: consistent hashing, sloppy quorums, hinted handoff, vector clocks. <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>
- Kleppmann, M., *Designing Data-Intensive Applications* (O'Reilly, 2017), Chapter 2 — the best single treatment of data models, including the document/relational history and graph models.
- Redis documentation, "Redis persistence" — the RDB/AOF trade-offs from the source, including the honest statements about what each mode can lose. https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
- Amazon DynamoDB Developer Guide — partition behavior, capacity units, GSI consistency, and the single-table design guidance under "best practices." <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/>
- Elhemali, M., et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," *USENIX ATC*, 2022 — the service's architecture ten years in, including adaptive capacity.
- Apache Cassandra documentation — data modeling and the CQL reference; the modeling section is an explicit statement of query-first discipline. <https://cassandra.apache.org/doc/latest/>
- MongoDB Manual — transactions, replication, and the data-modeling section on embedding versus referencing. <https://www.mongodb.com/docs/manual/>
- Lakshman, A. and Malik, P., "Cassandra — A Decentralized Structured Storage System," *ACM SIGOPS Operating Systems Review*, 2010 — Bigtable's data model on Dynamo's distribution, from the original authors.
- Nishtala, R., et al., "Scaling Memcache at Facebook," *NSDI*, 2013 — the canonical study of a cache tier at extreme scale, including invalidation and consistency machinery.
- Brewer, E., "CAP Twelve Years Later: How the 'Rules' Have Changed," *IEEE Computer*, 2012 — the CAP conjecture's author on what it does and does not claim; pairs with Gilbert and Lynch's 2002 proof. Depth in Volume 6, Chapter 4.

- McKinley, D., "Choose Boring Technology," 2015 — the innovation-token argument referenced in the decision framework. <https://mcfunley.com/choose-boring-technology>
- Volume 5, Chapter 2 — Storage Engines — the LSM machinery under Cassandra; Chapter 9 — Partitioning and Sharding — the skew and hot-key theory this chapter applies.
- Volume 6, Chapter 7 — Dynamo-style quorums, read repair, and anti-entropy in full; Chapter 11 — CRDTs, for the conflicts the AP lineage surfaces.

Chapter 12 — NewSQL and Distributed SQL

What this chapter covers. Chapter 11 ended on an uncomfortable note: the NoSQL generation bought horizontal scale by selling the two most valuable abstractions the database industry had ever built — declarative SQL and ACID transactions — and a decade of application-level joins, fan-out reads, and hand-rolled compensation logic was the price. This chapter examines the systems that asked whether that sale was ever necessary. The NewSQL / distributed-SQL claim is that you can have full SQL, serializable (in the best cases, externally consistent) transactions, *and* horizontal write scaling in one system — and that the real price is not semantics but **latency floors and operational complexity**. Every write pays a consensus round trip; every global guarantee is purchased with time infrastructure or extra coordination; and someone has to operate the machinery.

We take Google Spanner as the archetype, because the 2012 paper is the clearest statement of the architecture everyone else adapted: data in tablets, each tablet a Paxos group, two-phase commit layered over Paxos groups, and TrueTime — bounded clock uncertainty from GPS and atomic clocks — converted directly into external consistency via commit-wait. We then study CockroachDB as the no-special-hardware answer: Raft-replicated ranges, hybrid logical clocks instead of TrueTime, and the machinery (uncertainty windows, timestamp pushing, read refreshes) needed to get serializability without bounded clock error. We survey the rest of the field honestly — TiDB, YugabyteDB, the sharding-middleware path of Vitess and Citus, and Aurora, which is routinely miscategorized

and belongs to a different taxonomy entirely. Then we state the costs plainly, work the latency arithmetic from RTTs rather than benchmarks, and close with a decision ladder that begins, as it should, with a single Postgres node.

This is the chapter where Volume 5 and Volume 6 fuse. Almost nothing here is a new mechanism: Paxos and Raft are Volume 6, Chapters 5-6; clocks are Volume 6, Chapter 2; quorums are Volume 6, Chapter 7. What is new is seeing them *productized* behind a SQL prompt.

Learning goals — after this chapter you should be able to:

- State the NewSQL thesis precisely: which guarantees are retained, which costs are accepted, and why the NoSQL trade of Chapter 11 turned out to be contingent rather than fundamental.
- Describe Spanner's architecture: tablets as Paxos groups, replication-as-consensus, and 2PC over Paxos groups as the fault-tolerant version of Chapter 10's fragile coordinator.
- Explain TrueTime and commit-wait mechanically, and why bounded clock uncertainty converts into commit latency — and into external consistency.
- Distinguish external consistency (strict serializability) from plain serializability, using Chapter 6's vocabulary, and say why the difference is observable.
- Describe CockroachDB's design: ranges, Raft, leaseholders, HLCs, the uncertainty window, and how timestamp pushing and read refreshes substitute for commit-wait.
- Place TiDB, YugabyteDB, Vitess, Citus, Aurora, and Neon-style disaggregation correctly in the taxonomy, and say what each does *not* give you.
- Do honest latency arithmetic for single-region and multi-region deployments, and choose table localities and follower reads accordingly.
- Apply the decision ladder: when a single node, read replicas, sharding middleware, or a true distributed SQL engine is the right rung — and recognize the anti-pattern of paying consensus latency for a single-region CRUD app.

The thesis: was the NoSQL trade necessary?

Recall the shape of Chapter 11's bargain. Dynamo-style and Bigtable-style systems scaled writes horizontally by abandoning, in various combinations: the relational model and its declarative queries, multi-object transactions, strong consistency, and secondary indexes as a first-class feature. Applications inherited the abandoned work. Denormalization replaced joins; "read, modify, hope" replaced transactions; and every team maintaining a fan-out-on-write timeline was doing, badly and once per company, work a query planner does well and once per industry.

The historical justification was that the coordination required for cross-node transactions was believed incompatible with scale and availability — a belief the CAP discussion of the era hardened into slogan form. The NewSQL generation's contribution was to test the belief and find it overstated. Three observations made the difference:

1. **Consensus makes replication and commit fault-tolerant, and it got fast enough.** Chapter 8 treated replication and Chapter 10 treated atomic commit as separate problems with separate failure modes. Run replication *as* consensus — every shard a Paxos or Raft group — and both problems change character: replica failover becomes leader election, and the 2PC coordinator state, the fragile single point of Chapter 10, becomes a replicated, self-healing log entry.
2. **A partitioned keyspace under a SQL layer is still relational.** Chapter 9's hash- and range-partitioning did not stop working when transactions arrived; the hard part was never splitting data, it was coordinating across the splits.
3. **Time infrastructure can substitute for communication.** The deepest result, and the subject of most of this chapter: if you can bound clock error, you can order transactions globally without a global sequencer, paying with a small wait instead of a wide round trip.

So the honest restatement of the NewSQL claim is not "scale is free now." It is: **the semantics were never the obstacle; coordination latency was, and remains, the true cost — so pay it in the places you actually need it, and keep SQL and ACID everywhere.** Whether that

cost is acceptable depends on geography and workload, which is why this chapter ends with a ladder rather than a recommendation.

A note on terms. "NewSQL" is the older marketing label and covered some single-node-fast systems (VoltDB-style) that are out of scope here. "Distributed SQL" is the term the surviving systems use for the specific architecture this chapter treats: shared-nothing, consensus-replicated shards under a full SQL engine with distributed ACID transactions. We use the terms in that sense.

Spanner: the archetype

Google published Spanner in 2012 (OSDI), and it remains the reference design: nearly every system in this chapter is either an adaptation of it or defines itself against it. Spanner was built because Google's own teams kept demonstrating Chapter 11's failure mode internally — the paper is blunt that engineers were building consistency and transaction layers on top of Bigtable by hand, and that the database should do it instead.

Architecture: replication is consensus

A Spanner deployment (a *universe*) spans multiple datacenters organized into *zones*. Data lives in **tablets** — contiguous key ranges, the descendants of Bigtable's tablets and the cousins of Chapter 9's range partitions. The critical move is how tablets are replicated: **each tablet's replicas form a Paxos group**, with one replica elected leader holding a time-based lease. There is no separate replication subsystem in the Chapter 8 sense — no primary shipping WAL to standbys with its attendant failover ambiguity. The Paxos log *is* the replication stream, and everything Chapter 8 made you worry about (split brain, lost acknowledged writes, failover data loss) is handled by the consensus safety proof from Volume 6, Chapter 5. Placement — which zones hold replicas, how many, which may vote — is policy configured per *directory* (a bucket of keys sharing a prefix), which is also the unit the system moves around for load balancing.

A single-group transaction — one whose reads and writes all fall in one tablet's key range — commits by replicating its writes through that group's Paxos log. No 2PC is involved, and well-designed schemas (Spanner's interleaved tables exist precisely for this) keep most transactions in this fast path.

Cross-group transactions layer **two-phase commit over the Paxos groups**. Each participating group's leader acts as a participant; one is chosen as coordinator. This is exactly the resolution that Chapter 10 promised when it called classic 2PC "a protocol whose coordinator is a single point of blocking failure": here the coordinator's decision is itself a Paxos-replicated log entry, so a coordinator machine crashing does not strand participants holding locks — the group elects a new leader, which reads the decision (or its absence) from the replicated log and completes the protocol. 2PC's atomicity structure survives; its availability defect is repaired by putting consensus underneath every participant, coordinator included. The cost also survives: a cross-group commit is 2PC's extra round trips, each leg itself a consensus commit.

```

flowchart TB
    subgraph SQL["SQL and transaction layer"]
        C["Coordinator leader  
2PC coordinator for  
cross-group transactions"]
    end
    subgraph G1["Tablet A – Paxos group 1"]
        L1["Leader zone 1  
holds lease, holds locks"]
        F1a["Replica zone 2"]
        F1b["Replica zone 3"]
        L1 --> F1a
        L1 --> F1b
    end
    subgraph G2["Tablet B – Paxos group 2"]
        L2["Leader zone 2"]
        F2a["Replica zone 1"]
        F2b["Replica zone 3"]
        L2 --> F2a
        L2 --> F2b
    end
    C -->|"prepare / commit  
each message is a  
Paxos-replicated log entry"| L1
    C -->|"prepare / commit"| L2
    TT["TrueTime  
GPS + atomic clock  
time masters per datacenter"] -. ->|"TT.now gives interval  
earliest to latest"| C

```

TrueTime: bounded uncertainty as an API

Every distributed database needs to order transactions. Chapter 10's systems used a coordinator; TiDB (below) uses a centralized timestamp oracle; classic Lamport machinery (Volume 6, Chapter 2) gives causal order but says nothing about real time. Spanner's answer is unique because it changes the *hardware* question: Google equipped datacenters with time masters backed by GPS receivers and atomic clocks, cross-checked against each other, with every machine's local daemon polling them and tracking worst-case drift between polls.

The API is the insight. `TT.now()` does not return a timestamp; it returns an **interval** `[earliest, latest]` with the guarantee that the absolute time at which the call executed lies within it. The half-width of that interval, ϵ , is the instantaneous uncertainty: in the 2012 paper it sawtooths between roughly 1 and 7 ms as clocks drift between synchronizations, averaging around 4 ms. Two derived predicates matter: `TT.after(t)` is true once `t` is definitely in the past (`t < TT.now().earliest`), and `TT.before(t)` symmetrically.

Ordinary NTP gives you an estimate with unbounded, unreported error — Volume 6, Chapter 2's central warning. TrueTime gives you a *bounded, reported* error, and every guarantee in the rest of this section is manufactured from that bound. This is the sense in which Spanner's consistency is bought with infrastructure: the guarantee is only as good as the bound, and the paper is explicit that a TrueTime violation (uncertainty exceeding the reported interval) would break correctness, which is why the clock fleet is engineered and monitored like the storage fleet.

Commit-wait: turning clock uncertainty into latency, and latency into consistency

First, be precise about the guarantee being purchased, because it is stronger than Chapter 6's serializability. **Serializability** says the outcome equals *some* serial order of the transactions — but it does not promise that order respects real time. A serializable system may legally order a transaction that committed at 12:00:01 *before* one that committed at 12:00:00, as long as results are consistent with the chosen order; Chapter 6 flagged this when noting that a serializable database may still serve you a stale-but-consistent snapshot. **External consistency** closes that gap: if transaction T1 commits (in real, wall-clock time) before T2 begins, then

T1's commit timestamp is smaller than T2's, and T2 observes T1's effects. In Volume 6, Chapter 3's vocabulary this is **strict serializability** — linearizability lifted from single operations to whole transactions. The difference is observable by ordinary applications: commit an order in one request, read it back in the next request through a different frontend, and external consistency is exactly the promise that the read cannot miss the write — Chapter 8's replication-lag anomalies ruled out globally, by contract.

Here is how TrueTime delivers it. When a read-write transaction commits, the coordinator leader chooses a commit timestamp `s` no smaller than `TT.now().latest` at commit time (and above other lower bounds such as prepare timestamps). Then comes the step everything else in this chapter orbits:

Commit-wait. The leader holds the transaction — locks held, result not yet released — until `TT.after(s)` is true, i.e. until `s` is unambiguously in the past on every clock in the fleet. Only then does it apply the commit and reply to the client.

The wait is roughly 2ε in the worst case, a few milliseconds on average, and overlaps with Paxos replication of the commit record, so much of it is hidden. But its logical effect is total: when the client learns its transaction committed at `s`, real time is already past `s` everywhere. Any transaction that starts afterward — anywhere on the planet, through any replica — will call `TT.now()` and receive an interval entirely above `s`, so its own timestamp must exceed `s`, and the real-time order and the timestamp order agree. No communication established that agreement. The machines never exchanged a message to order these two transactions; they consulted physics.

```

sequenceDiagram
    participant CL as Client
    participant L as Coordinator leader
    participant P as Paxos group
    participant T2 as Later transaction T2

    CL->>L: commit T1
    L->>L: acquire locks, pick s = TT.now latest
    L->>P: replicate commit record at s
    P-->>L: Paxos majority ack
    Note over L: COMMIT-WAIT
    hold until TT.after s is true
    duration about 2 epsilon
        L->>L: TT.after s - s is in the past
    on every clock in the fleet
        L->>L: apply, release locks
        L-->>CL: committed at s
        Note over CL: only NOW does the client learn of s
        CL->>T2: causes T2 somehow - next request
        T2->>T2: TT.now earliest already above s
    so timestamp of T2 above s
        Note over T2: real-time order equals
        timestamp order - external consistency

```

Sit with the trade, because it is the profound one and this chapter's version of Volume 4, Chapter 3's arc that stronger consistency costs performance: **clock uncertainty becomes commit latency, linearly**. Better clocks (smaller ϵ) mean faster commits; degraded clocks mean Spanner slows down but stays correct — the system's response to a time master failure is rising ϵ and rising commit-wait, not corruption. Clock quality is a performance SLO, and consistency has a price denominated in milliseconds-of-uncertainty. It is the cleanest example in this suite of buying a semantic guarantee with an infrastructure investment.

Reads without locks: snapshots at a timestamp

The other half of Spanner's design is what timestamps make cheap. Because every commit carries a globally meaningful timestamp and storage is multi-versioned (Chapter 6's MVCC, keyed now by TrueTime rather than by a local transaction counter), a **read-only transaction** needs no locks and no 2PC: the system assigns it a timestamp — `TT.now().latest` for a fresh externally consistent read — and every replica can answer from its versioned store *as of* that timestamp.

The replica-side mechanism is **safe time**: each replica tracks a timestamp `t_safe` below which it has applied everything that could matter — bounded by the Paxos apply point and, crucially, by the prepare timestamps of any transactions sitting in 2PC's in-doubt window (Chapter 10's uncertainty period reappearing as a *read horizon*). A replica may serve a read at `t` iff `t ≤ t_safe`; otherwise it waits or the read goes elsewhere. Any sufficiently caught-up replica can serve the read — no leader round trip — which is what makes Spanner's snapshot reads scale and what CockroachDB's follower reads (below) reproduce. Clients that can tolerate staleness may ask for a bounded-staleness or exact past timestamp and get answered by the nearest replica; Chapter 8's lag anomalies return here *by explicit, bounded contract* rather than by accident, which is the difference between a feature and an incident.

CockroachDB: distributed SQL without the atomic clocks

Spanner's obvious criticism was that it presumes Google's clock fleet. CockroachDB (open-sourced 2015 onward, from ex-Google engineers) is the direct test of whether the architecture survives on commodity hardware and commodity NTP. Most of it does; the part that does not — bounded uncertainty — is replaced by machinery worth understanding precisely because it shows what TrueTime was *for*.

Ranges, Raft, leaseholders

The keyspace is one ordered map, split into **ranges** (default target on the order of hundreds of megabytes in recent versions), each replicated — typically 3 or 5 ways — by its own **Raft group** (Volume 6, Chapter 6; Raft rather than Paxos, same guarantees). Ranges split, merge, and rebalance automatically as data and load shift; this is Chapter 9's range partitioning with the rebalancing made autonomous. Every node can act as a SQL **gateway**: it parses and plans your Postgres-protocol query and routes KV operations to the relevant ranges. Writes go through Raft on each affected range; cross-range transactions use a 2PC-style protocol whose transaction record is itself Raft-replicated — the same "consensus underneath the coordinator" repair as Spanner's, with a parallel-commit optimization that shaves the extra consensus round from the common case.

One range replica holds the **lease** and is the *leaseholder*: the replica that serves reads and coordinates writes for that range. The point of the lease is read latency: a leaseholder can answer reads from local state *without a Raft round trip*, because the lease (time-bounded, and protected against the clock-skew failure modes Volume 6, Chapter 2 warns about) guarantees no other replica is concurrently committing writes it hasn't seen. This is the fencing-token discipline productized: reads cost one machine, writes cost a Raft quorum.

```
flowchart TB
    APP["Application  
Postgres wire protocol"] --> GW["Gateway node  
SQL parsing, planning,  
KV routing"]
    subgraph R42["Range 42 – key span users slash a to users slash m"]
        LH["Leaseholder + Raft leader  
serves reads locally  
no consensus round trip"]
        F1["Follower replica"]
        F2["Follower replica"]
        LH -->|"Raft log replication  
writes need a majority"| F1
        LH -->|"Raft log replication"| F2
    end
    GW -->|"read – one hop"| LH
    GW -->|"write – Raft quorum commit"| LH
    GW -->|"stale-tolerant read  
AS OF SYSTEM TIME  
follower read"| F2
```

HLCs and the uncertainty window: honesty without TrueTime

CockroachDB timestamps transactions with **hybrid logical clocks** (Kulkarni et al., 2014). An HLC is a physical clock reading welded to a logical counter: it stays close to wall time, but on every message exchange it ratchets forward to exceed any timestamp it has seen, so it *always* respects happens-before along communication paths — Lamport's relation from Volume 4, Chapter 3 and Volume 6, Chapter 2, carried inside a timestamp that still roughly means "when."

Be precise about what that buys and what it does not. **HLCs guarantee causal ordering: if information flowed from event A to event B, B's HLC exceeds A's.** They do **not** bound the gap between the clock and real time — commodity NTP offers no honest bound — so two transactions on *non-communicating* nodes can be timestamped in the

opposite of their real-time order. That is precisely the case TrueTime's commit-wait handled by waiting out the uncertainty, and it is the case CockroachDB must handle differently.

Its approach, described conceptually and honestly:

- **A configured maximum offset.** The cluster assumes clock skew never exceeds `max_offset` (500 ms by default). This is an *assumption*, not a measurement: nodes that observe skew beyond the bound self-terminate to protect correctness (this is why clock health is an operational SLO on CockroachDB — a misbehaving NTP fleet takes nodes down by design). Contrast TrueTime, where the bound is measured and reported per call.
- **The uncertainty window on reads.** A transaction at timestamp `t` that encounters a committed value with timestamp in `(t, t + max_offset]` cannot tell whether that write really happened after it or merely carries a fast clock. To avoid reading a snapshot that excludes a write which in real time preceded it, the reader must assume the worst: it **pushes its own timestamp** above the offending value and retries the read at the new time. Spanner pays uncertainty on every commit; CockroachDB pays it only on actual close encounters — but pays it as restarts rather than as a fixed wait.
- **Read refreshes.** Pushing a transaction's timestamp forward (whether by uncertainty or by conflicts with later readers via the timestamp cache) would naively force a full retry. Instead the gateway tracks the transaction's read set and attempts a **refresh**: re-verify that nothing it read changed between the old and new timestamps. If nothing did, the transaction slides forward and commits at the pushed timestamp; if something did, it restarts for real — surfacing to the application as a retryable serialization error, for which your Chapter 6 retry loop is again mandatory equipment.

The resulting guarantee, stated honestly: CockroachDB is **serializable by default** (for years it offered nothing weaker; recent versions added a READ COMMITTED mode — check current docs), and it prevents stale reads on any given key. It does **not** claim Spanner's full external consistency: without bounded uncertainty there is a narrow, known anomaly (the "causal reverse") in which two transactions on disjoint keys, ordered in real time but communicating only outside the database, can be timestamped in the wrong order. Serializability holds; strict serializability, in full generality, is exactly the thing TrueTime's hardware was buying,

and CockroachDB's documentation says so plainly. For most applications the distinction never bites; for audit-ordering-critical ones it is the honest fine print.

The Postgres surface and its limits

CockroachDB speaks the PostgreSQL wire protocol and a large subset of its SQL dialect, which is a genuinely consequential choice: drivers, ORMs, and muscle memory carry over. The limits matter too, and they cluster predictably. Compatibility is with the dialect, not the engine: the extension ecosystem (PostGIS as an extension, foreign data wrappers, custom C extensions) does not carry over, though some functionality is reimplemented natively; historically stored procedures, triggers, and event-driven features arrived late and partially; and performance intuition does not transfer — a Postgres-tuned workload full of sequential-integer primary keys, `SELECT ... FOR UPDATE` hot spots, or chatty multi-statement transactions runs headlong into range hot spots and per-statement latencies that a single-node engine never showed you. Treat "Postgres-compatible" as "your tools connect and most SQL runs," not "your database is interchangeable."

The rest of the field, briefly and honestly

TiDB (PingCAP) has the same architectural skeleton with different choices: a stateless SQL layer (TiDB servers) speaking the *MySQL* dialect, over **TiKV**, a Raft-replicated key-value store of regions (ranges), with a Placement Driver cluster handling metadata and — the notable difference — a centralized **timestamp oracle** (TSO) allocating transaction timestamps, in the lineage of Google's Percolator rather than of TrueTime. A central TSO makes ordering trivial and uncertainty windows unnecessary inside one deployment, at the cost of a round trip to the TSO's region — which is why the design is most comfortable when writes concentrate near the PD leader, and why multi-region TiDB involves explicit topology work. TiFlash adds columnar replicas for HTAP, foreshadowing Chapter 13.

YugabyteDB is shaped like CockroachDB — tablets, Raft per tablet, HLCs — with one deliberate divergence: rather than reimplement the SQL layer, it reuses PostgreSQL's actual query-engine code atop its distributed storage layer (DocDB), which buys deeper PG feature

compatibility at the price of coupling to that codebase. It also exposes a Cassandra-like API alongside SQL.

Vitess and Citus are the other path, and the contrast is the instructive part. Vitess (built at YouTube, now CNCF) is sharding middleware over stock MySQL: a routing proxy layer (vtgate) presents one logical database over many independent MySQL shards. Citus is the same idea as a PostgreSQL extension with a coordinator over worker PG nodes. Neither runs consensus per shard, and neither gives you general serializable cross-shard transactions — single-shard transactions are ordinary local ACID, cross-shard writes are limited, best-effort, or (in Vitess's optional 2PC) explicitly discouraged. That sounds like a defect and is often precisely the right trade: if your workload shards cleanly along a tenant or user key (Chapter 9's central design question), then virtually all transactions are single-shard, each shard is a boring, mature, locally-fast MySQL or Postgres, and you have scaled SQL *without* buying global coordination you would never use. The distributed-SQL engines earn their complexity only when transactions genuinely cross the partition key.

Aurora is a different beast entirely, and the taxonomy point is worth making carefully because "Aurora" and "Spanner" get said in the same breath constantly. Amazon Aurora (SIGMOD 2017 paper) keeps a *single-writer* MySQL- or Postgres-compatible compute node and reinvents the layer *below* it: the storage is a distributed, multi-tenant service holding six copies of every data segment across three AZs, and the instance ships **only the redo log** to it — "the log is the database." Writes are acknowledged at a 4-of-6 quorum (Chapter 7's WAL meeting Volume 6, Chapter 7's quorums); pages are materialized from the log by the storage fleet, not flushed by the database. This buys excellent durability, fast crash recovery, fast read-replica creation, and storage that scales independently of compute. It does **not** buy horizontal *write* scaling or distributed transactions: there is one writer, and the ceiling on write throughput is that node — Chapter 8's single-primary model with a spectacular storage subsystem, operated for you (Chapter 14's theme). Choosing Aurora because "we need to scale like Spanner" is a category error in both directions.

AlloyDB and Neon mark the emerging pattern this points to: storage-compute disaggregation for *Postgres itself*. Neon, for instance, splits the engine into stateless compute, a small consensus cluster

durably receiving the WAL (safekeepers — Paxos again, guarding the log), and pageservers that materialize versions from it, enabling copy-on-write branching and scale-to-zero; AlloyDB applies a similar log-centric disaggregation inside Google Cloud. The taxonomy now has three distinct columns, and conflating them causes real architectural mistakes:

	Sharding middleware	Disaggregated single-writer	Distributed SQL
Examples	Vitess, Citus	Aurora, AlloyDB, Neon	Spanner, CockroachDB, TiDB, YugabyteDB
Write scaling	Yes, per shard key	No — one writer	Yes, per range/ tablet
Cross-shard serializable txns	No / limited	N/A — one node	Yes
Consensus in the write path	No	Storage quorum / WAL consensus	Yes, per range
Failure/ consistency model	Per-shard classic	Classic engine, better durability	Chapter 6 + Volume 6 fused
Latency floor per write	Local commit	Storage quorum, same-region	Raft/Paxos quorum RTT, + geography

The costs, stated plainly

Latency floors: geography is physics

Reason from round trips, not vendor benchmarks. Within a region, inter-AZ RTT is on the order of 0.5–2 ms; across the continental US, roughly 60–70 ms; transatlantic, roughly 70–90 ms. A consensus write commits after the leader hears from a majority, so its floor is the RTT to the *nearest majority* of replicas — plus, for cross-range transactions, 2PC's additional consensus round(s), plus (Spanner) commit-wait's few milliseconds.

Run the arithmetic for three deployments of a 3-replica range:

- **Single region, three AZs.** Quorum RTT $\approx$ 1-2 ms. Write latency floor: single-digit milliseconds. This is the benign case — and still several times a single-node local commit, which is why per-statement chatty transactions that were free on one Postgres node become visible here.
- **Three regions, US-East / US-Central / US-West, leader in the East.** Majority = leader plus one of Central ($\sim$ 30 ms RTT) — floor $\approx$ 30-35 ms per consensus commit, doubled-ish for cross-range 2PC. Every write, forever, regardless of load.
- **Global — US / Europe / Asia.** Majority spans an ocean: 70-110 ms floors, and a transaction touching ranges whose leaders sit on different continents compounds them.

No implementation cleverness removes these floors; they are speed-of-light terms. The mitigations are all *topology*: place leaders near writers, and declare which data needs which geography. The system-level vocabulary (CockroachDB's is representative; Spanner's configurations are analogous) is **global tables** (read everywhere fast, write slow), **regional tables** (fast in their home region), and — the workhorse for user-partitioned data — **regional-by-row**, where each row homes itself:

```
-- CockroachDB multi-region sketch
ALTER DATABASE app PRIMARY REGION "us-east1";
ALTER DATABASE app ADD REGION "eu-west1";
ALTER DATABASE app ADD REGION "ap-southeast1";

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       STRING UNIQUE NOT NULL,
  home        crdb_internal_region NOT NULL DEFAULT default_to_databases_primary_region(gateway_region()),
  profile     JSONB
) LOCALITY REGIONAL BY ROW AS home;
-- each row's replicas and leaseholder follow its region column:
-- EU users commit against EU replicas at intra-region latency
```

This is Chapter 9's partition-key design decision returning at continental scale, and Volume 7, Chapter 10 treats the full multi-region design space. The complementary read-side lever is the **follower read**: explicitly stale, served by the nearest replica without touching the leaseholder — Chapter 8's read-replica lag, reborn with a contract:

```
-- served by the closest replica; bounded staleness, no leaseholder  
round trip  
SELECT balance  
  FROM accounts AS OF SYSTEM TIME follower_read_timestamp()  
 WHERE id = $1;
```

Skew, operations, and money

Distribution does not fix skew. Chapter 9's hot-partition pathology transfers intact: a monotonically increasing primary key turns the last range into a single-Raft-group bottleneck; a celebrity row is hot no matter how many ranges surround it. The engines mitigate — automatic load-based splitting, hash-sharded indexes — but a workload whose writes concentrate on one key gets one Raft group's throughput, full stop.

Operational complexity is real and novel. You now operate a consensus fleet: rebalancing churn, lease thrashing, snapshot transfer storms after node loss, upgrade orchestration across a quorum — and, distinctively, *clock health as a first-class SLO* in HLC-based systems, where NTP misbehavior doesn't degrade performance, it removes nodes. Managed offerings shift this work but not the latency physics or the schema-design obligations.

And it costs more. 3× or 5× storage, cross-region egress on every replication message, and — less visibly — engineering time spent on retry loops, hot-spot-aware schema design, and topology configuration. Against this, weigh what it replaces: the sharding middleware, the fan-out services, the reconciliation jobs, and the incident load of Chapter 11's hand-rolled consistency.

Choosing: the honest ladder

The anti-pattern to name first, because it is now common: **a single-region CRUD application on a distributed SQL engine**, paying quorum latency on every write, running retry loops it never needed, and operating (or renting) a consensus fleet — for a workload one Postgres node with a replica would serve with lower latency, richer features, and two decades of operational folklore. Distributed SQL is a rung you climb to, under pressure, not a default.

```

flowchart TB
    A["Single-node Postgres or MySQL  
plus the Chapter 14 levers:  
pooling, tuning, bigger box, caching"]
    B["Add read replicas – Chapter 8  
scales reads, accepts lag,  
writes still one node"]
    C["Writes exceed one node,  
or data exceeds one box?"]
    D["Do transactions cross  
the natural shard key?  
Is multi-region write  
a real requirement?"]
    E["Sharding middleware – Chapter 9  
Vitess, Citus  
per-shard classic SQL,  
no global transactions"]
    F["Distributed SQL  
Spanner, CockroachDB, TiDB, Yugabyte  
pay consensus latency,  
design topology deliberately"]
    A -->|"read pressure"| B
    B --> C
    C -->|"no – stay here"| B
    C -->|"yes"| D
    D -->|"no – clean shard key,  
single-region"| E
    D -->|"yes – cross-key transactions  
plus write scale plus geography"| F

```

Climb it in order. Chapter 14's operating levers first — a well-tuned single node with connection pooling handles more than most teams believe, with the simplest possible semantics. Read replicas next, accepting Chapter 8's documented anomalies for read scale. If write volume or data size genuinely exceeds one node, ask Chapter 9's question — *is there a clean partition key that transactions rarely cross?* — and if yes, sharding middleware keeps every component boring. The distributed SQL engines are the correct rung when three requirements **coexist**: write scale beyond one node, relational semantics with transactions that genuinely cross any shard key you could pick, and (usually) multi-region presence with real consistency needs. When all three hold, these systems are not over-engineering; they are the only honest answer, and everything else on the ladder is a pile of application-level consistency code waiting to be written badly. Aurora-style disaggregation sits *beside* the ladder, not on

it: choose it for durability, recovery, and elasticity around a single writer, not for write scale.

The distributed-systems lens

Every prior chapter's lens section pointed outward from a database topic toward distributed systems. This chapter *is* the meeting point, so the lens is a set of identifications rather than analogies.

Every mechanism here is a Volume 6 topic productized. Replication-as-consensus is Chapters 5–6 wearing a `CREATE TABLE` statement; leaseholder reads are leases and fencing from Chapter 2's neighborhood; safe-time snapshot reads are watermarking; Aurora's 4-of-6 writes are Chapter 7's quorum intersection; 2PC-over-Paxos is Chapter 10 of this volume repaired by Volume 6's machinery. If you can read this chapter's architectures fluently, you have passed Volume 6's practical exam.

TrueTime versus HLC is the definitive case study in buying consistency with time. Volume 4, Chapter 3 began an arc — stronger guarantees cost performance, and you choose where to pay — that reaches its logical conclusion here. Spanner pays in hardware and a fixed commit-wait, and receives external consistency: physics as a communication channel. CockroachDB pays nothing in hardware, assumes a skew bound, and pays in occasional restarts and a documented edge-case anomaly. Neither is wrong; together they are the cleanest demonstration in production systems that *time infrastructure is interchangeable with coordination*, at an exchange rate set by clock quality. External consistency itself is Volume 6, Chapter 3's linearizability, applied to transactions — one vocabulary, one design space, from CPU store buffers to transatlantic commits.

The meta-lesson is Codd's vindication. Chapter 1 argued that the relational model's deepest idea was data independence — programs state *what*, the system decides *how*. The NoSQL decade seemed to refute it at scale; this chapter records the refutation failing. SQL and ACID crossed the distribution transition intact: the *interface* survived while the *implementation* was rebuilt from consensus groups and clock fleets, which is precisely what a good abstraction is for. The applications atop Spanner do not know about Paxos, exactly as the applications of 1985 did not know about B-trees. Abstractions that encode *what* rather than *how*

are the ones that survive their implementations — a lesson worth carrying well beyond databases.

Key takeaways

- The NoSQL trade — semantics for scale — was contingent, not fundamental. Distributed SQL keeps full SQL and serializable transactions at horizontal scale; the real, permanent costs are **consensus latency floors and operational complexity**.
- Spanner's skeleton is the reference: tablets, **each a Paxos group** (replication is consensus), with **2PC layered over Paxos groups** so the coordinator of Chapter 10 is no longer a single point of blocking failure.
- **TrueTime** exposes bounded clock uncertainty as an interval; **commit-wait** holds each commit until its timestamp is unambiguously past, converting clock uncertainty directly into commit latency — and buying **external consistency** (strict serializability), which is strictly stronger than Chapter 6's serializability because it respects real-time order.
- Timestamps plus MVCC give **lock-free snapshot reads at any sufficiently old timestamp**, served by any replica past its safe time — the scalable read path of every system in this family.
- CockroachDB shows the no-special-hardware translation: ranges + Raft + leaseholders, **HLCs** (causal order, but *no bounded uncertainty*), an assumed `max_offset` with nodes self-evicting on violation, and uncertainty restarts / timestamp pushing / read refreshes in place of commit-wait. Serializable by default; honestly short of full external consistency.
- Taxonomy matters: **Vitess/Citus** scale SQL without global transactions (right when the shard key is clean); **Aurora/AlloyDB/Neon** disaggregate storage under a *single writer* ("the log is the database") and do not scale writes; only the Spanner family gives cross-shard serializable transactions.
- The costs are physics and skew: quorum RTT floors on every write, topology decisions (regional-by-row, leader placement, follower reads with contractual staleness), hot ranges that distribution cannot dissolve, and clock health as an SLO.

- Climb the ladder in order: tuned single node → read replicas → sharding middleware **or** distributed SQL — the latter only when write scale, cross-key transactions, and geography genuinely coexist. Paying global-consensus latency for a single-region CRUD app is the era's signature anti-pattern.
- The meta-lesson: SQL and ACID survived the distribution transition because they specify *what*, not *how* — data independence proved portable to planet scale.

Further reading

- Corbett, J. C., Dean, J., et al., "Spanner: Google's Globally-Distributed Database," *OSDI*, 2012 — the archetype paper: Paxos groups, TrueTime, commit-wait, external consistency. <https://research.google/pubs/spanner-googles-globally-distributed-database/>
- Kulkarni, S., Demirbas, M., Madappa, D., Avva, B., and Leone, M., "Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases" (Hybrid Logical Clocks), 2014 — the HLC design CockroachDB and YugabyteDB build on. <https://cse.buffalo.edu/tech-reports/2014-04.pdf>
- Taft, R., et al., "CockroachDB: The Resilient Geo-Distributed SQL Database," *SIGMOD*, 2020 — the peer-reviewed architecture description, including transactions, parallel commits, and clock handling.
- CockroachDB documentation and engineering blog — the architecture overview, "Living Without Atomic Clocks," and the multi-region and follower-reads guides. <https://www.cockroachlabs.com/docs/> and <https://www.cockroachlabs.com/blog/living-without-atomic-clocks/>
- Verbitski, A., et al., "Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases," *SIGMOD*, 2017 — "the log is the database," 6-way/4-of-6 quorum storage. <https://dl.acm.org/doi/10.1145/3035918.3056101>
- TiDB documentation — architecture (TiDB/TiKV/PD), the Percolator-derived transaction model, and the timestamp oracle. <https://docs.pingcap.com/>
- YugabyteDB documentation — DocDB architecture and the reuse of the PostgreSQL query layer. <https://docs.yugabyte.com/>

- Vitess documentation <https://vitess.io/docs/> and Citus documentation <https://docs.citusdata.com/> — the sharding-middleware path.
- Peng, D. and Dabek, F., "Large-scale Incremental Processing Using Distributed Transactions and Notifications" (Percolator), *OSDI*, 2010 — the ancestor of TiDB's transaction protocol.
- Volume 6, Chapters 2, 3, 5, 6, 7 — clocks, consistency models, Paxos, Raft, quorums: the mechanisms this chapter productized. Volume 7, Chapter 10 — multi-region architecture, where the topology decisions sketched here are treated in full.

Chapter 13 — Specialized Stores: Search, Time-Series, and Analytics

What this chapter covers. Chapters 1 through 12 built a complete picture of the general-purpose database: B-tree and LSM storage engines (Chapter 2), secondary indexes (Chapter 3), a query processor (Chapter 4), transactions and MVCC (Chapters 5–6), a WAL (Chapter 7), replication (Chapter 8), and partitioning (Chapter 9). That machinery is tuned for one workload shape: point and short-range reads and writes of rows, identified by key — OLTP. This chapter is about three workload shapes different enough that a purpose-built engine beats the general-purpose one by orders of magnitude: full-text relevance search, high-cardinality append-heavy time series, and scan-heavy analytical aggregation. For each, we work out *why* the row store loses from first principles, what structure the specialized store substitutes, and what new failure modes you inherit in the trade. We close with the architectural point that matters more than any individual store: these systems are almost never the source of truth. They are **derived views**, fed from an OLTP system by change-data-capture, and the properties that make that safe — rebuildability, idempotent apply, tolerated lag — are design obligations, not defaults. The intellectual anchor is Stonebraker's 2005 argument that "one size fits all" was over, plus the observation that every specialized store is assembled from the *same* primitives you already know — LSM structure, partitioning, replication, log-based recovery — tuned for a different access pattern.

Learning goals — after this chapter you should be able to:

- Explain the inverted index and the analysis pipeline, and say precisely why `LIKE '%term%'` cannot be rescued by any B-tree index.
- Describe BM25's shape — what term-frequency saturation and length normalization each contribute — and read a relevance score without mysticism.
- Explain Lucene's immutable-segment architecture as an LSM variant, and derive Elasticsearch's near-real-time visibility semantics from it.
- Characterize the time-series workload shape, explain Gorilla-style delta-of-delta and XOR compression, and identify label cardinality as the operational hazard it is.
- Derive the columnar I/O advantage arithmetically, name the four compression families it unlocks, and explain zone maps, late materialization, and vectorized execution.
- Sketch the modern OLTP-to-analytics pipeline: CDC into a warehouse or lakehouse, Parquet's physical layout, and what Iceberg/Delta add on top.
- Argue for and against adding a specialized store, in terms of the pipeline and lag SLOs you will pay for it.

One size does not fit all — and why

In 2005 Michael Stonebraker and Uğur Çetintemel published a paper with an unusually blunt title: *"One Size Fits All": An Idea Whose Time Has Come and Gone*. The argument: the commercial RDBMS of the day — row-oriented, B-tree-indexed, tuned over twenty-five years for OLTP — was being sold for every workload, while for several major workloads a purpose-built engine could beat it by one to two orders of magnitude. It was partly a research manifesto (Stonebraker's own C-Store and StreamBase were the exhibits), but two decades later it reads as simple description. Search runs on Lucene, not on `LIKE`. Metrics run on Prometheus or InfluxDB, not on MySQL. Analytics runs on ClickHouse, BigQuery, or Snowflake, not on the OLTP primary.

It is worth being precise about *why* specialization wins, because the reason is not magic. A storage engine is a bundle of bets: what is contiguous on disk, what the unit of I/O is, what is indexed, what is compressed against what, what stays hot in memory. Chapter 2's B-tree

row store bets that you read and write individual rows by key, that reads and writes interleaved, and that any column of a row is about as likely to be wanted as any other. Each workload in this chapter violates those bets so thoroughly that honoring them becomes pure overhead:

Dimension	OLTP (Ch 1-12)	Full-text search	Time series	Analytics/OLAP
Unit of interest	Row by key	Documents ranked by relevance	Points in a series over a time range	Aggregates over columns of many rows
Write pattern	Update-in-place, random keys	Index-once, rarely update	Append-only, time-ordered	Bulk load, append
Read pattern	Point/short range	Top-k by score	Range scan of one series, aggregate	Full-column scans
Deletes	Individual rows	Rare; tombstoned	By retention window, in bulk	Rare; by partition
Data layout that wins	Row-major, B-tree	Term-major: inverted index	Series-major, time-chunked	Column-major
Compression leverage	Modest (heterogeneous rows)	Postings compression	Extreme (adjacent points similar)	Extreme (homogeneous columns)

Read the last two rows together and the chapter's theme appears: **specialized stores win by choosing a physical layout in which the data a query touches is contiguous and compresses well, at the cost of making other access patterns awkward or impossible.** A row store spreads a term's occurrences, a series' points, and a column's values across the whole table; each specialized store gathers exactly one of those together. Nothing gathers all three — which is why you end up

running more than one store, and why this chapter ends with the pipelines between them.

Full-text search

The inverted index

The problem: given a few query terms, return the documents most *relevant* to them, from a corpus of millions, in tens of milliseconds. The structure that solves it is the inverted index, and the name says what it is. A forward index maps document → terms (that is just the document); an inverted index maps **term → postings list**: for every term in the corpus, a sorted list of the IDs of documents containing it, usually with the term's positions in each document and enough statistics to score matches.

A query for `quorum lease` becomes: fetch the two postings lists, intersect them (both are sorted by document ID, so this is a linear merge — Chapter 4's merge join), and score the survivors. The work is proportional to the lengths of two postings lists, not to the size of the corpus.

Contrast the row store. `WHERE body LIKE '%quorum%'` cannot use a B-tree index on `body` at all: Chapter 3 established that a B-tree serves prefixes and ranges of its sort order, and a substring anchored nowhere has no prefix. The engine's only option is a full scan of every row, running a substring search over every document — $O(\text{corpus})$ per query, plus it answers the wrong question, because `LIKE` knows nothing about word boundaries ("quorums"? "Quorum,"?), morphology ("leasing" vs "lease"), or ranking. The inverted index is not a faster `LIKE`; it is a different question answered by a different structure.

```

flowchart LR
  subgraph DOCS["Documents"]
    D1["doc 1: 'the leader holds a lease'"]
    D2["doc 2: 'quorum reads need no lease'"]
    D3["doc 3: 'a quorum of replicas'"]
  end
  subgraph IDX["Inverted index: term to postings"]
    T1["'leader' : 1"]
    T2["'lease' : 1, 2"]
    T3["'quorum' : 2, 3"]
    T4["'replica' : 3"]
  end
  DOCS -->|"analysis: tokenize, lowercase, stem"| IDX
  Q["query: quorum lease"] --> T2
  Q --> T3
  T2 --> M["intersect sorted postings: 1,2 with 2,3 gives 2"]
  T3 --> M
  M --> R["score doc 2 with BM25, return ranked hits"]

```

The analysis pipeline

Between raw text and the index sits the **analyzer**, and it is where most real-world search quality problems live. Analysis turns a string into a token stream through a pipeline that typically includes: tokenization (splitting on word boundaries — nontrivial for CJK languages, code identifiers, or IPv4 addresses), normalization (lowercasing, Unicode folding so `café` matches `cafe`), stop-word removal (dropping `the`, `a`, `of` — less fashionable than it was, since BM25's IDF term already down-weights them and removing them breaks phrase queries like "to be or not to be"), and stemming or lemmatization (reducing `leasing`, `leased`, `leases` to a common stem so they match each other). For substring-within-word and autocomplete needs there are **n-gram** and edge-n-gram token filters, which index every character window of a token — powerful, and an easy way to multiply index size several-fold if applied carelessly.

The iron rule is that the query must be analyzed the same way as the documents, or matches silently vanish: a document indexed with stemming and a query analyzed without it will miss every inflected form. This — not the index structure — is the source of most "search is broken" tickets.

A realistic Elasticsearch mapping makes the pipeline concrete. Note the standard idiom of indexing one source field two ways: analyzed `text` for matching, and un-analyzed `keyword` for exact filtering, sorting, and aggregation.

```
PUT /articles
{
  "settings": {
    "analysis": {
      "analyzer": {
        "english_articles": {
          "tokenizer": "standard",
          "filter": ["lowercase", "asciifolding", "english_stemmer"]
        }
      },
      "filter": {
        "english_stemmer": { "type": "stemmer", "language": "english" }
      }
    }
  },
  "mappings": {
    "dynamic": "strict",
    "properties": {
      "title": { "type": "text", "analyzer": "english_articles",
        "fields": { "raw": { "type": "keyword" } } },
      "body": { "type": "text", "analyzer": "english_articles" },
      "tags": { "type": "keyword" },
      "published": { "type": "date" }
    }
  }
}
```

Two deliberate choices deserve comment. `"dynamic": "strict"` disables dynamic mapping — by default Elasticsearch will add a field to the mapping for every new JSON key it sees, and a service that logs documents with user-controlled or ever-growing key sets (per-customer keys, timestamps as keys) will suffer a **mapping explosion**: thousands of fields, each with index structures, ballooning cluster state that must be replicated to every node. And `tags` is `keyword`, not `text`: running an aggregation on an analyzed field either fails or aggregates on stems, a classic analyzer pitfall.

Relevance: from TF-IDF to BM25

Boolean matching finds candidates; ranking decides what the user sees. The classical intuition is **TF-IDF**: a document matters more for a term the

more often the term occurs in it (term frequency), and a term matters more the fewer documents it occurs in (inverse document frequency — `lease` is informative, `the` is not). Multiply, sum over query terms, sort.

Raw TF-IDF has two defects that the modern standard, **BM25** (the Okapi weighting function, Robertson and Walker's line of work in the 1990s), repairs. The score of document D for query Q is, per query term q_i with frequency $f(q_i, D)$ in the document:

$$\text{score}(D, Q) = \sum_{q_i} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot |D| / \text{avgdl})}$$

At concept level, honestly, each piece does one job:

- **IDF(q_i)** carries over the rare-terms-matter intuition (in a smoothed logarithmic form).
- **Term-frequency saturation.** In raw TF, a document saying `lease` 200 times scores twice one saying it 100 times. In BM25 the TF factor is $f \cdot (k_1 + 1) / (f + k_1 \dots)$ — a saturating curve that approaches an asymptote as f grows. The second occurrence of a term adds a lot of evidence; the fiftieth adds almost none. k_1 (typically ~ 1.2) controls how fast saturation sets in; $k_1 = 0$ collapses TF to binary presence.
- **Length normalization.** Long documents contain more of everything, so raw TF favors them unfairly. The $(1 - b + b \cdot |D| / \text{avgdl})$ factor in the denominator penalizes documents longer than the corpus average `avgdl`. b (typically ~ 0.75) sets how strongly: $b = 0$ ignores length entirely, $b = 1$ normalizes fully.

That is the whole formula's shape: rare terms weighted up, repeated terms with diminishing returns, long documents handicapped. Lucene made BM25 its default similarity in version 6.0; it remains the baseline that learned ranking models are measured against, and for most applications it is what you should ship first.

Phrase and proximity queries are why postings lists store positions. `"leader lease"` as a phrase requires not just that both terms appear in a document but that a position of `lease` equals a position of `leader` plus one; the engine intersects postings and then walks the two position lists. Proximity queries (`"leader lease"~5` — within five positions) relax the offset check. Positions typically add substantially to index size, which is why they are optional per field.

Lucene: LSM for documents

Apache Lucene — the library inside Elasticsearch, OpenSearch, and Solr — has an architecture you already know, because Chapter 2 taught it under another name. A Lucene index is a collection of **immutable segments**. Incoming documents accumulate in an in-memory buffer; a **flush** writes the buffer out as a new segment — a complete, self-contained mini-index with its own term dictionary and postings. Background **merges** take several small segments and rewrite them as one larger segment. Substitute *memtable* for buffer and *SSTable* for segment and this is an LSM tree, adopted for the same reason: an inverted index is expensive to update in place (an insert touches a postings list for every distinct term in the document), and cheap to rebuild in bulk for a batch of documents. Sequential writes of immutable files, deferred consolidation by merge — the same bet, made for search in the early 2000s.

The consequences are the LSM consequences, translated:

- **Deletes are tombstones.** You cannot remove a document from an immutable segment, so a delete marks the document dead in a per-segment live-docs bitmap. Dead documents still occupy space and still appear in the statistics (skewing IDF slightly) until a merge rewrites the segment without them. An update is a delete plus a reinsert of the whole document.
- **Merges are the background tax.** Merge policy tuning in Elasticsearch is Chapter 2's compaction tuning with the serial numbers filed off, including the failure mode of merges falling behind under heavy indexing.
- **Visibility is decoupled from durability.** A newly indexed document is not searchable until a **refresh** opens a new point-in-time searcher over the current segment set (Elasticsearch refreshes every second by default); it is not durable in Lucene terms until a **commit** fsyncs the segment state (Elasticsearch layers its own translog — a WAL, Chapter 7 — so that acknowledged writes survive a crash between commits). This is the precise meaning of "near-real-time search": *visible after refresh, not after acknowledgment*. Index a document and immediately search for it and you will usually not find it. This is not a bug and cannot be configured away without destroying indexing throughput; applications must be written for it, a point we return to at the end of the chapter.

Elasticsearch and OpenSearch: distributed Lucene

Elasticsearch (and its fork OpenSearch) wraps Lucene in the distribution machinery of Chapters 8 and 9: an index is split into **primary shards** (each shard is one Lucene index), documents are routed to a shard by hash of the document ID — Chapter 9's hash partitioning — and each primary has zero or more **replica shards** that the primary feeds, serving both durability and read throughput. A search fans out to one copy of every shard, each shard returns its local top-k, and a coordinating node merges the per-shard results into a global top-k — scatter-gather, with the usual tail-latency implication that the query is as slow as the slowest shard.

Two honest caveats belong here. First, relevance scores are computed per shard by default, from per-shard term statistics; with few documents or skewed routing, the same document can score differently depending on which shard it landed on. Second, and more important: **Elasticsearch is not your system of record**. Its consistency story is tuned for search availability, not for the guarantees of Chapters 5–8; documents become visible on primaries and replicas at different refresh moments, and the project's long public history of resiliency issues (tracked for years against Jepsen findings) argues for keeping authoritative data in a store with stronger promises and treating the search cluster as a rebuildable view — which is how virtually every mature deployment uses it.

The "good enough" tier: Postgres full-text search

Before operating a search cluster, check whether the database you already run suffices. PostgreSQL ships genuine full-text search: `to_tsvector('english', body)` runs an analysis pipeline (tokenization, stemming, stop words) producing a `tsvector` of lexemes and positions; `@@ to_tsquery('quorum & lease')` matches; a **GIN index** on the `tsvector` — Chapter 3's generalized inverted index, the same term-to-postings idea inside Postgres — makes matching fast; `ts_rank` provides TF-style ranking (not BM25). You get all of it inside your existing transactions, backups, and replication — the index is updated in the same transaction as the row, so search is *consistent*, which Elasticsearch will never give you.

The limits are equally real: ranking quality below BM25, no fuzziness or sophisticated relevance tooling out of the box, analysis less rich than Lucene's, and — because GIN updates amplify writes — indexing cost on

hot tables. The honest engineering position: for moderate corpus sizes and utilitarian search (admin panels, catalogs in the hundreds-of-thousands of items, internal tools), Postgres FTS is the right answer, because the best pipeline is the one you do not have to build. Reach for a dedicated engine when relevance quality is a product surface, when the corpus or query load outgrows one node, or when you need the analysis zoo.

Time-series databases

The workload shape

A time series is a sequence of (timestamp, value) points belonging to an identified series — in the Prometheus data model, a metric name plus a set of label key-value pairs: `http_requests_total{service="checkout", method="POST", status="500"}`. The workload has a shape so consistent it can be listed:

- **Append-mostly, time-ordered.** New points arrive at "now," per series, at roughly regular intervals. Out-of-order arrivals exist but are the exception; updates and deletes of individual points essentially never happen.
- **High cardinality of series, small values.** Millions of active series, each point a timestamp and (usually) one float.
- **Recent-hot, old-cold.** Dashboards and alerts hammer the last hour; last month is occasionally queried; last year is compliance.
- **Aggregation-dominant reads.** Nobody reads one point. Queries are range scans over a time window of one or many series, folded through `rate`, `avg`, `max`, quantiles.
- **Retention as a first-class operation.** Data expires wholesale by age, and is often **downsampled** — replaced by lower-resolution rollups — on the way out.

Put this into Chapter 2's B-tree row store and everything grates. Each sample is a full row with its key overhead, so a 16-byte fact costs perhaps 60-100 bytes on disk. Inserts keyed by (series, time) all land at the right edge of each series' key range — fine — but with millions of series the insertion points are scattered across the tree, so the working set of dirty pages is enormous. Retention by `DELETE WHERE time < ...` is Chapter 6's nightmare: it churns MVCC versions and WAL for every expired row and

leaves the vacuum system to reclaim them. And an aggregation over one series reads pages that interleave many series' rows, wasting most of every I/O.

TSDBs restructure around the shape with two moves: partition by time, and compress within series.

Time partitioning: retention as `drop`

Every serious TSDB stores data in **time-partitioned chunks** — Prometheus's two-hour blocks, TimescaleDB's chunks, InfluxDB's shard groups. This is Chapter 9's range partitioning on the time dimension, and it converts the workload's hardest operations into trivial ones. Writes touch only the current ("head") chunk, so the hot working set is small and the older chunks can be compressed, compacted, and made immutable — the LSM shape again. Queries carry a time range, so the planner prunes to the overlapping chunks before reading anything. And retention becomes **dropping whole chunks**: unlink files, no per-row deletes, no MVCC churn, no vacuum debt. When TimescaleDB documentation tells you to use `drop_chunks` instead of `DELETE`, it is this exact mechanism.

TimescaleDB is worth singling out as the "specialization as an extension" design point: a **hypertable** is declarative time partitioning layered over ordinary PostgreSQL — `create_hypertable('metrics', 'time')` and the extension creates and prunes chunk tables automatically, adds native columnar compression within chunks, and provides continuous aggregates for rollups — while keeping full SQL, joins against your business tables, and your existing operational tooling. It is to time series roughly what Postgres FTS is to search: the tier to exhaust before running a separate system.

Gorilla compression: delta-of-delta and XOR

The second move is compression that exploits *similarity between adjacent points of one series*, and the canonical description is Facebook's **Gorilla** paper (VLDB 2015), describing the in-memory TSDB behind their monitoring. Its two techniques are now everywhere — Prometheus's chunk encoding, InfluxDB's TSM engine, TimescaleDB's compressed chunks, M3 — and both are simple enough to walk through honestly.

Timestamps: delta-of-delta. Scrape intervals are nearly constant, so consecutive deltas between timestamps are nearly identical. Store the

first timestamp; then for each point compute the delta, and the delta between this delta and the previous one. For a series scraped every 60 s, that second difference is almost always exactly zero — and Gorilla encodes a zero delta-of-delta as a **single bit**. Nonzero values are encoded with a short prefix code selecting a width bucket sized to hold them. In Facebook's measurements, about 96% of timestamps compressed to one bit.

```
flowchart TB
    subgraph RAW["Raw timestamps: 64 bits each"]
        R1["t0 = 02:00:00"] --> R2["02:01:00"] --> R3["02:02:00"] --> R4["02:03:02"] --> R5["02:04:02"]
    end
    end
    subgraph DELTA["First delta, seconds"]
        A1["60"] --> A2["60"] --> A3["62"] --> A4["60"]
    end
    end
    subgraph DOD["Delta-of-delta: what is stored"]
        B1["0 : one bit"] --> B2["plus 2 : few bits"] --> B3["minus 2 : few bits"]
    end
    end
    RAW --> DELTA --> DOD
    DOD --> N["regular series: about one bit per timestamp instead of 64"]
```

Values: XOR of adjacent floats. Successive samples of one series are usually close in value, so the IEEE-754 bit patterns share their sign, exponent, and high mantissa bits, and `current XOR previous` is a word that is mostly zeros. If the XOR is exactly zero (value unchanged — extremely common for gauges), store a single `0` bit. Otherwise store a control bit and the nonzero "meaningful" middle of the XOR, described by its count of leading and trailing zero bits — with a further optimization that if the meaningful bits fall inside the same window as the previous value's, the window description can be reused. Across Facebook's production data the two techniques together averaged **about 1.37 bytes per point** — roughly 12× smaller than the naive 16 bytes, before any general-purpose compression. That ratio is what makes "keep every point at full resolution for weeks, in RAM or on NVMe" economically sane, and it is only available because the layout put each series' points adjacent: compression leverage is a *consequence of layout*, a theme that returns at full strength in the columnar section.

Cardinality: the operational hazard

The failure mode that actually pages people is not compression or retention; it is **cardinality**. Every distinct combination of metric name and label values is its own series, with its own index entries, head-chunk memory, and per-series overhead. Total series count is *multiplicative* in label values: a metric with 10 services × 50 endpoints × 5 status classes × 200 pods is 500,000 series before you have measured anything twice. Add one label with unbounded values — user ID, request ID, a raw URL path with embedded IDs, an error message — and the series count goes vertical. Prometheus degrades in a characteristic way: memory grows with active head series; **churn** (pods restarting with new instance labels, deploys rolling) creates floods of new series whose old versions linger in the head block until it is cut; queries that touch many series slow down long before any one series is large.

```
# p99 request latency per service – a sane aggregation.
histogram_quantile(
  0.99,
  sum by (service, le) (rate(http_request_duration_seconds_bucket[5m]))
)
```

Two cardinality readings of that innocuous query. First, the `sum by (service, le)` is not cosmetic: it aggregates away `instance`, `pod`, and `endpoint` *before* the quantile, so the query's cost scales with services × buckets rather than with every underlying series — on a large fleet the difference between milliseconds and a timed-out dashboard panel. Second, the series the query reads exist per bucket (`le`) per label combination: a 12-bucket histogram multiplies the metric's cardinality by 12. Histograms are the most common accidental cardinality bomb. The rule to socialize on your team: **labels are for bounded, low-cardinality dimensions you will actually group by; anything identifier-shaped belongs in logs or traces, not labels**. Volume 11, Chapter 2 develops the operational side — detection, per-team series budgets, and the remote-write ecosystem (Thanos, Mimir, M3) that federates many Prometheus servers' data into long-term storage.

Prometheus itself makes an instructive design study in what the workload shape permits: it **pulls** metrics by scraping targets (which gives it target-health knowledge for free and puts load control in the collector), stores to a **local** TSDB with a WAL (Chapter 7's recovery-by-replay, verbatim) and two-hour blocks, and deliberately refuses to be a distributed or durable

long-term store — clustering is delegated to running two identical servers and to remote-write federation. It can refuse because monitoring data is a derived, lossy-tolerable view; nobody's balance is in it. InfluxDB sits elsewhere on the design spectrum — push ingest, its TSM storage engine (an LSM variant with Gorilla-style encodings), and a history of cardinality being *the* scaling pain point in the 1.x/2.x tag-index era, which its IOx columnar rewrite explicitly targeted.

Downsampling and retention tiers complete the picture: raw resolution for days, 5-minute rollups (min/max/avg/sum/count — keep the components, since you cannot average averages) for months, hourly for years. TimescaleDB's continuous aggregates, Thanos/Mimir compaction, and InfluxDB tasks all implement the same ladder, and the same caveat applies everywhere: a rollup discards the tails, so any percentile computed from downsampled data is an estimate resting on what the rollup preserved (store histogram buckets, not pre-computed quantiles, if you need percentiles to survive downsampling).

Analytics and OLAP

Row versus column, from first principles

An analytical query touches few columns of many rows: `SELECT region, sum(amount) FROM orders WHERE order_date >= '2026-01-01' GROUP BY region` reads three columns — `order_date`, `region`, `amount` — of a table that may have a hundred. A row store lays rows contiguously, so every page it reads carries all hundred columns; the query pays I/O for 100 columns and uses 3. A **column store** lays each column contiguously, so the query reads exactly the three columns it needs: a ~33× raw I/O reduction on this example before compression is even mentioned — and analytical tables are routinely wider and the queried subset routinely this narrow.


```

flowchart TB
    subgraph ROW["Row layout: pages carry whole rows"]
        RP["page: id,date,region,amount,+96 cols / id,date,region,amount,+96 cols / ..."]
    end
    end
    subgraph COL["Column layout: pages carry one column"]
        C1["date: 2026-01-03, 2026-01-03, 2026-01-04, ..."]
        C2["region: EU, EU, EU, US, US, ..."]
        C3["amount: 12.50, 8.00, 41.99, ..."]
        C4["96 other columns: never read"]
    end
    end
    Q["query touches 3 of 100 columns"] --> ROW
    Q --> COL
    ROW --> R10["reads 100 percent of table bytes"]
    COL --> C10["reads about 3 percent, then compression shrinks that 5 to 20x more"]

```

Compression is the second, larger win, and it falls out of homogeneity: a column is a run of values of one type, often sorted or clustered, with low local entropy. Four encoding families do most of the work. **Run-length encoding (RLE)**: a sorted or clustered column like `region` stores ("EU", 41200), ("US", 78911) instead of 120,111 strings. **Dictionary encoding**: map each distinct string to a small integer and store the integers — which also means predicates like `region = 'EU'` become integer comparisons. **Bit-packing**: a dictionary code with 200 distinct values needs 8 bits, not 32. **Delta encoding**: sorted numeric columns (IDs, timestamps) store small differences — the Gorilla timestamp trick generalized. These compose (dictionary, then RLE on the codes, then bit-pack), and 5-20× compression on real analytical data is routine. Row stores cannot get this because interleaving heterogeneous columns destroys the homogeneity the encodings feed on. The literature anchor is Stonebraker et al.'s **C-Store** paper (VLDB 2005) — the academic column store that became Vertica and whose ideas (columnar projections, compression-aware execution, a small write store merged into a read store) are visible in every system below.

Three execution techniques complete the columnar story. **Vectorized execution** — Chapter 4 flagged this — processes values in batches of a few thousand per operator call rather than one row at a time; on columnar data a batch is a contiguous array of one type, so the loop is branch-light, cache-friendly, SIMD-izable, and amortizes interpretation overhead — Volume 1's cache-behavior arguments doing query processing. Engines can often evaluate predicates *directly on compressed*

data (compare against the dictionary code; skip whole RLE runs). **Late materialization:** keep working with column-and-position vectors as long as possible, stitching values into row tuples only when the result demands it, so columns irrelevant to the filter are fetched only for the rows that survive it. And **zone maps:** per block of rows (per row group, per part granule), store min/max of each column; a query with `order_date >= '2026-01-01'` skips every block whose max date is older. This is Chapter 3's BRIN index generalized and made ubiquitous — it is why *sort order and partitioning are the most important physical design decisions in a column store*: pruning only works if the clustering makes block min/max ranges narrow for the columns you filter on.

Getting the data there: ETL, ELT, CDC

Analytics stores are loaded, not written to by applications, and the loading pipeline has its own history. Classic **ETL** transformed data in flight — nightly batch jobs reshaping OLTP rows into warehouse star schemas before loading — because warehouse compute was the scarce resource. Cheap elastic warehouse compute inverted this into **ELT**: land raw data in the warehouse first, transform it *inside* the warehouse with SQL (the dbt-style workflow), keeping the raw layer replayable when transform logic changes. The freshness frontier is **CDC streaming**: tail the OLTP system's WAL or binlog (Chapter 7's log, read by an outsider; Debezium is the standard tool), publish row-change events to a log (Volume 10, Chapter 6 covers CDC and the outbox pattern properly), and apply them continuously to the warehouse, taking freshness from "yesterday" to minutes — at the price of operating a pipeline that must handle schema changes, backfills, and exactly-once apply (in practice: idempotent upserts keyed by primary key and log position).

Parquet and the lakehouse

The columnar ideas escaped the database and became a file format. **Apache Parquet** is the de facto columnar interchange format, and its layout is a column store in a file: a file holds **row groups** (horizontal slices, typically 128 MB–1 GB); within a row group each column is a **column chunk**, encoded (dictionary, RLE/bit-packing, delta) and compressed in **pages**; and the file ends with a **footer** containing the schema, the offset of every column chunk, and per-chunk min/max statistics. The footer-at-end design means a reader fetches the footer first, then only the byte ranges of the columns and row groups the query

needs — projection and zone-map pruning against a dumb object store, no database process required.

```
orders.parquet
├─ row group 0                      (rows 0 .. 999,999)
│  └─ column chunk: order_date    [delta-encoded] min=2026-01-01
│                                     max=2026-01-09
│  └─ column chunk: region        [dictionary+RLE] min="APAC"
│                                     max="US"
│  └─ column chunk: amount        [dict/plain]      min=0.99
│                                     max=9410.00
│     └─ ... one chunk per column, each in compressed pages
├─ row group 1                      (rows 1,000,000 .. 1,999,999)
│  └─ ...
└─ footer: schema, chunk offsets, per-chunk min/max stats ← read this first
```

A directory of Parquet files is not yet a table: there is no atomic multi-file commit, no schema enforcement, no way to know which files constitute the current version. **Open table formats** — Apache Iceberg and Delta Lake, similar in aim — add exactly that missing metadata layer, and the result is called the **lakehouse**: warehouse-grade table semantics over object-store files. Conceptually (Iceberg's vocabulary): a table's current state is a metadata file pointing at a **snapshot**; a snapshot points, via manifest lists and manifests, at the exact set of data files that are the table; a commit writes new data files and swaps in a new snapshot with one atomic metadata operation. From that one design you get: atomic appends and rewrites (readers see either the old or new snapshot, never a half-written mix), **time travel** (query any retained snapshot by ID or timestamp), **schema evolution** done safely (Iceberg tracks columns by ID, not by name or position, so renames and drops do not silently corrupt old files), and pruning from manifest-level column statistics before a single data file is opened. The strategic effect is real: storage in an open format on S3/GCS, and Spark, Trino, Snowflake, BigQuery, and DuckDB all reading *the same table* — the first credible unbundling of database storage from database compute.

Exemplars: ClickHouse, the cloud warehouses, DuckDB

ClickHouse is the open-source performance benchmark of the category, and its MergeTree engine is — again — the LSM shape in columnar dress: inserts write immutable, sorted, columnar **parts**; background merges consolidate them; a sparse primary index stores one key entry

per *granule* (8,192 rows by default), which is enough to prune granule ranges for scan queries while remaining tiny. Note what the sparse index means: ClickHouse is not built for point lookups — the "primary key" defines sort order and pruning, not uniqueness.

```
CREATE TABLE events
(
    event_date    Date,
    event_time    DateTime,
    service       LowCardinality(String),
    endpoint      LowCardinality(String),
    user_id       UInt64,
    latency_ms    UInt32
)
ENGINE = MergeTree
PARTITION BY toYYYYMM(event_date)           -- Ch9 range partitioning:
retention = DROP PARTITION
ORDER BY (service, endpoint, event_time)    -- sort key: clustering for
pruning + compression
TTL event_date + INTERVAL 90 DAY;           -- declarative retention

-- Pre-aggregated rollup maintained at insert time:
CREATE MATERIALIZED VIEW events_hourly
ENGINE = SummingMergeTree
ORDER BY (service, endpoint, hour) AS
SELECT service, endpoint, toStartOfHour(event_time) AS hour,
       count() AS requests, sum(latency_ms) AS latency_sum
FROM events GROUP BY service, endpoint, hour;
```

Every clause maps to a concept from this chapter or an earlier one: the partition key is Chapter 9 (and makes retention a metadata drop, the TSDB trick); the `ORDER BY` is the clustering that makes zone-map pruning and RLE effective; `LowCardinality` is explicit dictionary encoding; the materialized view is the downsampling ladder, maintained incrementally as parts are written and merged.

BigQuery and Snowflake define the cloud warehouse's architectural contribution: **separation of storage and compute**. Data lives in replicated object storage in columnar form (Snowflake's micro-partitions with min/max metadata; BigQuery on Colossus, descended from the Dremel paper's architecture); compute is allocated per query or per "virtual warehouse," scaled elastically, and billed by use. Note the contrast with Chapter 12's Aurora, which also separates storage from compute but for OLTP — Aurora's storage tier speaks *WAL* and serves *pages* to row-oriented engines for low-latency transactions, while the

warehouses' storage speaks *columnar files* to stateless scan workers for throughput. Same slogan, different primitive, because the workload bet differs — the chapter's thesis restated inside one architectural pattern.

DuckDB is the honest counterweight to all the distributed machinery: an in-process analytical engine — the SQLite of analytics, linked into your program — with vectorized columnar execution and native Parquet reading. Its existence forces a sizing question teams skip: a single modern machine scans on the order of gigabytes per second per NVMe device with hundreds of gigabytes of RAM available, so a "big data" workload of 200 GB — which describes a great many companies' entire analytical estate — runs interactively on a laptop with zero pipeline, zero cluster, zero vendor. Distributed warehouses earn their complexity at terabytes-to-petabytes and high query concurrency, not before.

HTAP, briefly and skeptically

Hybrid transactional/analytical processing promises one system for both workloads. The credible designs concede this chapter's premise rather than refuting it: they keep **two layouts of the same data** — TiDB's TiFlash maintains columnar replicas of row-store regions (as Raft learners, so replication consistency is real); AlloyDB keeps an in-memory columnar representation alongside the Postgres row store; SQL Server's columnstore indexes are the same move in one box. You still pay for two representations in storage and write-path work; the columnar copy still trails the row store, however slightly; and heavy scans sharing hardware with OLTP tail latencies is a risk Chapter 12 should make you respect. HTAP compresses the next section's pipeline into one vendor's replication layer — genuinely valuable for freshness and simplicity at moderate scale — but it does not repeal the layout dichotomy, and at large scale the separate, asynchronously fed warehouse remains the norm.

The integration architecture: derived views of a source of truth

Step back from the individual stores and one architectural fact organizes everything: **the specialized stores are not sources of truth. They are derived views.** The catalog lives in Postgres; the search index, the metrics, and the warehouse tables are *projections* of it, maintained asynchronously. Volume 10, Chapter 6 supplies the machinery: CDC from

the OLTP WAL, or an outbox table written in the same transaction as the business write, published to a durable log (Kafka), consumed by one pipeline per derived store. Dual-writing from the application ("save to Postgres, then also POST to Elasticsearch") is the anti-pattern to name and ban: the second write fails independently, ordering across writers is undefined, and nothing reconciles the drift. The log gives every consumer the same ordered history and an offset to resume from.

```
flowchart LR
    APP["application"] -->|"transactions"| OLTP["OLTP source of truth  
Postgres: rows + WAL, Ch7"]
    OLTP -->|"CDC: WAL tailing  
or outbox, Vol 10 Ch6"| LOG["durable log  
Kafka"]
    LOG --> SPIPE["indexer"] --> SEARCH["search cluster  
Elasticsearch: inverted index"]
    LOG --> WPIPE["loader"] --> WH["warehouse or lakehouse  
ClickHouse / Iceberg: columnar"]
    APP -->|"metrics scraped, pull"| TSDB["Prometheus TSDB  
Gorilla chunks"]
    SEARCH -->|"lag: refresh + pipeline"| APP
    WH -->|"lag: minutes"| APP
```

Two properties make the architecture safe rather than merely fashionable, and both are design obligations on you:

Rebuildability. Every derived store must be reconstructible from the source of truth plus the log: re-run the indexer over a snapshot and the retained log, and the search index comes back. This is Chapter 7's recovery-by-replay promoted to system scale — *reindexing is replay* — and it is what makes derived stores forgiving: a corrupted index, a botched analyzer change, a warehouse schema migration are all handled the same way, by building the new view alongside the old and cutting over (for search, atomically, behind an index alias). The test is concrete: if losing the Elasticsearch cluster is an *incident*, you have an architecture; if it is *data loss*, you have accidentally created a second source of truth. Corollary: pipelines must be idempotent — replays deliver duplicates, so upsert by primary key rather than append.

Tolerated lag. Every arrow in that diagram is asynchronous, so every derived view answers questions about the recent past — search lags by refresh interval plus pipeline latency; the warehouse by minutes. This must surface in two places. In the product: a seller who saves a listing and immediately searches for it may not find it, so UIs read-back from the

source of truth for "your own" data — Chapter 8's read-your-writes discipline, recurring one level up. And in operations: lag is an SLO — measured per pipeline, ideally as end-to-end staleness via tracer records, with alerting — because a silently stalled indexer serving week-old results is worse than a down one. Volume 11 treats pipeline observability; the design-time obligation is that *someone owns each arrow*.

This, finally, is the honest accounting of polyglot persistence that Chapter 11 promised: each specialized store is individually excellent, and the tax is paid *between* them — in connectors, schema-evolution coordination, backfill tooling, lag monitoring, and the on-call load of N stores plus N pipelines. The architecture above is what makes the tax payable. The discipline of asking "does Postgres FTS / TimescaleDB / DuckDB suffice?" before adding a store is what keeps it small.

The distributed-systems lens

Three ideas from this chapter deserve their systems-level statement.

"One size does not fit all" is a statement about layout, and layout is destiny. Every order-of-magnitude win in this chapter came from one decision: put the data a query touches contiguously, and let compression exploit the resulting homogeneity. Term-major for search, series-major for metrics, column-major for analytics. No single layout serves all three, so at scale you *will* run multiple stores — and the interesting engineering moves from the stores to the pipelines between them.

Every specialized store is built from the primitives you already know — the same LEGO, different builds. Lucene segments, TSDB chunks, and MergeTree parts are all Chapter 2's LSM bet: immutable sorted runs, background merges, tombstoned deletes. Elasticsearch shards, Prometheus federation, and ClickHouse partitions are Chapter 9. Replica shards, remote-write, and TiFlash learners are Chapter 8. Zone maps are Chapter 3's BRIN; the translog and the Prometheus WAL are Chapter 7. What distinguishes the systems is not novel mechanisms but which guarantees each *dropped* to buy throughput for its shape: search dropped read-your-writes (refresh semantics); Prometheus dropped durable distributed storage (monitoring tolerates loss); warehouses dropped write latency (loading is batch or streaming, never a user's critical path). Reading a new store's architecture page becomes a fast exercise: find the primitives, find the dropped guarantees.

Derived-data architecture is the log-centric worldview, applied. Jay Kreps's essay *The Log* argued that a durable, ordered log is the natural integration point of a data ecosystem: the source of truth commits, the log carries history, and every specialized store is a materialized view maintained by replaying it — Chapter 7's "the log is the database; tables are caches of it," writ across systems. The properties that make single-node recovery work — replay from a known position, idempotent application — are exactly what makes cross-system pipelines safe; violating them (dual writes, non-idempotent consumers, unmonitored lag) is this decade's most reliable generator of "why does search disagree with the database" incidents. The lens matters organizationally too: in a many-team company the log is the *interface* between the team owning the source of truth and the teams owning the views, and lag SLOs are the contract between them.

Key takeaways

- **Specialized stores win on layout, not magic.** OLTP engines bet on row-major, update-in-place, point access. Search, time series, and analytics each violate the bet badly enough that a purpose-built layout — term-major, series-major, column-major — wins by orders of magnitude, chiefly via contiguity and the compression it unlocks.
- **The inverted index maps term → postings; the analyzer decides what a term is.** LIKE '%x%' has no indexable prefix and answers the wrong question anyway. Most search quality bugs are analyzer bugs — documents and queries must be analyzed identically.
- **BM25 = IDF × saturating TF × length normalization.** Rare terms matter more, repeated terms have diminishing returns (k_1), long documents are handicapped (b). It is Lucene's default and the right first thing to ship.
- **Lucene is LSM for documents.** Immutable segments, background merges, bitmap-tombstoned deletes — and therefore *near-real-time* semantics: documents are visible after refresh, not after acknowledgment. Elasticsearch distributes it with Chapter 8/9 machinery but is not a system of record. Postgres tsvector+GIN is the legitimate "good enough" tier.
- **TSDBs = time partitioning + per-series compression.** Chunk-by-time makes retention a file drop and keeps the hot set small; Gorilla's delta-of-delta timestamps and XOR'd floats reach ~1.37 bytes/point.

Cardinality is the operational hazard: series count is multiplicative in label values, so identifier-shaped labels are banned by rule.

- **Columnar wins twice: read only queried columns, then compress homogeneous data 5-20×** (RLE, dictionary, bit-packing, delta), executed vectorized with late materialization and zone-map pruning — so sort order and partitioning are the physical design decisions that matter.
- **Parquet is the columnar file; Iceberg/Delta make files a table** — atomic snapshot commits, schema evolution by field ID, time travel — enabling many engines over one copy in object storage. Storage-compute separation (BigQuery/Snowflake) is Aurora's slogan with a different primitive; DuckDB proves a laptop beats a cluster below the hundreds-of-gigabytes line. HTAP keeps two layouts of one dataset — useful, not a repeal of the dichotomy.
- **Specialized stores are derived views.** Feed them by CDC/outbox through a log, never by dual writes; require rebuildability (reindex = replay, Chapter 7 at system scale) and idempotent apply; treat lag as an SLO and design UIs to tolerate it. If losing the store is data loss rather than an incident, it has silently become a second source of truth.

Further reading

- Stonebraker, M. and Çetintemel, U., "'One Size Fits All': An Idea Whose Time Has Come and Gone," *ICDE*, 2005 — the manifesto this chapter tests against reality.
- Stonebraker, M. et al., "C-Store: A Column-oriented DBMS," *VLDB*, 2005 — the academic column store behind Vertica; projections, compression-aware execution, write/read store split.
- Abadi, D., Boncz, P., Harizopoulos, S., Idreos, S., and Madden, S., *The Design and Implementation of Modern Column-Oriented Database Systems*, Foundations and Trends in Databases, 2013 — the definitive survey of the columnar techniques in this chapter.
- Pelkonen, T. et al., "Gorilla: A Fast, Scalable, In-Memory Time Series Database," *VLDB*, 2015 — delta-of-delta timestamps, XOR value compression, and the 1.37 bytes/point result. <https://www.vldb.org/pvldb/vol8/p1816-teller.pdf>

- Robertson, S. and Zaragoza, H., "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, 2009 — BM25 from its authors, with the reasoning behind k_1 and b .
- Apache Lucene documentation — segments, merge policies, and scoring. <https://lucene.apache.org/core/documentation.html>
- Elasticsearch reference: "Near real-time search," analysis, and mapping chapters. <https://www.elastic.co/guide/en/elasticsearch/reference/current/near-real-time.html>
- PostgreSQL documentation, Chapter "Full Text Search" — tsvector, tsquery, GIN. <https://www.postgresql.org/docs/current/textsearch.html>
- Prometheus documentation — data model, storage (TSDB internals), and the practices pages on label cardinality. <https://prometheus.io/docs/>
- Apache Parquet format specification — row groups, column chunks, pages, footer metadata and statistics. <https://parquet.apache.org/docs/file-format/>
- Apache Iceberg specification — snapshots, manifests, schema evolution by field ID. <https://iceberg.apache.org/spec/>
- Armbrust, M. et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," *VLDB*, 2020 — the Delta variant of the table-format design.
- Melnik, S. et al., "Dremel: Interactive Analysis of Web-Scale Datasets," *VLDB*, 2010 — the ancestry of BigQuery and of Parquet's nested columnar encoding.
- ClickHouse documentation — MergeTree engine family, sparse primary indexes, materialized views. <https://clickhouse.com/docs/en/engines/table-engines/mergetree-family/mergetree>
- Kreps, J., "The Log: What every software engineer should know about real-time data's unifying abstraction," LinkedIn Engineering blog, 2013 — the derived-data worldview.
- Raasveldt, M. and Mühleisen, H., "DuckDB: an Embeddable Analytical Database," *SIGMOD* demo, 2019 — in-process OLAP.
- Volume 10, Chapter 6 — CDC and the outbox pattern, the feed for every store here.

- Volume 11, Chapter 2 — metrics pipelines and cardinality management in production.

Chapter 14 — Operating Databases: Pooling, Migrations, and Scaling

What this chapter covers. The previous thirteen chapters built the machine: relational foundations, storage engines, indexes, query processing, transactions, MVCC, the WAL, replication, partitioning, and the distributed variants. This closing chapter is about *running* the machine, and its thesis is blunt: most database pain in production is operational, not architectural. Teams migrate to exotic datastores to escape problems that were actually a mis-sized connection pool, an unbatched migration, a missing index, or an autovacuum that fell behind — problems with known, learnable fixes. We treat four operational levers quantitatively: connection pooling (why Postgres connections are expensive, why small pools beat big ones, and what each PgBouncer mode breaks), schema migrations as an engineering discipline (expand-and-contract, the Postgres lock-hazard catalogue, and the lock-queue pileup that turns a one-millisecond DDL into an outage), the single-node scaling ladder you must climb before reaching for Chapter 9's sharding, and the observability and backup practices that distinguish a database you operate from a database you merely hope at. A short field guide to recurring incident shapes closes the volume.

Learning goals — after this chapter you should be able to:

- Explain mechanically why a Postgres connection costs what it costs, and why `max_connections = 5000` degrades a server that would be healthy at 100.
- Size a connection pool from Little's law and the USL of Volume 4, Chapter 1, and defend a small number against the intuition that more connections mean more throughput.
- Choose between application-side pooling and a server-side pooler, select the right PgBouncer mode, and enumerate exactly what transaction pooling breaks.

- Design a zero-downtime schema change with expand-and-contract, batch a backfill safely, and wrap DDL in the `lock_timeout` -plus-retry pattern that prevents lock-queue pileups.
- Order the single-node scaling ladder correctly: query fixes, then caching, then read replicas, then vertical scaling, and only then partitioning.
- Translate the four golden signals into database terms and name the specific views and counters that measure each.
- State why replication is not a backup, and why a backup that has never been restored is a hope, not an artifact.

Connection pooling

Why connections are expensive in Postgres

PostgreSQL forks a dedicated operating-system **process** for every connection. That is not a historical accident to be embarrassed about — it buys memory isolation between sessions and predates robust threading — but it fixes the cost structure you operate under. Each backend process carries several megabytes of private memory before it does any work, and its working memory grows with activity: `work_mem` is not a per-connection cap but a per-sort, per-hash-node budget, so a single complex query can consume several multiples of it. A thousand mostly-idle connections can quietly hold gigabytes of RSS that you provisioned for the buffer cache.

Memory is the visible cost. The subtler cost is that several core operations scale with the *number of backends that exist*, not the number doing work. Taking an MVCC snapshot (Chapter 6) historically required scanning the shared process array to determine which transaction IDs were in flight — work proportional to connection count, performed at every statement in read-committed mode. PostgreSQL 14's snapshot-scalability work (Andres Freund's series of patches) dramatically reduced this penalty, and it is the reason PG14+ tolerates large idle-connection counts far better than PG12 did — but "tolerates better" is not "free": the shared lock table, the process array, and the kernel's scheduler all still carry per-backend weight. At `max_connections = 5000` on a 32-core machine you have also promised the OS scheduler up to 5000 runnable processes contending for 32 cores; when load arrives, throughput

collapses into context-switch thrash exactly as Volume 4, Chapter 1 predicts.

Connection *establishment* is expensive too — fork, TLS handshake, authentication, catalog cache warm-up — comfortably milliseconds. An application that opens a connection per request pays that tax on every request and hammers the postmaster with fork storms under load. Pooling exists to amortize establishment cost and, more importantly, to cap concurrency at the database.

Small pools outperform big ones

The counterintuitive core of pool sizing: **a smaller pool is usually faster**, not just safer. The connection pool is the canonical worked example of the Universal Scalability Law from Volume 4, Chapter 1. A database server has a finite number of cores and a finite number of I/O channels; past the point where those are saturated, each additional concurrent query adds no throughput while adding contention (α — lock waits, buffer partition locks) and coherence cost (β — cache-line traffic, context switches). The USL curve does not plateau, it *turns down*. The HikariCP project's "About Pool Sizing" page demonstrates this with an Oracle benchmark in which cutting a pool from 2,048 to 96 connections dropped query wait times from ~100 ms to ~2 ms — same hardware, same workload, more than an order of magnitude better, purely by refusing to oversubscribe.

The classical starting formula, from the PostgreSQL community via the HikariCP wiki, is:

$$\text{pool_size} \approx \text{core_count} \times 2 + \text{effective_spindle_count}$$

For a 16-core server on SSDs, that suggests something in the low tens — not hundreds. The formula is a heuristic, not a law; its real content is that the right number is proportional to the server's *parallel service capacity*, not to the client's *desire for concurrency*.

Little's law (Volume 4, Chapter 1) gives you the demand-side check. If your service issues $\lambda = 2,000$ queries/second and the mean time a connection is held per query is $W = 5$ ms, then the average number of busy connections is $L = \lambda \times W = 10$. A pool of 20 gives you $2\times$ headroom for bursts and variance. Note what the arithmetic punishes: **W is hold time, not query time**. A transaction that runs three 1 ms queries but

holds the connection for 200 ms while awaiting a downstream HTTP call has $W = 200$ ms, and your pool requirement just went up 40×. This is why Chapter 5's discipline — keep transactions short, never do external I/O inside one — is also pool-sizing discipline, and it is the same rule as "never hold a lock across I/O" from Volume 4, Chapter 2, because a pooled connection *is* a lock on a slice of the database's capacity.

When the pool is smaller than peak demand, excess requests queue at checkout. That is correct behavior: a bounded queue in front of a saturated resource is admission control, and the alternative — admitting everything — makes every request slow instead of making a few requests wait. We return to this in the distributed-systems lens.

Application-side pools: HikariCP as the exemplar

Every serious runtime has an in-process pool (HikariCP on the JVM, `pgxpool` in Go, `asyncpg`/SQLAlchemy pools in Python). HikariCP's configuration surface is a good checklist because each knob corresponds to a failure mode:

```

# HikariCP – the knobs that matter, annotated honestly
maximumPoolSize=10          # from Little's law + headroom, NOT from
thread count.
minimumIdle=10              # equal to max: a fixed-size pool. Elastic
pools re-pay

# connection setup exactly when load spikes – the worst time.
connectionTimeout=3000      # ms to wait at checkout before failing.
Fail fast: a
                             # request that waits 30s for a connection
times out
                             # upstream anyway and holds its own
resources meanwhile.
maxLifetime=1740000         # 29 min. Retire connections BEFORE any
external limit –

# server idle timeouts, LB/NAT flow expiry, failover DNS –
                             # kills them mid-query. Keep it below the
smallest of those.
keepaliveTime=60000         # periodic liveness probe so a dead peer is
noticed
                             # before a real query trips over it.
leakDetectionThreshold=10000 # log a stack trace when a connection is
held >10s.
                             # Leak detection is not optional in a
codebase where
                             # anyone can forget a close() on an error
path.

```

The three decisions worth internalizing: fix the pool size rather than letting it flex (elasticity adds connection-establishment latency precisely under load); make checkout timeout short and treat it as backpressure, not as an error to retry immediately; and run leak detection permanently, because a leaked connection is invisible until the day traffic rises and the pool arithmetic stops working.

Server-side poolers: PgBouncer and its modes

Application-side pools have a fleet-level flaw: they multiply. Forty pods with `maximumPoolSize=10` is 400 server connections — each pod's pool is individually modest and the sum still swamps the database, and autoscaling makes the sum a moving target. The fix is a **server-side pooler** between the fleet and the database. PgBouncer is the standard for Postgres: a single-threaded, event-driven proxy that speaks the Postgres wire protocol, accepts thousands of client connections cheaply (each is

just a socket and a small struct, not a process), and multiplexes them onto a small set of real server connections.

```
flowchart LR
    subgraph FLEET["App fleet - 40 pods, autoscaling"]
        P1["Pod 1"]
        HikariCP_pool_10["HikariCP pool: 10"]
        P2["Pod 2"]
        HikariCP_pool_10_2["HikariCP pool: 10"]
        PN["Pod 40"]
        HikariCP_pool_10_3["HikariCP pool: 10"]
    end
    subgraph PGB["PgBouncer - transaction mode"]
        CL["client side: up to 2000 cheap socket connections"]
        SV["server side: 20 pooled real backend connections"]
        DB["PostgreSQL"]
        max_connections["max_connections = 100"]
        busy_backends["20 busy backends, warm caches"]
        P1 --> CL
        P2 --> CL
        PN --> CL
        CL --> SV
        SV --> DB
        NOTE["transaction mode breaks: session prepared statements* session advisory locks SET without LOCAL LISTEN, WITH HOLD cursors, temp tables"]
    end
    PGB --.- NOTE
```

PgBouncer has three pooling modes, and the mode determines both the multiplexing win and what it silently breaks:

- **Session pooling.** A client keeps one server connection from connect to disconnect. Nothing breaks — full session semantics — but the multiplexing benefit collapses to amortizing connection establishment. Idle clients still pin server connections.
- **Transaction pooling.** A server connection is assigned at `BEGIN` and returned at `COMMIT / ROLLBACK` (or after each autocommit statement). This is the mode that delivers the big ratios — thousands of clients over tens of backends — because real workloads hold transactions for a small fraction of wall time. The price: **anything with session-**

scoped state breaks, because your next transaction runs on a different backend. Concretely: server-side prepared statements created with session lifetime (`PREPARE` , or driver-level prepared statements — though PgBouncer 1.21+ can track and replay protocol-level prepared statements via `max_prepared_statements` , closing the most common gap); session-level advisory locks (`pg_advisory_lock` — the transaction-scoped `pg_advisory_xact_lock` variants are safe); `SET` of session GUCs (`SET LOCAL` inside a transaction is safe; bare `SET` leaks onto whichever backend you happened to have, then vanishes); `LISTEN / NOTIFY` ; `WITH HOLD` cursors; and temporary tables that outlive a transaction.

- **Statement pooling.** The connection is returned after every statement; multi-statement transactions are rejected outright. Useful for pure autocommit workloads behind sharding middleware; rarely what you want.

Transaction pooling is the production default for high-connection-count fleets, and the breakage list above is not a footnote — it is a contract your application code must be audited against. The most common wound is self-inflicted: a driver configured to use server-side prepared statements against an old PgBouncer, producing sporadic `prepared statement "S_1" does not exist` errors that only appear under load, when transactions actually interleave across backends.

A minimal honest configuration:

```
[databases]
app = host=10.0.1.5 port=5432 dbname=app

[pgbouncer]
pool_mode = transaction
max_client_conn = 2000           ; cheap – sockets, not processes
default_pool_size = 20           ; the number that actually hits Postgres
reserve_pool_size = 5            ; burst headroom after reserve_pool_timeout
query_wait_timeout = 5           ; seconds a query may wait for a server
conn;                             ; fail fast instead of building an
invisible queue
server_idle_timeout = 300
max_prepared_statements = 200    ; 1.21+: protocol-level prepared stmt
support
```

PgBouncer's single-threaded design means one instance saturates one core; the standard scale-out is multiple instances (per-AZ, or with

`SO_REUSEPORT`). A newer generation addresses this and adds features: **pgcat** (multi-core, sharding-aware, load-balances across replicas), **Supervisor**, and cloud-managed poolers like **AWS RDS Proxy**, which adds IAM auth and faster failover but behaves like transaction pooling with its own "pinning" rules — session-state-touching queries pin a client to a backend and quietly forfeit multiplexing. The architecture and the trade-offs are the same; read the pinning rules of whichever proxy you adopt with the transaction-pooling breakage list in hand.

Pool exhaustion and timeout layering

Pools fail in two characteristic ways. **Leaks**: a code path checks out a connection and never returns it — typically an early return or exception path that skips the close. The pool shrinks one connection at a time over days, invisibly, until a traffic bump turns 100% utilization into an outage. Leak detection thresholds and metrics on `active / idle / waiting` counts are the countermeasures. **Starvation by long transactions**: a report query or an interactive session that runs `BEGIN` and then waits — on a lock, on a human, on a downstream — holds a connection for minutes. Ten of those in a pool of 20 halves your effective capacity. This is Chapter 5's transaction discipline surfacing as an operational constraint, and Postgres provides the backstop: `idle_in_transaction_session_timeout` kills sessions that sit inside an open transaction doing nothing (which also protects vacuum — see below).

Timeouts must be layered coherently, shortest at the innermost scope:

Layer	Knob (typical)	Guards against
TCP connect	driver <code>connect_timeout</code>	dead host, black-holed SYN
Pool checkout	<code>connectionTimeout</code> / <code>query_wait_timeout</code>	pool exhaustion; converts it to fast failure
Statement	<code>statement_timeout</code> (per-role or per-session)	runaway query holding a connection

Layer	Knob (typical)	Guards against
Idle-in-transaction	<code>idle_in_transaction_session_timeout</code>	abandoned transactions starving pool and vacuum
Connection lifetime	<code>maxLifetime</code> / <code>server_lifetime</code>	stale connections outliving NAT/LB/ failover

An inner timeout longer than the outer one that wraps it is a lie: if the HTTP handler deadline is 2 s, a 30 s `statement_timeout` guarantees orphaned queries that burn database capacity for clients that already hung up.

Schema migrations as an engineering discipline

Tooling: versioned, forward, in VCS

The baseline is not controversial: every schema change is a versioned migration file, committed to the same repository as the code that depends on it, applied by a tool that records applied versions in the database (Flyway's `flyway_schema_history`, Liquibase's `changelog`, or the framework-native equivalents in Rails, Django, and Alembic). Reviews happen on the SQL, not on a DBA's terminal. Down-migrations are worth writing for development, but production philosophy should be **forward-only in spirit**: you cannot "roll back" a migration that dropped data, and a failed deploy is fixed by rolling *forward* with a new migration, not by running `down` against a database that live traffic has already written to.

The hard part is not the tooling. It is that a migration is a change to shared mutable state that must be applied while N application versions are reading and writing it — which makes every nontrivial migration a compatibility problem, and the solution a choreography.

Expand-and-contract

The universal pattern for zero-downtime change is **expand-and-contract**: make the schema additively compatible with both old and new code, move the data and the traffic, then remove the old shape. Each

phase is a **separate deploy**, verified before the next begins. Compressing phases is how zero-downtime migrations cause downtime.

```
flowchart LR
    A["1. EXPAND  
add nullable column  
deploy: DDL only"] --> B["2. DUAL-WRITE  
app writes old + new  
deploy N+1"]
    B --> C["3. BACKFILL  
batched UPDATES  
no deploy - a job"]
    C --> D["4. VERIFY  
counts, checksums,  
NOT NULL check NOT VALID"]
    D --> E["5. SWITCH READS  
app reads new column  
deploy N+2"]
    E --> F["6. STOP OLD WRITES  
deploy N+3"]
    F --> G["7. CONTRACT  
drop old column  
releases later"]
```

Worked example — replacing a `numeric` money column with integer cents:

1. **Expand.** `ALTER TABLE orders ADD COLUMN total_cents bigint;` — nullable, no default. In Postgres this is a catalog-only change: it needs `ACCESS EXCLUSIVE` but holds it for milliseconds (the lock *hazard* is the queue, covered next).
2. **Dual-write.** Deploy code that writes both columns on insert and update. Old code still running elsewhere in the fleet writes only the old column — that is fine; backfill will catch those rows, which is why backfill comes *after* dual-write is fully rolled out, never before.
3. **Backfill, batched.** Never `UPDATE orders SET total_cents = ...` in one statement. A single-statement backfill of a large table (a) holds row locks and a transaction open for the duration, blocking writers and stalling vacuum's xmin horizon (Chapter 6), (b) generates the entire table's worth of WAL in one burst, spiking replication lag (Chapter 8), and (c) creates a dead tuple for every row at once, handing autovacuum a cliff. Batches bound all three:

```
#!/usr/bin/env bash
# Batched backfill: small transactions, lag-aware pacing.
set -euo pipefail
LAST_ID=0
while :; do
    LAST_ID=$(psql "$DB" -qtAX -c "
        WITH batch AS (
            SELECT id FROM orders
            WHERE id > ${LAST_ID} AND total_cents IS NULL
            ORDER BY id
            LIMIT 5000
        ),
        upd AS (
            UPDATE orders o
            SET total_cents = round(o.total * 100)::bigint
            FROM batch b WHERE o.id = b.id
            RETURNING o.id
        )
        SELECT coalesce(max(id), -1) FROM upd;")
    [ "${LAST_ID}" = "-1" ] && break # nothing left to do

    # Pace against the replicas: do not outrun apply on the standbys.
    LAG=$(psql "$DB" -qtAX -c "
        SELECT coalesce(max(extract(epoch FROM replay_lag)), 0)::int
        FROM pg_stat_replication;")
    if [ "${LAG}" -gt 10 ]; then sleep 30; else sleep 0.5; fi
done
```

Each batch is its own short transaction; the sleep yields to foreground load; the lag check is the difference between a backfill and a replication incident. 4. **Verify.** Count rows where the columns disagree; if the end state needs `NOT NULL`, add it as `ALTER TABLE orders ADD CONSTRAINT total_cents_nn CHECK (total_cents IS NOT NULL) NOT VALID`; then `VALIDATE CONSTRAINT` — validation scans the table under a lock that does not block reads or writes, and on PostgreSQL 12+ a subsequent `SET NOT NULL` can use the validated constraint as proof and skip its own full-table scan. 5-7. **Switch reads, stop old writes, contract.** Each its own deploy; the drop waits until no deployable artifact references the old column — including ad-hoc analytics and that one cron job nobody remembers.

The pattern generalizes: splitting a table, moving a column between tables, changing a type — all are expand, dual-write, backfill, verify, cut over, contract. It is slow by design. The speed of a schema change is not the time the DDL takes; it is the number of release cycles the

choreography takes, and attempting to shortcut that is the single most common cause of migration incidents.

The Postgres lock-hazard catalogue

Every `ALTER TABLE` takes a lock; the operational question is which lock and **for how long**. The taxonomy that matters:

Operation	Lock	Duration	Notes
<code>ADD COLUMN</code> (nullable, no default)	ACCESS EXCLUSIVE	ms — catalog only	safe with <code>lock_timeout</code>
<code>ADD COLUMN ... DEFAULT <constant></code>	ACCESS EXCLUSIVE	PG11+: ms (default stored as metadata)	pre-11: full table rewrite under the lock — a classic multi-hour outage
<code>ADD COLUMN ... DEFAULT <volatile fn></code>	ACCESS EXCLUSIVE	full rewrite, even on PG11+	e.g. <code>DEFAULT gen_random_uuid()</code>
<code>ALTER COLUMN TYPE</code> (non-binary-compatible)	ACCESS EXCLUSIVE	full rewrite + index rebuilds	use expand-and-contract instead
<code>SET NOT NULL</code>	ACCESS EXCLUSIVE	full scan (PG12+: skipped if a valid CHECK proves it)	use the NOT VALID two-step
<code>ADD FOREIGN KEY</code>	locks both tables	validation scan	<code>NOT VALID + VALIDATE CONSTRAINT</code> splits lock from scan
<code>CREATE INDEX</code>	SHARE	blocks all writes for the build	never on a live table
<code>CREATE INDEX CONCURRENTLY</code>	SHARE UPDATE EXCLUSIVE	writes proceed	see caveats below

Operation	Lock	Duration	Notes
DROP COLUMN	ACCESS EXCLUSIVE	ms — column is hidden, not erased	space reclaimed lazily by rewrites

`CREATE INDEX CONCURRENTLY` (Chapter 3) is the tool for indexing live tables, with three caveats that bite: it cannot run inside a transaction block (so your migration tool needs its non-transactional mode); it is slower — two table scans plus waits for every transaction that might use the index to finish; and **on failure it leaves behind an INVALID index** that consumes space and write overhead while serving no queries — you must `DROP INDEX CONCURRENTLY` and retry. Monitor for invalid indexes; they are a silent tax.

The catalogue above understates the real hazard, though, because duration-of-lock-held is not the whole story. The killer is duration-of-lock-*waited*:

The lock-queue pileup

Postgres lock queues are fair: a requested `ACCESS EXCLUSIVE` lock waits behind existing holders, and **everything requested after it waits behind it** — including plain `SELECT`s that only need `ACCESS SHARE`. So a "safe," millisecond, catalog-only `ADD COLUMN` deployed while one long-running report holds `ACCESS SHARE` becomes a plug: the `ALTER` waits on the report, every query on the table queues behind the `ALTER`, the connection pool fills with waiters, checkout timeouts fire fleet-wide, and you have a full outage caused by a DDL that "doesn't lock anything." This is among the most common self-inflicted database outages in industry.

```

sequenceDiagram
    participant R as Report query
    participant M as Migration
    participant W as Web traffic
    participant L as PG lock manager

    R->>L: SELECT on orders – holds ACCESS SHARE, runs 10 min
    M->>L: ALTER TABLE orders ADD COLUMN – needs ACCESS EXCLUSIVE
    Note over M,L: ALTER queues behind the report
    W->>L: ordinary SELECTs – need ACCESS SHARE
    Note over W,L: fair queue: they wait BEHIND the ALTER
    all traffic on orders now blocks
    Note over W: pool fills with waiters – checkout
    timeouts – fleet-wide outage
    L-->>M: lock_timeout 2s fires – ALTER aborts
    L-->>W: queue drains, traffic recovers in seconds
    Note over M: retry with backoff, off-peak

```

The safety net is mechanical and should be baked into your migration tooling, not left to memory: set a short `lock_timeout` for every DDL statement, and retry with backoff.

```

# lock_timeout + retry: the DDL either gets the lock fast or gets out
# of the way.
for attempt in 1 2 3 4 5; do
    if psql "$DB" -v ON_ERROR_STOP=1 -c "
        SET lock_timeout = '2s';
        ALTER TABLE orders ADD COLUMN total_cents bigint;"
    then
        break
    fi
    echo "lock_timeout hit, attempt ${attempt}; backing off"
    sleep $((attempt * 10))
done

```

With a 2-second `lock_timeout`, the worst case is a 2-second stall on the table, which the retries repeat until a gap in long-running queries lets the DDL through. Pair it with monitoring for long-running transactions before migrating, and with `statement_timeout` kept separate — `lock_timeout` bounds waiting for the lock, `statement_timeout` bounds the work after acquiring it, and rewriting DDL needs the second one generous while the first stays tight.

MySQL, briefly

InnoDB's online DDL landscape differs in mechanism but not in discipline. `ALTER TABLE` declares an algorithm: `INSTANT` (metadata-only — adding a column, available since 8.0.12 and generalized in 8.0.29), `INPLACE` (the table is modified without a full copy, often permitting concurrent DML), and `COPY` (full rebuild, blocking). When the built-in algorithms are insufficient or the risk is too high, the ecosystem's answer is the **shadow-table** approach: GitHub's `gh-ost` creates a ghost copy of the table, copies rows in throttled batches while tailing the binlog to apply concurrent changes, then performs an atomic rename cut-over; Percona's `pt-online-schema-change` does the same with triggers instead of binlog tailing. Both are expand-and-contract automated at the single-table level — the same idea, productized.

Destructive changes are quarantined

Renames and drops get their own rule: **never rename in place**. There is no deploy ordering under which `RENAME COLUMN` works with a fleet running two application versions — old code breaks the instant the rename lands, or new code breaks until it does. A rename is therefore always add-new → dual-write → migrate readers → drop-old: a multi-release choreography for what feels like a one-line change. Drops deserve a quarantine period even after code references reach zero — a release or two of the column existing unused, plus a check of query logs, before the irreversible statement runs. The cost of carrying a dead column for a month is nearly zero; the cost of discovering a forgotten consumer after the drop is a restore drill you did not schedule.

Finally, migration testing: a migration validated only against a developer database or a thin staging dataset has been tested for syntax, not for behavior. Lock duration, rewrite time, backfill runtime, and autovacuum interaction are all functions of data volume and shape. The staging-data-size lie — "it ran in 40 ms in staging" — is how five-hour table rewrites reach production. Test destructive or rewriting migrations against a restored production snapshot (which, usefully, doubles as your restore drill) or a masked production-shaped dataset, and record expected durations in the migration's review.

Scaling the single node first

Chapters 9 through 12 exist, and this section's job is to delay your need for them. A modern server — dozens of cores, hundreds of gigabytes of RAM, NVMe storage delivering hundreds of thousands of IOPS — runs a well-tuned Postgres at tens of thousands of transactions per second. Most systems that "outgrew Postgres" outgrew an untuned Postgres, and bought distributed-systems complexity (Chapter 10's coordination costs, Chapter 9's cross-shard queries) to avoid an afternoon of profiling. Climb the ladder in order:

```
flowchart TD
    S["Database is slow or saturated"] --> Q1["Examine top queries?  
pg_stat_statements"]
    Q1 -->|no| FIX["Step 0: fix queries  
indexes, N+1s, EXPLAIN  
cheapest, highest yield"]
    FIX --> S
    Q1 -->|yes| Q2["Read-heavy and  
tolerates staleness?"]
    Q2 -->|yes| CACHE["Cache or read replicas  
accept invalidation debt  
and lag discipline"]
    CACHE --> S
    Q2 -->|no| Q3["Bigger box still  
meaningfully bigger?"]
    Q3 -->|yes| VERT["Vertical scale  
hardware is cheaper than  
engineering time"]
    VERT --> S
    Q3 -->|no| SHARD["Now Chapter 9:  
partitioning and sharding  
the complexity is real"]
```

Step zero is query optimization, because it is the only step that *removes* load rather than redistributing it. `pg_stat_statements` (MySQL: `performance_schema` / `sys.statement_analysis`) aggregates every query by normalized fingerprint; the top ten by `total_exec_time` almost always contain most of the answer:

```

SELECT queryid,
       calls,
       round(mean_exec_time::numeric, 2) AS mean_ms,
       round(total_exec_time::numeric)   AS total_ms,
       rows,
       shared_blks_read
-- actual disk-ish reads, not cache hits
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

```

The recurring finds: a missing index turning a point lookup into a scan (Chapter 3), an ORM-generated N+1 issuing ten thousand 1 ms queries where one join would do (the API chattiness problem of Volume 8 in miniature — high `calls`, innocuous `mean_ms`, dominant `total_ms`), and a query whose plan regressed after data growth crossed a planner threshold (`EXPLAIN (ANALYZE, BUFFERS)` discipline — Chapter 4). A single fixed query can return more capacity than a hardware generation.

Caching is powerful and should be adopted with honest accounting. An app-level cache in front of the database converts read load into invalidation obligations — Volume 7, Chapter 3's territory — and every cached read is a consistency decision made implicitly. Two honest questions before caching a query: is this query *slow for a fixable reason* (in which case the cache is hiding a bug you will meet again at invalidation time), and what is the blast radius when this cache is cold (see the thundering-herd pattern in the incident guide below)?

Read replicas (Chapter 8) scale reads nearly linearly for workloads that tolerate replication lag, and the discipline is entirely about lag: read-your-writes violations appear exactly when lag spikes, which is exactly when load is high, which is exactly when the replicas are most needed. Route reads that require freshness to the primary explicitly; treat "read from replica" as a per-query decision with a staleness budget, not a global switch.

Vertical scaling is underrated by engineers and correctly rated by accountants. Doubling memory so the working set fits in cache, or moving to NVMe, is often a same-day change that buys years — and its fully-loaded cost is a fraction of one engineer-year spent sharding. Its limits are real: the price curve turns convex at the top of the instance range, failover of a huge single node is slower, and write throughput eventually meets the WAL's serial ceiling (Chapter 7). When you can no

longer buy a meaningfully bigger box — *and* the top of `pg_stat_statements` is clean — then partitioning and sharding (Chapter 9) and the NewSQL systems that automate them (Chapter 12) are the honest next rung, with their costs now justified.

Observability for databases

Volume 11's golden signals — latency, traffic, errors, saturation — translate directly; the work is knowing which counters embody them.

Saturation is the signal databases die by, and it is multi-dimensional: connection utilization ($\text{busy} \div \text{max}$, both at the pooler and at Postgres), CPU, I/O (device utilization and queue depth), and **replication lag** (`pg_stat_replication.replay_lag` on the primary, `pg_last_xact_replay_timestamp` deltas on standbys). Each has a different failure signature and all deserve alerts at levels below catastrophe.

Latency must be per-query-fingerprint, not global — a global p99 is the average of a bimodal distribution of cheap point reads and expensive reports, and it moves for reasons that identify nothing. `pg_stat_statements.queryid` is the fingerprint; exporting `mean_exec_time` and `calls` per fingerprint gives you per-query-shape latency trends, which is the database equivalent of per-endpoint SLIs. Complement the aggregates with **slow-query logging** — `log_min_duration_statement` set to a meaningful threshold — and on busy systems use `log_min_duration_sample` with `log_statement_sample_rate` (PG13+) so a latency regression does not turn the log volume itself into an incident (see disk-full, below).

Lock and wait-event monitoring. `pg_stat_activity.wait_event_type / wait_event` tell you what backends are waiting *on* — `Lock`, `IO`, `LWLock`, `Client` — and a sudden shift in the wait-event mix diagnoses incidents faster than any latency graph. For lock waits specifically, `pg_blocking_pids()` answers who-blocks-whom directly:

```
-- Who is blocked, on whom, and how old is the blocker's transaction?
SELECT w.pid                                AS waiting_pid,
       now() - w.query_start                AS waiting_for,
       left(w.query, 60)                    AS waiting_query,
       b.pid                                AS blocking_pid,
       b.state                              AS blocking_state,
       now() - b.xact_start                 AS blocking_xact_age,
       left(b.query, 60)                    AS blocking_query
FROM pg_stat_activity w
JOIN LATERAL unnest(pg_blocking_pids(w.pid)) AS blk(pid) ON true
JOIN pg_stat_activity b ON b.pid = blk.pid
ORDER BY waiting_for DESC;
```

Run under pressure, this query names the idle-in-transaction session or the queued DDL at the head of a pileup in one read. It belongs in a runbook, not in someone's memory.

Vacuum and bloat. Chapter 6 established that MVCC defers cleanup to vacuum; the operational consequence is that vacuum health is a first-class signal. `pg_stat_user_tables` exposes `n_dead_tup` and `last_autovacuum` per table; dead-tuple counts trending up while `last_autovacuum` ages means autovacuum is losing. The default `autovacuum_vacuum_scale_factor` of 0.2 means a billion-row table accumulates 200 million dead tuples before vacuum triggers — hot large tables need per-table overrides (a small `autovacuum_vacuum_scale_factor` or a flat `autovacuum_vacuum_threshold`), and a throttled cluster needs `autovacuum_vacuum_cost_limit` raised, because the default cost limits were chosen for spinning disks and are far too shy for NVMe. The **wraparound** scenario deserves sober description rather than panic: transaction IDs are 32-bit and comparisons are circular, so tables must be frozen before their oldest unfrozen XID falls ~2 billion transactions behind; autovacuum runs aggressive freezing at `autovacuum_freeze_max_age` (default 200 million), modern versions add a failsafe vacuum mode, and the database will refuse new writes rather than corrupt data if the horizon is nearly exhausted. In practice wraparound emergencies are always preceded by weeks of visible symptoms — `datfrozenxid` age climbing unbounded, usually because a forgotten replication slot, a `prepared transaction`, or an eternal idle-in-transaction session pinned the xmin horizon. Monitor XID age; the emergency is optional.

Capacity trending closes the loop: disk-space growth rate (with WAL and log volumes tracked separately — each has its own leak modes), WAL

bytes per second (Chapter 7 — it predicts both replication bandwidth and backup size), and connection counts over weeks. Databases rarely fail suddenly; they fail on trend lines nobody plotted.

Backup and restore: the discipline that actually matters

Everything else in this chapter degrades service; losing data ends it. The mechanics follow directly from earlier chapters. **Logical backups** (`pg_dump`) serialize schema and rows: portable across versions and architectures, selective, and human-auditable — but restores rebuild every index and are far too slow for large databases' recovery objectives, and a dump is a single point in time. **Physical backups** copy the data directory (`pg_basebackup`, or tools like `pgBackRest` and `WAL-G` that add compression, incrementals, and object-store targets) and pair with **continuous WAL archiving** (Chapter 7). The pairing is what buys **point-in-time recovery**: restore the base backup, replay archived WAL to any chosen instant — including the instant *before* the bad deploy ran `UPDATE` without a `WHERE`. Your recovery-point objective is set by archive frequency; your recovery-time objective by base-backup age (more WAL to replay) and restore bandwidth.

Two principles carry more weight than all the tooling:

Replication is not backup. Chapter 8's replicas protect against hardware failure and nothing else, because replication's entire purpose is to faithfully copy every write — including `DROP TABLE`. A delayed replica (`recovery_min_apply_delay`) widens the window for catching a mistake and can dramatically shorten recovery when caught in time, and it is a genuinely useful tool — but it is a complement to PITR, not a substitute, because the delay you configured is a bet about how fast you notice mistakes.

A backup you have not restored is a hope. Backup jobs fail silently in every way imaginable: credentials rot, object-store lifecycle rules delete what you meant to keep, the archive is complete but the one WAL segment needed for consistency is not, the restore works but takes nineteen hours against a four-hour RTO. The only test is a **scheduled restore drill**: periodically restore the latest backup to a scratch instance, replay WAL to a target time, run integrity checks, and *record the elapsed time* as your measured RTO. Automate it — the restore-for-migration-

testing practice above and this drill can be the same pipeline — and alert when the drill fails or the measured RTO drifts past the objective. Organizations discover their backups do not work at exactly one of two times: during a drill, or during the incident.

Incident patterns: a field guide

Database incidents cluster into a small number of shapes. Knowing them converts an hour of confused dashboard-staring into minutes of pattern-matching — which is the argument (Volume 11, Chapter 5) for writing each of these up as a runbook with the diagnostic queries inline.

- **Connection-pool exhaustion cascade.** Something slows the database → hold times rise → Little's law fills the pool → checkouts queue and time out → app threads block → upstream health checks fail → retries add load. The pool is rarely the cause; it is the messenger. Diagnosis: pool wait metrics plus `pg_stat_activity` state counts — many `active` means the database is slow (find the query); many `idle in transaction` means the app is holding (find the code path).
- **Lock-queue pileup behind DDL.** The sequence diagram above. Signature: sudden fleet-wide blocking scoped to one table, `wait_event_type = Lock`, a migration in the deploy log. The who-blocks-whom query names it; killing the DDL (or the long-runner ahead of it) drains the queue instantly. Prevention: `lock_timeout` + retry, always.
- **Autovacuum death spiral.** Load rises → autovacuum throttled or cancelled by lock conflicts → dead tuples accumulate → scans read more pages for the same rows → queries slow → load rises. Slow-burn onset over days, visible in `n_dead_tup` trends. Recovery usually requires raising vacuum cost limits and letting it catch up during a quiet window — and finding the xmin-pinning session, if any.
- **Replication-lag spiral.** A WAL burst (bulk update, unbatched backfill, rewriting DDL) exceeds replica apply speed → lag grows → lag-sensitive reads fail over to the primary → primary load rises → more WAL. Prevention is the lag-aware pacing shown in the backfill loop; response is shedding read load or relaxing freshness requirements temporarily.

- **Disk full.** The database's uniquely unforgiving resource. Recurring causes: WAL retention leaks — an **orphaned replication slot** pins WAL forever (monitor `pg_replication_slots` for inactive slots; `max_slot_wal_keep_size` is the modern backstop), a broken `archive_command` blocks WAL recycling; and log spam — an error loop at thousands of lines per second can fill a volume in hours (which is why sampled logging matters). Postgres stops accepting writes when WAL space is gone; recovery under pressure risks the classic fatal error of deleting WAL files by hand. Runbook this one *before* you need it.
- **Thundering-herd cache expiry.** A popular cache key expires → every concurrent request misses simultaneously → the database absorbs the full read load of a query it had not seen in hours. The fixes are Volume 4, Chapter 9's singleflight (collapse concurrent identical misses into one loader), jittered TTLs, and serve-stale-while-revalidating. The database-side signature: a familiar fingerprint in `pg_stat_statements` whose `calls` graph is a vertical line.

Managed databases: what "managed" actually covers

RDS, Cloud SQL, and their peers (Volume 12, Chapter 6 treats the platform side) own real work: hardware, OS patching, minor-version upgrades, backup execution, failover machinery, and replica provisioning. What they do not own is everything this chapter discussed: your schema, your queries, your indexes, your pool sizing, your migration choreography, your capacity planning, and — critically — **verifying that restores meet your RTO**. The provider guarantees a backup exists; the drill is still yours. "Managed" removes the operating system from your pager, not the database from your engineering. Two managed-specific habits: keep parameter groups in infrastructure-as-code, because console-edited parameters drift silently between environments and "staging behaves differently" is frequently a parameter-group diff; and learn the provider's failover behavior empirically (connection handling, DNS TTLs, typical duration) *before* the first unplanned one.

The distributed-systems lens

The database is usually the stateful heart of a service architecture — the one component that cannot be made stateless, restarted casually, or

trivially horizontally scaled. That position makes its operational ceilings *fleet-wide* facts. When the database saturates, the backpressure propagates outward through every service that touches it (Volume 10, Chapter 7): pools fill, then request queues, then upstream timeouts — one slow table can become an organization-wide brownout. This is why the data tier's admission control matters so much: **a bounded connection pool is a bulkhead** in exactly Volume 4, Chapter 9's sense, the same semaphore math applied at the data tier, deciding *where* the queueing happens (at checkout, visibly, with a timeout) rather than *whether* it happens.

Migrations, seen through the same lens, are distributed deployments. At any moment during a rollout, one schema version must interoperate with N application versions — plus canaries, plus the instance that failed to drain. That is precisely the compatibility discipline Volume 8 applies to API and message-schema evolution (Chapters 5 and 8): additive changes first, consumers before producers, destructive changes only after every reader is gone. Expand-and-contract is not a database trick; it is the universal protocol for changing a shared contract under rolling deployment, and DDL is simply the case where the contract is enforced by a lock manager.

Finally, the organizational lesson (Volume 15's territory): none of these levers should be rediscovered per-team. Mature platform organizations productize them — a golden connection-pool configuration baked into service templates; migration linting in CI that rejects `RENAME COLUMN`, `un-CONCURRENTLY` index builds, and DDL without `lock_timeout`; automated restore drills with RTO dashboards; runbooks for the six incident shapes above pre-written with the diagnostic queries inline. The difference between an organization where databases are a recurring source of outages and one where they are boring is rarely the database. It is whether these operational levers are individual heroics or paved road.

Key takeaways

- **Most production database pain is operational.** Pools, migrations, vacuum, observability, and backups are learnable levers; exhaust them before buying architectural complexity.

- Postgres connections are **processes**: per-backend memory plus per-backend costs in snapshots and shared structures. Huge `max_connections` degrades the whole server; cap concurrency with pools instead.
- **Small pools outperform big ones** — the connection pool is the canonical USL example. Size from Little's law ($\text{connections} \approx \lambda \times W$, with W as *hold* time) plus headroom, and treat checkout queueing as admission control, not failure.
- **PgBouncer transaction pooling** delivers the big multiplexing ratios and breaks session state: session prepared statements (pre-1.21), session advisory locks, bare `SET`, `LISTEN`, `WITH HOLD` cursors, temp tables. Audit code against that list.
- **Layer timeouts** — connect, checkout, statement, idle-in-transaction, lifetime — shortest innermost, and never let an inner timeout exceed the outer deadline.
- Zero-downtime schema change is **expand → dual-write → batched backfill → verify → switch reads → contract**, each phase a separate deploy. Backfills are batched because of lock scope, WAL/replication-lag bursts, and vacuum pressure.
- The DDL hazard is the **lock queue**, not the lock: a waiting ACCESS EXCLUSIVE blocks everything behind it. `lock_timeout` plus retry is mandatory, tool-enforced. Know the ALTER catalogue (PG11 made constant defaults metadata-only; `CREATE INDEX CONCURRENTLY` can leave INVALID indexes; `NOT VALID` + `VALIDATE` splits locks from scans). **Never rename in place.**
- Climb the **single-node ladder in order**: fix top queries (`pg_stat_statements`, N+1s, EXPLAIN), cache honestly, add read replicas with lag discipline, scale vertically — and only then shard.
- Database observability = golden signals translated: saturation is connections/CPU/IO/replication lag; latency is per-fingerprint; watch wait events, who-blocks-whom, dead tuples, XID age, and the capacity trend lines.
- **Replication is not backup** (deletes replicate), and **an unrestored backup is a hope**. PITR = base backup + WAL archive; scheduled restore drills measure the real RTO.
- Managed databases outsource the OS, not the engineering: schema, queries, pools, capacity, and restore verification remain yours.

- Pool limits are **bulkheads at the data tier**; migrations are **distributed deployments** governed by the same compatibility rules as APIs; platform teams should productize all of it as paved road.

Further reading

- Wooldridge, B., "About Pool Sizing," HikariCP wiki — the pool-sizing argument, the benchmark, and the `cores × 2 + spindles` heuristic. <https://github.com/brettwooldridge/HikariCP/wiki/About-Pool-Sizing>
- PgBouncer documentation — pooling modes and the features-per-mode compatibility table; read it before enabling transaction mode. <https://www.pgouncer.org/features.html>
- PostgreSQL documentation, "Explicit Locking" — the lock-mode conflict matrix that underlies every DDL hazard in this chapter. <https://www.postgresql.org/docs/current/explicit-locking.html>
- PostgreSQL documentation, `ALTER TABLE` — notes on which forms rewrite the table and which are metadata-only. <https://www.postgresql.org/docs/current/sql-altertable.html>
- PostgreSQL documentation, "Routine Vacuuming" — autovacuum tuning, freeze ages, and the authoritative description of transaction-ID wraparound. <https://www.postgresql.org/docs/current/routine-vacuuming.html>
- PostgreSQL documentation, "Continuous Archiving and Point-in-Time Recovery" — the base-backup-plus-WAL model in full. <https://www.postgresql.org/docs/current/continuous-archiving.html>
- gh-ost documentation — the binlog-based shadow-table design and its cut-over algorithm; the clearest description of online schema change as a system. <https://github.com/github/gh-ost/blob/master/doc/cheatsheet.md>
- MySQL 8.0 Reference Manual, "Online DDL Operations" — the `INSTANT/INPLACE/COPY` algorithm tables. <https://dev.mysql.com/doc/refman/8.0/en/innodb-online-ddl-operations.html>
- Kreps, J., "The Log: What every software engineer should know about real-time data's unifying abstraction" (2013) — the conceptual frame behind WAL shipping, replication, and PITR alike. <https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying>

- Beyer, B. et al., *Site Reliability Engineering* (O'Reilly, 2016), Chapter 6 — the golden signals this chapter translated into database terms.
- Volume 4, Chapter 1 — the USL and Little's law that pool sizing instantiates; Volume 4, Chapter 9 — bulkheads and singleflight.
- Chapter 6 — MVCC and vacuum; Chapter 7 — the WAL; Chapter 8 — replication and lag; Chapter 9 — partitioning, when the ladder truly runs out.